

راهنمای برنامه نویسی رقابتی

آنتی لاکسون

ترجمه فارسی

۱۴ مهر ۱۴۰۵

فهرست مطالب

پیش‌گفتار

ث

اول تکنیک‌های پایه

۱

۳	۱	مقدمه
۳	۱.۱	زبان‌های برنامه‌نویسی
۴	۲.۱	ورودی و خروجی
۶	۳.۱	کار با اعداد
۸	۴.۱	کوتاه‌نویسی کد
۱۰	۵.۱	ریاضیات
۱۵	۶.۱	مسابقات و منابع

۱۷	۲	پیچیدگی زمانی
۱۷	۱.۲	قواعد محاسبه
۲۰	۲.۲	رده‌های پیچیدگی
۲۱	۳.۲	تخمین کارایی
۲۱	۴.۲	بیشینه مجموع زیرآرایه

۲۵	۳	مرتب‌سازی
۲۵	۱.۳	نظریه مرتب‌سازی
۲۹	۲.۳	مرتب‌سازی در C++
۳۱	۳.۳	جستجوی دودویی

۳۵	۴	ساختمان داده‌ها
۳۵	۱.۴	آرایه‌های پویا
۳۷	۲.۴	ساختارهای مجموعه
۳۸	۳.۴	ساختارهای نگاشت (Map)
۳۹	۴.۴	تکرارکننده‌ها و بازه‌ها
۴۲	۵.۴	سایر ساختارها
۴۶	۶.۴	مقایسه با مرتب‌سازی

۴۹	۵	جستجوی کامل
۴۹	۱.۵	تولید زیرمجموعه‌ها
۵۱	۲.۵	تولید جایگشت‌ها
۵۲	۳.۵	عقب‌گرد
۵۴	۴.۵	هرس کردن جستجو

۵.۵ ملاقات در میانه (Meet the in middle) ۵۷

۶ الگوریتم‌های حریصانه

۱.۶ مسئله سکه ۵۹
۲.۶ زمان‌بندی ۶۰
۳.۶ وظایف و مهلت‌ها ۶۲
۴.۶ کمینه‌سازی مجموع‌ها ۶۳
۵.۶ فشردن‌سازی داده‌ها ۶۴

۷ برنامه‌نویسی پویا

۱.۷ مسئله سکه ۶۷
۲.۷ طولانی‌ترین زیردنباله صعودی ۷۲
۳.۷ مسیرها در جدول ۷۳
۴.۷ مسائل کوله‌پشتی ۷۴
۵.۷ فاصله ویرایشی ۷۶
۶.۷ شمارش کاشی‌کاری‌ها ۷۷

۸ تحلیل سرشکن

۱.۸ روش دو اشاره‌گر ۷۹
۲.۸ نزدیک‌ترین عناصر کوچک‌تر ۸۱
۳.۸ کمینه در پنجره لغزان ۸۲

۹ پرسش‌های بازه‌ای

۱.۹ پرسش‌های آرایه ایستا ۸۶
۲.۹ درخت ایندکس دودویی ۸۸
۳.۹ درخت بازه‌ای ۹۱
۴.۹ تکنیک‌های تکمیلی ۹۵

۱۰ عملیات بیتی

۱.۱۰ نمایش بیتی ۹۷
۲.۱۰ عملیات بیتی ۹۸
۳.۱۰ نمایش مجموعه‌ها ۱۰۰
۴.۱۰ بهینه‌سازی‌های بیتی ۱۰۲
۵.۱۰ برنامه‌نویسی پویا ۱۰۴

دوم الگوریتم‌های گراف

۱۱ مبانی گراف

۱.۱۱ اصطلاحات گراف ۱۱۱
۲.۱۱ نمایش گراف ۱۱۵

۱۲ پیمایش گراف

۱.۱۲ جستجوی اول عمق ۱۱۹
۲.۱۲ جستجوی اول سطح ۱۲۱
۳.۱۲ کاربردها ۱۲۳

۱۲۵	۱۳ کوتاه‌ترین مسیرها
۱۲۵	۱.۱۳ الگوریتم بلمن-فورد
۱۲۸	۲.۱۳ الگوریتم دایکسترا
۱۳۱	۳.۱۳ الگوریتم فلویید-وارشال
۱۳۵	۱۴ الگوریتم‌های درخت
۱۳۶	۱.۱۴ پیمایش درخت
۱۳۷	۲.۱۴ قطر
۱۳۹	۳.۱۴ همه طولانی‌ترین مسیرها
۱۴۱	۴.۱۴ درخت‌های دودویی
۱۴۳	۱۵ درخت‌های فراگیر
۱۴۴	۱.۱۵ الگوریتم کروسکال
۱۴۷	۲.۱۵ ساختار مجموعه‌های مجزا
۱۴۹	۳.۱۵ الگوریتم پریم
۱۵۱	۱۶ گراف‌های جهت‌دار
۱۵۱	۱.۱۶ مرتب‌سازی توپولوژیکی
۱۵۳	۲.۱۶ برنامه‌نویسی پویا
۱۵۶	۳.۱۶ مسیرهای جانشین
۱۵۷	۴.۱۶ تشخیص دور
۱۵۹	۱۷ همبندی قوی
۱۶۰	۱.۱۷ الگوریتم کوساراجو
۱۶۲	۲.۱۷ مسئله PSAT
۱۶۵	۱۸ پرسش‌های درختی
۱۶۵	۱.۱۸ یافتن نیاکان
۱۶۶	۲.۱۸ زیردرخت‌ها و مسیرها
۱۶۹	۳.۱۸ پایین‌ترین جد مشترک
۱۷۲	۴.۱۸ الگوریتم‌های برون‌خط
۱۷۵	۱۹ مسیرها و مدارها
۱۷۵	۱.۱۹ مسیرهای اویلری
۱۷۹	۲.۱۹ مسیرهای همیلتونی
۱۸۰	۳.۱۹ دنباله‌های دی بروین
۱۸۰	۴.۱۹ گشت‌های اسب
۱۸۳	۲۰ شاره‌ها و برش‌ها
۱۸۴	۱.۲۰ الگوریتم فورد-فالکرسون
۱۸۸	۲.۲۰ مسیرهای مجزا
۱۸۹	۳.۲۰ تطابق‌های بیشینه
۱۹۲	۴.۲۰ پوشش‌های مسیری

سوم مباحث پیشرفته

۱۹۵

۲۱ نظریه اعداد

۱۹۷

- ۱.۲۱ اعداد اول و مقسوم علیه ها ۱۹۷
 ۲.۲۱ حساب پیمانه ای ۲۰۱
 ۳.۲۱ حل معادلات ۲۰۴
 ۴.۲۱ سایر نتایج ۲۰۵

۲۲ ترکیبیات

۲۰۷

- ۱.۲۲ ضرایب دوجمله ای ۲۰۸
 ۲.۲۲ اعداد کاتالان ۲۱۰
 ۳.۲۲ اصل شمول و عدم شمول ۲۱۲
 ۴.۲۲ لم برنساید ۲۱۴
 ۵.۲۲ فرمول کیلی ۲۱۵

۲۳ ماتریس ها

۲۱۷

- ۱.۲۳ عملگرها ۲۱۷
 ۲.۲۳ رابطه های بازگشتی خطی ۲۲۰
 ۳.۲۳ گراف ها و ماتریس ها ۲۲۲

۲۴ احتمال

۲۲۵

- ۱.۲۴ محاسبه ۲۲۵
 ۲.۲۴ پیشامدها ۲۲۶
 ۳.۲۴ متغیرهای تصادفی ۲۲۸
 ۴.۲۴ زنجیره های مارکوف ۲۳۰
 ۵.۲۴ الگوریتم های تصادفی ۲۳۱

۲۵ نظریه بازی ها

۲۳۵

- ۱.۲۵ وضعیت های بازی ۲۳۵
 ۲.۲۵ بازی نیم ۲۳۷
 ۳.۲۵ قضیه اسپرگ-گراندى (Sprague-Grundy theorem) ۲۳۸

۲۶ الگوریتم های رشته

۲۴۳

- ۱.۲۶ اصطلاحات رشته ۲۴۳
 ۲.۲۶ ساختار ترای ۲۴۴
 ۳.۲۶ هشینگ رشته ۲۴۵
 ۴.۲۶ الگوریتم Z ۲۴۷

۲۷ الگوریتم های ریشه دوم

۲۵۱

- ۱.۲۷ ترکیب الگوریتم ها ۲۵۲
 ۲.۲۷ افرازهای عدد صحیح ۲۵۴
 ۳.۲۷ الگوریتم مو (Mo's algorithm) ۲۵۵

۲۸ بازی های درخت های بازه ای

۲۵۷

- ۱.۲۸ انتشار تنبل ۲۵۸
 ۲.۲۸ درخت های پویا ۲۶۱
 ۳.۲۸ ساختمان داده ها ۲۶۳
 ۴.۲۸ حالت دوبعدی ۲۶۴

۲۶۵	هندسه ۲۹
۲۶۶	۱.۲۹ اعداد مختلط
۲۶۸	۲.۲۹ نقاط و خطوط
۲۷۱	۳.۲۹ مساحت چندضلعی
۲۷۲	۴.۲۹ تابع‌های فاصله

۲۷۵	۳۰ الگوریتم‌های خط جاروب
۲۷۶	۱.۳۰ نقاط تقاطع
۲۷۷	۲.۳۰ مسئله نزدیک‌ترین جفت نقطه
۲۷۸	۳.۳۰ مسئله پوش محدب

۲۸۱	کتاب‌نامه
-----	-----------

پیش‌گفتار

هدف این کتاب ارائه یک مقدمه جامع بر برنامه‌نویسی رقابتی است. فرض بر این است که با مبانی اولیه‌ی برنامه‌نویسی آشنا هستید، اما به هیچ پیش‌زمینه‌ای در برنامه‌نویسی رقابتی نیاز نیست.

این کتاب به‌ویژه برای دانش‌آموزان و دانشجویانی در نظر گرفته شده است که می‌خواهند الگوریتم‌ها را بیاموزند و در المپیاد جهانی کامپیوتر (IOI) یا مسابقه بین‌المللی برنامه‌نویسی دانشجویی (ICPC) شرکت کنند. البته این کتاب برای هر فرد دیگری که به برنامه‌نویسی رقابتی علاقه‌مند است نیز کاملاً مناسب است.

رسیدن به سطح یک برنامه‌نویس رقابتی ماهر نیازمند زمان و تمرین بسیار است، اما در عین حال فرصتی بی‌نظیر برای یادگیری عمیق به شمار می‌رود. مطمئن باشید با اختصاص زمان به مطالعه‌ی این کتاب، حل مسائل و شرکت در مسابقات، به درک عمیق و جامعی از الگوریتم‌ها دست خواهید یافت.

این کتاب به‌طور پیوسته در حال توسعه و به‌روزرسانی است. می‌توانید نظرات و بازخوردهای خود درباره کتاب را به نشانی ahslaaks@cs.helsinki.fi ارسال کنید.

هلسینکی، آگوست ۲۰۱۹
آنتی لاکسونن

بخش اول

تکنیک‌های پایه

فصل ۱

مقدمه

برنامه‌نویسی رقابتی ترکیب دو مبحث است: (۱) طراحی الگوریتم‌ها و (۲) پیاده‌سازی الگوریتم‌ها. **طراحی الگوریتم‌ها** شامل حل مسئله و تفکر ریاضی است. مهارت‌های تحلیل مسائل و حل خلاقانه آن‌ها مورد نیاز است. الگوریتم حل مسئله باید هم درست و هم کارآمد باشد، و هسته مسئله اغلب ابداع الگوریتمی کارآمد است. دانش نظری الگوریتم‌ها برای برنامه‌نویسان رقابتی مهم است. معمولاً راه‌حل مسئله ترکیبی از روش‌های شناخته‌شده و بینش‌های جدید است. تکنیک‌های حاضر در برنامه‌نویسی رقابتی اساس تحقیقات علمی الگوریتم‌ها را نیز تشکیل می‌دهند. **پیاده‌سازی الگوریتم‌ها** نیازمند مهارت‌های برنامه‌نویسی خوب است. در برنامه‌نویسی رقابتی، راه‌حل‌ها با ارزیابی الگوریتم پیاده‌سازی‌شده توسط مجموعه‌ای از تست‌ها داوری می‌شوند. بنابراین، تنها درست بودن ایده الگوریتم کافی نیست، بلکه پیاده‌سازی نیز باید درست باشد. سبک کدنویسی خوب در مسابقات سراسر است و مختصر است. برنامه‌ها باید سریع نوشته شوند، زیرا زمان زیادی وجود ندارد. برخلاف مهندسی نرم‌افزار سنتی، برنامه‌ها کوتاه‌اند (معمولاً حداکثر چند صد خط کد)، و پس از مسابقه نیازی به نگهداری آن‌ها نیست.

۱.۱ زبان‌های برنامه‌نویسی

در حال حاضر، محبوب‌ترین زبان‌های برنامه‌نویسی در مسابقات، ++C پایتون و جاوا هستند. برای نمونه، در Jam Code Google ۲۰۱۷، میان ۳۰۰۰ شرکت‌کننده برتر، ۷۹٪ از ++C ۱۶٪ از پایتون و ۸٪ از جاوا استفاده کردند [۲۹]. برخی شرکت‌کنندگان از چند زبان نیز استفاده کردند.

بسیاری بر این باورند که ++C بهترین انتخاب برای برنامه‌نویس رقابتی است، و ++C تقریباً همیشه در سامانه‌های مسابقه در دسترس است. مزایای استفاده از ++C این است که زبانی بسیار کارآمد است و کتابخانه استاندارد آن شامل مجموعه‌ای بزرگ از داده‌ساختارها و الگوریتم‌ها است.

از سوی دیگر، تسلط بر چندین زبان و درک نقاط قوت آن‌ها مفید است. برای مثال، اگر اعداد صحیح بزرگ در مسئله نیاز باشند، پایتون می‌تواند انتخاب خوبی باشد، زیرا عملیات توکار برای محاسبات با اعداد صحیح بزرگ دارد. با این حال، بیشتر مسائل در مسابقات برنامه‌نویسی طوری طرح می‌شوند که استفاده از زبانی خاص، مزیت ناعادلانه نباشد.

تمام برنامه‌های نمونه در این کتاب به ++C نوشته شده‌اند، و اغلب از داده‌ساختارها و الگوریتم‌های کتابخانه استاندارد استفاده می‌شود. برنامه‌ها از استاندارد ++C۱۱ پیروی می‌کنند، که امروزه در اکثر مسابقات قابل استفاده است. اگر هنوز نمی‌توانید با ++C برنامه‌نویسی کنید، اکنون زمان مناسبی برای یادگیری آن است.

قالب کد C++

قالب کد معمول C++ برای برنامه‌نویسی رقابتی به این شکل است:

```
>h++.stdc/bits< include#  
  
;std namespace using  
  
{ ()main int  
here comes solution //  
}
```

خط `#include` در ابتدای کد قابلیت از کامپایلر `g++` است که اجازه می‌دهد کل کتابخانه استاندارد اضافه شود. بنابراین نیازی به اضافه کردن جداگانه کتابخانه‌هایی مانند `iostream`، `vector` و `algorithm` نیست، بلکه خودکار در دسترس قرار می‌گیرند. خط `using` اعلام می‌کند که کلاس‌ها و توابع کتابخانه استاندارد می‌توانند مستقیماً در کد استفاده شوند. بدون خط `using` باید مثلاً `std::cout` نوشته می‌شد، اما اکنون نوشتن `cout` کفایت می‌کند. کد را می‌توان با استفاده از دستور زیر کامپایل کرد:

```
test o- cpp.test Wall- 02- ++11c=std- ++g
```

این دستور فایل دودویی `test` را از کد منبع `test.cpp` تولید می‌کند. کامپایلر از استاندارد C++11 پیروی می‌کند (`-std=c++11`)، کد را بهینه‌سازی می‌کند (`-O2`) و هشدارهایی درباره خطاهای احتمالی نمایش می‌دهد (`-Wall`).

۲.۱ ورودی و خروجی

در بیشتر مسابقات، جریان‌های استاندارد برای خواندن ورودی و نوشتن خروجی استفاده می‌شوند. در C++ جریان‌های استاندارد عبارتند از `cin` برای ورودی و `cout` برای خروجی. علاوه بر این، توابع C یعنی `scanf` و `printf` نیز می‌توانند به کار روند. ورودی برنامه معمولاً شامل اعداد و رشته‌هایی است که با فاصله و خطوط جدید جدا شده‌اند. آن‌ها را می‌توان از جریان `cin` به صورت زیر خواند:

```
;b ,a int  
;x string  
;x >> b >> a >> cin
```

این نوع کد همیشه کار می‌کند، به این فرض که حداقل یک فاصله یا خط جدید بین هر عنصر در ورودی وجود داشته باشد. برای مثال، کد بالا هر دو ورودی زیر را می‌خواند:

```
monkey 456 123
```

```
456 123
monkey
```

می‌شود: استفاده زیر شکل به خروجی برای cout جریان

```
456; = b ,123 = a int
; "monkey" = x string
; "n\" << x << " " << b << " " << a << cout
```

ورودی و خروجی گاهی گلوگاه برنامه است. خطوط زیر در ابتدای کد، ورودی و خروجی را کارآمدتر می‌کنند:

```
(0);sync_with_stdio::ios
(0);tie.cin
```

توجه کنید که کاراکتر خط جدید "\n" سریع‌تر از endl عمل می‌کند، چون endl همیشه یک عملیات فلاش (تخلیه بافر) انجام می‌دهد. توابع C یعنی scanf و printf جایگزینی برای جریان‌های استاندارد C++ هستند. آن‌ها معمولاً کمی سریع‌ترند، ولی استفاده از آن‌ها سخت‌تر است. کد زیر دو عدد صحیح را از ورودی می‌خواند:

```
;b ,a int
);b& ,a& , "d% d%"(scanf
```

می‌کند: چاپ را صحیح عدد دو زیر کد

```
456; = b ,123 = a int
);b ,a , "n\d% d%"(printf
```

گاهی برنامه باید یک خط کامل را از ورودی بخواند که ممکن است شامل فاصله باشد. این کار با استفاده از تابع getline انجام می‌شود:

```
;s string
);s ,cin(getline
```

اگر مقدار داده‌ها نامشخص باشد، حلقه زیر مفید است:

```
{ )x >> cin( while
code //
}
```

در دیگری داده که زمانی تا می‌خواند، ورودی از دیگری از پس یکی را عناصر حلقه این باشد. نداشته وجود ورودی در برخی سامانه‌های مسابقه، از فایل‌ها برای ورودی و خروجی استفاده می‌شود. راه حل آسان برای این موضوع، نوشتن کد به شکل همیشگی با جریان‌های استاندارد و اضافه کردن

خطوط زیر به ابتدای کد است:

```
);stdin , "r" , "txt.input" (freopen  
);stdout , "w" , "txt.output" (freopen
```

"output.txt" فایل در را خروجی و خوانده "input.txt" فایل از را ورودی برنامه آن، از پس می‌نویسد.

۳.۱ کار با اعداد

اعداد صحیح

پرکاربردترین نوع عدد صحیح در برنامه‌نویسی رقابتی `int` است؛ نوعی ۳۲ بیتی با محدوده مقادیر $1 - 2^{31} \dots 2^{31} - 1$ یا حدود $2 \cdot 10^9 \dots 2 \cdot 10^9 - 2$. اگر نوع `int` کافی نباشد، می‌توان از نوع ۶۴ بیتی `long long` استفاده کرد. این نوع دارای محدوده مقادیر $1 - 2^{63} \dots 2^{63} - 1$ یا حدود $9 \cdot 10^{18} \dots 9 \cdot 10^{18} - 9$ است. کد زیر یک متغیر از نوع `long long` تعریف می‌کند:

```
;LL123456789123456789 = x long long
```

است. `long long` عدد نوع که می‌دهد نشان `LL` پسوند یک اشتباه متداول هنگام استفاده از نوع `long long` این است که نوع `int` همچنان در بخش دیگری از کد استفاده شود. برای نمونه، کد زیر حاوی یک خطای ظریف است:

```
123456789; = a int  
;a*a = b long long  
-1757895751 // ;"n\" << b << cout
```

اگرچه متغیر `b` از نوع `long long` است، اما هر دو عدد در عبارت `a*a` از نوع `int` هستند و نتیجه نیز از نوع `int` خواهد بود. به همین دلیل، متغیر `b` نتیجه نادرستی ذخیره می‌کند. مشکل با تغییر نوع `a` به `long long` یا تغییر عبارت به `(long) a*a` حل می‌شود. معمولاً مسائل مسابقات به گونه‌ای طراحی می‌شوند که نوع `long long` کافی باشد. با این حال، مفید است بدانید کامپایلر `g++` نوع ۱۲۸ بیتی `__int128_t` را نیز با محدوده مقادیر $1 - 2^{127} \dots 2^{127} - 1$ یا حدود $10^{38} \dots 10^{38} - 1$ ارائه می‌دهد. البته این نوع در تمام سامانه‌های مسابقه در دسترس نیست.

حساب پی‌مان‌های

باقی‌مانده تقسیم x بر m با $x \bmod m$ نشان داده می‌شود. برای مثال، $17 \bmod 5 = 2$ ، زیرا $17 = 3 \cdot 5 + 2$.

گاهی پاسخ مسئله عددی بسیار بزرگ است اما چاپ آن «به پیمانه m » کفایت می‌کند، یعنی باقی‌مانده تقسیم پاسخ بر m (برای نمونه، «به پیمانه $10^9 + 7$ »). ایده این است که حتی در صورت بسیار بزرگ بودن پاسخ نهایی، استفاده از نوع‌های `int` و `long long` کافی باشد.

یک ویژگی مهم باقی‌مانده این است که در جمع، تفریق و ضرب، می‌توان باقی‌مانده را قبل از انجام عملیات محاسبه کرد:

بنابراین، می‌توان پس از هر عملیات باقی‌مانده را محاسبه کرد تا اعداد هرگز بیش از حد بزرگ نشوند.

برای نمونه، کد زیر $n!$ ، یعنی فاکتوریل n ، را به پیمانه m محاسبه می‌کند:

معمولاً می‌خواهیم باقی‌مانده همواره بین $0 \dots m - 1$ باشد. اما در C++ و زبان‌های دیگر، باقی‌مانده یک عدد منفی یا صفر است یا منفی. یک روش آسان برای اطمینان از عدم وجود باقی‌مانده‌های منفی این است که ابتدا باقی‌مانده را به صورت معمول حساب کرده و در صورت منفی بودن نتیجه، m را به آن اضافه کنیم:

و باشد داشته وجود تفریق عملیات کد در که است لازم زمانی تنها کار این حال، این با
برود. باقی‌مانده شدن منفی احتمال

نوع‌های داده متمیز شناور معمول در برنامه‌نویسی رقابتی عبارتند از نوع ۶۴ بیتی `double`، و به عنوان یک افزونه در کامپایلر `g++`، نوع ۸۰ بیتی `double long`. در بیشتر موارد، `double` کافی است، اما `double long` دقت بالاتری دارد.

دقت مورد نیاز برای پاسخ معمولاً در صورت مسئله ذکر می‌شود. یک روش آسان برای چاپ پاسخ، استفاده از تابع `printf` و تعیین تعداد ارقام اعشار در رشته قالب‌بندی است. برای نمونه، کد زیر مقدار x را با ۹ رقم اعشار چاپ می‌کند:

یکی از دشواری‌های کار با اعداد ممیز شناور این است که برخی اعداد را نمی‌توان به طور دقیق به صورت ممیز شناور نمایش داد، و خطاهای گرد کردن رخ خواهد داد. برای نمونه، نتیجه کد زیر غافلگیرکننده است:

به دلیل خطای گرد کردن، مقدار x کمی کوچک‌تر از ۱ است، در حالی که مقدار درست باید ۱ باشد.

مقایسه اعداد ممیز شناور با عملگر `==` پرخطر است، زیرا ممکن است مقادیر باید برابر باشند اما به دلیل خطاهای دقت، برابر نباشند. روش بهتر برای مقایسه اعداد ممیز شناور این است که فرض کنیم دو عدد برابرند اگر اختلاف بین آن‌ها کمتر از ϵ باشد، که در آن یک عدد کوچک است.

در عمل، اعداد را می‌توان به شکل زیر مقایسه کرد ($\epsilon = 10^{-9}$):

```
if (abs(b-a) < 1e-9)
    // equal are b and a
}
```

توجه کنید با این که اعداد ممیز شناور نادقیق هستند، اعداد صحیح تا حد مشخصی دقیق ذخیره می‌شوند. مثال: نوع `double` تمام اعداد صحیح با قدر مطلق حداکثر 2^{53} را دقیق نمایش می‌دهد.

۴.۱ کوتاه‌نویسی کد

کد کوتاه در برنامه‌نویسی رقابتی ایده‌آل است، چون کدها باید سریع نوشته شوند. به همین دلیل، برنامه‌نویسان نام‌های کوتاه‌تری برای انواع داده و اجزای کد تعریف می‌کنند.

نام‌های نوع داده

دستور `typedef` نام کوتاه‌تر برای یک نوع داده می‌سازد. مثال: نام `long long` طولانی است؛ نام کوتاه‌تر `ll` را تعریف می‌کنیم:

```
typedef long long ll;
```

کد: این، از بعد

```
long long a = 123456789;
long long b = 987654321;
cout << b*a << "\n";
```

می‌شود: کوتاه این‌گونه

```
ll a = 123456789;
ll b = 987654321;
cout << b*a << "\n";
```

دستور `typedef` برای انواع داده پیچیده‌تر نیز کار می‌کند. کد زیر نام `vi` را برای بردای از اعداد صحیح و نام `pi` را برای زوج مرتب دو عدد صحیح تعریف می‌کند:

```
typedef vector<int> vi;
typedef pair<int,int> pi;
```

ماکروها

روش دیگر کوتاه‌نویسی، تعریف **ماکروها** است. ماکرو رشته‌های مشخصی از کد را قبل از کامپایل جایگزین می‌کند. در C++ ماکروها با کلمه کلیدی #define تعریف می‌شوند. مثال: تعریف ماکروهای زیر:

```
first F define#  
second S define#  
push_back PB define#  
make_pair MP define#
```

کد: این، از بعد

```
));x1,y1(make_pair(push_back.v  
));x2,y2(make_pair(push_back.v  
;second].i[v+first].i[v = d int
```

می‌شود: کوتاه این‌گونه

```
));x1,y1(MP(PB.v  
));x2,y2(MP(PB.v  
;S].i[v+F].i[v = d int
```

ماکرو پارامتر هم می‌گیرد که حلقه‌ها و ساختارها را کوتاه می‌کند. مثال: تعریف ماکروی زیر:

```
++)i ; b <= i ; a = i int( for )b,a,i(REP define#
```

کد: این، از بعد

```
{ ++i ; n <= i 1; = i int( for  
);i(search  
}
```

می‌شود: کوتاه این‌گونه

```
{ )n,,1i(REP  
);i(search  
}
```

گاهی ماکروها خطاهایی می‌سازند که پیدا کردنشان سخت است. مثال: ماکروی زیر برای محاسبه مربع یک عدد:

```
a*a )a(SQ define#
```

کد مثال: نمی‌کند. کار درست همیشه ماکرو این

```
; "n\" << (3+3)SQ << cout
```

است: کد این معادل

```
cout << 3+3*3+3 << "n\" ; // 15
```

نسخه بهتر ماکرو به این صورت است:

```
#define a(a) (a*(a))
```

کد اکنون

```
cout << (3+3)SQ << "n\" ;
```

است: کد این معادل

```
cout << (3+3)*(3+3) << "n\" ; // 36
```

۵.۱ ریاضیات

ریاضیات نقش مهمی در برنامه‌نویسی رقابتی دارد. موفقیت در برنامه‌نویسی رقابتی بدون مهارت‌های قوی ریاضی ممکن نیست. این بخش به مفاهیم و فرمول‌های مهم ریاضی می‌پردازد که در ادامه کتاب استفاده می‌شوند.

فرمول‌های مجموع

هر مجموع به فرم

$$\sum_{x=1}^n x^k = 1^k + 2^k + 3^k + \dots + n^k,$$

که در آن k عدد صحیح مثبت است، فرمول بسته دارد که چندجمله‌ای با درجه $k+1$ است. برای مثال^۱،

$$\sum_{x=1}^n x = 1 + 2 + 3 + \dots + n = \frac{n(n+1)}{2}$$

9

$$\sum_{x=1}^n x^2 = 1^2 + 2^2 + 3^2 + \dots + n^2 = \frac{n(n+1)(2n+1)}{6}.$$

یک **تصادد حسابی** دنباله‌ای از اعداد است که تفاضل هر دو عدد متوالی در آن ثابت است. برای مثال،

3, 7, 11, 15

^۱ فرمول کلی برای این مجموع‌ها با نام **فرمول فالهابر** وجود دارد، ولی ارائه آن در اینجا پیچیده است.

یک تصاعد حسابی با قدر نسبت (مقدار ثابت) ۴ است. مجموع جملات تصاعد حسابی با فرمول زیر محاسبه می‌شود:

$$\underbrace{a + \dots + b}_{n \text{ عدد}} = \frac{n(a+b)}{2}$$

که a عدد اول، b عدد آخر و n تعداد اعداد است. برای مثال،

$$3 + 7 + 11 + 15 = \frac{4 \cdot (3 + 15)}{2} = 36.$$

اساس فرمول این است: مجموع از n عدد تشکیل شده و میانگین مقدار هر عدد $(a+b)/2$ است.

یک **تصاعد هندسی** دنباله‌ای از اعداد است که نسبت هر دو عدد متوالی در آن ثابت است. برای مثال،

$$3, 6, 12, 24$$

یک تصاعد هندسی با قدر نسبت ۲ است. مجموع جملات تصاعد هندسی با فرمول زیر محاسبه می‌شود:

$$a + ak + ak^2 + \dots + b = \frac{bk - a}{k - 1}$$

که a عدد اول، b عدد آخر و نسبت اعداد متوالی k است. برای مثال،

$$3 + 6 + 12 + 24 = \frac{24 \cdot 2 - 3}{2 - 1} = 45.$$

اثبات فرمول به این صورت است. فرض کنید:

$$S = a + ak + ak^2 + \dots + b.$$

با ضرب دو طرف در k :

$$kS = ak + ak^2 + ak^3 + \dots + bk,$$

حل معادله

$$kS - S = bk - a$$

فرمول را به دست می‌دهد.

حالت خاص مجموع تصاعد هندسی فرمول زیر است:

$$1 + 2 + 4 + 8 + \dots + 2^{n-1} = 2^n - 1.$$

یک **مجموع هارمونیک** مجموعی به فرم زیر است:

$$\sum_{x=1}^n \frac{1}{x} = 1 + \frac{1}{2} + \frac{1}{3} + \dots + \frac{1}{n}.$$

یک کران بالا برای مجموع هارمونیک برابر $\log_2(n) + 1$ است. به طور مشخص، می‌توانیم هر جمله $1/k$ را طوری تغییر دهیم که k تبدیل به نزدیک‌ترین توان دو شود که از k بزرگ‌تر نیست. برای نمونه، وقتی $n = 6$ است، می‌توان مجموع را به صورت زیر تخمین زد:

$$1 + \frac{1}{2} + \frac{1}{3} + \frac{1}{4} + \frac{1}{5} + \frac{1}{6} \leq 1 + \frac{1}{2} + \frac{1}{2} + \frac{1}{4} + \frac{1}{4} + \frac{1}{4}.$$

این کران بالا از $\log_2(n) + 1$ بخش تشکیل می‌شود (۱، $2 \cdot 1/2$ ، $4 \cdot 1/4$ و غیره)، و مقدار هر بخش حداکثر ۱ است.

نظریه مجموعه‌ها

یک مجموعه گردایه‌ای از عناصر است. برای نمونه، مجموعه

$$X = \{2, 4, 7\}$$

شامل عناصر ۲، ۴ و ۷ است. نماد $\emptyset$ نشان‌دهنده مجموعه تهی است، و $|S|$ اندازه یک مجموعه S را نشان می‌دهد، یعنی تعداد عناصر موجود در مجموعه. برای نمونه، در مجموعه بالا، $|X| = 3$.

اگر یک مجموعه S شامل عنصری به نام x باشد، می‌نویسیم $x \in S$ ، و در غیر این صورت می‌نویسیم $x \notin S$. برای نمونه، در مجموعه بالا

$$4 \in X \text{ و } 5 \notin X.$$

مجموعه‌های جدید را می‌توان با استفاده از عملگرهای مجموعه‌ای ساخت:

• **اشتراک** $A \cap B$ شامل عنصری است که هم در A و هم در B هستند. برای نمونه، اگر $A = \{1, 2, 5\}$ و $B = \{2, 4\}$ ، آنگاه $A \cap B = \{2\}$.

• **اجتماع** $A \cup B$ شامل عنصری است که در A یا B یا هر دو هستند. برای نمونه، اگر $A = \{3, 7\}$ و $B = \{2, 3, 8\}$ ، آنگاه $A \cup B = \{2, 3, 7, 8\}$.

• **متمم** $\bar{A}$ شامل عنصری است که در A نیستند. تفسیر متمم بستگی به **مجموعه مرجع** دارد، که شامل تمام عناصر ممکن است. برای نمونه، اگر $A = \{1, 2, 5, 7\}$ و مجموعه مرجع $\{1, 2, \dots, 10\}$ باشد، آنگاه $\bar{A} = \{3, 4, 6, 8, 9, 10\}$.

• **تفاضل** $A \setminus B = A \cap \bar{B}$ شامل عنصری است که در A هستند ولی در B نیستند. دقت کنید که B می‌تواند شامل عنصری باشد که در A نیستند. برای نمونه، اگر $A = \{2, 3, 7, 8\}$ و $B = \{3, 5, 8\}$ ، آنگاه $A \setminus B = \{2, 7\}$.

اگر هر عنصر از A به S نیز تعلق داشته باشد، می‌گوییم A یک **زیرمجموعه** از S است، و با $A \subset S$ نشان داده می‌شود. یک مجموعه S همواره شامل $2^{|S|}$ زیرمجموعه است، شامل مجموعه تهی. برای نمونه، زیرمجموعه‌های مجموعه $\{2, 4, 7\}$ عبارتند از

$$\emptyset, \{2\}, \{4\}, \{7\}, \{2, 4\}, \{2, 7\}, \{4, 7\} \text{ و } \{2, 4, 7\}.$$

برخی از مجموعه‌های پرکاربرد عبارتند از $\mathbb{N}$ (اعداد طبیعی)، $\mathbb{Z}$ (اعداد صحیح)، $\mathbb{Q}$ (اعداد گویا) و $\mathbb{R}$ (اعداد حقیقی). مجموعه $\mathbb{N}$ بسته به موقعیت می‌تواند به دو صورت تعریف شود: $\mathbb{N} = \{0, 1, 2, \dots\}$ یا $\mathbb{N} = \{1, 2, 3, \dots\}$.

همچنین می‌توان مجموعه‌ای را با استفاده از قاعده‌ای به شکل

$$\{f(n) : n \in S\},$$

ساخت که در آن $f(n)$ یک تابع است. این مجموعه شامل تمام عنصری به شکل $f(n)$ است که n عضوی از S باشد. برای مثال، مجموعه

$$X = \{2n : n \in \mathbb{Z}\}$$

شامل تمام اعداد صحیح زوج است.

منطق

مقدار یک عبارت منطقی یا درست (۱) یا نادرست (۰) است. مهم‌ترین عملگرهای منطقی عبارت‌اند از $\neg$ (نقیض)، $\wedge$ (عطف)، $\vee$ (فصل)، $\Rightarrow$ (استلزام) و $\Leftrightarrow$ (هم‌ارزی). جدول زیر معنای این عملگرها را نشان می‌دهد:

$A \Leftrightarrow B$	$A \Rightarrow B$	$A \vee B$	$A \wedge B$	$\neg B$	$\neg A$	B	A
۱	۱	۰	۰	۱	۱	۰	۰
۰	۱	۱	۰	۰	۱	۱	۰
۰	۰	۱	۰	۱	۰	۰	۱
۱	۱	۱	۱	۰	۰	۱	۱

عبارت $\neg A$ مقداری مخالف با A دارد. عبارت $A \wedge B$ درست است اگر هر دو A و B درست باشند، و عبارت $A \vee B$ درست است اگر A یا B یا هر دو درست باشند. عبارت $A \Rightarrow B$ درست است هرگاه هر زمان که A درست باشد، B نیز درست باشد. عبارت $A \Leftrightarrow B$ درست است اگر A و B هر دو درست یا هر دو نادرست باشند.

یک **محمول** عبارتی است که بسته به پارامترهای خود درست یا نادرست است. محمول‌ها معمولاً با حروف بزرگ نشان داده می‌شوند. به عنوان مثال، می‌توان محمول $P(x)$ را تعریف کرد که دقیقاً زمانی درست است که x یک عدد اول باشد. با استفاده از این تعریف، $P(7)$ درست است ولی $P(8)$ نادرست است.

یک **سور** یک عبارت منطقی را به عناصر یک مجموعه مرتبط می‌کند. مهم‌ترین سورها $\forall$ (به ازای هر) و $\exists$ (وجود دارد) هستند. برای مثال،

$$\forall x(\exists y(y < x))$$

بدین معناست که به ازای هر عنصر x در مجموعه، عنصری مانند y در مجموعه وجود دارد به‌طوری‌که y کوچک‌تر از x است. این گزاره در مجموعه اعداد صحیح درست است، اما در مجموعه اعداد طبیعی نادرست است.

با استفاده از نمادگذاری شرح داده شده، می‌توان انواع گزاره‌های منطقی را بیان کرد. برای مثال،

$$\forall x((x > 1 \wedge \neg P(x)) \Rightarrow (\exists a(\exists b(a > 1 \wedge b > 1 \wedge x = ab))))$$

بدین معناست که اگر عدد x بزرگ‌تر از ۱ و غیر اول باشد، آنگاه اعدادی مانند a و b وجود دارند که بزرگ‌تر از ۱ هستند و حاصل ضرب آن‌ها برابر x است. این گزاره در مجموعه اعداد صحیح درست است.

توابع

تابع $\lfloor x \rfloor$ عدد x را به سمت پایین به یک عدد صحیح گرد می‌کند، و تابع $\lceil x \rceil$ عدد x را به سمت بالا به یک عدد صحیح گرد می‌کند. برای مثال،

$$\lfloor 3/2 \rfloor = 1 \quad \lceil 3/2 \rceil = 2$$

توابع $\max(x_1, x_2, \dots, x_n)$ و $\min(x_1, x_2, \dots, x_n)$ کوچک‌ترین و بزرگ‌ترین مقادیر میان $x_1, x_2, \dots, x_n$ را به دست می‌دهند. برای نمونه،

$$\min(1, 2, 3) = 1 \quad \max(1, 2, 3) = 3$$

فاکتوریل $n!$ می‌تواند به این صورت تعریف شود:

$$\prod_{x=1}^n x = 1 \cdot 2 \cdot 3 \cdot \dots \cdot n$$

یا به صورت بازگشتی:

$$\begin{aligned} 0! &= 1 \\ n! &= n \cdot (n-1)! \end{aligned}$$

اعداد فیبوناچی در موقعیت‌های بسیاری پدیدار می‌شوند. آن‌ها می‌توانند به صورت بازگشتی بدین شکل تعریف شوند:

$$\begin{aligned} f(0) &= 0 \\ f(1) &= 1 \\ f(n) &= f(n-1) + f(n-2) \end{aligned}$$

نخستین اعداد فیبوناچی عبارتند از:

$$0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, \dots$$

همچنین یک فرمول بسته برای محاسبه اعداد فیبوناچی وجود دارد که گاهی به آن **فرمول بینه** می‌گویند:

$$f(n) = \frac{(1 + \sqrt{5})^n - (1 - \sqrt{5})^n}{2^n \sqrt{5}}.$$

لگاریتم‌ها

لگاریتم یک عدد x به صورت $\log_k(x)$ نشان داده می‌شود که در آن k پایه لگاریتم است. طبق تعریف، $\log_k(x) = a$ دقیقاً زمانی برقرار است که $k^a = x$ باشد. یک ویژگی کاربردی لگاریتم‌ها این است که $\log_k(x)$ برابر است با تعداد دفعاتی که باید x را بر k تقسیم کنیم تا به عدد ۱ برسیم. برای نمونه، $\log_2(32) = 5$ زیرا ۵ بار تقسیم بر ۲ نیاز است:

$$32 \rightarrow 16 \rightarrow 8 \rightarrow 4 \rightarrow 2 \rightarrow 1$$

لگاریتم‌ها اغلب در تحلیل الگوریتم‌ها به کار می‌روند، زیرا بسیاری از الگوریتم‌های کارآمد در هر مرحله مقداری را نصف می‌کنند. بنابراین، می‌توانیم کارایی این الگوریتم‌ها را با استفاده از لگاریتم تخمین بزنیم. لگاریتم حاصل ضرب عبارت است از:

$$\log_k(ab) = \log_k(a) + \log_k(b),$$

و در نتیجه،

$$\log_k(x^n) = n \cdot \log_k(x).$$

علاوه بر این، لگاریتم خارج قسمت برابر است با:

$$\log_k\left(\frac{a}{b}\right) = \log_k(a) - \log_k(b).$$

فرمول کاربردی دیگر به این صورت است:

$$\log_u(x) = \frac{\log_k(x)}{\log_k(u)},$$

و با استفاده از این، محاسبه لگاریتم در هر پایه‌ای امکان‌پذیر است اگر روشی برای محاسبه لگاریتم در یک پایه ثابت وجود داشته باشد.

لگاریتم طبیعی $\ln(x)$ یک عدد x لگاریتمی است که پایه آن $e \approx 2.71828$ می‌باشد. ویژگی دیگر لگاریتم‌ها این است که تعداد ارقام یک عدد صحیح x در مبنای b برابر است با $\lceil \log_b(x) + 1 \rceil$. برای نمونه، نمایش 123 در مبنای 2 برابر با ۱۱۱۰۱۱ است و $\lceil \log_2(123) + 1 \rceil = 7$.

۶.۱ مسابقات و منابع

IOI

المپیاد جهانی کامپیوتر (IOI) یک مسابقه سالانه برنامه‌نویسی برای دانش‌آموزان دوره متوسطه است. هر کشور مجاز به اعزام تیمی متشکل از چهار دانش‌آموز به مسابقه است. معمولاً حدود ۳۰۰ شرکت‌کننده از ۸۰ کشور حضور دارند.

مسابقه IOI شامل دو آزمون پنج‌ساعته است. در هر دو آزمون، شرکت‌کنندگان باید سه مسئله الگوریتمی با دشواری‌های مختلف را حل کنند. مسائل به زیرمسئله تقسیم می‌شوند و هر زیرمسئله نمره مشخص دارد. حتی اگر شرکت‌کنندگان در قالب تیم اعزام شوند، رقابت انفرادی است.

سرفصل‌های IOI [۴۱] مباحث مجاز در آزمون را تعیین می‌کند. تقریباً تمام مباحث سرفصل IOI در این کتاب پوشش داده شده است.

شرکت‌کنندگان IOI از طریق مسابقات ملی انتخاب می‌شوند. پیش از IOI، مسابقات منطقه‌ای متعددی برگزار می‌شود، مانند المپیاد انفورماتیک بالتیک (BOI)، المپیاد انفورماتیک اروپای مرکزی (CEOI) و المپیاد انفورماتیک آسیا-اقیانوسیه (APIO). برخی کشورها مسابقات تمرینی برخط برای شرکت‌کنندگان آینده IOI برگزار می‌کنند، مانند مسابقات آزاد انفورماتیک کرواسی [۱۱] و المپیاد کامپیوتر آمریکا [۶۸]. همچنین، مجموعه بزرگی از مسائل مسابقات لهستان به صورت برخط در دسترس است [۶۰].

ICPC

مسابقه بین‌المللی برنامه‌نویسی دانشجویی (ICPC) یک مسابقه سالانه برنامه‌نویسی برای دانشجویان است. هر تیم شامل سه دانشجو است. برخلاف IOI، اعضا با یکدیگر همکاری می‌کنند؛ تنها یک رایانه برای هر تیم وجود دارد.

مسابقه ICPC شامل چند مرحله است و در نهایت بهترین تیم‌ها به مرحله نهایی جهانی دعوت می‌شوند. با وجود ده‌ها هزار شرکت‌کننده، تنها تعداد اندکی سهمیه نهایی^۲ وجود دارد، بنابراین صعود به مرحله نهایی در برخی منطقه‌ها دستاوردی بزرگ محسوب می‌شود. در هر مسابقه، ICPC تیم‌ها پنج ساعت زمان دارند تا حدود ده مسئله الگوریتمی را حل کنند. راه‌حل مسئله تنها در صورتی پذیرفته می‌شود که همه آزمایش‌ها را به طور بهینه حل کند. در طول مسابقه، رقبا می‌توانند نتایج تیم‌های دیگر را ببینند، اما در ساعت پایانی جدول امتیازات منجمد می‌شود و دیدن نتایج آخرین ارسال‌ها ممکن نیست. مباحث ممکن در ICPC به اندازه IOI مشخص نیست. در هر صورت، برای ICPC به دانش بیشتری نیاز است، به ویژه مهارت‌های ریاضیاتی قوی‌تر.

^۲تعداد دقیق سهمیه‌های نهایی هر سال متغیر است؛ سال ۲۰۱۷، تعداد ۱۳۳ سهمیه نهایی وجود داشت.

مسابقات برخط

مسابقات برخط متعددی نیز برای همگان در دسترس است. در حال حاضر فعال‌ترین پایگاه مسابقات Codeforces است که تقریباً هفتگی مسابقه برگزار می‌کند. در Codeforces شرکت‌کنندگان دو بخش هستند: تازه‌کارها در Div2 و برنامه‌نویسان باتجربه‌تر در Div1 رقابت می‌کنند. سایر پایگاه‌های مسابقه عبارتند از: HackerRank Academy، CS AtCoder، و Topcoder.

برخی شرکت‌ها مسابقات برخط با مرحله نهایی حضوری برگزار می‌کنند. نمونه‌های آن Hacker Facebook Cup، Jam Code Google Cup و Yandex.Algorithm است. البته شرکت‌ها از این مسابقات برای استخدام نیرو نیز استفاده می‌کنند؛ عملکرد خوب در مسابقه روش مناسبی برای اثبات مهارت‌هاست.

کتاب‌ها

پیش از این نیز کتاب‌هایی (به جز این کتاب) با تمرکز بر برنامه‌نویسی رقابتی و حل مسئله الگوریتمی تالیف شده‌اند:

• Program- The Challenges: Programming Revilla: A. M. and Skiena S. S. [۵۹] Manual Training Contest ming

• Lower New The :۳ Programming Competitive Halim: F. and Halim S. [۳۳] Contests Programming of Bound

• from Set Problem Ultimate The Challenge? a for Looking al.: et Diks K. [۱۵] Competitions Programming Warsaw of University the

دو کتاب نخست برای افراد مبتدی در نظر گرفته شده‌اند، در حالی که کتاب آخر شامل مباحث پیشرفته است.

البته کتاب‌های عمومی الگوریتم نیز برای برنامه‌نویسان رقابتی مناسب هستند. برخی از کتاب‌های محبوب عبارتند از:

• to Introduction Stein: C. and Rivest L. R. Leiserson، E. C. Cormen، H. T. [۱۳] Algorithms

• [۴۵] Design Algorithm Tardos: É. and Kleinberg J.

• [۵۸] Manual Design Algorithm The Skiena: S. S.

فصل ۲

پیچیدگی زمانی

کارایی الگوریتم‌ها در برنامه‌نویسی رقابتی بسیار مهم است. معمولاً طراحی الگوریتمی که مسئله را حل کند آسان است، اما چالش واقعی، ابداع الگوریتمی سریع است. اگر الگوریتم بیش از حد کند باشد، تنها بخشی از امتیاز یا هیچ امتیازی دریافت نخواهد کرد.

پیچیدگی زمانی یک الگوریتم تخمین می‌زند که الگوریتم برای یک ورودی مشخص، چه مقدار زمان مصرف خواهد کرد. ایده اصلی این است که کارایی به عنوان تابعی نشان داده شود که پارامتر آن، اندازه ورودی است. با محاسبه پیچیدگی زمانی، می‌توان بدون پیاده‌سازی متوجه شد آیا الگوریتم به اندازه کافی سریع است یا خیر.

۱.۲ قواعد محاسبه

پیچیدگی زمانی یک الگوریتم با نماد $O(\dots)$ نشان داده می‌شود که در آن سه نقطه نشان‌دهنده یک تابع است. معمولاً متغیر n بیانگر اندازه ورودی است. برای مثال، اگر ورودی یک آرایه از اعداد باشد، n اندازه آرایه خواهد بود، و اگر ورودی یک رشته باشد، n طول رشته خواهد بود.

حلقه‌ها

دلیل رایج برای کند بودن یک الگوریتم این است که شامل حلقه‌های متعددی است که ورودی را پیمایش می‌کنند. هرچه الگوریتم شامل حلقه‌های تودرتوی بیشتری باشد، کندتر است. اگر k حلقه تودرتو وجود داشته باشد، پیچیدگی زمانی $O(n^k)$ است. برای مثال، پیچیدگی زمانی کد زیر $O(n)$ است:

```
for (i = 1; i <= n; ++i) {
    // code
}
```

و پیچیدگی زمانی کد زیر $O(n^2)$ است:

```
for (i = 1; i <= n; ++i) {
    for (j = 1; j <= n; ++j) {
        // code
    }
}
```

مرتبه بزرگی

پیچیدگی زمانی تعداد دقیق دفعات اجرای کد درون حلقه را بیان نمی‌کند، بلکه تنها مرتبه بزرگی را نشان می‌دهد. در مثال‌های زیر، کد درون حلقه به ترتیب $3n$ ، $n + 5$ و $\lceil n/2 \rceil$ بار اجرا می‌شود، اما پیچیدگی زمانی هر کد $O(n)$ است.

```
{ ++i ; n3* <= i 1; = i int( for
code //
}
```

```
{ ++i +5; n <= i 1; = i int( for
code //
}
```

```
{ 2) += i ; n <= i 1; = i int( for
code //
}
```

به عنوان مثالی دیگر، پیچیدگی زمانی کد زیر $O(n^2)$ است:

```
{ ++i ; n <= i 1; = i int( for
{ ++j ; n <= j +1; i = j int( for
code //
}
}
```

مراحل

اگر الگوریتم از مراحل متوالی تشکیل شده باشد، پیچیدگی زمانی کل برابر با بیشترین پیچیدگی زمانی در میان تک‌تک مراحل است. دلیل این امر این است که کندترین مرحله معمولاً گلوگاه کد محسوب می‌شود.

برای مثال، کد زیر از سه مرحله با پیچیدگی‌های زمانی $O(n)$ ، $O(n)$ و $O(n^2)$ تشکیل شده است. بنابراین، پیچیدگی زمانی کل $O(n^2)$ است.

```
{ ++i ; n <= i 1; = i int( for
code //
}

{ ++i ; n <= i 1; = i int( for
{ ++j ; n <= j 1; = j int( for
code //
```

```

    }
}
{ ++)i ;n <= i 1; = i int( for
code //
}

```

چند متغیر

گاهی پیچیدگی زمانی به چندین عامل بستگی دارد. در این حالت، فرمول پیچیدگی زمانی شامل چند متغیر است. برای نمونه، پیچیدگی زمانی کد زیر $O(nm)$ است:

```

{ ++)i ;n <= i 1; = i int( for
{ ++)j ;m <= j 1; = j int( for
code //
}
}

```

بازگشت

پیچیدگی زمانی تابع بازگشتی به تعداد دفعات فراخوانی تابع و پیچیدگی زمانی یک فراخوانی بستگی دارد. پیچیدگی زمانی کل برابر حاصل ضرب این مقادیر است. برای نمونه، تابع زیر را در نظر بگیرید:

```

{ )n int(f void
;return 1) == n( if
-1);n(f
}

```

$O(1)$ فراخوانی هر زمانی پیچیدگی و می شود تابع فراخوانی بار n موجب $f(n)$ فراخوانی است. $O(n)$ کل زمانی پیچیدگی بنابراین، است. به عنوان نمونه ای دیگر، تابع زیر را در نظر بگیرید:

```

{ )n int(g void
;return 1) == n( if
-1);n(g
-1);n(g
}

```

بررسی $n = 1$ برای مگر می کند، ایجاد دیگر فراخوانی دو تابع فراخوانی هر حالت این در ایجاد تابع فراخوانی های زیر جدول می دهد. رخ چه n پارامتر با g فراخوانی هنگام می کنیم می دهد: نشان را تک فراخوانی این توسط شده

فراخوانی تابع	تعداد فراخوانی‌ها
$g(n)$	۱
$g(n-1)$	۲
$g(n-2)$	۴
$\dots$	$\dots$
$g(1)$	2^{n-1}

بر این اساس، پیچیدگی زمانی برابر است با:

$$1 + 2 + 4 + \dots + 2^{n-1} = 2^n - 1 = O(2^n).$$

۲.۲ رده‌های پیچیدگی

فهرست زیر شامل پیچیدگی‌های زمانی متداول در الگوریتم‌ها است:

$O(1)$ زمان اجرای الگوریتم **زمان ثابت** به اندازه ورودی بستگی ندارد. یک الگوریتم زمان ثابت نوعی، فرمول مستقیمی است که پاسخ را محاسبه می‌کند.

$O(\log n)$ یک الگوریتم **لگاریتمی** معمولاً اندازه ورودی را در هر گام نصف می‌کند. زمان اجرای چنین الگوریتمی لگاریتمی است، زیرا $\log_2 n$ برابر با تعداد دفعاتی است که n باید بر ۲ تقسیم شود تا به ۱ برسد.

$O(\sqrt{n})$ یک الگوریتم **ریشه دوم** از $O(\log n)$ کندتر ولی از $O(n)$ سریع‌تر است. ویژگی خاص ریشه دوم این است که $\sqrt{n} = n / \sqrt{n}$ ، بنابراین ریشه دوم $\sqrt{n}$ به نوعی در میانه ورودی قرار دارد.

$O(n)$ یک الگوریتم **خطی** ورودی را به تعداد ثابتی پیمایش می‌کند. این پیچیدگی زمانی اغلب بهترین حالت ممکن است، زیرا معمولاً دسترسی به هر عنصر ورودی حداقل یک بار پیش از اعلام پاسخ ضروری است.

$O(n \log n)$ این پیچیدگی زمانی اغلب نشان می‌دهد که الگوریتم ورودی را مرتب‌سازی می‌کند، زیرا پیچیدگی زمانی الگوریتم‌های کارآمد مرتب‌سازی $O(n \log n)$ است. احتمال دیگر این است که الگوریتم از ساختمان داده‌ای استفاده می‌کند که در آن هر عملیات $O(\log n)$ زمان می‌برد.

یک الگوریتم **درجه دوم** اغلب شامل دو حلقه تودرتو است. بررسی تمام جفت‌های عناصر ورودی در زمان $O(n^2)$ امکان‌پذیر است.

یک الگوریتم **درجه سوم** اغلب شامل سه حلقه تودرتو است. بررسی تمام سه‌تایی‌های عناصر ورودی در زمان $O(n^3)$ امکان‌پذیر است.

این پیچیدگی زمانی اغلب نشان می‌دهد که الگوریتم روی تمام زیرمجموعه‌های عناصر ورودی پیمایش می‌کند. برای نمونه، زیرمجموعه‌های $\{1, 2, 3\}$ عبارتند از $\emptyset$ ، $\{1\}$ ، $\{2\}$ ، $\{3\}$ ، $\{1, 2\}$ ، $\{1, 3\}$ و $\{2, 3\}$.

این پیچیدگی زمانی اغلب نشان می‌دهد که الگوریتم روی تمام جایگشت‌های عناصر ورودی پیمایش می‌کند. برای نمونه، جایگشت‌های $\{1, 2, 3\}$ عبارتند از $(1, 2, 3)$ ، $(1, 3, 2)$ ، $(2, 1, 3)$ ، $(2, 3, 1)$ و $(3, 1, 2)$.

یک الگوریتم **چندجمله‌ای** است اگر پیچیدگی زمانی آن حداکثر $O(n^k)$ باشد که در آن k یک مقدار ثابت است. تمام پیچیدگی‌های زمانی بالا به جز $O(2^n)$ و $O(n!)$ چندجمله‌ای هستند. در عمل، ثابت k معمولاً کوچک است، و بنابراین پیچیدگی زمانی چندجمله‌ای تقریباً به این معناست که الگوریتم کارآمد است. بیشتر الگوریتم‌های این کتاب چندجمله‌ای هستند. با این حال، مسائل مهم زیادی وجود دارند که هیچ الگوریتم چندجمله‌ای برای آن‌ها شناخته نشده است؛ به این معنی که هیچ‌کس راه حل کارآمدی برای آن‌ها نمی‌داند. مسائل **سخت-NP** دسته مهمی از مسائل هستند که هیچ الگوریتم چندجمله‌ای برای آن‌ها شناخته نشده است.^۱

۳.۲ تخمین کارایی

با محاسبه پیچیدگی زمانی یک الگوریتم، می‌توان پیش از پیاده‌سازی بررسی کرد که آیا الگوریتم برای حل مسئله به اندازه کافی کارآمد است یا خیر. نقطه شروع تخمین‌ها این واقعیت است که یک رایانه مدرن می‌تواند چند صد میلیون عملیات را در یک ثانیه انجام دهد. برای نمونه، فرض کنید محدودیت زمانی یک مسئله یک ثانیه و اندازه ورودی $n = 10^5$ است. اگر پیچیدگی زمانی $O(n^2)$ باشد، الگوریتم حدود $10^{10} = (10^5)^2$ عملیات انجام خواهد داد. این کار حداقل به چند ده ثانیه زمان نیاز دارد، بنابراین الگوریتم برای حل مسئله بیش از حد کند به نظر می‌رسد. از سوی دیگر، با توجه به اندازه ورودی، می‌توان پیچیدگی زمانی مورد نیاز الگوریتم حل‌کننده مسئله را حدس زد. جدول زیر شامل چند تخمین کاربردی با فرض محدودیت زمانی یک ثانیه است.

اندازه ورودی	پیچیدگی زمانی مورد نیاز
$n \leq 10$	$O(n!)$
$n \leq 20$	$O(2^n)$
$n \leq 500$	$O(n^3)$
$n \leq 5000$	$O(n^2)$
$n \leq 10^6$	$O(n)$ یا $O(n \log n)$
n بزرگ است	$O(1)$ یا $O(\log n)$

به عنوان مثال، اگر اندازه ورودی $n = 10^5$ باشد، احتمالاً انتظار می‌رود که پیچیدگی زمانی الگوریتم $O(n)$ یا $O(n \log n)$ باشد. این اطلاعات طراحی الگوریتم را ساده‌تر می‌کند، زیرا روش‌هایی را که منجر به الگوریتمی با پیچیدگی زمانی بدتر می‌شوند کنار می‌گذارد. با این حال، باید به یاد داشت که پیچیدگی زمانی تنها برآوردی از کارایی است، زیرا ضرایب ثابت را پنهان می‌کند. برای نمونه، الگوریتمی با زمان اجرای $O(n)$ ممکن است $n/2$ یا $5n$ عملیات انجام دهد. این موضوع اثر مهمی بر زمان اجرای واقعی الگوریتم دارد.

۴.۲ بیشینه مجموع زیرآرایه

اغلب چندین الگوریتم با پیچیدگی‌های زمانی متفاوت برای حل یک مسئله وجود دارد. این بخش مسئله‌ای کلاسیک را بررسی می‌کند که یک راه حل مستقیم با زمان اجرای $O(n^3)$ دارد. با این وجود، با طراحی الگوریتمی بهتر می‌توان مسئله را در زمان $O(n^2)$ و حتی $O(n)$ حل کرد.

^۱ یک کتاب کلاسیک در این زمینه، کتاب Intractability: and Computers A Guide to the Theory of NP-Completeness اثر R. M. Garey و S. D. Johnson است [۲۸].

به ازای یک آرایه شامل n عدد، هدف محاسبه بیشینه مجموع زیرآرایه است؛ یعنی بزرگ‌ترین مجموع ممکن از دنباله‌ای از مقادیر متوالی در آرایه^۲. این مسئله زمانی جالب می‌شود که مقادیر منفی در آرایه وجود داشته باشند. به عنوان مثال، در آرایه

-1	2	4	-3	5	2	-5	2
----	---	---	----	---	---	----	---

زیرآرایه زیر بیشینه مجموع 10 را تولید می‌کند:

-1	2	4	-3	5	2	-5	2
----	---	---	----	---	---	----	---

فرض می‌کنیم زیرآرایه خالی مجاز است، بنابراین بیشینه مجموع زیرآرایه همیشه حداقل 0 است.

الگوریتم ۱

یک روش مستقیم برای حل مسئله، بررسی تمام زیرآرایه‌های ممکن، محاسبه مجموع مقادیر هر زیرآرایه و نگهداری بیشینه مجموع است. کد زیر این الگوریتم را پیاده‌سازی می‌کند:

```

0; = best int
{ ++)a ;n < a 0; = a int( for
{ ++)b ;n < b ;a = b int( for
0; = sum int
{ ++)k ;b <= k ;a = k int( for
];k[array += sum
}
);sum,best(max = best
}
}
;"n\" << best << cout

```

متغیرهای a و b اندیس‌های ابتدا و انتهای زیرآرایه را مشخص می‌کنند، و مجموع مقادیر در متغیر sum محاسبه می‌شود. متغیر $best$ شامل بیشینه مجموع یافت‌شده در طول جستجو است.

پیچیدگی زمانی الگوریتم $O(n^3)$ است، زیرا از سه حلقه تودرتو تشکیل شده که ورودی را پیمایش می‌کنند.

الگوریتم ۲

بهینه‌تر کردن الگوریتم ۱ با حذف یک حلقه آسان است. این کار با محاسبه مجموع هم‌زمان با حرکت انتهای راست زیرآرایه ممکن می‌شود. نتیجه، کد زیر است:

```

0; = best int
{ ++)a ;n < a 0; = a int( for
0; = sum int
{ ++)b ;n < b ;a = b int( for

```

^۲ کتاب Pearls Programming نوشته J. Bentley [۸] این مسئله را معروف کرد.


```

];b[array += sum
);sum,best(max = best
}
}
; "n\\n" << best << cout

```

می‌شود. $O(n^2)$ زمانی پیچیدگی تغییر، این از پس

الگوریتم ۳

به شکلی شگفت‌آور، حل این مسئله در زمان $O(n)$ ممکن است^۳؛ یعنی فقط یک حلقه کافی است. ایده این است که برای هر جایگاه آرایه، بیشینه مجموع زیرآرایه‌ای محاسبه شود که در آن جایگاه پایان می‌یابد. پس از آن، پاسخ مسئله، بیشینه این مجموع‌ها خواهد بود. زیرمسئله یافتن زیرآرایه با بیشینه مجموع را در نظر بگیرید که در جایگاه k پایان می‌یابد. دو حالت وجود دارد:

۱. زیرآرایه تنها شامل عنصر در جایگاه k است.

۲. زیرآرایه از یک زیرآرایه تشکیل شده که در جایگاه $k - 1$ پایان می‌یابد و به دنبال آن عنصر در جایگاه k می‌آید.

در حالت دوم، از آنجا که به دنبال زیرآرایه‌ای با بیشینه مجموع هستیم، زیرآرایه‌ای که در جایگاه $k - 1$ پایان می‌یابد نیز باید بیشینه مجموع را داشته باشد. بنابراین، می‌توان مسئله را با محاسبه بیشینه مجموع زیرآرایه برای هر جایگاه پایانی از چپ به راست، به صورت کارآمد حل کرد. کد زیر الگوریتم را پیاده‌سازی می‌کند:

```

0; = sum ,0 = best int
{ ++)k ;n < k 0; = k int( for
]);k[array+sum,]k[array(max = sum
);sum,best(max = best
}
; "n\\n" << best << cout

```

الگوریتم تنها شامل یک حلقه است که ورودی را پیمایش می‌کند، بنابراین پیچیدگی زمانی $O(n)$ است. این مقدار، بهترین پیچیدگی زمانی ممکن نیز هست؛ زیرا هر الگوریتمی برای این مسئله باید تمام عناصر آرایه را دست‌کم یک بار بررسی کند.

مقایسه کارایی

بررسی میزان کارایی الگوریتم‌ها در عمل جالب است. جدول زیر زمان‌های اجرای الگوریتم‌های فوق را به ازای مقادیر مختلف n روی یک رایانه امروزی نشان می‌دهد. در هر آزمایش، ورودی به صورت تصادفی تولید شده است. زمان لازم برای خواندن ورودی اندازه‌گیری نشده است.

^۳در [۸]، این الگوریتم زمان‌خطی به جی. بی. کادین نسبت داده شده و الگوریتم گاهی الگوریتم کادین نامیده می‌شود.

اندازه آرایه n	الگوریتم ۱	الگوریتم ۲	الگوریتم ۳
10^2	s 0.0	s 0.0	s 0.0
10^3	s 0.1	s 0.0	s 0.0
10^4	s 10.0 <	s 0.1	s 0.0
10^5	s 10.0 <	s 5.3	s 0.0
10^6	s 10.0 <	s 10.0 <	s 0.0
10^7	s 10.0 <	s 10.0 <	s 0.0

مقایسه نشان می‌دهد همه الگوریتم‌ها در اندازه‌های کوچک ورودی کارآمد هستند، اما ورودی‌های بزرگ‌تر تفاوت‌های چشمگیری در زمان اجرای الگوریتم‌ها آشکار می‌کنند. الگوریتم ۱ در $n = 10^4$ کند می‌شود و الگوریتم ۲ در $n = 10^5$ کند می‌شود. تنها الگوریتم ۳ می‌تواند حتی بزرگ‌ترین ورودی‌ها را نیز به صورت آبی پردازش کند.

فصل ۳

مرتب‌سازی

مرتب‌سازی مسئله‌ای بنیادین در طراحی الگوریتم است. بسیاری از الگوریتم‌های کارآمد از مرتب‌سازی به عنوان یک زیرروال استفاده می‌کنند، زیرا پردازش داده‌ها در صورتی که عناصر مرتب باشند، اغلب آسان‌تر است.

به عنوان مثال، مسئله «آیا آرایه شامل دو عنصر برابر است؟» با استفاده از مرتب‌سازی به سادگی حل می‌شود. اگر آرایه شامل دو عنصر برابر باشد، آن‌ها پس از مرتب‌سازی کنار یکدیگر قرار می‌گیرند، بنابراین یافتن آن‌ها آسان است. همچنین مسئله «پرتکرارترین عنصر در یک آرایه چیست؟» را می‌توان به شیوه مشابهی حل کرد.

الگوریتم‌های زیادی برای مرتب‌سازی وجود دارد و آن‌ها نمونه‌های خوبی برای چگونگی به‌کارگیری تکنیک‌های مختلف طراحی الگوریتم هستند. الگوریتم‌های عمومی و کارآمد مرتب‌سازی در زمان $O(n \log n)$ عمل می‌کنند، و بسیاری از الگوریتم‌هایی که از مرتب‌سازی به عنوان زیرروال استفاده می‌کنند نیز همین پیچیدگی زمانی را دارند.

۱.۳ نظریه مرتب‌سازی

مسئله پایه‌ای در مرتب‌سازی به شرح زیر است:

آرایه‌ای شامل n عنصر به شما داده شده است، وظیفه شما مرتب‌سازی عناصر به ترتیب صعودی است.

به عنوان مثال، آرایه

1	3	8	2	9	2	5	6
---	---	---	---	---	---	---	---

پس از مرتب‌سازی به صورت زیر درخواهد آمد:

1	2	2	3	5	6	8	9
---	---	---	---	---	---	---	---

الگوریتم‌های $O(n^2)$

الگوریتم‌های ساده مرتب‌سازی آرایه در زمان $O(n^2)$ اجرا می‌شوند. چنین الگوریتم‌هایی کوتاه هستند و معمولاً از دو حلقه تو در تو تشکیل می‌شوند. یک الگوریتم معروف مرتب‌سازی با

زمان $O(n^2)$ ، **مرتب‌سازی حبابی** است که در آن عناصر با توجه به مقادیرشان در آرایه «حباب» می‌زنند و بالا می‌آیند.

مرتب‌سازی حبابی از n دور تشکیل شده است. در هر دور، الگوریتم در میان عناصر آرایه پیمایش می‌کند. هر زمان که دو عنصر متوالی با ترتیب نادرست یافت شوند، الگوریتم جای آن‌ها را عوض می‌کند. الگوریتم را می‌توان به صورت زیر پیاده‌سازی کرد:

```
int( for i = 0; i < n; ++i)
{
    for( j = 0; j < n - 1; ++j)
    {
        if( array[j] > array[j + 1])
            swap( array[j], array[j + 1]);
    }
}
```

پس از دور اول الگوریتم، بزرگ‌ترین عنصر در جایگاه صحیح قرار خواهد گرفت، و به طور کلی پس از k دور، k عنصر بزرگ‌تر در جایگاه‌های صحیح خواهند بود. بنابراین پس از n دور، کل آرایه مرتب خواهد شد.
به عنوان مثال، در آرایه

1	3	8	2	9	2	5	6
---	---	---	---	---	---	---	---

دور نخست مرتب‌سازی حبابی عناصرها را به این ترتیب جابه‌جا می‌کند:

1	3	2	8	9	2	5	6
---	---	---	---	---	---	---	---



1	3	2	8	2	9	5	6
---	---	---	---	---	---	---	---



1	3	2	8	2	5	9	6
---	---	---	---	---	---	---	---



1	3	2	8	2	5	6	9
---	---	---	---	---	---	---	---



وارونگی‌ها

مرتب‌سازی حبابی نمونه‌ای از الگوریتم مرتب‌سازی است که همواره عنصرهای مجاور در آرایه را جابه‌جا می‌کند. مشخص می‌شود پیچیدگی زمانی چنین الگوریتمی همواره حداقل $O(n^2)$ است، زیرا در بدترین حالت، $O(n^2)$ جابه‌جایی برای مرتب‌سازی آرایه نیاز است.

یک مفهوم کاربردی در تحلیل الگوریتم‌های مرتب‌سازی، **وارونگی** (inversion) است: جفتی از عناصر آرایه $(array[a], array[b])$ به طوری که $a < b$ و $array[a] > array[b]$ باشد؛ یعنی عناصر ترتیب نادرست دارند. برای نمونه، آرایه

1	2	2	6	3	5	9	8
---	---	---	---	---	---	---	---

سه وارونگی دارد: $(6, 3)$ ، $(6, 5)$ و $(9, 8)$. تعداد وارونگی‌ها نشان می‌دهد مرتب‌سازی آرایه چقدر کار نیاز دارد. آرایه زمانی کاملاً مرتب است که هیچ وارونگی وجود نداشته باشد. از طرف دیگر، اگر عناصر آرایه ترتیب معکوس داشته باشند، تعداد وارونگی‌ها بیشترین مقدار ممکن است:

$$1 + 2 + \dots + (n - 1) = \frac{n(n - 1)}{2} = O(n^2)$$

جابه‌جایی یک جفت عنصر متوالی با ترتیب نادرست، دقیقاً یک وارونگی از آرایه کم می‌کند. بنابراین، اگر الگوریتم مرتب‌سازی فقط بتواند عناصر متوالی را جابه‌جا کند، هر جابه‌جایی حداکثر یک وارونگی حذف می‌کند و پیچیدگی زمانی الگوریتم حداقل $O(n^2)$ است.

الگوریتم‌های $O(n \log n)$

مرتب‌سازی کارآمد آرایه در زمان $O(n \log n)$ با الگوریتم‌هایی ممکن است که به جابه‌جایی عناصر متوالی محدود نیستند. یکی از این الگوریتم‌ها **مرتب‌سازی ادغامی**^۱ است که بر پایه بازگشت کار می‌کند.

مرتب‌سازی ادغامی، زیرآرایه $[a \dots b]$ array را به این شکل مرتب می‌کند:

۱. اگر $a = b$ باشد، کاری انجام ندهید، زیرا زیرآرایه از قبل مرتب است.

۲. موقعیت عنصر میانی را محاسبه کنید: $k = \lfloor (a + b) / 2 \rfloor$.

۳. زیرآرایه $[a \dots k]$ array را بازگشتی مرتب کنید.

۴. زیرآرایه $[k + 1 \dots b]$ array را بازگشتی مرتب کنید.

۵. زیرآرایه‌های مرتب‌شده $[a \dots k]$ array و $[k + 1 \dots b]$ array را در زیرآرایه مرتب‌شده $[a \dots b]$ array ادغام کنید.

مرتب‌سازی ادغامی کارآمد است، زیرا طول زیرآرایه را در هر مرحله نصف می‌کند. بازگشت شامل $O(\log n)$ سطح است و پردازش هر سطح به زمان $O(n)$ نیاز دارد. ادغام زیرآرایه‌های $[a \dots k]$ array و $[k + 1 \dots b]$ array در زمان خطی ممکن است، زیرا از قبل مرتب هستند. برای نمونه، مرتب‌سازی آرایه زیر را در نظر بگیرید:

1	3	6	2	8	2	5	9
---	---	---	---	---	---	---	---

آرایه به صورت زیر به دو زیرآرایه تقسیم می‌شود:

1	3	6	2
8	2	5	9

سپس، زیرآرایه‌ها به صورت بازگشتی به این شکل مرتب می‌شوند:

1	2	3	6
2	5	8	9

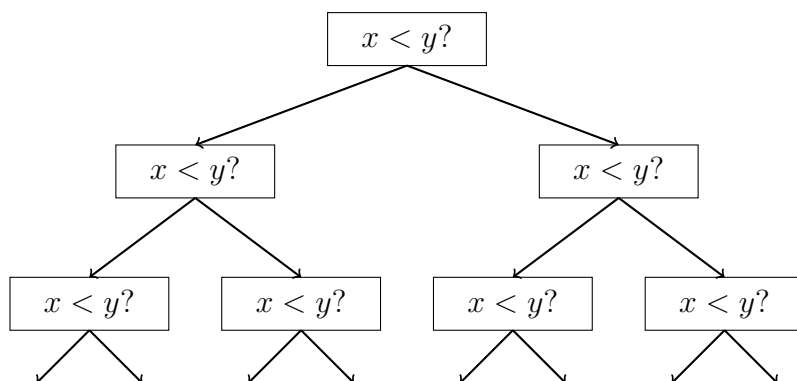
در نهایت، الگوریتم زیرآرایه‌های مرتب‌شده را ادغام کرده و آرایه مرتب‌شده نهایی را تولید می‌کند:

1	2	2	3	5	6	8	9
---	---	---	---	---	---	---	---

^۱بر اساس [۴۷]، مرتب‌سازی ادغامی در سال ۱۹۴۵ توسط جان فون نویمان ابداع شد.

کران پای‌ن مرتب‌سازی

آیا مرتب‌سازی یک آرایه سریع‌تر از زمان $O(n \log n)$ ممکن است؟ مشخص شده که اگر الگوریتم به مقایسه عناصر آرایه محدود باشد، این کار امکان‌پذیر نیست. کران پایین پیچیدگی زمانی را می‌توان با در نظر گرفتن مرتب‌سازی به عنوان فرآیندی اثبات کرد که در آن هر مقایسه دو عنصر، اطلاعات بیشتری درباره محتویات آرایه می‌دهد. این فرآیند درخت زیر را تولید می‌کند:



در اینجا " $x < y?$ " به معنی مقایسه دو عنصر x و y است. اگر $x < y$ باشد، فرآیند به سمت چپ ادامه می‌یابد و در غیر این صورت به سمت راست می‌رود. نتایج این فرآیند، حالت‌های ممکن برای مرتب‌سازی آرایه هستند که در مجموع $n!$ حالت است. به همین دلیل، ارتفاع درخت باید حداقل برابر با مقدار زیر باشد:

$$\log_2(n!) = \log_2(1) + \log_2(2) + \dots + \log_2(n).$$

با انتخاب $n/2$ عنصر آخر و تغییر مقدار هر عنصر به $\log_2(n/2)$ ، یک کران پایین برای این مجموع به دست می‌آوریم. این کار تخمین زیر را نتیجه می‌دهد:

$$\log_2(n!) \geq (n/2) \cdot \log_2(n/2),$$

بنابراین ارتفاع درخت و کمترین تعداد مراحل ممکن در یک الگوریتم مرتب‌سازی در بدترین حالت، حداقل $n \log n$ است.

مرتب‌سازی شمارشی

کران پایین $n \log n$ برای الگوریتم‌هایی که عناصر آرایه را با یکدیگر مقایسه نمی‌کنند بلکه از اطلاعات دیگری بهره می‌برند، اعمال نمی‌شود. یک نمونه از این الگوریتم‌ها **مرتب‌سازی شمارشی** است که آرایه را در زمان $O(n)$ مرتب می‌کند، با این فرض که هر عنصر آرایه یک عدد صحیح بین $0 \dots c$ بوده و $c = O(n)$ باشد. این الگوریتم یک آرایه شمارش می‌سازد که اندیس‌های آن، عناصر آرایه اصلی هستند. الگوریتم روی آرایه اصلی پیمایش کرده و تعداد دفعات تکرار هر عنصر در آرایه را محاسبه می‌کند.

برای نمونه، آرایه

1	3	6	9	9	3	5	9
---	---	---	---	---	---	---	---

متناظر با آرایه شمارش زیر است:

1	2	3	4	5	6	7	8	9
1	0	2	0	1	1	0	0	3

برای مثال، مقدار در موقعیت ۳ در آرایه شمارش برابر ۲ است، زیرا عنصر ۳ دقیقاً ۲ بار در آرایه اصلی آمده است. ساخت آرایه شمارش زمان $O(n)$ می‌برد. پس از آن، آرایه مرتب‌شده را می‌توان در زمان $O(n)$ ساخت، زیرا تعداد رخدادهای هر عنصر از آرایه شمارش قابل بازیابی است. بنابراین، پیچیدگی زمانی کل مرتب‌سازی شمارشی $O(n)$ است. مرتب‌سازی شمارشی الگوریتمی بسیار کارآمد است اما تنها زمانی قابل استفاده است که ثابت c به اندازه کافی کوچک باشد، به طوری که عناصر آرایه بتوانند به عنوان اندیس در آرایه شمارش به کار روند.

۲.۳ مرتب‌سازی در C++

تقریباً هرگز ایده خوبی نیست که در مسابقه از یک الگوریتم مرتب‌سازی دست‌ساز استفاده کنید، زیرا پیاده‌سازی‌های خوبی در زبان‌های برنامه‌نویسی در دسترس است. برای مثال، کتابخانه استاندارد C++ شامل تابع `sort` است که می‌توان از آن به راحتی برای مرتب‌سازی آرایه‌ها و سایر داده‌ساختارها استفاده کرد. استفاده از توابع کتابخانه‌ای مزایای زیادی دارد. نخست، در زمان صرفه‌جویی می‌شود زیرا نیازی به پیاده‌سازی تابع نیست. دوم، پیاده‌سازی کتابخانه قطعاً درست و کارآمد است؛ بعید است که یک تابع مرتب‌سازی دست‌ساز عملکرد بهتری داشته باشد. در این بخش نحوه استفاده از تابع `sort` در C++ را بررسی می‌کنیم. کد زیر یک بردار را به صورت صعودی مرتب می‌کند:

```
int v[4] = {2, 5, 3, 8};  
vector<int> v2(v, v+4);  
sort(v2.begin(), v2.end());
```

پیش‌فرض ترتیب بود. خواهد [2, 3, 3, 4, 5, 5, 8] صورت به بردار محتویات مرتب‌سازی، از پس است: امکان‌پذیر زیر شکل به نیز نزولی مرتب‌سازی اما است، صعودی مرتب‌سازی

```
sort(v2.rbegin(), v2.rend());
```

کرد: مرتب زیر شکل به می‌توان را معمولی آرایه یک

```
int n = 7; // size array  
int a[] = {2, 5, 3, 8};  
sort(a, a+n);
```

کد زیر رشته s را مرتب می‌کند:

```
;"monkey" = s string  
( ));end.s ,()begin.s(sort
```

برای می‌شوند. مرتب رشته آن نویسه‌های که معناست این به رشته یک مرتب‌سازی می‌شود. "ekmnoy" به تبدیل "monkey" رشته مثال،

عمل‌گرهای مقایسه

تابع sort نیازمند آن است که یک **عملگر مقایسه** برای نوع داده عناصر مورد نظر تعریف شده باشد. هنگام مرتب‌سازی، هر جا که تعیین ترتیب دو عنصر لازم باشد از این عملگر استفاده می‌شود.

بیشتر انواع داده C++ عملگر مقایسه توکار دارند و عناصر این نوع‌ها را می‌توان به صورت خودکار مرتب کرد. برای مثال، اعداد بر اساس مقادیر عددی‌شان و رشته‌ها به ترتیب الفبایی مرتب می‌شوند.

جفت‌ها (pair) در درجه اول بر اساس عنصر اول (first) مرتب می‌شوند. با این حال، اگر عنصر اول دو جفت برابر باشد، بر اساس عنصر دوم (second) مرتب خواهند شد:

```
;v >>int,int<pair<vector  
,5});({1push_back.v  
,3});({2push_back.v  
,2});({1push_back.v  
( ));end.v ,()begin.v(sort
```

بود. خواهد (2, 3) و (1, 5)، (1, 2) صورت به جفت‌ها ترتیب کار، این از پس به طور مشابه، چندتایی‌ها (tuple) در درجه اول با عنصر نخست، سپس با عنصر دوم و الی آخر مرتب می‌شوند^۲:

```
;v >>int,int,int<tuple<vector  
,4});,1({2push_back.v  
,3});,5({1push_back.v  
,3});,1({2push_back.v  
( ));end.v ,()begin.v(sort
```

بود. خواهد (2, 1, 4) و (2, 1, 3)، (1, 5, 3) صورت به چندتایی‌ها ترتیب کار، این از پس

ساختارهای تعریف‌شده توسط کاربر

ساختارهای تعریف‌شده توسط کاربر به طور خودکار عملگر مقایسه ندارند. عملگر باید درون ساختار به عنوان تابع operator< تعریف شود، که پارامتر آن عنصر دیگری از همان نوع است. اگر عنصر کوچکتر از پارامتر باشد، عملگر باید true بازگرداند، و در غیر این صورت false.

برای مثال، ساختار P زیر شامل مختصات x و y یک نقطه است. عملگر مقایسه طوری تعریف شده است که نقاط در وهله اول بر اساس مختصات x و در وهله دوم بر اساس مختصات y مرتب شوند.

^۲توجه داشته باشید که در برخی کامپایلرهای قدیمی‌تر، برای ساخت یک چندتایی باید از تابع make_tuple به جای آکولاد استفاده کرد (برای مثال، make_tuple(2, 1, 4) به جای {2, 1, 4}).


```

        { P struct
            ;y ,x int
        { )p& P const<(operator bool
;x.p < x return )x.p != x( if
        ;y.p < y return else
        }
    };

```

توابع مقایسه

همچنین می‌توان یک **تابع مقایسه** خارجی را به عنوان تابع بازگشتی (callback) به تابع sort داد. برای مثال، تابع مقایسه comp زیر رشته‌ها را ابتدا بر اساس طول و سپس بر اساس ترتیب الفبایی مرتب می‌کند:

```

        { )b string ,a string(comp bool
        ();size.b < ()size.a return ())size.b != ()size.a( if
        ;b < a return
        }

```

کرد: مرتب زیر صورت به می‌توان را رشته‌ها از برداری اکنون

```

        );comp ,()end.v ,()begin.v(sort

```

۳.۳ جستجوی دودویی

یک روش کلی برای جستجوی عنصر در آرایه، استفاده از حلقه for است که روی عناصر آرایه پیمایش می‌کند. برای مثال، کد زیر به دنبال عنصر x در یک آرایه می‌گردد:

```

        { ++)i ;n < i 0; = i int( for
        { )x == ]i[array( if
        i index at found x //
        }
    }

```

پیچیدگی زمانی این رویکرد $O(n)$ است، زیرا در بدترین حالت، بررسی تمام عناصر آرایه ضروری است. اگر ترتیب عناصر تصادفی باشد، این بهترین رویکرد ممکن نیز هست، زیرا اطلاعات اضافی درباره محل جستجوی عنصر x در دسترس نیست.

اما اگر آرایه مرتب‌شده باشد، اوضاع فرق می‌کند. در این حالت می‌توان جستجو را بسیار سریع‌تر انجام داد، زیرا ترتیب عناصر در آرایه، جستجو را هدایت می‌کند. الگوریتم **جستجوی دودویی** زیر یک عنصر را در آرایه مرتب‌شده در زمان $O(\log n)$ جستجو می‌کند.

روش ۱

روش معمول برای پیاده‌سازی جستجوی دودویی مشابه جستجوی کلمه در فرهنگ لغت است. جستجو یک محدوده فعال را در آرایه نگه می‌دارد، که در ابتدا شامل تمام عناصر آرایه است. سپس مراحل انجام می‌شود که هر کدام اندازه محدوده را نصف می‌کنند. در هر مرحله، عنصر میانی محدوده فعال بررسی می‌شود. اگر عنصر میانی همان عنصر هدف باشد، جستجو پایان می‌یابد. در غیر این صورت، جستجو بسته به مقدار عنصر میانی، به صورت بازگشتی در نیمه چپ یا راست محدوده ادامه می‌یابد.

ایده بالا این‌گونه پیاده‌سازی می‌شود:

```
int a = 0, b = n - 1;
while (b <= a)
{
    int k = (a + b) / 2;
    if (array[k] == x)
        return k;
    else if (array[k] < x)
        a = k + 1;
    else
        b = k - 1;
}
```

در این پیاده‌سازی، ناحیه فعال $a \dots b$ است و در ابتدا این ناحیه $0 \dots n - 1$ است. الگوریتم در هر گام اندازه ناحیه را نصف می‌کند، بنابراین پیچیدگی زمانی $O(\log n)$ است.

روش ۲

روش دیگر برای پیاده‌سازی جستجوی دودویی مبتنی بر شیوه‌ای کارآمد برای پیمایش عناصر آرایه است. ایده کار، جهش به جلو و کاهش سرعت جهش هنگام نزدیک شدن به عنصر هدف است.

جستجو در آرایه از چپ به راست حرکت می‌کند، و طول جهش اولیه $n/2$ است. در هر گام، طول جهش نصف می‌شود: ابتدا $n/4$ ، سپس $n/8$ ، $n/16$ و غیره، تا اینکه در نهایت طول جهش برابر با ۱ شود. پس از پایان جهش‌ها، یا عنصر هدف پیدا شده است یا معلوم می‌شود درون آرایه وجود ندارد.

کد زیر ایده بالا را پیاده‌سازی می‌کند:

```
int k = 0;
for (int b = 1; b <= n; b *= 2)
{
    while (array[b + k] <= x)
        k += b;
    if (array[k] == x)
        return k;
}
```

در طول جستجو، متغیر b طول جهش کنونی را نگه می‌دارد. پیچیدگی زمانی الگوریتم $O(\log n)$ است، زیرا کد درون حلقه `while` برای هر طول جهش حداکثر دو بار اجرا می‌شود.

توابع ++C

کتابخانه استاندارد ++C شامل توابع زیر است که بر پایه جستجوی دودویی بوده و در زمان لگاریتمی عمل می‌کنند:

- تابع `lower_bound` اشاره‌گری به نخستین عنصر آرایه برمی‌گرداند که مقدار آن حداقل x است.
- تابع `upper_bound` اشاره‌گری به نخستین عنصر آرایه برمی‌گرداند که مقدار آن بزرگ‌تر از x است.
- تابع `equal_range` هر دو اشاره‌گر بالا را برمی‌گرداند.

این توابع فرض می‌کنند که آرایه مرتب است. اگر چنین عنصری وجود نداشته باشد، اشاره‌گر به عنصر پس از آخرین عنصر آرایه اشاره می‌کند. برای نمونه، کد زیر مشخص می‌کند آیا آرایه دارای عنصری با مقدار x هست یا خیر:

```
auto lower_bound = k;
if (x == k) {
    // index at found x
}
```

سپس، کد زیر شمار عناصر دارای مقدار x را به دست می‌آورد:

```
auto lower_bound = a;
auto upper_bound = b;
cout << a-b << "\n";
```

با استفاده از `equal_range`، کد کوتاه‌تر می‌شود:

```
auto equal_range = r;
cout << first.r-second.r << "\n";
```

یافتن کوچک‌ترین جواب

کاربرد مهم جستجوی دودویی یافتن نقطه‌ای است که مقدار یک تابع در آن تغییر می‌کند. فرض کنید می‌خواهیم کوچک‌ترین مقدار k را بیابیم که جواب معتبری برای مسئله باشد. تابع $ok(x)$ به ما داده شده است که اگر x جوابی معتبر باشد، مقدار `true` و در غیر این صورت `false` برمی‌گرداند. همچنین می‌دانیم به ازای $x < k$ مقدار $ok(x)$ برابر `false` و به ازای $x \geq k$ برابر `true` است. وضعیت به صورت زیر است:

...	$k+1$	k	$k-1$	...	1	0	x
...	true	true	false	...	false	false	$ok(x)$

اکنون مقدار k با جستجوی دودویی به دست می‌آید:

```

-1; = x int
{ 2) /= b 1; >= b ;z = b int( for
; b += x ))b+x(ok(! while
}
+1;x = k int

```

این جستجو بزرگترین مقدار x را می‌یابد که به ازای آن $ok(x)$ برابر `false` است. در نتیجه مقدار بعدی، یعنی $k = x + 1$ ، کوچکترین مقدار ممکن است که به ازای آن $ok(k)$ برابر `true` می‌شود. طول پرش اولیه z باید به اندازه کافی بزرگ باشد؛ مثلاً مقداری که از قبل می‌دانیم $ok(z)$ برای آن `true` است. الگوریتم تابع ok را $O(\log z)$ بار فراخوانی می‌کند، بنابراین پیچیدگی زمانی کل به تابع ok وابسته است. برای نمونه، اگر زمان اجرای این تابع $O(n)$ باشد، پیچیدگی زمانی کل $O(n \log z)$ خواهد بود.

یافتن بیشینه مقدار

جستجوی دودویی برای یافتن بیشینه مقدار تابعی که ابتدا صعودی و سپس نزولی است نیز به کار می‌رود. هدف یافتن موقعیت k است به طوری که:

• به ازای $x < k$ داشته باشیم $f(x) < f(x + 1)$ ، و

• به ازای $x \geq k$ داشته باشیم $f(x) > f(x + 1)$.

ایده کار استفاده از جستجوی دودویی برای یافتن بزرگترین مقدار x است که در آن $f(x) < f(x + 1)$ باشد. این نتیجه می‌دهد $k = x + 1$ ، زیرا $f(x + 1) > f(x + 2)$ است. کد زیر جستجو را پیاده‌سازی می‌کند:

```

-1; = x int
{ 2) /= b 1; >= b ;z = b int( for
; b += x +1))b+x(f < )b+x(f( while
}
+1;x = k int

```

برخلاف جستجوی دودویی معمولی، اینجا مقادیر متوالی تابع نباید برابر باشند. در صورت برابری، تعیین جهت ادامه جستجو ناممکن می‌شود.

فصل ۴

ساختمان داده‌ها

یک **ساختمان داده** روشی برای ذخیره داده‌ها در حافظه کامپیوتر است. انتخاب ساختمان داده مناسب برای هر مسئله اهمیت بالایی دارد، زیرا هر ساختمان داده مزایا و معایب خاص خود را دارد. پرسش اساسی این است: کدام عملیات‌ها در ساختمان داده انتخاب‌شده کارآمد هستند؟

این فصل مهم‌ترین ساختمان داده‌های موجود در کتابخانه استاندارد C++ را معرفی می‌کند. استفاده از کتابخانه استاندارد تا جای ممکن توصیه می‌شود، زیرا زمان زیادی را صرفه‌جویی می‌کند. در ادامه کتاب، ساختمان داده‌های پیشرفته‌تری را بررسی خواهیم کرد که در کتابخانه استاندارد در دسترس نیستند.

۱.۴ آرایه‌های پویا

یک **آرایه پویا** آرایه‌ای است که اندازه آن در طول اجرای برنامه قابل تغییر است. محبوب‌ترین آرایه پویا در C++ ساختار vector است که می‌توان از آن تقریباً مانند یک آرایه معمولی استفاده کرد.

کد زیر یک وکتور خالی ایجاد کرده و سه عنصر به آن اضافه می‌کند:

```
vector<int> v;
v.push_back(3); // [3]
v.push_back(2); // [3, 2]
v.push_back(5); // [3, 2, 5]
```

پس از این، دسترسی به عناصر مانند یک آرایه معمولی امکان‌پذیر است:

```
cout << v[0] << "\n"; // 3
cout << v[1] << "\n"; // 2
cout << v[2] << "\n"; // 5
```

تابع size تعداد عناصر موجود در وکتور را برمی‌گرداند. کد زیر روی وکتور پیمایش کرده و تمام عناصر آن را چاپ می‌کند:

```
for (int i = 0; i < v.size(); ++i)
    cout << v[i] << "\n";
```

```
}
```

روش کوتاه‌تر برای پیمایش وکتور به صورت زیر است:

```
{ }v : x auto( for  
; "n\" << x << cout  
}
```

تابع `back` آخرین عنصر وکتور را برمی‌گرداند و تابع `pop_back` آخرین عنصر را حذف می‌کند:

```
;v >int<vector  
(5);push_back.v  
(2);push_back.v  
2 // ; "n\" << ()back.v << cout  
( );pop_back.v  
5 // ; "n\" << ()back.v << cout
```

کد زیر یک وکتور با پنج عنصر ایجاد می‌کند:

```
,1};,5,2,4{2 = v >int<vector
```

روش دیگر ساخت وکتور، تعیین تعداد عناصر و مقدار اولیه هر عنصر است:

```
0 value initial ,10 size //  
(10);v >int<vector
```

```
5 value initial ,10 size //  
5); , (10v >int<vector
```

پیاده‌سازی داخلی وکتور از یک آرایه معمولی استفاده می‌کند. اگر اندازه وکتور افزایش یابد و آرایه بیش از حد کوچک شود، آرایه جدیدی تخصیص داده شده و تمام عناصر به آرایه جدید منتقل می‌شوند. با این حال، این اتفاق مکرراً رخ نمی‌دهد و پیچیدگی زمانی میانگین $O(1)$ است.

ساختار `string` هم آرایه پویا است، تقریباً مثل بردار کار می‌کند. رشته‌ها نحو اختصاصی دارند که در سایر ساختارهای داده نیست. الحاق رشته‌ها با نماد `+` انجام می‌شود. تابع `substr(k, x)` زیررشته از موقعیت k با طول x برمی‌گرداند، تابع `find(t)` موقعیت اولین وقوع زیررشته t پیدا می‌کند. کد زیر عملیات رشته را نشان می‌دهد:

```
; "hatti" = a string  
; a+a = b string
```

```

hattihatti // ;"n\" << b << cout
                ;'v' = [5]b
hattivatti // ;"n\" << b << cout
                ,4);(3substr.b = c string
tiva // ;"n\" << c << cout

```

۲.۴ ساختارهای مجموعه

یک **مجموعه** داده ساختاری است که دسته‌ای از عناصر را نگه می‌دارد. عملیات اصلی: درج، جستجو، حذف عنصر.

کتابخانه استاندارد C++ دو پیاده‌سازی دارد: ساختار set بر پایه درخت دودویی متوازن است، عملیات آن در زمان $O(\log n)$ اجرا می‌شود. ساختار unordered_set از درهم‌سازی استفاده می‌کند، عملیات آن در حالت میانگین در زمان $O(1)$ اجرا می‌شود.

انتخاب نوع پیاده‌سازی سلیقه‌ای است. مزیت ساختار set: حفظ ترتیب عناصر، ارائه توابعی که در unordered_set نیست. در مقابل، unordered_set می‌تواند کارآمدتر باشد. کد زیر مجموعه اعداد صحیح می‌سازد، برخی عملیات را نشان می‌دهد. تابع insert عنصر به مجموعه اضافه می‌کند، تابع count تعداد تکرار عنصر در مجموعه را برمی‌گرداند، تابع erase عنصر از مجموعه حذف می‌کند.

```

; s >int<set
(3);insert.s
(2);insert.s
(5);insert.s
1 // ;"n\" << (3)count.s << cout
0 // ;"n\" << (4)count.s << cout
(3);erase.s
(4);insert.s
0 // ;"n\" << (3)count.s << cout
1 // ;"n\" << (4)count.s << cout

```

مجموعه مثل بردار استفاده می‌شود، اما دسترسی به عناصر با نماد [] ممکن نیست. کد زیر مجموعه می‌سازد، تعداد عناصر را چاپ می‌کند، سپس روی همه عناصر پیمایش می‌کند:

```

,8};,6,5{2 = s >int<set
4 // ;"n\" << ()size.s << cout
{ )s : x auto( for
;"n\" << x << cout
}

```

ویژگی مهم مجموعه: تمام عناصر متمایز هستند. بنابراین تابع count همیشه ۰ (عنصر نیست) یا ۱ (عنصر هست) برمی‌گرداند، تابع insert عنصری که قبلاً باشد را دوباره اضافه نمی‌کند. کد زیر این رفتار را نشان می‌دهد:

```

; s >int<set

```

```
(5);insert.s
(5);insert.s
(5);insert.s
1 // ;"n\" << (5)count.s << cout
```

سی‌پلاس‌پلاس همچنین شامل ساختارهای `multiset` و `unordered_multiset` است که عملکردی مشابه `set` و `unordered_set` دارند، اما می‌توانند شامل چندین نسخه از یک عنصر باشند. به عنوان مثال، در کد زیر هر سه نسخه از عدد ۵ به یک چندمجموعه اضافه می‌شوند:

```
;s >int<multiset
(5);insert.s
(5);insert.s
(5);insert.s
3 // ;"n\" << (5)count.s << cout
```

می‌کند: حذف چندمجموعه از را عنصر یک نسخه‌های تمام `erase` تابع

```
(5);erase.s
0 // ;"n\" << (5)count.s << cout
```

شود: انجام زیر صورت به می‌تواند که است نظر مورد عنصر از نسخه یک حذف تنها اغلب،

```
(5));find.s(erase.s
2 // ;"n\" << (5)count.s << cout
```

۳.۴ ساختارهای نگاشت (Map)

یک **نگاشت** یک آرایه تعمیم‌یافته است که از جفت‌های کلید-مقدار تشکیل می‌شود. در حالی که کلیدها در یک آرایه معمولی همیشه اعداد صحیح متوالی $0, 1, \dots, n-1$ هستند (که n اندازه آرایه است)، کلیدها در یک نگاشت می‌توانند از هر نوع داده‌ای باشند و نیازی نیست که مقادیری متوالی باشند.

کتابخانه استاندارد C++ شامل دو پیاده‌سازی نگاشت است که متناظر با پیاده‌سازی‌های مجموعه‌اند: ساختار `map` بر پایه یک درخت دودویی متوازن است و دسترسی به عناصر زمان $O(\log n)$ می‌برد، در حالی که ساختار `unordered_map` از درهم‌سازی استفاده می‌کند و دسترسی به عناصر در حالت میانگین زمان $O(1)$ می‌برد. کد زیر یک نگاشت ایجاد می‌کند که در آن کلیدها رشته و مقادیر اعداد صحیح هستند:

```
;m >int,string<map
4; = ] "monkey" [m
3; = ] "banana" [m
9; = ] "harpsichord" [m
3 // ;"n\" << ] "banana" [m << cout
```


اگر مقدار یک کلید درخواست شود اما نگاشت شامل آن نباشد، کلید به طور خودکار با یک مقدار پیش‌فرض به نگاشت اضافه می‌شود. برای مثال در کد زیر، کلید "aybantu" با مقدار ۰ به نگاشت اضافه می‌شود.

```

; m > int, string<map
0 // ; "n\" << ] "aybantu" [ m << cout

```

خیر: یا دارد وجود نگاشت در کلید یک آیا که می‌کند بررسی count تابع

```

{ }) "aybantu" (count.m( if
exists key //
}

```

می‌کند: چاپ را نگاشت یک در موجود مقادیر و کلیدها تمام زیر کد

```

{ ) m : x auto( for
; "n\" << second.x << " " << first.x << cout
}

```

۴.۴ تکرارکننده‌ها و بازه‌ها

بسیاری از توابع در کتابخانه استاندارد C++ با تکرارکننده‌ها کار می‌کنند. یک **تکرارکننده** متغیری است که به یک عنصر در یک ساختار داده اشاره می‌کند. تکرارکننده‌های پرکاربرد begin و end بازه‌ای را تعریف می‌کنند که شامل تمام عناصر موجود در یک ساختار داده است. تکرارکننده begin به اولین عنصر در ساختار داده اشاره می‌کند، و تکرارکننده end به موقعیت بعد از آخرین عنصر اشاره دارد. وضعیت به شکل زیر است:

```

{          ۱۷ ، ۱۴ ، ۱۳ ، ۱۲ ، ۸ ، ۶ ، ۴ ، ۳ }
↑                ↑
s.end()          s.begin()

```

به عدم تقارن در تکرارکننده‌ها دقت کنید: s.begin() به عنصری درون ساختمان داده اشاره دارد، در حالی که s.end() به خارج از ساختمان داده اشاره می‌کند. بنابراین، بازه تعریف‌شده توسط تکرارکننده‌ها نیمه‌باز است.

کار با بازه‌ها

تکرارکننده‌ها در توابع کتابخانه استاندارد C++ کاربرد دارند، توابعی که بازه‌ای از عناصر یک ساختمان داده به آن‌ها داده می‌شود. معمولاً پردازش همه عناصر یک ساختمان داده مد نظر است، بنابراین تکرارکننده‌های begin و end به تابع ورودی داده می‌شوند. برای نمونه، کد زیر یک بردار را با استفاده از تابع sort مرتب می‌کند، سپس ترتیب عناصر را با تابع reverse معکوس می‌سازد، و در نهایت ترتیب عناصر را به کمک تابع random_shuffle برمی‌زند.

```
());end.v ,()begin.v(sort  
());end.v ,()begin.v(reverse  
());end.v ,()begin.v(random_shuffle
```

این توابع برای آرایه‌های معمولی نیز کاربرد دارند. در این حالت، به جای تکرارکننده، اشاره‌گرهایی به آرایه ورودی داده می‌شوند:

```
);n+a ,a(sort
);n+a ,a(reverse
);n+a ,a(random_shuffle
```

تکرارکننده‌های مجموعه

تکرارکننده‌ها اغلب برای دسترسی به عناصر یک مجموعه استفاده می‌شوند. کد زیر تکرارکننده `it` را ایجاد می‌کند که به کوچک‌ترین عنصر مجموعه اشاره دارد:

```
() ;begin.s = it iterator>::int<set
```

کد: این نوشتن برای کوتاه‌تر روش

```
() ;begin.s = it auto
```

زیر کد نمونه، برای است. دسترسی قابل `*` نماد با می‌کند اشاره آن به تکرارکننده که عنصری می‌کند: چاپ را مجموعه عنصر نخستین

```
() ;begin.s = it auto
;"n\" << it* << cout
```

تکرارکننده‌ها را می‌توان با عملگرهای `++` (به جلو) و `--` (به عقب) حرکت داد، بدین معنی که تکرارکننده به عنصر بعدی یا قبلی درون مجموعه جابه‌جا می‌شود. کد زیر تمام عناصر را به ترتیب صعودی چاپ می‌کند:

```
{ ++)it ();end.s != it ();begin.s = it auto( for
;"n\" << it* << cout
}
```

می‌کند: چاپ را مجموعه عنصر بزرگ‌ترین زیر کد

```
--;-it ();end.s = it auto
;"n\" << it* << cout
```

تابع `find(x)` تکرارکننده‌ای برمی‌گرداند که به عنصری با مقدار x اشاره دارد. با این حال، اگر مجموعه شامل x نباشد، خروجی تکرارکننده `end` خواهد بود.

```
);x(find.s = it auto
{ ()end.s == it( if
x //
{
```

تابع `lower_bound(x)` یک تکرارکننده به کوچک‌ترین عنصر مجموعه برمی‌گرداند که مقدار آن حداقل x باشد، و تابع `upper_bound(x)` یک تکرارکننده به کوچک‌ترین عنصر

مجموعه برمی‌گرداند که مقدار آن بزرگ‌تر از x باشد. در هر دو تابع، اگر چنین عنصری وجود نداشته باشد، مقدار بازگشتی `end` است. این توابع در ساختار داده `unordered_set` که ترتیب عناصر را حفظ نمی‌کند، پشتیبانی نمی‌شوند. برای نمونه، کد زیر نزدیک‌ترین عنصر به x را می‌یابد:

```
);x(lower_bound.s = it auto
{ ()begin.s == it( if
; "n\" << it* << cout
{ ()end.s == it( if else }
-;-it
; "n\" << it* << cout
{ else }
-;-it ;it* = a int
;it* = b int
; "n\" << b << cout )x-a < b-x( if
; "n\" << a << cout else
}
```

این کد فرض می‌کند که مجموعه خالی نیست، و با استفاده از تکرارکننده `it` تمام حالت‌های ممکن را بررسی می‌کند. ابتدا، تکرارکننده به کوچک‌ترین عنصری اشاره می‌کند که مقدار آن حداقل x است. اگر `it` برابر با `begin` باشد، عنصر متناظر نزدیک‌ترین مقدار به x است. اگر `it` برابر با `end` باشد، بزرگ‌ترین عنصر مجموعه نزدیک‌ترین مقدار به x است. اگر هیچ‌یک از حالت‌های قبلی رخ ندهد، عنصر نزدیک‌تر به x یا عنصر متناظر با `it` است یا عنصر پیش از آن.

۵.۴ سایر ساختارها

بیت‌ست (Bitset)

یک **بیت‌ست** آرایه‌ای است که هر مقدار آن ۰ یا ۱ است. برای نمونه، کد زیر یک بیت‌ست با ۱۰ عنصر می‌سازد:

```
;s ><10bitset
1; = [1]s
1; = [3]s
1; = [4]s
1; = [7]s
1 // ; "n\" << [4]s << cout
0 // ; "n\" << [5]s << cout
```

مزیت استفاده از بیت‌ست‌ها مصرف حافظه کمتر آن‌ها نسبت به آرایه‌های معمولی است، زیرا هر عنصر در بیت‌ست تنها یک بیت حافظه می‌گیرد. برای نمونه، اگر n بیت درون یک آرایه `int` ذخیره شود، $32n$ بیت حافظه مصرف خواهد شد، اما بیت‌ست متناظر تنها نیازمند n بیت حافظه است. افزون بر این، مقادیر بیت‌ست را می‌توان به شکل کارآمد با عملگرهای بیتی دستکاری کرد، که این کار بهینه‌سازی الگوریتم‌ها به کمک بیت‌ست را ممکن

می‌سازد.

کد زیر روش دیگری برای ساخت بیت‌ست بالا نشان می‌دهد:

```
left to right from // ));"0010011010"(string(s ><10bitset
1 // ;"n\" << [4]s << cout
0 // ;"n\" << [5]s << cout
```

تابع count تعداد یک‌ها را در بیت‌ست برمی‌گرداند:

```
));"0010011010"(string(s ><10bitset
4 // ;"n\" << ()count.s << cout
```

کد زیر نمونه‌هایی از به‌کارگیری عملگرهای بیتی را نشان می‌دهد:

```
));"0010110110"(string(a ><10bitset
));"1011011000"(string(b ><10bitset
0010010000 // ;"n\" << )b&a( << cout
1011111110 // ;"n\" << )b|a( << cout
1001101110 // ;"n\" << )b^a( << cout
```

دک (Deque)

یک **deque** (دک یا صف دوطرفه) آرایه‌ای پویا است که اندازه آن می‌تواند به شکلی کارآمد از هر دو سر آرایه تغییر کند. همانند بردار، یک deque توابع `push_back` و `pop_back` را ارائه می‌دهد، اما همچنین شامل توابع `push_front` و `pop_front` است که در بردار وجود ندارند.

یک deque می‌تواند به صورت زیر استفاده شود:

```
;d >int<deque
[5] // (5);push_back.d
,2] [5 // (2);push_back.d
,2],5[3 // (3);push_front.d
,5] [3 // ();pop_back.d
[5] // ();pop_front.d
```

پیاده‌سازی داخلی یک deque پیچیده‌تر از یک بردار است، و به همین دلیل، deque کندتر از بردار است. با این حال، هر دو عملیات افزودن و حذف عناصر در هر دو سر به طور میانگین به زمان $O(1)$ نیاز دارند.

پشته

یک **پشته** (stack) ساختمان داده‌ای است که دو عملیات با زمان $O(1)$ ارائه می‌دهد: افزودن یک عنصر به بالا، و حذف یک عنصر از بالا. تنها دسترسی به عنصر بالای پشته امکان‌پذیر است.

کد زیر نحوه استفاده از پشته را نشان می‌دهد:

```
int<stack> s;
(3);push.s
(2);push.s
(5);push.s
5 // ();top.s << cout
();pop.s
2 // ();top.s << cout
```

صف

یک صف (queue) نیز دو عملیات با زمان $O(1)$ ارائه می‌دهد: افزودن یک عنصر به انتهای صف، و حذف عنصر اول از صف. تنها دسترسی به عناصر اول و آخر صف امکان‌پذیر است. کد زیر نحوه استفاده از صف را نشان می‌دهد:

```
int<queue> q;
(3);push.q
(2);push.q
(5);push.q
3 // ();front.q << cout
();pop.q
2 // ();front.q << cout
```

صف اولویت‌دار

یک صف اولویت‌دار (priority queue) مجموعه‌ای از عناصر را نگهداری می‌کند. عملیات پشتیبانی‌شده شامل درج و، بسته به نوع صف، بازیابی و حذف کوچک‌ترین یا بزرگ‌ترین عنصر است. درج و حذف به زمان $O(\log n)$ نیاز دارند، و بازیابی در زمان $O(1)$ انجام می‌شود. در حالی که یک مجموعه مرتب تمام عملیات یک صف اولویت‌دار را به صورت کارآمد پشتیبانی می‌کند، مزیت استفاده از صف اولویت‌دار در این است که ضرایب ثابت کوچک‌تری دارد. یک صف اولویت‌دار معمولاً با استفاده از ساختار هرم پیاده‌سازی می‌شود که بسیار ساده‌تر از درخت دودویی متعادل مورد استفاده در مجموعه مرتب است. به طور پیش‌فرض، عناصر در یک صف اولویت‌دار در ++C به صورت نزولی مرتب شده‌اند، و یافتن و حذف بزرگ‌ترین عنصر در صف ممکن است. کد زیر این موضوع را نشان می‌دهد:

```
int<priority_queue> q;
(3);push.q
(5);push.q
(7);push.q
(2);push.q
7 // ;"n\" << ();top.q << cout
();pop.q
5 // ;"n\" << ();top.q << cout
```

```

        ();pop.q
        (6);push.q
        6 // ;"n\" << ()top.q << cout
        ();pop.q

```

اگر بخواهیم یک صف اولویت‌دار ایجاد کنیم که از یافتن و حذف کوچک‌ترین عنصر پشتیبانی کند، می‌توانیم به صورت زیر عمل کنیم:

```

; q >> int<greater, >int<vector, int<priority_queue

```

ساخت‌مان داده‌ای مبتنی بر سی‌است

کامپایلر g++ از برخی ساختارهای داده که بخشی از کتابخانه استاندارد ++C نیستند نیز پشتیبانی می‌کند. این ساختارها مبتنی بر خط‌مشی (policy-based) نامیده می‌شوند. برای استفاده از این ساختارها، خطوط زیر باید به کد اضافه شوند:

```

>hpp.assoc_container/pb_ds/ext< include#
;__gnu_pbds namespace using

```

مثل ولی است set مانند که کرد تعریف را indexed_set داده ساختار می‌توان سپس int: مقادیر برای تعریف است. اندیس‌پذیر آرایه

```

,rb_tree_tag,>int<less,null_type,int<tree typedef
;indexed_set >tree_order_statistics_node_update

```

مجموعه: ساخت

```

; s indexed_set
(2);insert.s
(3);insert.s
(7);insert.s
(9);insert.s

```

تابع می‌داشتند. مرتب‌شده آرایه در عناصر که اندیس‌هایی به دسترسی مجموعه: این ویژگی find_by_order تکرارکننده (iterator) در عنصر به برمی‌گرداند: داده‌شده موقعیت

```

(2);find_by_order.s = x auto
7 // ;"n\" << x* << cout

```

برمی‌گرداند: را داده‌شده عنصر موقعیت order_of_key تابع

```

2 // ;"n\" << (7)order_of_key.s << cout

```

برمی‌گردد: می‌داشت حضور صورت در عنصر که موقعیتی نباشد، مجموعه در عنصر اگر

```

2 // ;"n\" << (6)order_of_key.s << cout
3 // ;"n\" << (8)order_of_key.s << cout

```

می‌شوند. اجرا لگاریتمی زمان در تابع دو هر

۶.۴ مقایسه با مرتب‌سازی

حل مسئله با استفاده از ساختار داده‌ها یا مرتب‌سازی معمولاً ممکن است. گاهی تفاوت‌های چشمگیری در کارایی واقعی این روش‌ها وجود دارد که در پیچیدگی زمانی آن‌ها پنهان است. مسئله: دو لیست A و B با n عنصر داده شده است. هدف: شمارش تعداد عناصری که در هر دو لیست حضور دارند. مثال برای لیست‌های

$$A = [5, 2, 8, 9] \quad B = [3, 2, 9, 5]$$

پاسخ ۳ است زیرا اعداد ۲، ۵ و ۹ در هر دو لیست هستند. راه حل مستقیم: بررسی تمام جفت‌های عناصر در زمان $O(n^2)$. الگوریتم‌های کارآمدتر:

الگوریتم ۱

ساخت مجموعه از عناصر A ، سپس پیمایش عناصر B و بررسی حضور هر عنصر در A . کارآمد است چون عناصر A داخل یک مجموعه‌اند. با ساختار set، پیچیدگی زمانی الگوریتم $O(n \log n)$ است.

الگوریتم ۲

حفظ ترتیب عناصر لازم نیست، پس به جای set می‌توان از unordered_set استفاده کرد. روش ساده برای افزایش کارایی چون فقط ساختار داده تغییر می‌کند. پیچیدگی زمانی الگوریتم جدید $O(n)$ است.

الگوریتم ۳

به جای ساختمان داده‌ها، می‌توان از مرتب‌سازی استفاده کرد. ابتدا، هر دو لیست A و B را مرتب می‌کنیم. سپس، هم‌زمان روی هر دو لیست پیمایش کرده و عناصر مشترک را پیدا می‌کنیم. پیچیدگی زمانی مرتب‌سازی $O(n \log n)$ است و باقی الگوریتم در زمان $O(n)$ اجرا می‌شود، بنابراین پیچیدگی زمانی کل $O(n \log n)$ است.

مقایسه کارایی

جدول زیر میزان کارایی الگوریتم‌های فوق را به ازای مقادیر مختلف n نشان می‌دهد، در حالی که عناصر لیست‌ها اعداد صحیح تصادفی بین $1 \dots 10^9$ هستند:

n	الگوریتم ۱	الگوریتم ۲	الگوریتم ۳
10^6	1.5 ثانیه	0.3 ثانیه	0.2 ثانیه
$2 \cdot 10^6$	3.7 ثانیه	0.8 ثانیه	0.3 ثانیه
$3 \cdot 10^6$	5.7 ثانیه	1.3 ثانیه	0.5 ثانیه
$4 \cdot 10^6$	7.7 ثانیه	1.7 ثانیه	0.7 ثانیه
$5 \cdot 10^6$	10.0 ثانیه	2.3 ثانیه	0.9 ثانیه

الگوریتم‌های ۱ و ۲ یکسان هستند به جز اینکه از ساختارهای مجموعه متفاوتی استفاده می‌کنند. در این مسئله، این انتخاب تاثیر مهمی روی زمان اجرا دارد، زیرا الگوریتم ۲ بین ۴ تا ۵ برابر سریع‌تر از الگوریتم ۱ است.

با این حال، کارآمدترین الگوریتم، الگوریتم ۳ است که از مرتب‌سازی استفاده می‌کند. این الگوریتم تنها نیمی از زمان الگوریتم ۲ را مصرف می‌کند. نکته جالب این است که پیچیدگی زمانی هر دو الگوریتم ۱ و الگوریتم ۳ برابر با $O(n \log n)$ است، اما با این وجود، الگوریتم ۳ ده برابر سریع‌تر است. دلیل این موضوع ساده بودن فرآیند مرتب‌سازی است که تنها یک بار در ابتدای الگوریتم ۳ انجام می‌شود و باقی الگوریتم در زمان خطی کار می‌کند. در مقابل، الگوریتم ۱ یک درخت دودویی متوازن پیچیده را در تمام طول اجرای الگوریتم نگهداری می‌کند.

فصل ۵

جستجوی کامل

جستجوی کامل روشی کلی برای حل تقریباً هر مسئله الگوریتمی است. ایده اصلی، تولید تمام پاسخ‌های ممکن برای مسئله با استفاده از روش نیروی بی‌رحم (brute force)، (force و سپس انتخاب بهترین پاسخ یا شمارش تعداد پاسخ‌ها، بسته به صورت مسئله است. اگر زمان کافی برای بررسی همه پاسخ‌ها وجود داشته باشد، جستجوی کامل تکنیک مناسبی است؛ زیرا پیاده‌سازی این جستجو معمولاً آسان است و همیشه پاسخ درست می‌دهد. اگر جستجوی کامل بسیار کند باشد، تکنیک‌های دیگر نظیر الگوریتم‌های حریصانه یا برنامه‌نویسی پویا ممکن است مورد نیاز باشند.

۱.۵ تولید زیرمجموعه‌ها

ابتدا مسئله تولید تمام زیرمجموعه‌های یک مجموعه n عضوی را بررسی می‌کنیم. به عنوان مثال، زیرمجموعه‌های $\{0, 1, 2\}$ عبارتند از: $\emptyset$ ، $\{0\}$ ، $\{1\}$ ، $\{2\}$ ، $\{0, 1\}$ ، $\{0, 2\}$ ، $\{1, 2\}$ و $\{0, 1, 2\}$. دو روش رایج برای تولید زیرمجموعه‌ها وجود دارد: می‌توان از یک جستجوی بازگشتی استفاده کرد یا از نمایش بیتی اعداد صحیح بهره برد.

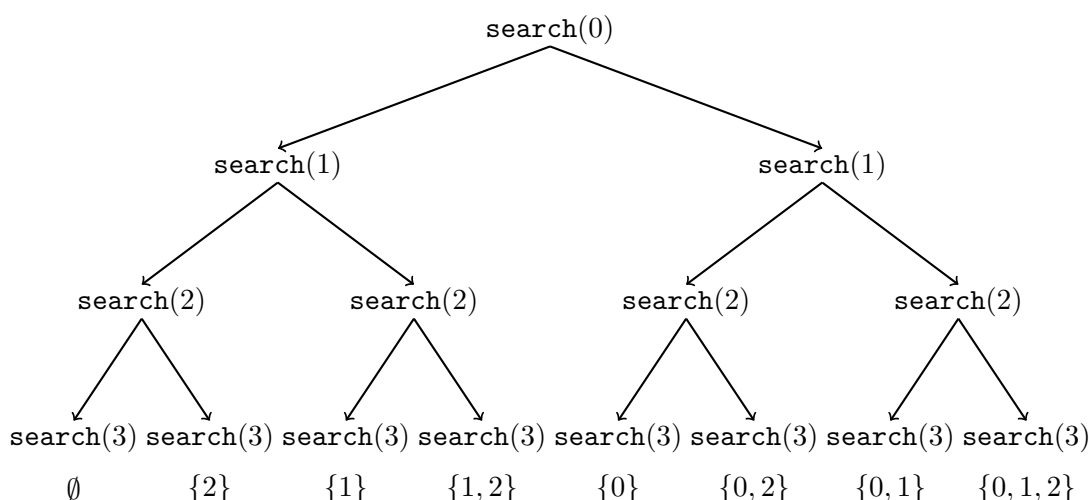
روش ۱

یک روش هوشمندانه برای پیمایش تمام زیرمجموعه‌های یک مجموعه، استفاده از بازگشت است. تابع زیر با نام search زیرمجموعه‌های مجموعه $\{0, 1, \dots, n-1\}$ را تولید می‌کند. این تابع یک بردار به نام subset را نگه می‌دارد که شامل عناصر هر زیرمجموعه خواهد بود. جستجو زمانی آغاز می‌شود که تابع با پارامتر فراخوانی شود.

```
void search(int k) {
    if (n == k) {
        subset process //
    } else {
        k(search + 1);
        subset.push_back(k);
        k(search + 1);
        subset.pop_back();
    }
}
```

هنگامی که تابع `search` با پارامتر k فراخوانی می‌شود، تصمیم می‌گیرد که آیا عنصر k را در زیرمجموعه قرار دهد یا خیر، و در هر دو حالت، سپس خود را با پارامتر $k + 1$ فراخوانی می‌کند. با این حال، اگر $k = n$ باشد، تابع متوجه می‌شود که همه عناصر پردازش شده‌اند و یک زیرمجموعه تولید شده است.

درخت زیر فراخوانی‌های تابع را در حالتی که $n = 3$ است نشان می‌دهد. ما همواره می‌توانیم شاخهٔ چپ (عنصر k در زیرمجموعه قرار نمی‌گیرد) یا شاخهٔ راست (عنصر k در زیرمجموعه قرار می‌گیرد) را انتخاب کنیم.



روش ۲

روش دیگر تولید زیرمجموعه‌ها بر پایه نمایش بیتی اعداد صحیح است. هر زیرمجموعه از یک مجموعه n عضوی را می‌توان به شکل دنباله‌ای از n بیت نشان داد، که متناظر با یک عدد صحیح بین $0 \dots 2^n - 1$ است. یک‌ها در دنباله بیتی مشخص می‌کنند کدام عضوها در زیرمجموعه حضور دارند.

قرارداد معمول این است که بیت آخر متناظر با عضو ۰، بیت ماقبل آخر متناظر با عضو ۱، و به همین ترتیب باشد. برای نمونه، نمایش بیتی عدد ۲۵ برابر ۱۱۰۰۱ است که با زیرمجموعه $\{0, 3, 4\}$ تناظر دارد.

کد زیر تمام زیرمجموعه‌های یک مجموعه n عضوی را پیمایش می‌کند:

```

for (int b = 0; b < (1 << n); ++b) {
    //
}
  
```

کد زیر نشان می‌دهد چگونه می‌توان اعضای زیرمجموعه متناظر با یک دنباله بیتی را به دست آورد. هنگام پردازش هر زیرمجموعه، کد یک بردار شامل اعضای موجود در زیرمجموعه می‌سازد.

```

for (int b = 0; b < (1 << n); ++b) {
    vector<int> subset;
    for (int i = 0; i < n; ++i) {
  
```

```

        ); i(push_back.subset )) i<< (1b( if
    }
}

```

۲.۵ تولید جایگشت‌ها

در ادامه مسئله تولید تمام جایگشت‌های یک مجموعه n عضوی بررسی می‌شود. برای نمونه، جایگشت‌های $\{0, 1, 2\}$ عبارتند از $(0, 1, 2)$ ، $(0, 2, 1)$ ، $(1, 0, 2)$ ، $(1, 2, 0)$ ، $(2, 0, 1)$ و $(2, 1, 0)$. اینجا نیز دو رویکرد وجود دارد: می‌توان از بازگشت بهره برد یا جایگشت‌ها را به روش تکراری پیمایش کرد.

روش ۱

همانند زیرمجموعه‌ها، جایگشت‌ها را نیز می‌توان با بازگشت تولید کرد. تابع `search` زیر جایگشت‌های مجموعه $\{0, 1, \dots, n-1\}$ را پیمایش می‌کند. تابع بردار `permutation` حاوی جایگشت را می‌سازد، و جستجو با فراخوانی تابع بدون آرگومان آغاز می‌شود.

```

void search() {
    if (n == permutation.size()) {
        permutation process //
    } else {
        for (int i = 0; i < n; ++i) {
            if (i[chosen] == true) continue;
            permutation[i] = i; search();
            permutation[i] = i[chosen];
        }
    }
}

```

هر فراخوانی تابع یک عنصر جدید به `permutation` اضافه می‌کند. آرایه `chosen` مشخص می‌کند کدام عناصر قبلاً در جایگشت قرار گرفته‌اند. اگر اندازه `permutation` برابر با اندازه مجموعه شود، یک جایگشت تولید شده است.

روش ۲

روش دیگر برای تولید جایگشت‌ها، شروع با جایگشت $\{0, 1, \dots, n-1\}$ و استفاده متوالی از تابعی است که جایگشت بعدی را به ترتیب صعودی می‌سازد. کتابخانه استاندارد C++ شامل تابع `next_permutation` است که می‌تواند برای این کار استفاده شود:

```

vector<int> permutation;
for (int i = 0; i < n; ++i)

```

```

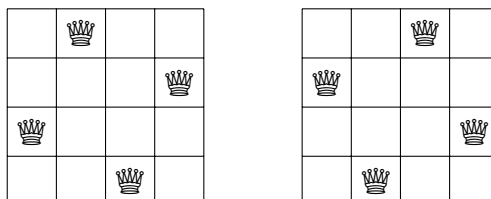
);i(push_back.permutation
}
{ do
permutation process //
while }
end.permutation,()begin.permutation(next_permutation(
)))

```

۳.۵ عقب‌گرد

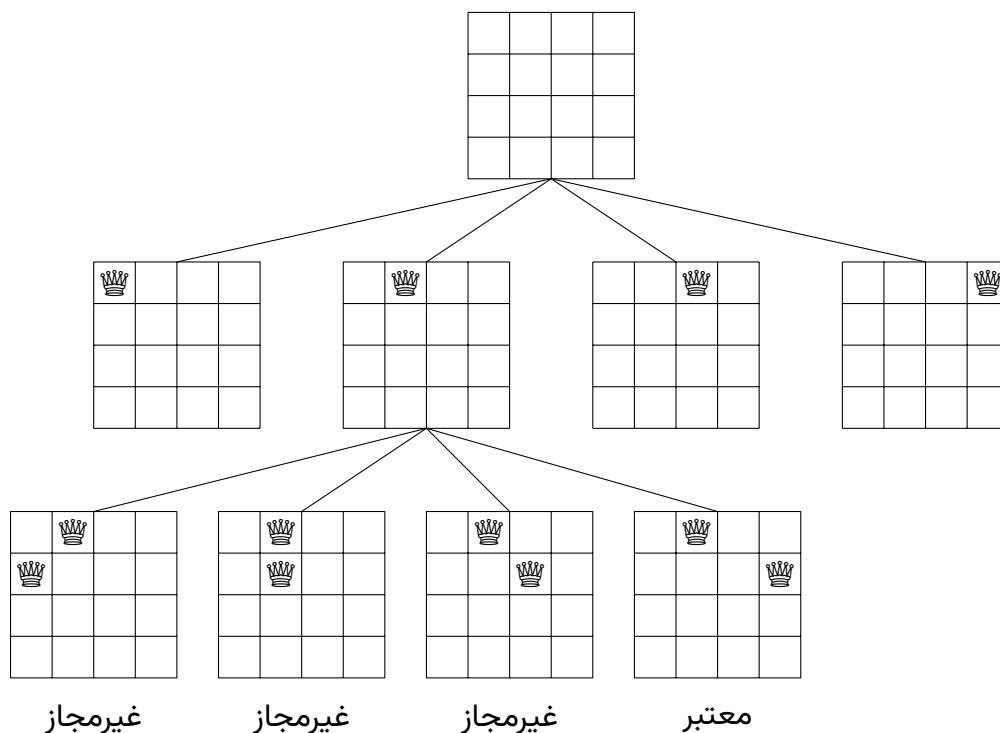
یک الگوریتم **عقب‌گرد** (backtracking) با یک جواب خالی شروع می‌شود و جواب را گام به گام گسترش می‌دهد. جستجو به صورت بازگشتی تمام روش‌های مختلف ساخت یک جواب را بررسی می‌کند.

به عنوان مثال، مسئله محاسبه تعداد روش‌های قرار دادن n وزیر در یک صفحه شطرنج $n \times n$ را در نظر بگیرید، به طوری که هیچ دو وزیری یکدیگر را تهدید نکنند. برای نمونه، زمانی که $n = 4$ است، دو جواب ممکن وجود دارد:



این مسئله را می‌توان با عقب‌گرد و قرار دادن سطر به سطر وزیرها در صفحه حل کرد. به طور دقیق‌تر، دقیقاً یک وزیر در هر سطر قرار می‌گیرد، به طوری که هیچ وزیری به وزیرهای قرار داده شده قبلی حمله نکند. هنگامی که همه n وزیر در صفحه قرار گرفتند، یک جواب پیدا شده است.

برای نمونه، وقتی $n = 4$ است، برخی از جواب‌های جزئی تولید شده توسط الگوریتم عقب‌گرد به صورت زیر هستند:



در پایین‌ترین سطح، سه پیکربندی اول غیرمجاز هستند، زیرا وزیرها به یکدیگر حمله می‌کنند. با این حال، پیکربندی چهارم معتبر است و می‌توان آن را با قرار دادن دو وزیر دیگر در صفحه به یک پاسخ کامل گسترش داد. تنها یک راه برای قرار دادن دو وزیر باقی‌مانده وجود دارد.

الگوریتم را می‌توان به صورت زیر پیاده‌سازی کرد:

```
void search(int y) {
    if (y == n) {
        ++count;
        return;
    }
    for (int x = 0; x < n; ++x) {
        if (!is_valid(x, y)) continue;
        column[x] = y;
        diag1[y + x] = 1;
        diag2[y - x + n] = 1;
        search(y + 1);
    }
}
```

تعداد کد و است $n \times n$ صفحه اندازه می‌شود. آغاز $\text{search}(0)$ فراخوانی با جستجو می‌کند. محاسبه count متغیر در را پاسخ‌ها کد فرض می‌کند که سطرها و ستون‌های صفحه از ۰ تا $n - 1$ شماره‌گذاری شده‌اند. هنگامی که تابع search با آرگومان y فراخوانی می‌شود، وزیری را در سطر y قرار می‌دهد و سپس خود را با آرگومان $y + 1$ فراخوانی می‌کند. سپس، اگر $y = n$ باشد، یک پاسخ یافت شده است و متغیر count یک واحد افزایش می‌یابد.

آرایه column ستون‌هایی را که شامل وزیر هستند پیگیری می‌کند، و آرایه‌های diag1 و diag2 قطرها را دنبال می‌کنند. اضافه کردن وزیری دیگر به ستون یا قطری که از قبل

شامل وزیر است، مجاز نیست. برای مثال، ستون‌ها و قطرهای صفحه 4×4 به صورت زیر شماره‌گذاری می‌شوند:

0	1	2	3
0	1	2	3
0	1	2	3
0	1	2	3

column

0	1	2	3
1	2	3	4
2	3	4	5
3	4	5	6

diag1

3	4	5	6
2	3	4	5
1	2	3	4
0	1	2	3

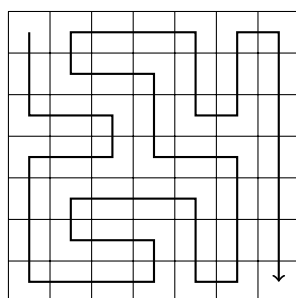
diag2

فرض کنید $q(n)$ نشان‌دهنده تعداد روش‌های قرار دادن n وزیر در یک صفحه شطرنج $n \times n$ باشد. الگوریتم عقب‌گرد بالا نشان می‌دهد که به عنوان مثال، $q(8) = 92$ است. با افزایش n ، جستجو به سرعت کند می‌شود، زیرا تعداد جواب‌ها به صورت نمایی رشد می‌کند. برای نمونه، محاسبه $q(16) = 14772512$ با الگوریتم بالا روی یک رایانه امروزی حدود یک دقیقه زمان می‌برد^۱.

۴.۵ هرس کردن جستجو

اغلب می‌توان عقب‌گرد را با هرس کردن درخت جستجو بهینه‌سازی کرد. ایده اصلی، افزودن «هوشمندی» به الگوریتم است تا در سریع‌ترین زمان ممکن تشخیص دهد که آیا یک جواب جزئی قابلیت تبدیل به یک جواب کامل را ندارد یا خیر. چنین بهینه‌سازی‌هایی می‌توانند تأثیر چشمگیری بر کارایی جستجو داشته باشند.

مسئله شمارش تعداد مسیرها در یک شبکه $n \times n$ از گوشه بالا-چپ به گوشه پایین-راست را در نظر بگیرید، به طوری که مسیر دقیقاً یک بار از هر خانه عبور کند. به عنوان مثال، در یک شبکه 7×7 ، تعداد ۱۱۱۷۱۲ مسیر از این نوع وجود دارد. یکی از این مسیرها به صورت زیر است:



تمرکز ما روی حالت 7×7 است، زیرا سطح دشواری آن متناسب با نیاز ماست. کار را با الگوریتم عقب‌گرد ساده آغاز می‌کنیم و سپس با مشاهداتی درباره نحوه هرس جستجو، آن را گام به گام بهینه‌سازی می‌کنیم. پس از هر بهینه‌سازی، زمان اجرای الگوریتم و تعداد فراخوانی‌های بازگشتی را اندازه می‌گیریم تا تأثیر هر بهینه‌سازی بر کارایی جستجو مشخص شود.

^۱روش کارآمد شناخته‌شده‌ای برای محاسبه مقادیر بزرگتر $q(n)$ وجود ندارد. رکورد فعلی $q(27) = 234907967154122528$ است که در سال ۲۰۱۶ محاسبه شد [۵۵].

الگوریتم پایه

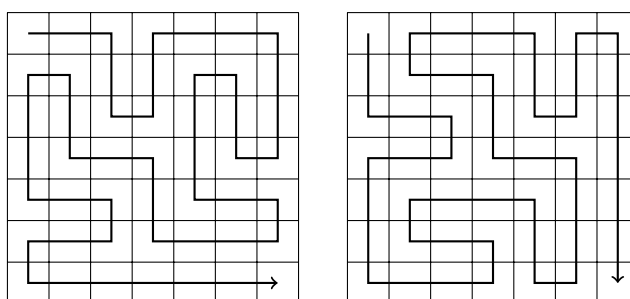
نسخه نخست الگوریتم فاقد هرگونه بهینه‌سازی است. صرفاً با عقب‌گرد، تمام مسیرهای ممکن از گوشه بالا-چپ به گوشه پایین-راست تولید و شمارش می‌شوند.

• زمان اجرا: ۴۸۳ ثانیه

• تعداد فراخوانی‌های بازگشتی: ۷۶ میلیارد

بهینه‌سازی ۱

در هر پاسخ، گام نخست حرکت به پایین یا راست است. همواره دو مسیر وجود دارند که نسبت به قطر شبکه پس از گام اول متقارن هستند. برای نمونه، مسیرهای زیر متقارنند:



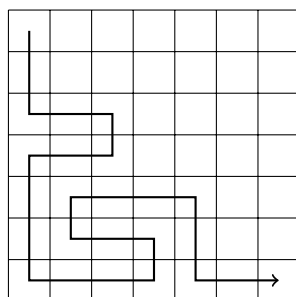
بنابراین، می‌توان شرط کرد که گام اول همیشه به سمت پایین (یا راست) باشد و در پایان، تعداد پاسخ‌ها دو برابر شود.

• زمان اجرا: ۲۴۴ ثانیه

• تعداد فراخوانی‌های بازگشتی: ۳۸ میلیارد

بهینه‌سازی ۲

اگر مسیر پیش از پیمایش تمام خانه‌های دیگر شبکه به خانه پایین-راست برسد، واضح است که تکمیل پاسخ ناممکن خواهد بود. نمونه‌ای از این حالت مسیر زیر است:



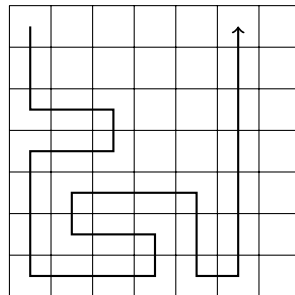
با تکیه بر این نکته، در صورت رسیدن زودهنگام به خانه پایین-راست، جستجو را فوراً متوقف می‌کنیم.

• زمان اجرا: ۱۱۹ ثانیه

• تعداد فراخوانی‌های بازگشتی: ۲۰ میلیارد

بهبود سه‌سازی ۳

اگر مسیر به دیوار برخورد کند و بتواند به چپ یا راست بپیچد، شبکه به دو بخش مجزا با خانه‌های دیده‌نشده تقسیم می‌شود. برای مثال، در وضعیت زیر مسیر می‌تواند به چپ یا راست برود:

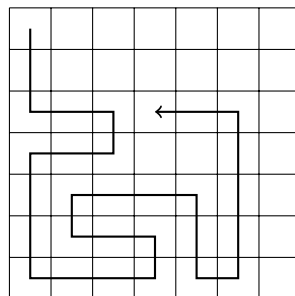


در این حالت، پیمایش تمام خانه‌ها دیگر امکان‌پذیر نیست؛ پس جستجو متوقف می‌شود. این بهینه‌سازی بسیار کارآمد است:

- زمان اجرا: ۸.۱ ثانیه
- تعداد فراخوانی‌های بازگشتی: ۲۲۱ میلیون

بهبود سه‌سازی ۴

ایده بهینه‌سازی ۳ را می‌توان تعمیم داد: اگر مسیر امکان حرکت رو به جلو نداشته باشد اما بتواند به چپ یا راست گردش کند، شبکه به دو بخش شامل خانه‌های دیده‌نشده تقسیم خواهد شد. برای مثال، مسیر زیر را ببینید:



روشن است که دیگر نمی‌توان همه خانه‌ها را پیمایش کرد؛ بنابراین جستجو قطع می‌شود. پس از این بهینه‌سازی، جستجو بسیار سریع عمل می‌کند:

- زمان اجرا: ۶.۰ ثانیه
- تعداد فراخوانی‌های بازگشتی: ۶۹ میلیون

اکنون زمان مناسبی است تا بهینه‌سازی الگوریتم متوقف و نتایج بررسی شود. زمان اجرای الگوریتم اولیه ۴۸۳ ثانیه بود؛ حال با اعمال بهینه‌سازی‌ها، زمان اجرا به ۶.۰ ثانیه رسیده است. بنابراین الگوریتم نزدیک به ۱۰۰۰ برابر سریع‌تر شد.

این پدیده‌ای رایج در عقب‌گرد است، زیرا درخت جستجو معمولاً بزرگ است و حتی مشاهدات ساده نیز می‌توانند به شکلی مؤثر جستجو را هرس کنند. به‌ویژه بهینه‌سازی‌هایی که در گام‌های اولیه الگوریتم رخ می‌دهند، یعنی در بالای درخت جستجو، بسیار مفید هستند.

۵.۵ ملاقات در میانه (Meet the middle)

ملاقات در میانه تکنیکی است که در آن فضای جستجو به دو بخش با اندازه‌های تقریباً برابر تقسیم می‌شود. یک جستجوی جداگانه برای هر دو بخش انجام می‌گیرد، و در نهایت نتایج جستجوها با یکدیگر ترکیب می‌شوند.

از این تکنیک می‌توان استفاده کرد اگر روشی کارآمد برای ترکیب نتایج جستجوها وجود داشته باشد. در چنین حالتی، این دو جستجو ممکن است زمان کمتری نسبت به یک جستجوی بزرگ نیاز داشته باشند. به‌طور معمول، با استفاده از تکنیک ملاقات در میانه می‌توان ضریب 2^n را به ضریب $2^{n/2}$ تبدیل کرد.

به‌عنوان مثال، مسئله‌ای را در نظر بگیرید که در آن لیستی از n عدد و یک عدد x داده شده است، و می‌خواهیم بررسی کنیم آیا می‌توان چند عدد از لیست را طوری انتخاب کرد که مجموع آن‌ها برابر x شود. برای نمونه، با فرض لیست $[2, 4, 5, 9]$ و $x = 15$ ، می‌توان اعداد $[2, 4, 9]$ را انتخاب کرد تا $2 + 4 + 9 = 15$ به دست آید. با این حال، اگر برای همین لیست $x = 10$ باشد، تشکیل چنین مجموعی امکان‌پذیر نیست.

یک الگوریتم ساده برای این مسئله آن است که تمام زیرمجموعه‌های عناصر را بررسی کنیم و ببینیم آیا مجموع هر یک از زیرمجموعه‌ها برابر x است یا خیر. زمان اجرای چنین الگوریتمی $O(2^n)$ است، زیرا 2^n زیرمجموعه وجود دارد. اما با استفاده از تکنیک ملاقات در میانه، می‌توان به الگوریتم کارآمدتری با زمان $O(2^{n/2})$ دست یافت^۲. توجه داشته باشید که $O(2^n)$ و $O(2^{n/2})$ پیچیدگی‌های متفاوتی هستند، زیرا $2^{n/2}$ برابر با $\sqrt{2^n}$ است.

ایده کار تقسیم لیست به دو لیست A و B است، به طوری که هر دو لیست تقریباً نیمی از اعداد را در بر داشته باشند. جستجوی اول تمام زیرمجموعه‌های A را تولید کرده و مجموع آن‌ها را در لیستی به نام S_A ذخیره می‌کند. به همین ترتیب، جستجوی دوم لیست S_B را از B ایجاد می‌کند. پس از این، کافی است بررسی شود که آیا می‌توان یک عنصر از S_A و عنصر دیگری از S_B انتخاب کرد به طوری که مجموع آن‌ها برابر x شود. این حالت دقیقاً زمانی ممکن است که راهی برای تشکیل مجموع x با استفاده از اعداد لیست اصلی وجود داشته باشد.

برای مثال، فرض کنید لیست $[2, 4, 5, 9]$ و $x = 15$ باشد. ابتدا لیست را به $A = [2, 4]$ و $B = [5, 9]$ تقسیم می‌کنیم. سپس، لیست‌های $S_A = [0, 2, 4, 6]$ و $S_B = [0, 5, 9, 14]$ را می‌سازیم. در این حالت، تشکیل مجموع $x = 15$ امکان‌پذیر است، زیرا S_A شامل مجموع 6 است، S_B شامل مجموع 9 است، و $6 + 9 = 15$. این متناظر با پاسخ $[2, 4, 9]$ است. می‌توان الگوریتم را طوری پیاده‌سازی کرد که پیچیدگی زمانی آن $O(2^{n/2})$ باشد. ابتدا، لیست‌های مرتب‌شده S_A و S_B را تولید می‌کنیم، که با استفاده از تکنیکی شبیه به ادغام در زمان $O(2^{n/2})$ قابل انجام است. پس از این، با توجه به مرتب بودن لیست‌ها، می‌توان در زمان $O(2^{n/2})$ بررسی کرد که آیا مجموع x می‌تواند از S_A و S_B ساخته شود یا خیر.

^۲ این ایده در سال ۱۹۷۴ توسط E. Horowitz و S. Sahni معرفی شد [۳۹].

فصل ۶

الگوریتم‌های حریصانه

یک الگوریتم حریصانه با انجام انتخابی که در لحظه بهترین به نظر می‌رسد، راه‌حلی برای مسئله می‌سازد. الگوریتم حریصانه هرگز از انتخاب‌های خود عقب‌نشینی نمی‌کند، بلکه مستقیماً راه‌حل نهایی را می‌سازد. به همین دلیل، الگوریتم‌های حریصانه معمولاً بسیار کارآمد هستند.

دشواری در طراحی الگوریتم‌های حریصانه، یافتن راهبردی حریصانه است که همیشه راه‌حل بهینه مسئله را تولید کند. انتخاب‌های بهینه محلی در یک الگوریتم حریصانه باید از نظر سراسری نیز بهینه باشند. اثبات درستی عملکرد یک الگوریتم حریصانه اغلب دشوار است.

۱.۶ مسئله سکه

به عنوان نخستین مثال، مسئله‌ای را بررسی می‌کنیم که در آن مجموعه‌ای از سکه‌ها داده شده است و هدف، ساختن مبلغ n با استفاده از این سکه‌ها است. ارزش سکه‌ها برابر با $\text{coins} = \{c_1, c_2, \dots, c_k\}$ است و از هر سکه می‌توان به تعداد دلخواه استفاده کرد. حداقل تعداد سکه‌های مورد نیاز چقدر است؟ برای نمونه، اگر سکه‌ها سکه‌های یورو (به سنت) باشند:

$$\{1, 2, 5, 10, 20, 50, 100, 200\}$$

و $n = 520$ باشد، حداقل به چهار سکه نیاز داریم. راه‌حل بهینه، انتخاب سکه‌های $200 + 100 + 20$ است که مجموع آن‌ها ۵۲۰ می‌شود.

الگوریتم حریصانه

یک الگوریتم حریصانه ساده برای این مسئله، همواره بزرگ‌ترین سکه ممکن را برمی‌گزیند تا زمانی که مبلغ مورد نظر ساخته شود. این الگوریتم در مثال بالا کار می‌کند، زیرا ابتدا دو سکه ۲۰۰ سنتی، سپس یک سکه ۱۰۰ سنتی و در پایان یک سکه ۲۰ سنتی انتخاب می‌کنیم. اما آیا این الگوریتم همواره کار می‌کند؟

مشخص می‌شود که اگر سکه‌ها از نوع یورو باشند، الگوریتم حریصانه همیشه کار می‌کند؛ یعنی همواره راه‌حلی با کمترین تعداد سکه ممکن به دست می‌دهد. درستی این الگوریتم را می‌توان به صورت زیر نشان داد:

نخست، هر یک از سکه‌های ۱، ۵، ۱۰، ۵۰ و ۱۰۰ حداکثر یک بار در یک راه‌حل بهینه ظاهر می‌شوند، زیرا اگر راه‌حل شامل دو عدد از این سکه‌ها باشد، می‌توان آن‌ها را با یک سکه

جایگزین کرد و راه حل بهتری به دست آورد. برای نمونه، اگر راه حل شامل سکه های $5 + 5$ باشد، می توان آن ها را با سکه 10 جایگزین کرد.

به همین ترتیب، سکه های ۲ و ۲۰ حداکثر دو بار در یک راه حل بهینه ظاهر می شوند، زیرا می توان سکه های $2 + 2 + 2$ را با سکه های $5 + 1$ و سکه های $20 + 20 + 20$ را با سکه های $50 + 10$ جایگزین کرد. علاوه بر این، یک راه حل بهینه نمی تواند شامل سکه های $2 + 2 + 1$ یا $20 + 20 + 10$ باشد، زیرا می توان آن ها را با سکه های 5 و 50 جایگزین کرد.

با استفاده از این مشاهدات، می توان برای هر سکه x نشان داد که ساختن بهینه مجموع x یا هر مبلغ بزرگ تر از آن، تنها با استفاده از سکه های کوچک تر از x غیرممکن است. برای نمونه، اگر $x = 100$ باشد، بزرگ ترین مجموع بهینه با استفاده از سکه های کوچک تر برابر است با: $99 = 50 + 20 + 20 + 5 + 2 + 2$. بنابراین، الگوریتم حریصانه ای که همیشه بزرگ ترین سکه را انتخاب می کند، راه حل بهینه را به دست می دهد.

این مثال نشان می دهد که اثبات درستی یک الگوریتم حریصانه می تواند دشوار باشد، حتی اگر خود الگوریتم ساده باشد.

حالات کلی

در حالت کلی، مجموعه سکه ها می تواند شامل هر مقداری باشد و الگوریتم حریصانه لزوماً راه حل بهینه را تولید نمی کند.

می توان اثبات کرد که یک الگوریتم حریصانه کار نمی کند با نشان دادن یک مثال نقض که در آن الگوریتم پاسخ نادرست می دهد. در این مسئله می توان به سادگی یک مثال نقض پیدا کرد: اگر سکه ها $\{1, 3, 4\}$ باشند و مجموع هدف برابر ۶ باشد، الگوریتم حریصانه پاسخ $1 + 1 + 4$ را تولید می کند در حالی که پاسخ بهینه $3 + 3$ است.

مشخص نیست که آیا مسئله سکه عمومی با هیچ الگوریتم حریصانه ای قابل حل است یا خیر^۱. اما همان طور که در فصل ۷ خواهیم دید، در برخی موارد، مسئله عمومی را می توان به طور کارآمد با استفاده از یک الگوریتم برنامه نویسی پویا که همیشه پاسخ درست می دهد حل کرد.

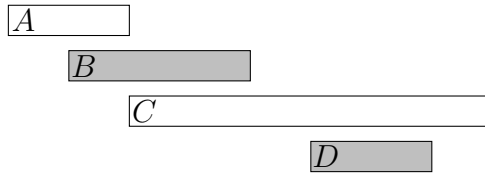
۲.۶ زمان بندی

بسیاری از مسائل زمان بندی را می توان با استفاده از الگوریتم های حریصانه حل کرد. یک مسئله کلاسیک به شرح زیر است: با فرض n رویداد همراه با زمان های شروع و پایان آن ها، برنامه ای پیدا کنید که شامل بیشترین تعداد رویداد ممکن باشد. انتخاب بخشی از یک رویداد امکان پذیر نیست. برای مثال، رویدادهای زیر را در نظر بگیرید:

رویداد	زمان شروع	زمان پایان
A	۱	۳
B	۲	۵
C	۳	۹
D	۶	۸

در این حالت بیشترین تعداد رویدادها دو است. برای مثال، می توانیم رویدادهای B و D را به صورت زیر انتخاب کنیم:

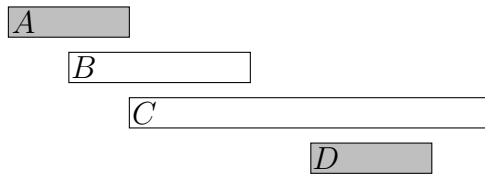
^۱با این حال، ممکن است در زمان چندجمله ای بررسی کرد که آیا الگوریتم حریصانه ارائه شده در این فصل برای مجموعه مشخصی از سکه ها کار می کند یا خیر [۵۳].



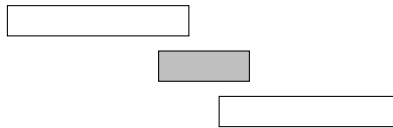
می‌توان چندین الگوریتم حریصانه برای مسئله ابداع کرد، اما کدام یک در هر حالتی درست کار می‌کند؟

الگوریتم ۱

ایده اول انتخاب رویدادهایی است که تا حد امکان کوتاه باشند. در مثال نمونه، این الگوریتم رویدادهای زیر را انتخاب می‌کند:



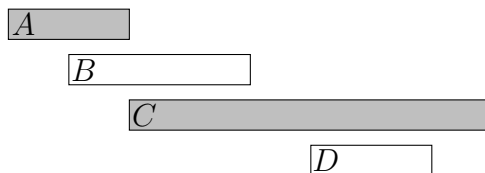
اما انتخاب رویدادهای کوتاه همیشه استراتژی درستی نیست. برای مثال، الگوریتم در حالت زیر شکست می‌خورد:



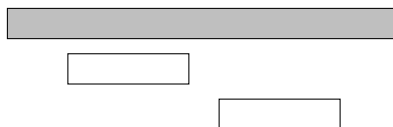
اگر رویداد کوتاه را انتخاب کنیم، فقط می‌توانیم یک رویداد را انتخاب کنیم. با این حال، امکان انتخاب هر دو رویداد بلند وجود داشت.

الگوریتم ۲

ایده دیگر این است که همیشه رویداد ممکن بعدی را انتخاب کنیم که تا حد امکان زودتر شروع می‌شود. این الگوریتم رویدادهای زیر را انتخاب می‌کند:



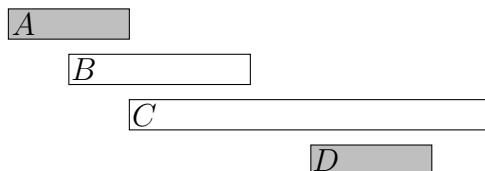
با این حال، می‌توانیم برای این الگوریتم نیز یک مثال نقض پیدا کنیم. به عنوان مثال، در حالت زیر، الگوریتم فقط یک رویداد را انتخاب می‌کند:



اگر رویداد اول را انتخاب کنیم، انتخاب هیچ رویداد دیگری ممکن نخواهد بود. در حالی که انتخاب دو رویداد دیگر ممکن بود.

الگوریتم ۳

ایده سوم این است که همیشه رویداد ممکن بعدی را انتخاب کنیم که تا حد امکان زودتر پایان می‌یابد. این الگوریتم رویدادهای زیر را انتخاب می‌کند:



مشخص می‌شود که این الگوریتم همیشه یک پاسخ بهینه تولید می‌کند. دلیل این امر آن است که انتخاب رویدادی که در زودترین زمان ممکن پایان می‌یابد، در گام اول همواره یک انتخاب بهینه است. پس از آن نیز انتخاب رویداد بعدی با استفاده از همین استراتژی انتخابی بهینه خواهد بود، و به همین ترتیب، تا زمانی که دیگر نتوان رویدادی را انتخاب کرد. یک روش برای اثبات درستی الگوریتم این است که بررسی کنیم چه اتفاقی می‌افتد اگر ابتدا رویدادی را انتخاب کنیم که دیرتر از رویداد با زودترین زمان پایان، تمام می‌شود. در این وضعیت، برای انتخاب رویداد بعدی حداکثر به همان اندازه (یا کمتر) انتخاب خواهیم داشت. بنابراین، انتخاب رویدادی که دیرتر تمام می‌شود هرگز نمی‌تواند پاسخ بهتری ایجاد کند، و الگوریتم حریصانه درست است.

۳.۶ وظایف و مهلت‌ها

اکنون مسئله‌ای را در نظر بگیرید که در آن n وظیفه با مدت زمان و مهلت معین داده شده‌اند و هدف ما انتخاب ترتیبی برای انجام این وظایف است. به ازای هر وظیفه، $d - x$ امتیاز کسب می‌کنیم که در آن d مهلت انجام وظیفه و x زمانی است که وظیفه را به پایان می‌رسانیم. بیشترین مجموع امتیاز ممکن که می‌توانیم به دست آوریم چقدر است؟ برای مثال، فرض کنید کارها به صورت زیر باشند:

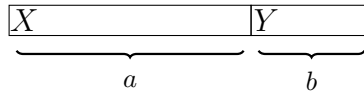
کار	مدت زمان	مهلت زمانی
A	۴	۲
B	۳	۵
C	۲	۷
D	۴	۵

در این حالت، یک زمان‌بندی بهینه برای کارها به صورت زیر است:

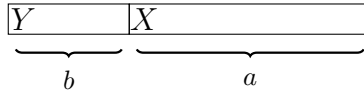


در این راه‌حل، C به میزان ۵ امتیاز، B به میزان ۰ امتیاز، A به میزان ۷- امتیاز و D به میزان ۸- امتیاز به همراه دارند، بنابراین مجموع امتیازات ۱۰- است.

به‌طور شگفت‌آوری، راه‌حل بهینه این مسئله اصلاً به مهلت‌های زمانی بستگی ندارد؛ یک راهبرد حریصانه صحیح آن است که کارها صرفاً بر اساس مدت زمان آن‌ها به صورت صعودی مرتب‌سازی شوند و سپس انجام گیرند. دلیل این موضوع آن است که اگر هر بار دو کار را پشت سر هم انجام دهیم طوری که انجام کار اول طولانی‌تر از کار دوم باشد، با جابه‌جا کردن این دو کار می‌توان به راه‌حل بهتری دست یافت. برای مثال، زمان‌بندی زیر را در نظر بگیرید:



در اینجا $a > b$ است، بنابراین باید کارها را جابه‌جا کنیم:



اکنون X به میزان b امتیاز کمتر و Y به میزان a امتیاز بیشتر کسب می‌کند، در نتیجه امتیاز کل به اندازه $a - b > 0$ افزایش می‌یابد. در یک راه‌حل بهینه، برای هر دو کار متوالی باید این شرط برقرار باشد که کار کوتاه‌تر قبل از کار طولانی‌تر بیاید. بنابراین، کارها باید با مرتب‌سازی بر اساس مدت زمانشان انجام شوند.

۴.۶ کمینه‌سازی مجموع‌ها

در ادامه مسئله‌ای را بررسی می‌کنیم که در آن n عدد $a_1, a_2, \dots, a_n$ داده شده است و هدف یافتن مقداری برای x است که مجموع زیر را کمینه کند:

$$|a_1 - x|^c + |a_2 - x|^c + \dots + |a_n - x|^c.$$

تمرکز ما بر روی حالت‌های $c = 1$ و $c = 2$ است.

حالت $c = 1$

در این حالت، باید مجموع زیر را کمینه کنیم:

$$|a_1 - x| + |a_2 - x| + \dots + |a_n - x|.$$

به عنوان مثال، اگر اعداد $[1, 2, 9, 2, 6]$ باشند، بهترین راه‌حل انتخاب $x = 2$ است که مجموع زیر را تولید می‌کند:

$$|1 - 2| + |2 - 2| + |9 - 2| + |2 - 2| + |6 - 2| = 12.$$

در حالت کلی، بهترین انتخاب برای x میانه اعداد است، یعنی عدد وسط پس از مرتب‌سازی. به عنوان مثال، لیست $[1, 2, 9, 2, 6]$ پس از مرتب‌سازی به $[1, 2, 2, 6, 9]$ تبدیل می‌شود، بنابراین میانه ۲ است.

میانه انتخابی بهینه است، زیرا اگر x کوچک‌تر از میانه باشد، مجموع با افزایش x کمتر می‌شود، و اگر x بزرگ‌تر از میانه باشد، مجموع با کاهش x کمتر می‌شود. از این رو، راه‌حل بهینه این است که x میانه باشد. اگر n زوج باشد و دو میانه وجود داشته باشد، هر دو میانه و تمام مقادیر بین آن‌ها انتخاب‌های بهینه‌ای هستند.

حالت $c = 2$

در این حالت، باید مجموع زیر را کمینه کنیم:

$$(a_1 - x)^2 + (a_2 - x)^2 + \dots + (a_n - x)^2.$$

به عنوان مثال، اگر اعداد $[1, 2, 9, 2, 6]$ باشند، بهترین راه حل انتخاب $x = 4$ است که مجموع زیر را تولید می‌کند:

$$(1 - 4)^2 + (2 - 4)^2 + (9 - 4)^2 + (2 - 4)^2 + (6 - 4)^2 = 46.$$

در حالت کلی، بهترین انتخاب برای x میانگین اعداد است. در این مثال، میانگین برابر است با $4 = (1 + 2 + 9 + 2 + 6)/5$. این نتیجه را می‌توان با نمایش مجموع به شکل زیر به دست آورد:

$$nx^2 - 2x(a_1 + a_2 + \dots + a_n) + (a_1^2 + a_2^2 + \dots + a_n^2)$$

بخش آخر به x وابسته نیست، بنابراین می‌توانیم از آن صرف نظر کنیم. بخش‌های باقیمانده تابع $nx^2 - 2xs$ را تشکیل می‌دهند که در آن $s = a_1 + a_2 + \dots + a_n$ است. این یک سهمی رو به بالا با ریشه‌های $x = 0$ و $x = 2s/n$ است، و مقدار کمینه آن، میانگین ریشه‌ها یعنی $x = s/n$ است؛ یعنی میانگین اعداد $a_1, a_2, \dots, a_n$.

۵.۶ فشرده‌سازی داده‌ها

یک **کد دودویی** برای هر نویسه از یک رشته، یک **کلمه‌کد** متشکل از بیت‌ها تعیین می‌کند. می‌توانیم با جایگزینی هر نویسه با کلمه‌کد متناظر، رشته را فشرده‌سازی کنیم. به عنوان مثال، کد دودویی زیر کلمه‌کدهایی را برای نویسه‌های A تا D تعیین می‌کند:

نویسه	کلمه‌کد
A	۰۰
B	۰۱
C	۱۰
D	۱۱

این یک کد با **طول ثابت** است به این معنی که طول هر کلمه‌کد یکسان است. برای مثال، می‌توانیم رشته‌ی AABACDACA را به صورت زیر فشرده کنیم:

00 00 01 00 10 11 00 10 00

با استفاده از این کد، طول رشته‌ی فشرده‌شده ۱۸ بیت است. اما می‌توانیم رشته را بهتر فشرده کنیم اگر از یک کد با **طول متغیر** استفاده کنیم که در آن کلمه‌کدها می‌توانند طول‌های متفاوتی داشته باشند. آنگاه می‌توانیم کلمه‌کدهای کوتاه را به نویسه‌هایی اختصاص دهیم که تکرار بالایی دارند و کلمه‌کدهای بلند را به نویسه‌هایی که به ندرت ظاهر می‌شوند اختصاص دهیم. مشخص می‌شود که یک کد **بهینه** برای رشته‌ی فوق به صورت زیر است:

نویسه	کلمه‌کد
A	۰
B	۱۱۰
C	۱۰
D	۱۱۱

یک کد بهینه، رشته‌ی فشرده‌شده‌ای با کوتاه‌ترین طول ممکن تولید می‌کند. در این حالت، رشته‌ی فشرده‌شده با استفاده از کد بهینه عبارت است از:

001100101110100,

بنابراین به جای ۱۸ بیت، تنها به ۱۵ بیت نیاز است. بدین ترتیب، به لطف یک کد بهتر، امکان صرفه‌جویی ۳ بیت در رشته‌ی فشرده‌شده فراهم شد.

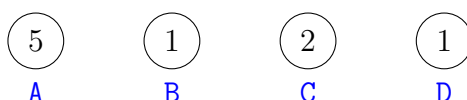
شرط لازم این است که هیچ کلمه‌کدی پیشوند کلمه‌کد دیگری نباشد. به عنوان مثال، مجاز نیست که یک کد شامل هر دو کلمه‌کد ۱۰ و ۱۰۱۱ باشد. دلیل این موضوع این است که می‌خواهیم امکان بازسازی رشته‌ی اصلی از رشته‌ی فشرده‌شده وجود داشته باشد. اگر یک کلمه‌کد می‌توانست پیشوند کلمه‌کد دیگری باشد، این کار همیشه امکان‌پذیر نبود. به عنوان مثال، کد زیر معتبر نیست:

نویسه	کلمه‌کد
A	۱۰
B	۱۱
C	۱۰۱۱
D	۱۱۱

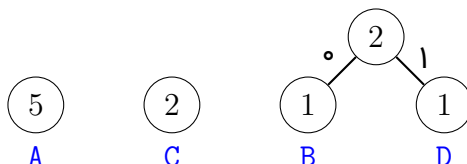
با استفاده از این کد، تشخیص این‌که آیا رشته‌ی فشرده‌شده‌ی ۱۰۱۱ متناظر با رشته‌ی AB است یا رشته‌ی C، ممکن نخواهد بود.

کدگذاری هافمن

کدگذاری هافمن^۲ یک الگوریتم حریصانه است که یک کد بهینه برای فشرده‌سازی یک رشته‌ی مشخص می‌سازد. این الگوریتم یک درخت دودویی بر اساس بسامد نویسه‌ها در رشته بنا می‌کند، و کلمه‌کد هر نویسه را می‌توان با دنبال کردن یک مسیر از ریشه تا گره متناظر خواند. حرکت به چپ متناظر با بیت ۰، و حرکت به راست متناظر با بیت ۱ است. در ابتدا، هر نویسه از رشته با گرهی نمایش داده می‌شود که وزنش برابر با تعداد دفعات تکرار آن نویسه در رشته است. سپس در هر مرحله، دو گره با کمترین وزن با ایجاد یک گره جدید ترکیب می‌شوند که وزنش برابر با مجموع وزن‌های گره‌های اولیه است. این فرایند تا زمانی که تمام گره‌ها ترکیب شوند ادامه می‌یابد. در ادامه خواهیم دید چگونه کدگذاری هافمن کد بهینه را برای رشته AABACDACA ایجاد می‌کند. در ابتدا، چهار گره متناظر با نویسه‌های رشته وجود دارد:

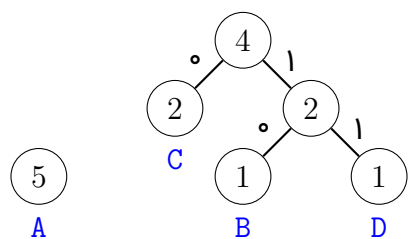


گره‌ی A که نویسه A را نشان می‌دهد دارای وزن ۵ است زیرا نویسه A ۵ بار در رشته آمده است. سایر وزن‌ها نیز به همین ترتیب محاسبه شده‌اند. مرحله نخست، ترکیب گره‌های متناظر با نویسه‌های B و D است که هر دو وزن ۱ دارند. نتیجه به این صورت است:

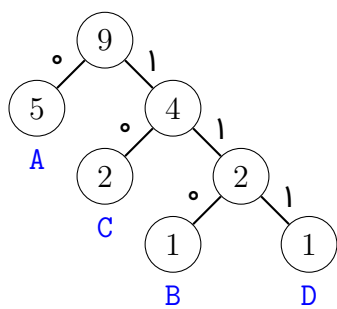


پس از این، گره‌های با وزن ۲ ترکیب می‌شوند:

^۲دیوید هافمن این روش را هنگام حل یک تمرین درسی دانشگاهی کشف کرد و الگوریتم را در سال ۱۹۵۲ منتشر نمود [۴۰].



در نهایت، دو گره باقی مانده ترکیب می شوند:



اکنون همه گره ها در درخت هستند، بنابراین کد آماده است. کلمه کدهای زیر از درخت قابل خواندن هستند:

نویسه	کلمه کد
A	۰
B	۱۱۰
C	۱۰
D	۱۱۱

فصل ۷

برنامه‌نویسی پویا

برنامه‌نویسی پویا تکنیکی است که درستی جستجوی کامل را با کارایی الگوریتم‌های حریصانه ترکیب می‌کند. برنامه‌نویسی پویا زمانی قابل استفاده است که مسئله را بتوان به زیرمسئله‌های هم‌پوشان تقسیم کرد؛ زیرمسئله‌هایی که مستقل از یکدیگر حل می‌شوند. کاربرد برنامه‌نویسی پویا دو بخش است:

- **یافتن پاسخ بهینه:** هدف، یافتن پاسخی با بیشترین یا کمترین مقدار ممکن است.
- **شمارش تعداد پاسخ‌ها:** هدف، محاسبه تعداد کل پاسخ‌های ممکن است.

نخست نحوه استفاده از برنامه‌نویسی پویا برای یافتن پاسخ بهینه بررسی می‌شود، سپس همان ایده برای شمارش پاسخ‌ها به کار می‌رود. درک برنامه‌نویسی پویا نقطه عطفی در مسیر هر برنامه‌نویس رقابتی است. ایده پایه ساده است، اما چالش اصلی، پیاده‌سازی برنامه‌نویسی پویا روی مسائل گوناگون است. این فصل مجموعه‌ای از مسائل کلاسیک را به عنوان نقطه شروع مناسب معرفی می‌کند.

۱.۷ مسئله سکه

نخست مسئله‌ای بررسی می‌شود که در فصل ۶ مطرح شد: مجموعه‌ای از ارزش‌های سکه $\text{coins} = \{c_1, c_2, \dots, c_k\}$ و مبلغ هدف n داده شده است. هدف، ساخت مبلغ n با کمترین تعداد سکه ممکن است.

در فصل ۶ مسئله با الگوریتم حریصانه حل شد؛ الگوریتمی که همیشه بزرگ‌ترین سکه ممکن را برمی‌گزیند. الگوریتم حریصانه برای نمونه روی سکه‌های یورو پاسخ می‌دهد، اما در حالت کلی لزوماً پاسخ بهینه تولید نمی‌کند.

اکنون زمان حل بهینه مسئله با برنامه‌نویسی پویاست، به شکلی که الگوریتم برای هر مجموعه‌ای از سکه‌ها کار کند. الگوریتم برنامه‌نویسی پویا بر پایه تابعی بازگشتی عمل می‌کند که تمام حالت‌های ساخت مبلغ را مشابه الگوریتم جستجوی کامل بررسی می‌نماید. با این وجود، الگوریتم برنامه‌نویسی پویا به دلیل استفاده از یادداشت‌برداری (memoization) و محاسبه پاسخ هر زیرمسئله تنها در یک نوبت، کارآمد است.

فرمول‌بندی بازگشتی

ایده برنامه‌نویسی پویا، فرمول‌بندی بازگشتی مسئله است، به طوری که پاسخ مسئله از روی پاسخ زیرمسئله‌های کوچک‌تر محاسبه شود. در مسئله سکه، شکل بازگشتی طبیعی مسئله چنین است: کمترین تعداد سکه مورد نیاز برای ساخت مبلغ x چقدر است؟

فرض کنید $\text{solve}(x)$ نشان‌دهنده کمترین تعداد سکه مورد نیاز برای مبلغ x باشد. مقادیر تابع به ارزش سکه‌ها وابسته است. برای نمونه، اگر $\text{coins} = \{1, 3, 4\}$ باشد، نخستین مقادیر تابع چنین خواهد بود:

$\text{solve}(0)$	$=$	0
$\text{solve}(1)$	$=$	1
$\text{solve}(2)$	$=$	2
$\text{solve}(3)$	$=$	1
$\text{solve}(4)$	$=$	1
$\text{solve}(5)$	$=$	2
$\text{solve}(6)$	$=$	2
$\text{solve}(7)$	$=$	2
$\text{solve}(8)$	$=$	2
$\text{solve}(9)$	$=$	3
$\text{solve}(10)$	$=$	3

برای نمونه، $\text{solve}(10) = 3$ ، زیرا حداقل ۳ سکه نیاز است تا مجموع ۱۰ تشکیل شود. پاسخ بهینه $3 + 3 + 4 = 10$ است. ویژگی بنیادین solve این است که مقادیر آن را می‌توان به صورت بازگشتی از مقادیر کوچک‌تر آن محاسبه کرد. ایده کار تمرکز بر اولین سکه‌ای است که برای مجموع انتخاب می‌کنیم. برای نمونه، در سناریوی بالا، اولین سکه می‌تواند ۱، ۳ یا ۴ باشد. اگر ابتدا سکه ۱ را انتخاب کنیم، کار باقی‌مانده تشکیل مجموع ۹ با استفاده از حداقل تعداد سکه است، که یک زیرمسئله از مسئله اصلی به شمار می‌رود. همین موضوع برای سکه‌های ۳ و ۴ نیز صدق می‌کند. بنابراین، می‌توان از فرمول بازگشتی زیر برای محاسبه حداقل تعداد سکه‌ها استفاده کرد:

$$\begin{aligned} \text{solve}(x) = \min(&\text{solve}(x-1) + 1, \\ &\text{solve}(x-3) + 1, \\ &\text{solve}(x-4) + 1). \end{aligned}$$

حالت پایه بازگشت $\text{solve}(0) = 0$ است، زیرا برای تشکیل مجموع صفر هیچ سکه‌ای لازم نیست. برای نمونه،

$$\text{solve}(10) = \text{solve}(7) + 1 = \text{solve}(4) + 2 = \text{solve}(0) + 3 = 3.$$

اکنون آماده‌ایم تا یک تابع بازگشتی عمومی ارائه دهیم که حداقل تعداد سکه‌های لازم برای تشکیل مجموع x را محاسبه کند:

$$\text{solve}(x) = \begin{cases} \infty & x < 0 \\ 0 & x = 0 \\ \min_{c \in \text{coins}} \text{solve}(x-c) + 1 & x > 0 \end{cases}$$

نخست، اگر $x < 0$ باشد، مقدار برابر با ∞ است، زیرا تشکیل مجموع منفی ممکن نیست. سپس، اگر $x = 0$ باشد، مقدار برابر با 0 است، زیرا برای مجموع تهی نیازی به سکه نیست. در نهایت، اگر $x > 0$ باشد، متغیر c تمام حالت‌های ممکن برای انتخاب اولین سکه از مجموع را بررسی می‌کند.

پس از یافتن تابع بازگشتی حل مسئله، می‌توان راه‌حل را مستقیماً در ++C پیاده‌سازی کرد (ثابت INF نشان‌دهنده بی‌نهایت است):

```

        { }x int(solve int
;INF return 0) < x( if
0; return 0) == x( if
;INF = best int
{ }coins : c auto( for
)+1);c-x(solve ,best(min = best
}
;best return
}

```

با این حال، این تابع کارا نیست، چون ممکن است تعداد روش‌های ساخت مجموع به صورت نمایی باشد. در ادامه خواهیم دید چگونه با استفاده از تکنیکی به نام یادداشت‌برداری (memoization) این تابع را کارا کنیم.

استفاده از یادداشت‌برداری

ایده برنامه‌نویسی پویا استفاده از یادداشت‌برداری برای محاسبه کارای مقادیر یک تابع بازگشتی است. این یعنی مقادیر تابع پس از محاسبه درون یک آرایه ذخیره می‌شوند. برای هر پارامتر، مقدار تابع تنها یک بار به صورت بازگشتی محاسبه می‌شود و پس از آن، مقدار مستقیماً از آرایه بازخوانی خواهد شد. در این مسئله، از آرایه‌های زیر استفاده می‌کنیم:

```

];N[ready bool
];N[value int

```

که در آن $ready[x]$ مشخص می‌کند آیا مقدار $solve(x)$ محاسبه شده است یا خیر، و در صورت محاسبه، $value[x]$ شامل این مقدار است. ثابت N طوری انتخاب شده است که تمام مقادیر مورد نیاز در آرایه‌ها جا شوند. اکنون تابع را می‌توان به صورت کارآمد به شکل زیر پیاده‌سازی کرد:

```

        { }x int(solve int
;INF return 0) < x( if
0; return 0) == x( if
];x[value return ];x[ready( if
;INF = best int
{ }coins : c auto( for
)+1);c-x(solve ,best(min = best
}
;best = ];x[value
;true = ];x[ready
;best return
}

```

تابع حالت‌های پایه $x < 0$ و $x = 0$ را مانند قبل مدیریت می‌کند. سپس با بررسی

$ready[x]$ چک می‌کند آیا $solve(x)$ قبلاً در $value[x]$ ذخیره شده است یا خیر، و در صورت وجود، تابع مستقیماً آن را برمی‌گرداند. در غیر این صورت، تابع مقدار $solve(x)$ را به صورت بازگشتی محاسبه کرده و در $value[x]$ ذخیره می‌کند.

این تابع به صورت کارآمد عمل می‌کند، زیرا پاسخ برای هر پارامتر x فقط یک بار به صورت بازگشتی محاسبه می‌شود. پس از ذخیره مقدار $solve(x)$ در $value[x]$ ، هر زمان که تابع دوباره با پارامتر x فراخوانی شود، می‌توان آن را سریع بازیابی کرد. پیچیدگی زمانی الگوریتم $O(nk)$ است، که در آن n مجموع هدف و k تعداد سکه‌ها است.

توجه کنید که می‌توان آرایه $value$ را به صورت حلقه‌ای (پایین به بالا) با استفاده از یک حلقه که تمام مقادیر $solve$ را برای پارامترهای $0 \dots n$ محاسبه می‌کند نیز ساخت:

```

0; = [0]value
{ ++ )x ; n <= x 1; = x int( for
    ; INF = ]x[value
    { )coins : c auto( for
        { 0 ) >= c-x( if
    ]+1);c-x[value ,]x[value(min = ]x[value
    }
}
}
```

در واقع، اکثر برنامه‌نویسان رقابتی این پیاده‌سازی را ترجیح می‌دهند، زیرا کوتاه‌تر است و ضریب ثابت کمتری دارد. از این پس، در مثال‌هایمان از پیاده‌سازی‌های حلقه‌ای استفاده می‌کنیم. با این حال، اغلب فکر کردن به راه حل‌های برنامه‌نویسی پویا در قالب توابع بازگشتی ساده‌تر است.

ساختن یک جواب

گاهی اوقات از ما خواسته می‌شود هم مقدار جواب بهینه را بیابیم و هم نمونه‌ای از نحوه ساخت چنین جوابی ارائه دهیم. برای مثال در مسئله سکه، می‌توانیم آرایه دیگری تعریف کنیم که برای هر مقدار پول، اولین سکه در یک جواب بهینه را مشخص کند:

```
];N[first int
```

داد: تغییر زیر شکل به را الگوریتم می‌توان سپس،

```

0; = [0]value
{ ++ )x ; n <= x 1; = x int( for
    ; INF = ]x[value
    { )coins : c auto( for
        { ]x[value < ]+1c-x[value && 0 >= c-x( if
    ]+1;c-x[value = ]x[value
        ;c = ]x[first
    }
}
}
```

n مجموع برای بهینه جواب یک در موجود سکه‌های چاپ برای می‌تواند زیر کد این، از پس

شود: استفاده

```
while (n > 0) {
    cout << "n\" << n[first << cout
    ]; n[first -= n
}
```

شم‌ارش تعداد جواب‌ها

حال نسخه دیگری از مسئله سکه را بررسی می‌کنیم که در آن هدف محاسبه تعداد کل روش‌های ساخت مجموع x با استفاده از سکه‌ها است. برای مثال اگر $\text{coins} = \{1, 3, 4\}$ و $x = 5$ باشد، در مجموع ۶ روش وجود دارد:

$3 + 1 + 1 \cdot$	$1 + 1 + 1 + 1 + 1 \cdot$
$1 + 4 \cdot$	$1 + 1 + 3 \cdot$
$4 + 1 \cdot$	$1 + 3 + 1 \cdot$

باز هم می‌توان مسئله را بازگشتی حل کرد. فرض کنید $\text{solve}(x)$ بیانگر تعداد روش‌های ساخت مجموع x باشد. برای مثال اگر $\text{coins} = \{1, 3, 4\}$ باشد، آنگاه $\text{solve}(5) = 6$ و رابطه بازگشتی چنین است:

$$\begin{aligned} \text{solve}(x) = & \text{solve}(x-1) + \\ & \text{solve}(x-3) + \\ & \text{solve}(x-4). \end{aligned}$$

تابع بازگشتی کلی به صورت زیر است:

$$\text{solve}(x) = \begin{cases} 0 & x < 0 \\ 1 & x = 0 \\ \sum_{c \in \text{coins}} \text{solve}(x-c) & x > 0 \end{cases}$$

اگر $x < 0$ باشد، مقدار برابر ۰ است زیرا هیچ پاسخی وجود ندارد. اگر $x = 0$ باشد، مقدار برابر ۱ است زیرا تنها یک روش برای ساخت مجموع خالی وجود دارد. در غیر این صورت، مجموع تمام مقادیر به فرم $\text{solve}(x-c)$ را محاسبه می‌کنیم که در آن c عضوی از coins است.

کد زیر آرایه count را طوری می‌سازد که $\text{count}[x]$ برابر با مقدار $\text{solve}(x)$ به ازای $0 \leq x \leq n$ باشد:

```
int count[n+1];
count[0] = 1;
for (int x = 1; x <= n; ++x) {
    for (auto c : coins) {
        if (c <= x) {
            count[x] += count[x-c];
        }
    }
}
```

معمولاً تعداد پاسخ‌ها چنان بزرگ است که نیازی به محاسبه مقدار دقیق نیست بلکه ارائه پاسخ به پیمانه m کافی است، که برای مثال $m = 10^9 + 7$ است. این کار با تغییر کد ممکن است تا همه محاسبات به پیمانه m انجام شوند. در کد بالا، اضافه کردن سطر

```
;m %= x[count]
```

سطر از بعد

```
];c-x[count += x[count]
```

می‌کند. کفایت

اکنون تمام ایده‌های پایه برنامه‌نویسی پویا را بررسی کردیم. از آنجا که برنامه‌نویسی پویا در موقعیت‌های گوناگونی کاربرد دارد، مجموعه‌ای از مسائل را بررسی خواهیم کرد تا نمونه‌های بیشتری از توانایی‌های برنامه‌نویسی پویا نشان داده شود.

۲.۷ طولانی‌ترین زیردنباله صعودی

مسئله اول، یافتن **طولانی‌ترین زیردنباله صعودی** در آرایه‌ای با n عنصر است. این زیردنباله، دنباله‌ای با طول بیشینه از عناصر آرایه از چپ به راست است که در آن هر عنصر از عنصر قبلی خود بزرگ‌تر است. برای مثال در آرایه

0	1	2	3	4	5	6	7
6	2	5	1	7	4	8	3

طولانی‌ترین زیردنباله صعودی شامل ۴ عنصر است:

0	1	2	3	4	5	6	7
6	2	5	1	7	4	8	3



فرض کنید $\text{length}(k)$ نشان‌دهنده طول طولانی‌ترین زیردنباله صعودی باشد که در موقعیت k خاتمه می‌یابد. بنابراین، اگر تمام مقادیر $\text{length}(k)$ را به ازای $0 \leq k \leq n-1$ محاسبه کنیم، طول طولانی‌ترین زیردنباله صعودی را به دست خواهیم آورد. برای مثال، مقادیر تابع برای آرایه فوق به شرح زیر است:

```
length(0) = 1
length(1) = 1
length(2) = 2
length(3) = 1
length(4) = 3
length(5) = 2
length(6) = 4
length(7) = 2
```

برای نمونه، $\text{length}(6) = 4$ است، زیرا طولانی‌ترین زیردنباله صعودی منتهی به موقعیت ۶ متشکل از ۴ عنصر است.

جهت محاسبه مقدار $length(k)$ ، باید موقعیتی مانند $i < k$ بیابیم که در آن $array[i] < array[k]$ باشد و $length(i)$ بیشترین مقدار ممکن را داشته باشد. سپس می‌دانیم که $length(k) = length(i) + 1$ خواهد بود، زیرا این روش بهینه برای افزودن $array[k]$ به یک زیردنباله است. اما اگر چنین موقعیتی مانند i وجود نداشته باشد، آنگاه $length(k) = 1$ می‌شود، به این معنی که زیردنباله تنها شامل $array[k]$ است. از آنجا که تمام مقادیر تابع از روی مقادیر کوچک‌تر آن قابل محاسبه است، می‌توان از برنامه‌نویسی پویا بهره برد. در کد زیر، مقادیر تابع درون آرایه‌ای به نام $length$ ذخیره خواهد شد.

```
for (int k = 0; k < n; ++k) {
    int length = 1;
    for (int i = 0; i < k; ++i) {
        if (array[i] < array[k]) {
            length = max(length, i[length] + 1);
        }
    }
}
```

این کد در زمان $O(n^2)$ اجرا می‌شود، زیرا شامل دو حلقه تودرتو است. با این حال، می‌توان محاسبات برنامه‌نویسی پویا را به شکل کارآمدتری در زمان $O(n \log n)$ نیز پیاده‌سازی کرد. آیا می‌توانید روشی برای این کار بیابید؟

۳.۷ مسیرها در جدول

مسئله بعدی یافتن مسیری از گوشه بالا-چپ به گوشه پایین-راست در یک جدول $n \times n$ است، به طوری که تنها مجاز به حرکت به سمت پایین و راست باشیم. هر خانه شامل یک عدد صحیح مثبت است و مسیر باید به گونه‌ای انتخاب شود که مجموع مقادیر در طول مسیر بیشترین مقدار ممکن باشد. تصویر زیر یک مسیر بهینه را در جدول نشان می‌دهد:

۳	۷	۹	۲	۷
۹	۸	۳	۵	۵
۱	۷	۹	۸	۵
۳	۸	۶	۴	۱۰
۶	۳	۹	۷	۸

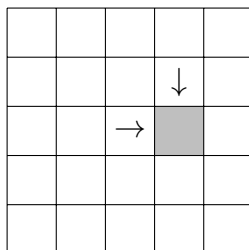
مجموع مقادیر در این مسیر برابر ۶۷ است و این بیشترین مجموع ممکن برای مسیری از گوشه بالا-چپ به گوشه پایین-راست می‌باشد.

فرض کنید سطرها و ستون‌های جدول از ۱ تا n شماره‌گذاری شده‌اند و $value[y][x]$ برابر با مقدار خانه (y, x) است. فرض کنید $sum(y, x)$ بیشترین مجموع در مسیری از گوشه بالا-چپ به خانه (y, x) را نشان دهد. در این صورت $sum(n, n)$ بیشترین مجموع از گوشه بالا-چپ تا گوشه پایین-راست را مشخص می‌کند. برای نمونه، در جدول بالا داریم: $sum(5, 5) = 67$.

می‌توان مجموع‌ها را به صورت بازگشتی به شکل زیر محاسبه کرد:

$$sum(y, x) = \max(sum(y, x-1), sum(y-1, x)) + value[y][x]$$

فرمول بازگشتی مبتنی بر این مشاهده است که مسیر منتهی به خانه (y, x) فقط می‌تواند از خانه $(y, x - 1)$ یا از خانه $(y - 1, x)$ بیاید:



بنابراین، جهتی انتخاب می‌شود که مجموع را بیشینه کند. فرض می‌شود $\text{sum}(y, x) = 0$ اگر $y = 0$ یا $x = 0$ باشد (چون چنین مسیری وجود ندارد)، بنابراین فرمول بازگشتی برای $y = 1$ یا $x = 1$ نیز کار می‌کند. چون تابع sum دو پارامتر دارد، آرایه برنامه‌نویسی پویا نیز دو بعدی است. برای مثال، می‌توان از آرایه زیر استفاده کرد:

```
int sum[N][N];
```

کرد: محاسبه صورت این به را مجموع‌ها و

```
for (int y = 1; y <= n; ++y)
    for (int x = 1; x <= n; ++x)
        sum[y][x] = max(sum[y][x-1], sum[y-1][x]) + value[x];
```

است. $O(n^2)$ الگوریتم زمانی پیچیدگی

۴.۷ مسائل کوله‌پشتی

اصطلاح **کوله‌پشتی** به مسائلی اشاره دارد که مجموعه‌ای از اشیاء داده شده و باید زیرمجموعه‌هایی با ویژگی‌های مشخص پیدا شوند. مسائل کوله‌پشتی اغلب با برنامه‌نویسی پویا حل می‌شوند. در این بخش، تمرکز روی این مسئله است: به ازای فهرستی از وزن‌ها $[w_1, w_2, \dots, w_n]$ ، تمام مجموعه‌هایی که می‌توان با این وزن‌ها ساخت مشخص شوند. برای مثال، اگر وزن‌ها $[1, 3, 3, 5]$ باشند، این مجموعه‌ها ممکن هستند:

۱۲	۱۱	۱۰	۹	۸	۷	۶	۵	۴	۳	۲	۱	۰
X	X		X	X	X	X	X	X	X		X	X

در این حالت، تمام مجموعه‌های بین $0 \dots 12$ به جز ۲ و ۱۰ ممکن هستند. برای مثال، مجموع ۷ ممکن است چون می‌توان وزن‌های $[1, 3, 3]$ را انتخاب کرد. برای حل مسئله، تمرکز روی زیرمسائلی است که فقط از k وزن اول ساخت مجموعه‌ها استفاده می‌کنند. فرض کنید $\text{possible}(x, k) = \text{true}$ باشد اگر بتوان مجموع x را با استفاده از k وزن اول ساخت، و در غیر این صورت $\text{possible}(x, k) = \text{false}$. مقادیر تابع به صورت بازگشتی این‌گونه محاسبه می‌شوند:

$$\text{possible}(x, k) = \text{possible}(x - w_k, k - 1) \vee \text{possible}(x, k - 1)$$

فرمول بر این اساس است که وزن w_k در مجموع استفاده شود یا نشود. اگر w_k استفاده شود، کار باقی‌مانده ساخت مجموع $x - w_k$ با $k - 1$ وزن اول است، و اگر w_k استفاده نشود، کار باقی‌مانده ساخت مجموع x با $k - 1$ وزن اول است. به عنوان حالت‌های پایه:

$$\text{possible}(x, 0) = \begin{cases} \text{true} & x = 0 \\ \text{false} & x \neq 0 \end{cases}$$

چون اگر هیچ وزنی استفاده نشود، فقط مجموع ۰ قابل ساخت است. جدول زیر همه مقادیر تابع را برای وزن‌های $[1, 3, 3, 5]$ نشان می‌دهد (علامت "X" نشان‌دهنده مقادیر درست است):

$k \backslash x$	۰	۱	۲	۳	۴	۵	۶	۷	۸	۹	۱۰	۱۱	۱۲
۰	X												
۱	X	X											
۲	X	X	X	X									
۳	X	X	X	X	X	X	X						
۴	X	X	X	X	X	X	X	X	X	X	X	X	X

پس از محاسبه این مقادیر، $\text{possible}(x, n)$ مشخص می‌کند آیا می‌توان با استفاده از همه وزن‌ها به مجموع x رسید یا خیر. فرض کنید W مجموع کل وزن‌ها باشد. راه‌حل برنامه‌نویسی پویا زیر با پیچیدگی زمانی $O(nW)$ متناظر با تابع بازگشتی است:

```

;true = [0][0]possible
{ ++)k ;n <= k 1; = k int( for
{ ++)x ;W <= x 0; = x int( for
-1];k]] [k[w-x[possible |= ]k] [x[possible 0) >= ]k[w-x( if
-1];k] [x[possible |= ]k] [x[possible
}
}

```

با این حال، پیاده‌سازی بهتری وجود دارد که تنها از یک آرایه یک‌بعدی $\text{possible}[x]$ استفاده می‌کند؛ آرایه‌ای که مشخص می‌سازد آیا ساخت زیرمجموعه‌ای با مجموع x ممکن است یا خیر. ترفند کار، به‌روزرسانی آرایه از راست به چپ برای هر وزن جدید است:

```

;true = [0]possible
{ ++)k ;n <= k 1; = k int( for
{ -)-x 0; >= x ;W = x int( for
;true = ]]k[w+x[possible ]x[possible( if
}
}

```

توجه داشته باشید که ایده کلی ارائه‌شده در اینجا در بسیاری از مسائل کوله‌پشتی کاربرد دارد. به عنوان مثال، اگر اشیایی با وزن‌ها و ارزش‌های مشخص داده شوند، می‌توانیم به ازای هر مجموع وزن، بیشترین مجموع ارزش یک زیرمجموعه را تعیین کنیم.

۵.۷ فاصله ویرایشی

فاصله ویرایشی یا **فاصله لون‌اشتاین**^۱ کمینه تعداد عملیات‌های ویرایشی مورد نیاز برای تبدیل یک رشته به رشته‌ای دیگر است. عملیات‌های ویرایشی مجاز عبارتند از:

• درج یک نویسه (مثال: $ABCA \rightarrow ABC$)

• حذف یک نویسه (مثال: $AC \rightarrow ABC$)

• تغییر یک نویسه (مثال: $ADC \rightarrow ABC$)

برای نمونه، فاصله ویرایشی بین LOVE و MOVIE برابر ۲ است، زیرا ابتدا می‌توان عملیات $MOVE \rightarrow LOVE$ (تغییر) و سپس عملیات $MOVIE \rightarrow MOVE$ (درج) را انجام داد. این کمترین تعداد عملیات ممکن است، زیرا مشخص است که تنها یک عملیات کافی نیست. فرض کنید رشته x با طول n و رشته y با طول m داده شده‌اند و هدف، محاسبه فاصله ویرایشی بین x و y است. برای حل مسئله، تابع $distance(a, b)$ را تعریف می‌کنیم که فاصله ویرایشی بین پیشوندهای $x[0 \dots a]$ و $y[0 \dots b]$ را می‌دهد. بنابراین با استفاده از این تابع، فاصله ویرایشی بین x و y برابر با $distance(n-1, m-1)$ است. مقادیر $distance$ را می‌توان به این صورت محاسبه کرد:

$$\begin{aligned} distance(a, b) = \min(&distance(a, b-1) + 1, \\ &distance(a-1, b) + 1, \\ &distance(a-1, b-1) + cost(a, b)). \end{aligned}$$

در اینجا اگر $x[a] = y[b]$ باشد مقدار $cost(a, b) = 0$ ، و در غیر این صورت $cost(a, b) = 1$ است. این رابطه حالات زیر را برای ویرایش رشته x در نظر می‌گیرد:

• $distance(a, b-1)$: درج یک نویسه در انتهای x

• $distance(a-1, b)$: حذف آخرین نویسه از x

• $distance(a-1, b-1)$: تطابق یا تغییر آخرین نویسه x

در دو حالت اول، یک عملیات ویرایش نیاز است (درج یا حذف). در حالت آخر، اگر $x[a] = y[b]$ باشد، می‌توان آخرین نویسه‌ها را بدون ویرایش تطابق داد، و در غیر این صورت یک عملیات ویرایش نیاز است (تغییر).

جدول زیر مقادیر $distance$ را در مثال نمونه نشان می‌دهد:

	M	O	V	I	E	
L O V E	0	1	2	3	4	5
	1	1	2	3	4	5
	2	2	1	2	3	4
	3	3	2	1	2	3
	4	4	3	2	2	2

^۱این فاصله به نام وی. آی. لون‌اشتاین نام‌گذاری شده است که آن را در ارتباط با کدهای دودویی بررسی کرد [۴۹].

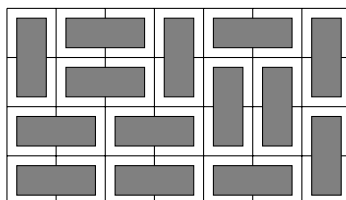
گوشه پایین و راست جدول نشان می‌دهد فاصله ویرایشی بین LOVE و MOVIE برابر ۲ است. همچنین، جدول نحوه ساخت کوتاه‌ترین دنباله از عملیات ویرایشی را نشان می‌دهد. در این حالت، مسیر به صورت زیر است:

		M	O	V	I	E
	0	1	2	3	4	5
L	1	1	2	3	4	5
O	2	2	1	2	3	4
V	3	3	2	1	2	3
E	4	4	3	2	2	2

نویسه‌های آخر MOVIE و LOVE برابرند، بنابراین فاصله ویرایشی بین آن‌ها برابر با فاصله ویرایشی بین LOV و MOVI است. می‌توان با یک عمل ویرایشی نویسه I را از MOVI حذف کرد. در نتیجه، فاصله ویرایشی یک واحد بیشتر از فاصله ویرایشی بین LOV و MOV است و به همین ترتیب.

۶.۷ شمارش کاشی‌کاری‌ها

گاهی حالت‌های یک راهکار برنامه‌نویسی پویا پیچیده‌تر از ترکیب‌های ثابتی از اعداد هستند. به‌عنوان مثال، مسئله محاسبه تعداد روش‌های متمایز برای پر کردن یک جدول $n \times m$ با استفاده از کاشی‌های با ابعاد 1×2 و 2×1 را در نظر بگیرید. به‌عنوان نمونه، یک جواب معتبر برای جدول 4×7 عبارت است از:



و تعداد کل جواب‌ها برابر ۷۸۱ است. این مسئله را می‌توان با برنامه‌نویسی پویا و با پیمایش سطر به سطر جدول حل کرد. هر سطر در یک جواب را می‌توان به صورت رشته‌ای شامل m نویسه از مجموعه $\{\sqcup, \sqcup, \sqsubset, \sqsupset\}$ نشان داد. برای مثال، جواب بالا شامل چهار سطر است که متناظر با رشته‌های زیر هستند:

$\sqsubset \sqsubset \sqsubset \sqsubset \sqsubset \sqsubset \sqsubset$ •

$\sqcup \sqsubset \sqsubset \sqcup \sqsubset \sqsubset \sqcup$ •

$\sqsubset \sqsubset \sqsubset \sqsubset \sqcup \sqcup \sqsubset$ •

$\sqsubset \sqsubset \sqsubset \sqsubset \sqsubset \sqsubset \sqcup$ •

فرض کنید $\text{count}(k, x)$ بیانگر تعداد روش‌های ساخت یک جواب برای سطرهای $1 \dots k$ از جدول باشد به طوری که رشته x متناظر با سطر k باشد. استفاده از برنامه‌نویسی پویا در اینجا ممکن است، زیرا وضعیت یک سطر فقط توسط وضعیت سطر قبلی محدود می‌شود.

یک پاسخ معتبر است اگر سطر 1 شامل نویسه □ نباشد، سطر n شامل نویسه □ نباشد، و تمام سطرهای متوالی سازگار باشند. برای نمونه، سطرهای □□□□□□□□ و □□□□□□□□ سازگارند، در حالی که سطرهای □□□□□□□□ و □□□□□□□□ سازگار نیستند. چون هر سطر شامل m نویسه است و برای هر نویسه چهار انتخاب وجود دارد، تعداد سطرهای متمایز حداکثر 4^m است. بنابراین، پیچیدگی زمانی راه حل $O(n4^{2m})$ است؛ چون برای هر سطر می‌توان $O(4^m)$ حالت ممکن را پیمایش کرد، و برای هر حالت، $O(4^m)$ حالت ممکن برای سطر قبلی وجود دارد. در عمل، بهتر است جدول چرخ Mel شود تا ضلع کوتاه‌تر طول m داشته باشد، زیرا ضریب 4^{2m} بر پیچیدگی زمانی غالب است. می‌توان با استفاده از نمایش فشرده‌تر برای سطرها، کارایی راه حل را افزایش داد. مشخص می‌شود کافی است بدانیم کدام ستون‌های سطر قبل شامل مربع بالایی یک کاشی عمودی هستند. بنابراین، می‌توان یک سطر را تنها با استفاده از نویسه‌های □ و □ نشان داد، که در آن □ ترکیبی از نویسه‌های □، □ و □ است. با استفاده از این نمایش، تنها 2^m سطر متمایز وجود دارد و پیچیدگی زمانی $O(n2^{2m})$ خواهد بود. به عنوان نکته پایانی، یک فرمول مستقیم شگفت‌انگیز نیز برای محاسبه تعداد کاشی‌کاری‌ها وجود دارد^۲:

$$\prod_{a=1}^{\lceil n/2 \rceil} \prod_{b=1}^{\lceil m/2 \rceil} 4 \cdot \left(\cos^2 \frac{\pi a}{n+1} + \cos^2 \frac{\pi b}{m+1} \right)$$

این فرمول بسیار کارآمد است، زیرا تعداد کاشی‌کاری‌ها را در زمان $O(nm)$ محاسبه می‌کند، اما چون پاسخ حاصل ضرب اعداد حقیقی است، چالش هنگام استفاده از فرمول، ذخیره دقیق نتایج میانی است.

^۲ به طرز شگفت‌آوری، این فرمول در سال ۱۹۶۱ توسط دو تیم پژوهشی [۴۳، ۶۷] که به طور مستقل کار می‌کردند کشف شد.

فصل ۸

تحلیل سرشکن

پیچیدگی زمانی یک الگوریتم اغلب تنها با بررسی ساختار الگوریتم به سادگی تحلیل می‌شود: الگوریتم شامل چه حلقه‌هایی است و این حلقه‌ها چند بار اجرا می‌شوند. با این حال، گاهی اوقات تحلیل مستقیم تصویر درستی از کارایی الگوریتم ارائه نمی‌دهد. می‌توان از **تحلیل سرشکن** برای تحلیل الگوریتم‌هایی استفاده کرد که شامل عملیاتی با پیچیدگی زمانی متغیر هستند. ایده این است که به جای تمرکز بر تک تک عملیات، کل زمان صرف شده برای تمام این عملیات در طول اجرای الگوریتم برآورد شود.

۱.۸ روش دو اشاره‌گر

در **روش دو اشاره‌گر**، از دو اشاره‌گر برای پیمایش مقادیر آرایه استفاده می‌شود. هر دو اشاره‌گر فقط می‌توانند در یک جهت حرکت کنند، که کارایی الگوریتم را تضمین می‌کند. سپس به دو مسئله می‌پردازیم که می‌توان با روش دو اشاره‌گر حل کرد.

مجموع زیرآرایه

به عنوان اولین مثال، مسئله‌ای را در نظر بگیرید که در آن آرایه‌ای از n عدد صحیح مثبت و یک مجموع هدف x داده شده است، و می‌خواهیم زیرآرایه‌ای پیدا کنیم که مجموع آن برابر با x باشد یا اعلام کنیم چنین زیرآرایه‌ای وجود ندارد. برای نمونه، آرایه

1	3	2	5	1	1	2	3
---	---	---	---	---	---	---	---

شامل زیرآرایه‌ای است که مجموع آن برابر ۸ است:

1	3	2	5	1	1	2	3
---	---	---	---	---	---	---	---

این مسئله را می‌توان در زمان $O(n)$ با روش دو اشاره‌گر حل کرد. ایده کار نگهداری اشاره‌گرهایی است که به مقدار اول و آخر یک زیرآرایه اشاره می‌کنند. در هر نوبت، اشاره‌گر چپ یک گام به راست حرکت می‌کند، و اشاره‌گر راست تا زمانی که مجموع زیرآرایه حاصل حداکثر x باشد به راست حرکت می‌کند. اگر مجموع دقیقاً برابر x شود، جواب پیدا شده است.

به عنوان مثال، آرایه زیر و مجموع هدف $x = 8$ را در نظر بگیرید:

1	3	2	5	1	1	2	3
---	---	---	---	---	---	---	---

زیرآرایه اولیه شامل مقادیر ۱، ۳ و ۲ است که مجموع آن‌ها برابر ۶ می‌شود:

1	3	2	5	1	1	2	3
↑		↑					

سپس اشاره‌گر چپ یک گام به راست می‌رود. اشاره‌گر راست حرکت نمی‌کند، چون در غیر این صورت مجموع زیرآرایه از x بیشتر می‌شود.

1	3	2	5	1	1	2	3
	↑	↑					

مجدداً اشاره‌گر چپ یک گام به راست حرکت می‌کند، و این بار اشاره‌گر راست سه گام به راست می‌رود. مجموع زیرآرایه $8 = 2 + 5 + 1$ است، بنابراین زیرآرایه‌ای با مجموع x پیدا شده است.

1	3	2	5	1	1	2	3
		↑	↑				

زمان اجرای الگوریتم به تعداد گام‌های حرکت اشاره‌گر راست وابسته است. گرچه کران بالای مفیدی برای تعداد گام‌های حرکت اشاره‌گر در یک نوبت تک وجود ندارد، می‌دانیم اشاره‌گر در مجموع $O(n)$ گام در طول الگوریتم حرکت می‌کند، زیرا فقط به سمت راست می‌رود.

از آنجا که هر دو اشاره‌گر چپ و راست در طول الگوریتم $O(n)$ گام حرکت می‌کنند، الگوریتم در زمان $O(n)$ اجرا می‌شود.

مسئله 2SUM

مسئله دیگری که با روش دو اشاره‌گر حل می‌شود مسئله زیر است، که با نام **مسئله 2SUM** نیز شناخته می‌شود: با فرض یک آرایه شامل n عدد و مجموع هدف x ، دو مقدار از آرایه بیابید که مجموع آن‌ها x شود، یا اعلام کنید چنین مقادیری وجود ندارند.

برای حل مسئله، ابتدا مقادیر آرایه را به ترتیب صعودی مرتب می‌کنیم. پس از آن، با استفاده از دو اشاره‌گر روی آرایه پیمایش می‌کنیم. اشاره‌گر چپ از مقدار نخست آغاز شده و در هر نوبت یک گام به راست می‌رود. اشاره‌گر راست از مقدار آخر شروع شده و تا زمانی که مجموع مقادیر چپ و راست حداکثر برابر x شود به چپ می‌رود. اگر مجموع دقیقاً x باشد، پاسخ پیدا شده است.

برای نمونه، آرایه زیر و مجموع هدف $x = 12$ را در نظر بگیرید:

1	4	5	6	7	9	9	10
---	---	---	---	---	---	---	----

موقعیت اولیه اشاره‌گرها به شکل زیر است. مجموع مقادیر $1 + 10 = 11$ است که از x کوچک‌تر است.

1	4	5	6	7	9	9	10
↑							↑

سپس اشاره‌گر چپ یک گام به راست می‌رود. اشاره‌گر راست سه گام به چپ حرکت می‌کند و مجموع برابر با $4 + 7 = 11$ می‌شود.

1	4	5	6	7	9	9	10
	↑			↑			

پس از این، اشاره‌گر چپ دوباره یک گام به راست می‌رود. اشاره‌گر راست حرکت نمی‌کند و جواب $5 + 7 = 12$ پیدا می‌شود.

1	4	5	6	7	9	9	10
		↑		↑			

زمان اجرای الگوریتم $O(n \log n)$ است، زیرا ابتدا آرایه در زمان $O(n \log n)$ مرتب شده و سپس هر دو اشاره‌گر $O(n)$ گام جابه‌جا می‌شوند.

توجه کنید حل مسئله به روش دیگر در زمان $O(n \log n)$ با جستجوی دودویی نیز ممکن است. در این روش، روی آرایه پیمایش کرده و به ازای هر مقدار آرایه، تلاش می‌کنیم مقدار دیگر سازنده مجموع x را بیابیم. این کار با انجام n جستجوی دودویی ممکن است که هر کدام $O(\log n)$ زمان می‌برد.

مسئله دشوارتر، **مسئله ۳SUM** است که یافتن سه مقدار از آرایه با مجموع x را می‌خواهد. با ایده الگوریتم بالا، این مسئله در زمان $O(n^2)$ حل می‌شود.^۱ روش آن را می‌دانید؟

۲.۸ نزدیک‌ترین عناصر کوچک‌تر

تحلیل سرشکن اغلب برای تخمین تعداد عملیات انجام‌شده روی ساختمان داده به کار می‌رود. عملیات ممکن است نامتوازن توزیع شوند؛ به طوری که بیشتر عملیات در فاز خاصی از الگوریتم رخ دهند، اما تعداد کل عملیات محدود است.

به عنوان مثال، مسئله یافتن **نزدیک‌ترین عنصر کوچک‌تر** برای هر عنصر آرایه را در نظر بگیرید؛ یعنی اولین عنصر کوچک‌تر پیش از آن در آرایه. ممکن است چنین عنصری وجود نداشته باشد که در این حالت الگوریتم باید این را گزارش کند. در ادامه روش حل کارآمد مسئله با ساختار پشته بررسی می‌شود.

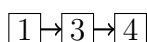
آرایه را از چپ به راست پیمایش کرده و پشته‌ای از عناصر آرایه نگه می‌داریم. در هر موقعیت آرایه، عناصر پشته را تا جایی حذف می‌کنیم که عنصر بالای پشته کوچک‌تر از عنصر فعلی شود یا پشته خالی گردد. سپس گزارش می‌دهیم عنصر بالای پشته نزدیک‌ترین عنصر کوچک‌تر به عنصر فعلی است؛ اگر پشته خالی باشد، چنین عنصری وجود ندارد. در نهایت، عنصر فعلی را به پشته اضافه می‌کنیم. به عنوان مثال، آرایه زیر را در نظر بگیرید:

1	3	4	2	5	3	4	2
---	---	---	---	---	---	---	---

ابتدا عناصر ۱، ۳ و ۴ به پشته اضافه می‌شوند، زیرا هر عنصر بزرگ‌تر از عنصر قبلی است. بنابراین نزدیک‌ترین عنصر کوچک‌تر به ۴ برابر ۳، و نزدیک‌ترین عنصر کوچک‌تر به ۳ برابر ۱ است.

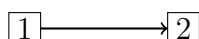
^۱مدت‌ها گمان می‌رفت حل مسئله ۳SUM سریع‌تر از زمان $O(n^2)$ ممکن نیست. اما در سال ۲۰۱۴ مشخص شد [۳۰] که چنین نیست.

1	3	4	2	5	3	4	2
---	---	---	---	---	---	---	---



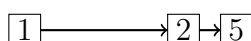
عنصر بعدی ۲ از دو عنصر بالای پشته کوچکتر است. بنابراین، عناصر ۳ و ۴ از پشته حذف می‌شوند و سپس عنصر ۲ به پشته اضافه می‌شود. نزدیک‌ترین عنصر کوچکتر به آن ۱ است:

1	3	4	2	5	3	4	2
---	---	---	---	---	---	---	---



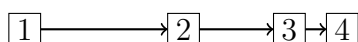
سپس، عنصر ۵ از عنصر ۲ بزرگتر است، پس به پشته اضافه می‌شود و نزدیک‌ترین عنصر کوچکتر به آن ۲ است:

1	3	4	2	5	3	4	2
---	---	---	---	---	---	---	---



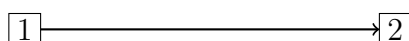
پس از این، عنصر ۵ از پشته حذف می‌شود و عناصر ۳ و ۴ به پشته اضافه می‌شوند:

1	3	4	2	5	3	4	2
---	---	---	---	---	---	---	---



در نهایت، همه عناصر به جز ۱ از پشته حذف می‌شوند و آخرین عنصر ۲ به پشته اضافه می‌شود:

1	3	4	2	5	3	4	2
---	---	---	---	---	---	---	---



کارایی الگوریتم به تعداد کل عملیات‌های پشته وابسته است. اگر عنصر کنونی بزرگتر از عنصر بالای پشته باشد، مستقیماً به پشته اضافه می‌شود که کارآمد است. با این حال، گاهی پشته شامل چندین عنصر بزرگتر است و حذف آن‌ها زمان می‌برد. با این وجود، هر عنصر دقیقاً یک بار به پشته اضافه می‌شود و حداکثر یک بار از پشته حذف می‌گردد. بنابراین، هر عنصر موجب $O(1)$ عملیات پشته می‌شود و الگوریتم در زمان $O(n)$ اجرا می‌شود.

۳.۸ کمینه در پنجره لغزان

یک **پنجره لغزان** زیرآرایه‌ای با اندازه ثابت است که از چپ به راست روی آرایه حرکت می‌کند. در هر موقعیت پنجره، می‌خواهیم اطلاعاتی درباره عناصر درون پنجره محاسبه کنیم. در این بخش، تمرکز روی مسئله نگهداری **کمینه در پنجره لغزان** است، به این معنی که باید کوچک‌ترین مقدار درون هر پنجره را گزارش کنیم.

کمینه در پنجره لغزان را می‌توان با ایده‌ای مشابه محاسبه نزدیک‌ترین عناصر کوچکتر به دست آورد. یک صف نگهداری می‌کنیم که در آن هر عنصر از عنصر قبلی بزرگتر است، و عنصر اول همیشه متناظر با کمینه عنصر درون پنجره است. پس از هر حرکت پنجره، عناصر را از انتهای صف حذف می‌کنیم تا زمانی که آخرین عنصر صف از عنصر جدید پنجره کوچکتر

شود، یا صف خالی گردد. همچنین اگر عنصر اول صف دیگر درون پنجره نباشد، آن را حذف می‌کنیم. در نهایت، عنصر جدید پنجره را به انتهای صف اضافه می‌کنیم. به عنوان مثال، آرایه زیر را در نظر بگیرید:

2	1	4	5	3	4	1	2
---	---	---	---	---	---	---	---

فرض کنید اندازه پنجره لغزان برابر ۴ است. در موقعیت اول پنجره، کوچک‌ترین مقدار ۱ است:

2	1	4	5	3	4	1	2
---	---	---	---	---	---	---	---

1 → 4 → 5

سپس پنجره یک گام به راست حرکت می‌کند. عنصر جدید ۳ از عناصر ۴ و ۵ داخل صف کوچک‌تر است، بنابراین عناصر ۴ و ۵ از صف حذف می‌شوند و عنصر ۳ به صف اضافه می‌شود. کوچک‌ترین مقدار همچنان ۱ است.

2	1	4	5	3	4	1	2
---	---	---	---	---	---	---	---

1 → 3

پس از این، پنجره دوباره حرکت می‌کند و کوچک‌ترین عنصر یعنی ۱ دیگر به پنجره تعلق ندارد. بنابراین، از صف حذف می‌شود و کوچک‌ترین مقدار اکنون ۳ است. همچنین عنصر جدید ۴ به صف افزوده می‌شود.

2	1	4	5	3	4	1	2
---	---	---	---	---	---	---	---

3 → 4

عنصر جدید بعدی ۱ از تمام عناصر صف کوچک‌تر است. بنابراین، تمام عناصر از صف حذف می‌شوند و صف تنها شامل عنصر ۱ خواهد بود:

2	1	4	5	3	4	1	2
---	---	---	---	---	---	---	---

1

در نهایت پنجره به آخرین موقعیت خود می‌رسد. عنصر ۲ به صف اضافه می‌شود، اما کوچک‌ترین مقدار درون پنجره همچنان ۱ است.

2	1	4	5	3	4	1	2
---	---	---	---	---	---	---	---

1 → 2

از آنجا که هر عنصر آرایه دقیقاً یک بار به صف اضافه می‌شود و حداکثر یک بار از صف حذف می‌گردد، این الگوریتم در زمان $O(n)$ عمل می‌کند.

فصل ۹

پرسش‌های بازه‌ای

در این فصل داده‌ساختارهایی بررسی می‌شوند که پردازش کارآمد پرسش‌های بازه‌ای را ممکن می‌سازند. در یک **پرسش بازه‌ای**، هدف محاسبه یک مقدار بر اساس زیرآرایه‌ای از یک آرایه است. پرسش‌های بازه‌ای متداول عبارتند از:

• $\text{sum}_q(a, b)$: محاسبه مجموع مقادیر در بازه $[a, b]$

• $\text{min}_q(a, b)$: یافتن مقدار کمینه در بازه $[a, b]$

• $\text{max}_q(a, b)$: یافتن مقدار بیشینه در بازه $[a, b]$

برای مثال، بازه $[3, 6]$ در آرایه زیر را در نظر بگیرید:

0	1	2	3	4	5	6	7
1	3	8	4	6	1	3	4

در این حالت، $\text{sum}_q(3, 6) = 14$ ، $\text{min}_q(3, 6) = 1$ و $\text{max}_q(3, 6) = 6$ است. روشی ساده برای پردازش پرسش‌های بازه‌ای استفاده از حلقه‌ای است که تمام مقادیر آرایه در بازه را پیمایش می‌کند. برای نمونه، تابع زیر می‌تواند برای پردازش پرسش‌های مجموع روی آرایه استفاده شود:

```
int sum(int a, int b) {
    int s = 0;
    for (int i = a; i <= b; ++i)
        s += array[i];
    return s;
}
```

این تابع در زمان $O(n)$ اجرا می‌شود، که n اندازه آرایه است. بنابراین با این تابع می‌توان q پرسش را در زمان $O(nq)$ پردازش کرد. اما اگر n و q بزرگ باشند، این روش کند است. روش‌هایی وجود دارند که پرسش‌های بازه‌ای را بسیار کارآمدتر پردازش می‌کنند.

۱.۹ پرسش‌های آرایه ایستا

ابتدا حالتی بررسی می‌شود که آرایه ایستا است؛ یعنی مقادیر آرایه هرگز بین پرسش‌ها به‌روزرسانی نمی‌شوند. در این حالت ساختن داده‌ساختاری ایستا که پاسخ هر پرسش ممکن را بدهد کافی است.

پرسش‌های مجموع

پرسش‌های مجموع روی آرایه ایستا با ساخت یک **آرایه مجموع پیشوندی** به سادگی پردازش می‌شوند. هر مقدار در آرایه مجموع پیشوندی برابر با مجموع مقادیر در آرایه اصلی تا آن جایگاه است؛ یعنی مقدار در جایگاه k برابر با $\text{sum}_q(0, k)$ است. آرایه مجموع پیشوندی در زمان $O(n)$ ساخته می‌شود. برای مثال، آرایه زیر را در نظر بگیرید:

0	1	2	3	4	5	6	7
1	3	4	8	6	1	4	2

آرایه مجموع پیشوندی متناظر به صورت زیر است:

0	1	2	3	4	5	6	7
1	4	8	16	22	23	27	29

از آنجا که آرایه مجموع پیشوندی شامل تمام مقادیر $\text{sum}_q(0, k)$ است، می‌توان هر مقدار از $\text{sum}_q(a, b)$ را در زمان $O(1)$ به صورت زیر محاسبه کرد:

$$\text{sum}_q(a, b) = \text{sum}_q(0, b) - \text{sum}_q(0, a - 1)$$

با تعریف $\text{sum}_q(0, -1) = 0$ ، فرمول فوق برای حالت $a = 0$ نیز برقرار است. به عنوان مثال، بازه $[3, 6]$ را در نظر بگیرید:

0	1	2	3	4	5	6	7
1	3	4	8	6	1	4	2

در این حالت $\text{sum}_q(3, 6) = 8 + 6 + 1 + 4 = 19$. این مجموع را می‌توان از دو مقدار آرایه مجموع پیشوندی به دست آورد:

0	1	2	3	4	5	6	7
1	4	8	16	22	23	27	29

$$\text{sum}_q(3, 6) = \text{sum}_q(0, 6) - \text{sum}_q(0, 2) = 27 - 8 = 19$$

بنابراین، همچنین امکان تعمیم این ایده به ابعاد بالاتر وجود دارد. به عنوان مثال، می‌توان یک آرایه مجموع پیشوندی دو بعدی ساخت که برای محاسبه مجموع هر زیرآرایه مستطیلی در زمان $O(1)$ استفاده شود. هر مجموع در چنین آرایه‌ای متناظر با زیرآرایه‌ای است که از گوشه بالا سمت چپ آرایه آغاز می‌شود.

تصویر زیر ایده را نشان می‌دهد:

		D				C			
		B				A			

مجموع زیرآرایه خاکستری با فرمول زیر محاسبه می‌شود:

$$S(A) - S(B) - S(C) + S(D),$$

که در آن $S(X)$ بیانگر مجموع مقادیر در زیرآرایه مستطیلی از گوشه بالا سمت چپ تا موقعیت X است.

پرس‌وجوه‌ای کمی‌ن

پردازش پرس‌وجوهای کمینه دشوارتر از پرس‌وجوهای مجموع است. با این حال، روش پیش‌پردازش ساده‌ای با زمان $O(n \log n)$ وجود دارد که پس از آن می‌توان به هر پرس‌وجوی کمینه در زمان $O(1)$ پاسخ داد.^۱ چون پرس‌وجوهای کمینه و بیشینه یکسان پردازش می‌شوند، روی پرس‌وجوهای کمینه تمرکز می‌کنیم.

ایده: پیش‌محاسبه تمام مقادیر $\min_q(a, b)$ که در آن‌ها $b - a + 1$ (طول بازه) توانی از دو است. برای نمونه، برای آرایه

0	1	2	3	4	5	6	7
1	3	4	8	6	1	4	2

مقادیر زیر محاسبه می‌شوند:

$\min_q(a, b)$	b	a	$\min_q(a, b)$	b	a	$\min_q(a, b)$	b	a
۱	۳	۰	۱	۱	۰	۱	۰	۰
۳	۴	۱	۳	۲	۱	۳	۱	۱
۱	۵	۲	۴	۳	۲	۴	۲	۲
۱	۶	۳	۶	۴	۳	۸	۳	۳
۱	۷	۴	۱	۵	۴	۶	۴	۴
۱	۷	۰	۱	۶	۵	۱	۵	۵
			۲	۷	۶	۴	۶	۶
						۲	۷	۷

تعداد مقادیر پیش‌محاسبه‌شده $O(n \log n)$ است، زیرا $O(\log n)$ طول بازه توانی از دو داریم. این مقادیر با فرمول بازگشتی زیر بهینه محاسبه می‌شوند:

$$\min_q(a, b) = \min(\min_q(a, a + w - 1), \min_q(a + w, b)),$$

^۱ این روش در [۷] معرفی شد و گاهی روش **table sparse** (جدول تنک) نامیده می‌شود. روش‌های پیچیده‌تری نیز وجود دارند [۲۲] که زمان پیش‌پردازش در آن‌ها $O(n)$ است، اما این الگوریتم‌ها در برنامه‌نویسی رقابتی نیاز نیستند.

که در آن $b - a + 1$ توانی از دو و $w = (b - a + 1)/2$ است. محاسبه همه مقادیر زمان $O(n \log n)$ می‌برد.

پس از این، هر مقدار $\min_q(a, b)$ در زمان $O(1)$ به صورت کمینه دو مقدار از پیش محاسبه شده به دست می‌آید. فرض کنید k بزرگ‌ترین توان دو باشد که از $b - a + 1$ بیشتر نیست. می‌توان مقدار $\min_q(a, b)$ را با رابطه زیر محاسبه کرد:

$$\min_q(a, b) = \min(\min_q(a, a + k - 1), \min_q(b - k + 1, b)).$$

در رابطه بالا، بازه $[a, b]$ به صورت اجتماع بازه‌های $[a, a + k - 1]$ و $[b - k + 1, b]$ نشان داده شده است که طول هر دو k است. به عنوان مثال، بازه $[1, 6]$ را در نظر بگیرید:

0	1	2	3	4	5	6	7
1	3	4	8	6	1	4	2

طول این بازه ۶ است، و بزرگ‌ترین توان دو که از ۶ بیشتر نیست ۴ است. بنابراین بازه $[1, 6]$ اجتماع بازه‌های $[1, 4]$ و $[3, 6]$ است:

0	1	2	3	4	5	6	7
1	3	4	8	6	1	4	2

0	1	2	3	4	5	6	7
1	3	4	8	6	1	4	2

از آنجا که $\min_q(1, 4) = 3$ و $\min_q(3, 6) = 1$ نتیجه می‌گیریم $\min_q(1, 6) = 1$.

۲.۹ درخت ایندکس دودویی

یک درخت ایندکس دودویی یا درخت فنویک^۲ را می‌توان نسخه پویای آرایه مجموع پیشوندی دانست. این ساختار داده از دو عملیات با پیچیدگی زمانی $O(\log n)$ روی آرایه پشتیبانی می‌کند: پاسخ به پرسش مجموع بازه‌ای و به‌روزرسانی یک مقدار.

مزیت درخت ایندکس شده دودویی (فنویک) امکان به‌روزرسانی کارآمد مقادیر آرایه بین پرسش‌های مجموع است. این کار با آرایه مجموع پیشوندی ممکن نیست، زیرا پس از هر به‌روزرسانی، ساخت دوباره کل آرایه مجموع پیشوندی در زمان $O(n)$ نیاز است.

ساختار

گرچه نام ساختار درخت ایندکس شده دودویی است، معمولاً به صورت یک آرایه نمایش داده می‌شود. در این بخش فرض می‌کنیم اندیس آرایه‌ها از یک شروع می‌شود، زیرا پیاده‌سازی را ساده‌تر می‌کند.

فرض کنید $p(k)$ بزرگ‌ترین توان دو که k را عادی می‌کند. درخت ایندکس شده دودویی را در آرایه $tree$ ذخیره می‌کنیم به طوری که

$$tree[k] = \text{sum}_q(k - p(k) + 1, k),$$

^۲ساختار داده درخت ایندکس دودویی توسط پی. ام. فنویک در سال ۱۹۹۴ معرفی شد [۲۱].

یعنی هر موقعیت k شامل مجموع مقادیر بازه‌ای از آرایه اصلی با طول $p(k)$ است که در موقعیت k پایان می‌یابد. برای نمونه، چون $p(6) = 2$ است، $\text{tree}[6]$ شامل مقدار $\text{sum}_q(5, 6)$ است.

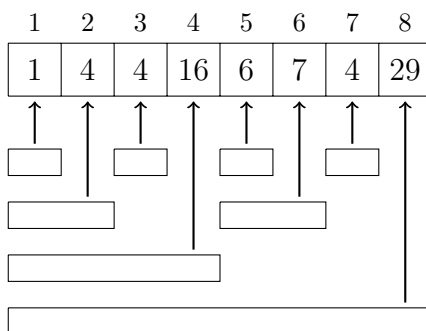
برای نمونه، آرایه زیر را در نظر بگیرید:

1	2	3	4	5	6	7	8
1	3	4	8	6	1	4	2

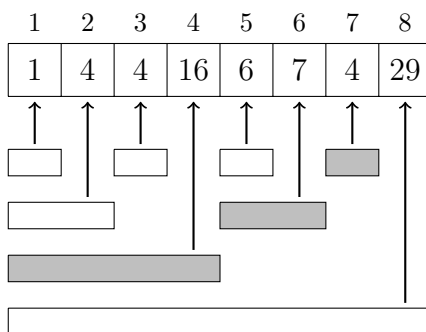
درخت ایندکس‌شده دودویی متناظر به این صورت است:

1	2	3	4	5	6	7	8
1	4	4	16	6	7	4	29

تصویر زیر واضح‌تر نشان می‌دهد که هر مقدار در درخت ایندکس‌شده دودویی چگونه با بازه‌ای در آرایه اصلی متناظر است:



با درخت بندانگشتی دودویی، هر مقدار $\text{sum}_q(1, k)$ در زمان $O(\log n)$ محاسبه می‌شود، زیرا بازه $[1, k]$ همیشه به $O(\log n)$ بازه با مجموع ذخیره‌شده در درخت تقسیم‌پذیر است. برای نمونه، بازه $[1, 7]$ شامل این بازه‌هاست:



بنابراین مجموع متناظر چنین محاسبه می‌شود:

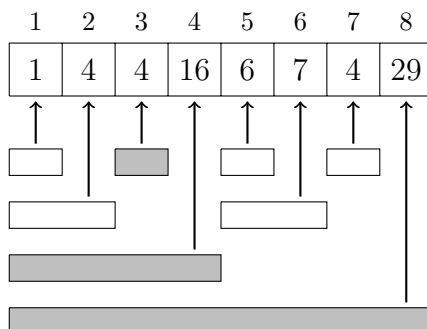
$$\text{sum}_q(1, 7) = \text{sum}_q(1, 4) + \text{sum}_q(5, 6) + \text{sum}_q(7, 7) = 16 + 7 + 4 = 27$$

برای محاسبه $\text{sum}_q(a, b)$ با شرط $a > 1$ ، همان شگرد آرایه‌های مجموع پیشوندی کار می‌کند:

$$\text{sum}_q(a, b) = \text{sum}_q(1, b) - \text{sum}_q(1, a - 1).$$

محاسبه هر دو عبارت $\text{sum}_q(1, a-1)$ و $\text{sum}_q(1, b)$ در زمان $O(\log n)$ انجام می‌شود؛ پیچیدگی زمانی کل $O(\log n)$ است.

سپس، پس از به‌روزرسانی مقداری در آرایه اصلی، چندین مقدار در درخت ایندکس‌شده باینری باید به‌روزرسانی شوند. برای مثال، اگر مقدار در جایگاه ۳ تغییر کند، مجموع بازه‌های زیر تغییر می‌کند:



از آنجا که هر عنصر آرایه به $O(\log n)$ بازه در درخت ایندکس‌شده باینری تعلق دارد، به‌روزرسانی $O(\log n)$ مقدار در درخت کافی است.

پیاده‌سازی

عملیات درخت ایندکس‌شده باینری را می‌توان با استفاده از عملیات بیتی به‌صورت کارا پیاده‌سازی کرد. نکته کلیدی این است که می‌توانیم هر مقدار $p(k)$ را با استفاده از فرمول زیر محاسبه کنیم:

$$p(k) = k \& - k.$$

تابع زیر مقدار $\text{sum}_q(1, k)$ را محاسبه می‌کند:

```
int sum(int k) {
    int s = 0;
    while (k > 0) {
        s += tree[k];
        k -= k & -k;
    }
    return s;
}
```

تابع زیر مقدار آرایه در جایگاه k را به اندازه x افزایش می‌دهد (x می‌تواند مثبت یا منفی باشد):

```
void add(int k, int x) {
    while (k <= n) {
        tree[k] += x;
        k += k & -k;
    }
}
```

پیچیدگی زمانی هر دو تابع $O(\log n)$ است، زیرا توابع به $O(\log n)$ مقدار در درخت ایندکس شده باینری دسترسی دارند، و هر انتقال به جایگاه بعدی به زمان $O(1)$ نیاز دارد.

۳.۹ درخت بازهای

یک **درخت بازهای**^۳ داده ساختاری است که از دو عملیات پشتیبانی می‌کند: پردازش پرس‌وجوی بازهای و به‌روزرسانی مقدار آرایه. درخت‌های بازهای می‌توانند پرس‌و‌جوه‌های مجموع، کمینه، بیشینه و بسیاری پرس‌و‌جوه‌های دیگر را پشتیبانی کنند به طوری که هر دو عملیات در زمان $O(\log n)$ اجرا شوند.

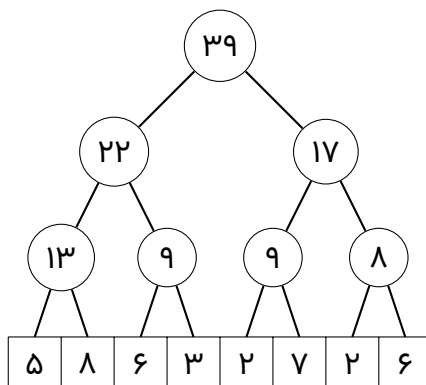
در مقایسه با درخت اندیس‌گذاری شده دودویی، مزیت درخت بازهای عمومی‌تر بودن آن است. در حالی که درخت‌های اندیس‌گذاری شده دودویی فقط از پرس‌وجوی مجموع پشتیبانی می‌کنند^۴، درخت‌های بازهای پرس‌و‌جوه‌های دیگر را نیز پوشش می‌دهند. از سوی دیگر، درخت بازهای حافظه بیشتری مصرف می‌کند و پیاده‌سازی آن کمی دشوارتر است.

ساختار

درخت بازهای یک درخت دودویی است که گره‌های سطح پایینی آن متناظر با عناصر آرایه هستند، و گره‌های دیگر حاوی اطلاعات لازم برای پردازش پرس‌و‌جوه‌های بازهای می‌باشند. در این بخش، فرض می‌کنیم اندازه آرایه توانی از دو است و از اندیس‌گذاری بر پایه صفر استفاده می‌شود، زیرا ساخت درخت بازهای برای چنین آرایه‌ای ساده‌تر است. اگر اندازه آرایه توانی از دو نباشد، همیشه می‌توان عناصر اضافی به آن افزود. ابتدا به بررسی درخت‌های بازهای با پشتیبانی از پرس‌وجوی مجموع می‌پردازیم. به عنوان مثال، آرایه زیر را در نظر بگیرید:

0	1	2	3	4	5	6	7
5	8	6	3	2	7	2	6

درخت بازهای متناظر به صورت زیر است:



هر گره داخلی درخت متناظر با بازهای از آرایه است که اندازه‌اش توانی از دو است. در درخت بالا، مقدار هر گره داخلی برابر با مجموع مقادیر آرایه مربوطه است و می‌توان آن را به صورت مجموع مقادیر گره‌های فرزند چپ و راست محاسبه کرد.

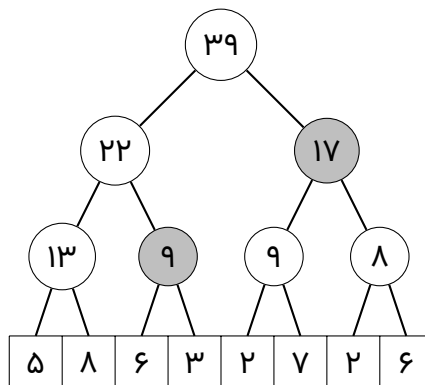
^۳ پیاده‌سازی پایین به بالا در این فصل منطبق بر [۶۲] است. ساختارهای مشابه در اواخر دهه ۱۹۷۰ برای حل مسائل هندسی استفاده شدند [۹].

^۴ در واقع، با استفاده از دو درخت اندیس‌گذاری شده دودویی امکان پشتیبانی از پرس‌وجوی کمینه وجود دارد [۱۶]، ولی این روش پیچیده‌تر از استفاده از یک درخت بازهای است.

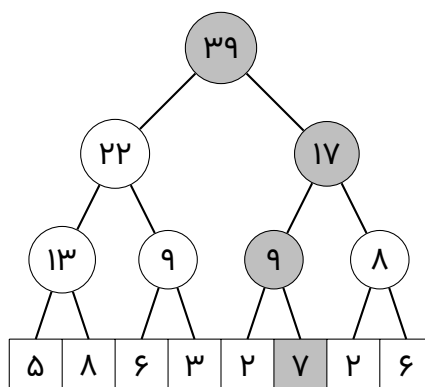
مشخص می‌شود که هر بازه $[a, b]$ را می‌توان به $O(\log n)$ بازه تقسیم کرد که مقادیر آن‌ها در گره‌های درخت ذخیره شده‌اند. برای مثال، بازه $[2, 7]$ را در نظر بگیرید:

0	1	2	3	4	5	6	7
۵	۸	۶	۳	۲	۷	۲	۶

در اینجا $\text{sum}_q(2, 7) = 6 + 3 + 2 + 7 + 2 + 6 = 26$. در این حالت، دو گره زیر درخت متناظر با این بازه هستند:



بنابراین، روش دیگر برای محاسبه مجموع $9 + 17 = 26$ است. وقتی مجموع با استفاده از گره‌های تا حد امکان بالا در درخت محاسبه شود، در هر سطح حداکثر دو گره نیاز است. بنابراین، تعداد کل گره‌ها $O(\log n)$ است. پس از به‌روزرسانی آرایه، باید تمام گره‌هایی که مقدارشان به مقدار به‌روزشده وابسته است را به‌روزرسانی کنیم. این کار با پیمایش مسیر از عنصر به‌روزشده آرایه به گره بالا و به‌روزرسانی گره‌های طول مسیر انجام می‌شود. تصویر زیر نشان می‌دهد در صورت تغییر مقدار آرایه ۷، کدام گره‌های درخت تغییر می‌کنند:

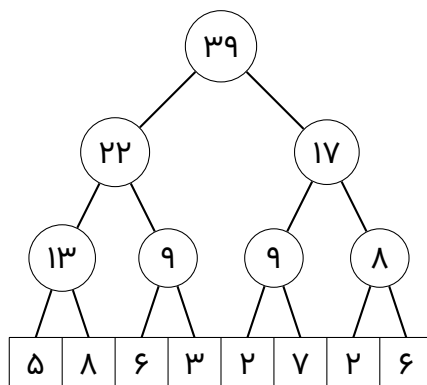


مسیر از پایین به بالا همیشه شامل $O(\log n)$ گره است، بنابراین هر به‌روزرسانی $O(\log n)$ گره در درخت را تغییر می‌دهد.

پی‌اده‌سازی

درخت بازه‌ای را به عنوان یک آرایه با $2n$ عنصر ذخیره می‌کنیم که n اندازه آرایه اصلی و توانی از دو است. گره‌های درخت از بالا به پایین ذخیره می‌شوند: $\text{tree}[1]$ گره ریشه است، $\text{tree}[2]$ و

$tree[3]$ فرزندان آن هستند و به همین ترتیب. در نهایت، مقادیر از $tree[n]$ تا $tree[2n-1]$ متناظر با مقادیر آرایه اصلی در پایین‌ترین سطح درخت هستند. برای نمونه، درخت بازه‌ای



به صورت زیر ذخیره می‌شود:

1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
39	22	17	13	9	9	8	5	8	6	3	2	7	2	6

با این نمایش، والد $tree[k]$ برابر $tree[\lfloor k/2 \rfloor]$ و فرزندان آن $tree[2k]$ و $tree[2k+1]$ هستند. موقعیت گره فرزند چپ زوج و فرزند راست فرد است. تابع زیر مقدار $sum_q(a, b)$ را محاسبه می‌کند:

```

int sum(int a, int b) {
    int n = a + b;
    int s = 0;
    while (b <= a) {
        if (a[tree + 1] == 2*a) ++a;
        if (b[tree + 0] == 2*b) --b;
        a /= 2; b /= 2;
    }
    return s;
}

```

بازه مرحله، هر در سپس است. $[a+n, b+n]$ ابتدا در که می‌دارد نگه را بازه‌ای تابع به ندارند تعلق بالاتر بازه به که گره‌هایی مقادیر آن، از قبل و می‌رود بالا درخت در سطح یک می‌شوند. اضافه مجموع

تابع زیر مقدار آرایه را در موقعیت k به اندازه x افزایش می‌دهد:

```

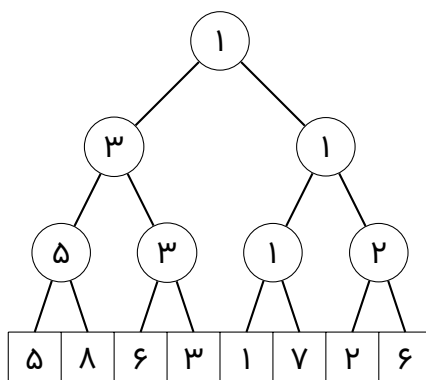
void add(int k, int x) {
    k += k;
    k[tree] += x;
    for (k /= 2; k >= 1; k /= 2) {
        k[tree] = k[2*tree] + k[2*tree + 1];
    }
}

```

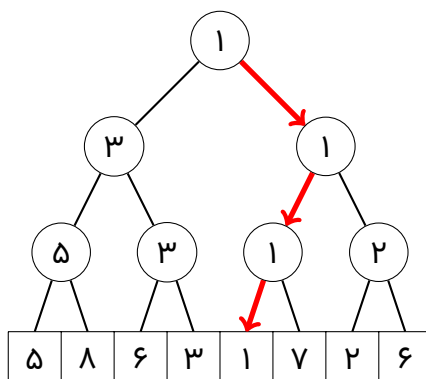
تمام مقادیر سپس، می‌کند. به‌روزرسانی درخت سطح پایین‌ترین در را مقدار تابع ابتدا می‌کند. به‌روز درخت گره بالاترین به رسیدن تا را درخت داخلی گره‌های هر دو تابع بالا در زمان $O(\log n)$ کار می‌کنند، زیرا یک درخت بازه‌ای با n عنصر از $O(\log n)$ سطح تشکیل شده است، و توابع در هر مرحله یک سطح در درخت بالا می‌روند.

سایر پرس‌وجوها

درخت‌های بازه‌ای می‌توانند از تمام پرس‌وجوهای بازه‌ای پشتیبانی کنند که در آن‌ها تقسیم بازه به دو بخش، محاسبه پاسخ به‌طور جداگانه برای هر دو بخش، و سپس ترکیب کارآمد پاسخ‌ها امکان‌پذیر باشد. نمونه‌هایی از این پرس‌وجوها عبارتند از کمینه و بیشینه، بزرگ‌ترین مقسوم‌علیه مشترک، و عملگرهای بیتی، or ، and و xor .
به عنوان مثال، درخت بازه‌ای زیر از پرس‌وجوهای کمینه پشتیبانی می‌کند:



در این حالت، هر گره درخت شامل کوچک‌ترین مقدار در بازه متناظر از آرایه است. بالاترین گره درخت شامل کوچک‌ترین مقدار کل آرایه است. عملیات را می‌توان مشابه قبل پیاده‌سازی کرد، اما به جای مجموع، کمینه‌ها محاسبه می‌شوند.
ساختار درخت بازه‌ای همچنین امکان استفاده از جستجوی دودویی برای یافتن موقعیت عناصر آرایه را فراهم می‌کند. به عنوان مثال، اگر درخت از پرس‌وجوهای کمینه پشتیبانی کند، می‌توان موقعیت عنصری با کمترین مقدار را در زمان $O(\log n)$ پیدا کرد.
به عنوان نمونه، در درخت بالا، عنصری با کمترین مقدار ۱ را می‌توان با پیمایش مسیری رو به پایین از بالاترین گره پیدا کرد:



۴.۹ تکنیک‌های تکمیلی

فشرده‌سازی اندیس‌ها

محدودیت ساختارهای داده مبتنی بر آرایه: اندیس‌ها اعداد صحیح متوالی هستند. اندیس‌های بزرگ مشکل می‌سازند. مثال: برای اندیس 10^9 ، آرایه باید 10^9 عنصر داشته باشد؛ حافظه زیادی هدر می‌رود.

راه‌حل: استفاده از **فشرده‌سازی اندیس‌ها**. جایگزینی اندیس‌های اصلی با اندیس‌های $1, 2, 3$ و غیره. شرط اجرا: آگاهی قبلی از تمام اندیس‌های موردنیاز الگوریتم. ایده: تبدیل هر اندیس اصلی x به $c(x)$ با تابع فشرده‌ساز c . شرط حفظ ترتیب: اگر $a < b$ باشد، باید $c(a) < c(b)$ بماند. پرس‌وجوها روی اندیس‌های فشرده‌شده نیز کار می‌کنند. مثال: برای اندیس‌های اصلی 555 ، 10^9 و 8 ، اندیس‌های جدید:

$$\begin{aligned}c(8) &= 1 \\c(555) &= 2 \\c(10^9) &= 3\end{aligned}$$

به‌روزرسانی‌های بازه‌ای

تا اینجا: ساختارهای داده با پرس‌وجوی بازه‌ای و به‌روزرسانی تک‌عنصری. حالت معکوس: به‌روزرسانی بازه‌ها و دریافت مقدار تک‌عنصرها. عملیات هدف: افزایش تمام عناصر بازه $[a, b]$ به مقدار x .

در کمال شگفتی، داده‌ساختارهای ارائه‌شده در این فصل در این وضعیت نیز قابل استفاده هستند. برای این کار، یک **آرایه تفاضلی** (difference array) می‌سازیم که مقادیر آن نشان‌دهنده تفاضل بین مقادیر متوالی در آرایه اصلی است. بنابراین، آرایه اصلی همان آرایه مجموع پیشوندی آرایه تفاضلی است. برای مثال، آرایه زیر را در نظر بگیرید:

0	1	2	3	4	5	6	7
3	3	1	1	1	5	2	2

آرایه تفاضلی برای آرایه بالا به صورت زیر است:

0	1	2	3	4	5	6	7
3	0	-2	0	0	4	-3	0

برای نمونه، مقدار ۲ در موقعیت ۶ آرایه اصلی متناظر با مجموع $3 - 2 + 4 - 3 = 2$ در آرایه تفاضلی است.

مزیت آرایه تفاضلی این است که می‌توان یک بازه در آرایه اصلی را صرفاً با تغییر دو عنصر در آرایه تفاضلی به‌روزرسانی کرد. به عنوان مثال، اگر بخواهیم مقادیر آرایه اصلی بین موقعیت‌های ۱ تا ۴ را به اندازه ۵ واحد افزایش دهیم، کافی است مقدار آرایه تفاضلی در موقعیت ۱ را ۵ واحد افزایش داده و مقدار در موقعیت ۵ را ۵ واحد کاهش دهیم. نتیجه به صورت زیر خواهد بود:

0	1	2	3	4	5	6	7
3	5	-2	0	0	-1	-3	0

به طور کلی‌تر، برای افزایش مقادیر در بازه $[a, b]$ به اندازه x ، مقدار در موقعیت a را x واحد افزایش داده و مقدار در موقعیت $b + 1$ را x واحد کاهش می‌دهیم. بدین ترتیب تنها نیاز به به‌روزرسانی مقادیر تکی و پردازش پرس‌وجوهای مجموع است، بنابراین می‌توان از درخت اندیس‌گذاری‌شده دودویی یا درخت بازه‌ای استفاده کرد.

مسئله دشوارتر، پشتیبانی هم‌زمان از پرس‌وجوهای بازه‌ای و به‌روزرسانی‌های بازه‌ای است. در فصل ۲۸ خواهیم دید که حتی این کار نیز امکان‌پذیر است.

در نمایش بدون علامت، فقط اعداد نامنفی ذخیره می‌شوند، اما کران بالای مقادیر بزرگ‌تر است. متغیر بدون علامت n بیتی می‌تواند مقادیر صحیح بین 0 تا $2^n - 1$ را نگه دارد. به عنوان مثال در C++، متغیر نوع `int unsigned` مقادیر بین 0 تا $2^{32} - 1$ را پوشش می‌دهد. بین این دو نمایش رابطه ریاضی برقرار است: عدد علامت‌دار $-x$ برابر با عدد بدون علامت $2^n - x$ است. کد زیر نشان می‌دهد عدد علامت‌دار $x = -43$ برابر عدد بدون علامت $y = 2^{32} - 43$ است:

```
int x = -43;
unsigned int y = x;
cout << x << "n\" << endl; // -43
cout << y << "n\" << endl; // 4294967253
```

اگر عددی از کران بالای نمایش بیتی فراتر رود، سرریز رخ می‌دهد. در نمایش علامت‌دار، عدد بعد از $2^{n-1} - 1$ برابر با -2^{n-1} است و در نمایش بدون علامت، عدد بعد از $2^n - 1$ برابر با 0 خواهد بود. کد زیر این رفتار را نشان می‌دهد:

```
int x = 2147483647;
cout << x << "n\" << endl; // 2147483647
x++;
cout << x << "n\" << endl; // -2147483648
```

در ابتدا، مقدار x برابر $2^{31} - 1$ است. این بزرگ‌ترین مقداری است که می‌توان در یک متغیر `int` ذخیره کرد، بنابراین عدد بعدی پس از $2^{31} - 1$ برابر -2^{31} است.

۲.۱۰ عملیات بیتی

عملگر And

عملگر **and** یعنی x & y عددی تولید می‌کند که در جایگاه‌هایی دارای بیت یک است که هر دو عدد x و y در آن جایگاه‌ها بیت یک داشته باشند. به عنوان مثال، $26 \& 22 = 18$ ، زیرا

$$\begin{array}{r} (22) \quad 10110 \\ (26) \quad 11010 \quad \& \\ \hline (18) \quad 10010 \quad = \end{array}$$

با استفاده از عملگر **and** می‌توان بررسی کرد که آیا عدد x زوج است یا خیر؛ چرا که اگر x زوج باشد $x \& 1 = 0$ و اگر x فرد باشد $x \& 1 = 1$ می‌شود. به طور کلی‌تر، x بر 2^k بخش‌پذیر است دقیقاً زمانی که $x \& (2^k - 1) = 0$ باشد.

عملگر Or

عملگر **or** یعنی $x | y$ عددی تولید می‌کند که در جایگاه‌هایی دارای بیت یک است که حداقل یکی از مقادیر x یا y در آن جایگاه‌ها بیت یک داشته باشد. به عنوان مثال، $26 | 22 = 30$ ، زیرا

پیاده‌سازی مجموعه

کد زیر متغیر `int` به نام x تعریف می‌کند که می‌تواند زیرمجموعه‌ای از $\{0, 1, 2, \dots, 31\}$ نگه دارد. سپس اعضای ۱، ۳، ۴ و ۸ را به مجموعه اضافه کرده و اندازه مجموعه را چاپ می‌کند.

```
int x = 0;
x |= 1;
x |= 3;
x |= 4;
x |= 8;
cout << x(__builtin_popcount) << "\n"; // 4
```

می‌کند: چاپ را مجموعه به متعلق اعضای تمام زیر کد سپس،

```
for (int i = 0; i < 32; ++i)
    if (x & (1 << i))
        cout << i << " ";
// output: 1 3 4 8
```

عملیات روی مجموعه‌ها

عملیات روی مجموعه‌ها با عملگرهای بیتی پیاده‌سازی می‌شوند:

نحو مجموعه‌ای	نحو بیتی	
$a \cap b$	$b \& a$	اشتراک
$a \cup b$	$b a$	اجتماع
$\bar{a}$	$a \sim$	متمم
$a \setminus b$	$(b \sim) \& a$	تفاضل

برای نمونه، کد زیر ابتدا مجموعه‌های $x = \{1, 3, 4, 8\}$ و $y = \{3, 6, 8, 9\}$ و سپس مجموعه $z = x \cup y = \{1, 3, 4, 6, 8, 9\}$ را می‌سازد:

```
int x = 1 | (1 << 3) | (1 << 1) | (1 << 4);
int y = 1 | (1 << 3) | (1 << 6) | (1 << 8) | (1 << 9);
int z = x | y;
cout << z(__builtin_popcount) << "\n"; // 6
```

پیمایش زیرمجموعه‌ها

کد زیر زیرمجموعه‌های $\{0, 1, \dots, n-1\}$ را پیمایش می‌کند:

```
for (int b = 0; b < (1 << n); ++b)
    process_subset(b);
```

می‌کند: پیمایش را عضو k دارای دقیقاً زیرمجموعه‌های زیر کد

```

        { ++b ); n << (1 < b 0; = b int( for
    { )k == )b( __builtin_popcount( if
        b subset process //
    }
}

```

می‌کند: پیمایش را x مجموعه زیرمجموعه‌های زیر کد

```

0; = b int
    { do
        b subset process //
    );x)&x-b=(b( while }

```

۴.۱۰ بهینه‌سازی‌های بیتی

بسیاری الگوریتم‌ها با عملیات بیتی بهینه می‌شوند. این بهینه‌سازی‌ها پیچیدگی زمانی الگوریتم را تغییر نمی‌دهند، ولی بر زمان اجرای واقعی کد اثر زیاد دارند. در این بخش مثال‌های این موضوع بررسی می‌شوند.

فاصله هم‌ی‌نگ

فاصله هم‌ی‌نگ $\text{hamming}(a, b)$ بین دو رشته هم‌طول a و b ، تعداد جایگاه‌هایی است که دو رشته در آن اختلاف دارند. برای نمونه:

$$\text{hamming}(01101, 11001) = 2.$$

مسئله: با داشتن فهرستی شامل n رشته بیتی با طول k ، کمینه فاصله هم‌ی‌نگ بین دو رشته از فهرست محاسبه شود. برای نمونه، پاسخ برای $[00111, 01101, 11110]$ برابر ۲ است، زیرا:

$$\begin{aligned} \text{hamming}(00111, 01101) &= 2 \\ \text{hamming}(00111, 11110) &= 3 \\ \text{hamming}(01101, 11110) &= 3 \end{aligned}$$

یک روش سراسر برای حل مسئله، بررسی تمام جفت‌های رشته‌ها و محاسبه فاصله هم‌ی‌نگ آن‌ها است که الگوریتمی با زمان $O(n^2k)$ به دست می‌دهد. تابع زیر می‌تواند برای محاسبه فاصله‌ها استفاده شود:

```

    { )b string ,a string(hamming int
        0; = d int
    { ++i ;k < i 0; = i int( for
        ++;d ])i[b != ]i[a( if
    }
    ;d return
}

```


با این حال، اگر k کوچک باشد، می‌توان با ذخیره رشته‌های بیتی به صورت اعداد صحیح و محاسبه فواصل همینگ با استفاده از عملیات بیتی، کد را بهینه‌سازی کرد. به‌ویژه اگر $k \leq 32$ باشد، می‌توان رشته‌ها را صرفاً به صورت مقادیر `int` ذخیره کرد و از تابع زیر برای محاسبه فاصله‌ها بهره برد:

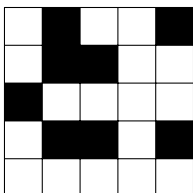
```
int hamming(int a, int b) {
    return a ^ b;
}
```

دارند، تفاوت a و b که موقعیت‌هایی در که می‌سازد بیتی رشته یک XOR عملگر بالا، تابع در محاسبه `__builtin_popcount` تابع از استفاده با بیت‌ها تعداد سپس، دارد. یک بیت می‌شود.

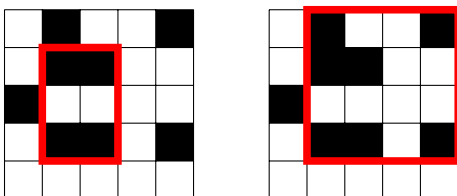
برای مقایسه پیاده‌سازی‌ها، فهرستی شامل ۱۰۰۰۰ رشته بیتی تصادفی به طول ۳۰ تولید شد. با روش اول، جستجو ۵.۱۳ ثانیه طول کشید، و پس از بهینه‌سازی بیتی، فقط ۵.۰ ثانیه زمان برد. بنابراین، کد بهینه‌شده با بیت تقریباً ۳۰ برابر سریع‌تر از کد اولیه بود.

شمارش زیرشبکه‌ها

به عنوان مثالی دیگر، مسئله زیر را در نظر بگیرید: یک شبکه $n \times n$ داده شده است که هر خانه آن سیاه (۱) یا سفید (۰) است. تعداد زیرشبکه‌هایی را محاسبه کنید که تمام گوشه‌های آن‌ها سیاه هستند. برای نمونه، شبکه



شامل دو زیرشبکه از این نوع است:



الگوریتمی با زمان $O(n^3)$ برای حل مسئله وجود دارد: تمام $O(n^2)$ جفت سطر را پیمایش کنید و برای هر جفت (a, b) ، تعداد ستون‌هایی را که در هر دو سطر دارای خانه سیاه هستند در زمان $O(n)$ محاسبه کنید. کد زیر فرض می‌کند که `color[y][x]` نشان‌دهنده رنگ در سطر y و ستون x است:

```
int count = 0;
for (int i = 0; i < n; ++i)
    for (int j = 0; j < n; ++j)
        if (color[i][j] == 1)
            count++;
```

هستند، سیاه گوشه‌های با زیرشبکه $(count - 1)/2$ شامل ستون‌ها این سپس، کرد. انتخاب زیرشبکه یک تشکیل برای آن‌ها از ستون دو هر می‌توان زیرا برای بهینه‌سازی این الگوریتم، شبکه را به بلوک‌هایی از ستون‌ها تقسیم می‌کنیم به طوری که هر بلوک شامل N ستون متوالی باشد. سپس، هر سطر به صورت فهرستی از اعداد N بیتی ذخیره می‌شود که رنگ‌های خانه‌ها را توصیف می‌کنند. اکنون می‌توانیم N ستون را به طور همزمان با استفاده از عملیات بیتی پردازش کنیم. در کد زیر، $color[y][k]$ نشان‌دهنده یک بلوک از N رنگ به صورت بیت است.

```

0; = count int
{ ++)i ;N/n <= i 0; = i int( for
]);i [b[color]&i] [a[color(__builtin_popcount += count
}
```

می‌کند. کار $O(n^3/N)$ زمان در حاصل الگوریتم ما یک شبکه تصادفی به ابعاد 2500×2500 تولید کردیم و پیاده‌سازی اولیه را با نسخه بهینه‌شده بیتی مقایسه نمودیم. در حالی که کد اولیه 29.6 ثانیه زمان برد، نسخه بهینه‌شده با بیت با $N = 32$ (اعداد int) تنها 3.1 ثانیه و با $N = 64$ (اعداد long long) حدود 1.7 ثانیه زمان برد.

۵.۱۰ برنامه‌نویسی پویا

عملیات بیتی روشی کارآمد و مناسب برای پیاده‌سازی الگوریتم‌های برنامه‌نویسی پویا ارائه می‌دهند که وضعیت‌های آن‌ها شامل زیرمجموعه‌هایی از عناصر است، زیرا چنین وضعیت‌هایی را می‌توان به صورت اعداد صحیح ذخیره کرد. در ادامه به بررسی مثال‌هایی از ترکیب عملیات بیتی و برنامه‌نویسی پویا می‌پردازیم.

انتخاب بهینه

به عنوان مثال اول، مسئله زیر را در نظر بگیرید: قیمت‌های k محصول در طول n روز داده شده است و می‌خواهیم هر محصول را دقیقاً یک بار بخریم. با این حال، در هر روز مجاز به خرید حداکثر یک محصول هستیم. حداقل قیمت کل چقدر است؟ به عنوان مثال، سناریوی زیر را در نظر بگیرید ($k = 3$ و $n = 8$):

	۰	۱	۲	۳	۴	۵	۶	۷
محصول ۰	۶	۹	۵	۲	۸	۹	۱	۶
محصول ۱	۸	۲	۶	۲	۷	۵	۷	۲
محصول ۲	۵	۳	۹	۷	۳	۵	۱	۴

در این سناریو، کمینه قیمت کل برابر 5 است:

	۰	۱	۲	۳	۴	۵	۶	۷
محصول ۰	۶	۹	۵	۲	۸	۹	۱	۶
محصول ۱	۸	۲	۶	۲	۷	۵	۷	۲
محصول ۲	۵	۳	۹	۷	۳	۵	۱	۴

فرض کنید $price[x][d]$ بیانگر قیمت محصول x در روز d باشد. برای مثال، در سناریوی بالا $price[2][3] = 7$ است. سپس، فرض کنید $total(S, d)$ کمینه قیمت کل برای خرید زیرمجموعه S از محصولات تا روز d باشد. با استفاده از این تابع، پاسخ مسئله $k-0 \dots$ $total(\{0 \dots k-1\}, n-1)$ خواهد بود.

ابتدا، $total(\emptyset, d) = 0$ ، زیرا خرید یک مجموعه تهی هزینه‌ای ندارد، و $total(\{x\}, 0) = price[x][0]$ ، زیرا برای خرید یک محصول در روز اول تنها یک راه وجود دارد. سپس، رابطه بازگشتی زیر قابل استفاده است:

$$total(S, d) = \min(total(S, d-1), \min_{x \in S} (total(S \setminus x, d-1) + price[x][d]))$$

این بدان معناست که یا در روز d هیچ محصولی نمی‌خریم یا محصول x متعلق به S را خریداری می‌کنیم. در حالت دوم، x را از S حذف کرده و قیمت x را به قیمت کل اضافه می‌کنیم. گام بعدی محاسبه مقادیر تابع با استفاده از برنامه‌نویسی پویا است. برای ذخیره مقادیر تابع، آرایه‌ای تعریف می‌کنیم:

```
int total[K][N];
```

نمایش با متناظر آرایه اول بعد هستند. بزرگ کافی اندازه به ثابت‌های N و K آن در که است. زیرمجموعه یک بیتی

ابتدا، حالت‌های $d = 0$ این‌گونه پردازش می‌شوند:

```
for (int x = 0; x < K; ++x)
    total[x][0] = price[x][0];
```

می‌شود: تبدیل زیر کد به بازگشتی رابطه سپس،

```
for (int d = 1; d < N; ++d)
    for (int s = 0; s < K; ++s)
        total[s][d] = total[s][d-1];
        for (int x = 0; x < K; ++x)
            if (total[s][d-1] < total[s][d-1] + price[x][d])
                total[s][d] = total[s][d-1] + price[x][d];
```

است. $O(n2^k k)$ الگوریتم زمانی پیچیدگی

از جای‌گشت‌ها به زیرمجموعه‌ها

با برنامه‌نویسی پویا، پیمایش روی جایگشت‌ها را اغلب می‌توان به پیمایش روی زیرمجموعه‌ها تبدیل کرد^۱. علت: تعداد جایگشت‌ها ($n!$) بسیار بزرگ‌تر از تعداد زیرمجموعه‌ها (2^n) است.

^۱ این روش سال ۱۹۶۲ توسط ام. هلد (M. Held) و آر. ام. کarp (R. M. Karp) معرفی شد [۳۴].

مثال: برای $n = 20$ مقدار $n! \approx 2.4 \cdot 10^{18}$ ، اما $2^n \approx 10^6$. بنابراین برای برخی مقادیر n ، پیمایش زیرمجموعه‌ها کارا است اما پیمایش جایگشت‌ها غیرممکن است.

مثال: آسانسوری با ظرفیت وزن x وجود دارد. n نفر با وزن‌های مشخص می‌خواهند از همکف به بالاترین طبقه بروند. حداقل تعداد دفعات حرکت آسانسور چقدر است اگر افراد با ترتیب بهینه سوار شوند؟

فرض کنید $x = 10$ و $n = 5$ و وزن‌ها چنین باشند:

وزن	شخص
۲	۰
۳	۱
۳	۲
۵	۳
۶	۴

اینجا حداقل دفعات حرکت ۲ است. یک ترتیب بهینه $\{0, 2, 3, 1, 4\}$ است که افراد را به دو حرکت تفکیک می‌کند: ابتدا $\{0, 2, 3\}$ (مجموع وزن ۱۰)، سپس $\{1, 4\}$ (مجموع وزن ۹). مسئله با بررسی همه جایگشت‌های n نفر در زمان $O(n!n)$ قابل حل است. اما با برنامه‌نویسی پویا الگوریتمی در زمان $O(2^n n)$ به دست می‌آید. ایده: برای هر زیرمجموعه از افراد دو مقدار محاسبه شود: حداقل دفعات حرکت لازم و حداقل وزن افراد در آخرین گروه. فرض کنید $weight[p]$ وزن شخص p باشد. دو تابع تعریف می‌کنیم: $rides(S)$ حداقل دفعات حرکت برای زیرمجموعه S ، و $last(S)$ حداقل وزن افراد در حرکت آخر. برای نمونه در سناریوی بالا:

$$rides(\{1, 3, 4\}) = 2x! \quad \text{و} \quad last(\{1, 3, 4\}) = 5,$$

زیرا حرکت‌های بهینه $\{1, 4\}$ و $\{3\}$ هستند، و حرکت دوم ۵ واحد وزن دارد. هدف نهایی محاسبه $rides(\{0 \dots n-1\})$ است.

می‌توانیم مقادیر تابع‌ها را به صورت بازگشتی محاسبه و سپس از برنامه‌نویسی پویا استفاده کنیم. ایده کار، بررسی تمام افراد متعلق به S و انتخاب بهینه آخرین فرد p است که وارد آسانسور می‌شود. هر انتخاب منجر به یک زیرمسئله برای زیرمجموعه کوچکتری از افراد می‌شود. اگر $last(S \setminus p) + weight[p] \leq x$ باشد، می‌توانیم p را به آخرین سفر اضافه کنیم. در غیر این صورت، باید سفر جدیدی رزرو کنیم که در ابتدا فقط شامل p است. برای پیاده‌سازی برنامه‌نویسی پویا، آرایه‌ای تعریف می‌کنیم:

```
int, int <pair> best[1<N];
```

مقدار می‌کند. نگهداری $(rides(S), last(S))$ جفت یک S ، زیرمجموعه هر برای که می‌کنیم: اولیه مقداردهی این‌گونه را خالی گروه به مربوط

```
best[0] = {1, 0};
```

می‌کنیم: پر شکل این به آرایه سپس،

```
for (int s = 1; s < 1<N; ++s) {
    int initial = 0;
    for (int p = 0; p < s; ++p) {
        if (last[s] + weight[p] <= x) {
            initial = min(initial, rides[s] + 1);
        }
    }
    best[s] = {initial, last[s]};
}
```

```

{ }x <= ]p[weight+second.option( if
ride existing an to p add //
];p[weight += second.option
{ else }
p for ride new a reserve //
++;first.option
];p[weight = second.option
}
);option ,]s[best(min = ]s[best
}
}
}

```

زیرمجموعه باشد، $S_1 \subset S_2$ که S_1 و S_2 زیرمجموعه دو هر برای می‌کند تضمین بالا حلقه محاسبه صحیح ترتیب با پویا برنامه‌نویسی مقادیر بنابراین، می‌شود. پردازش S_2 از قبل S_1 می‌شوند.

شمارش زیرمجموعه‌ها

آخرین مسئله این فصل به شرح زیر است: فرض کنید $X = \{0 \dots n-1\}$ باشد و به هر زیرمجموعه $S \subset X$ یک عدد صحیح $\text{value}[S]$ نسبت داده شده باشد. هدف ما محاسبه مقدار زیر برای هر S است:

$$\text{sum}(S) = \sum_{A \subset S} \text{value}[A],$$

یعنی مجموع مقادیر زیرمجموعه‌های S .

به عنوان مثال، فرض کنید $n = 3$ باشد و مقادیر به صورت زیر داده شده باشند:

$$\begin{array}{ll}
\text{value}[\{2\}] = 5 & \cdot \quad \text{value}[\emptyset] = 3 \\
\text{value}[\{0, 2\}] = 1 & \cdot \quad \text{value}[\{0\}] = 1 \\
\text{value}[\{1, 2\}] = 3 & \cdot \quad \text{value}[\{1\}] = 4 \\
\text{value}[\{0, 1, 2\}] = 3 & \cdot \quad \text{value}[\{0, 1\}] = 5
\end{array}$$

در این حالت، برای نمونه:

$$\begin{aligned}
\text{sum}(\{0, 2\}) &= \text{value}[\emptyset] + \text{value}[\{0\}] + \text{value}[\{2\}] + \text{value}[\{0, 2\}] \\
&= 3 + 1 + 5 + 1 = 10.
\end{aligned}$$

به دلیل وجود 2^n زیرمجموعه، یک راه‌حل ممکن بررسی تمام جفت‌های زیرمجموعه در زمان $O(2^{2n})$ است. اما با برنامه‌نویسی پویا، می‌توان مسئله را در زمان $O(2^n n)$ حل کرد. ایده این است که روی مجموعه‌هایی تمرکز کنیم که اعضای قابل حذف از S در آن‌ها محدود شده‌اند.

فرض کنید $\text{partial}(S, k)$ برابر با مجموع مقادیر زیرمجموعه‌هایی از S باشد، با این شرط که تنها عناصر $0 \dots k$ اجازه دارند از S حذف شوند. برای مثال،

$$\text{partial}(\{0, 2\}, 1) = \text{value}[\{2\}] + \text{value}[\{0, 2\}],$$

زیرا تنها اجازه حذف عناصر $0 \dots 1$ وجود دارد. می‌توان مقادیر `sum` را با استفاده از مقادیر `partial` محاسبه کرد، زیرا

$$\text{sum}(S) = \text{partial}(S, n - 1).$$

حالت‌های پایه برای این تابع عبارتند از

$$\text{partial}(S, -1) = \text{value}[S],$$

زیرا در این حالت هیچ عنصری نمی‌تواند از S حذف شود. سپس، در حالت کلی می‌توان از رابطه بازگشتی زیر استفاده کرد:

$$\text{partial}(S, k) = \begin{cases} \text{partial}(S, k - 1) & k \notin S \\ \text{partial}(S, k - 1) + \text{partial}(S \setminus \{k\}, k - 1) & k \in S \end{cases}$$

در اینجا تمرکز روی عنصر k است. اگر $k \in S$ باشد، دو انتخاب وجود دارد: می‌توان k را در S نگه داشت یا آن را از S حذف کرد.

روش بسیار هوشمندانه‌ای برای پیاده‌سازی محاسبه مجموعه‌ها وجود دارد. می‌توان آرایه‌ای تعریف کرد:

```
int sum[1<N];
```

زیر شکل به آرایه اولیه مقداردهی کرد. خواهد نگه‌داری را زیرمجموعه هر مجموعه که می‌شود: انجام

```
for (int s = 0; s < 1<N; ++s) {
    sum[s] = value[s];
}
```

کرد: پر صورت این به را آرایه می‌توان سپس،

```
for (int k = 0; k < 1<N; ++k) {
    for (int s = 0; s < 1<N; ++s) {
        if (s & (1<N - 1 - k))
            sum[s] += value[s ^ (1<N - 1 - k)];
    }
}
```

می‌کند. محاسبه `sum` آرایه درون $k = 0 \dots n - 1$ ازای به را $\text{partial}(S, k)$ مقادیر کد این آرایه همان از می‌توان است، $\text{partial}(S, k - 1)$ پایه بر همواره $\text{partial}(S, k)$ که آنجا از می‌دهد. دست به را کارآمدی بسیار پیاده‌سازی که امری کرد؛ استفاده مجدداً `sum`

بخش دوم

الگوریتم‌های گراف

فصل ۱۱

مبانی گراف

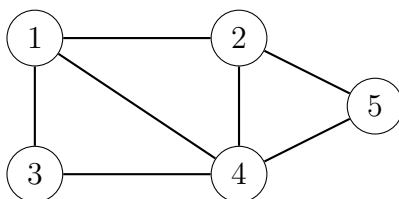
بسیاری از مسائل برنامه‌نویسی با مدل‌سازی مسئله به عنوان یک مسئله گراف و استفاده از الگوریتم گرافی مناسب حل می‌شوند. یک مثال متداول از گراف، شبکه جاده‌ها و شهرهای یک کشور است. با این حال، گاهی گراف درون مسئله پنهان است و تشخیص آن دشوار خواهد بود.

این بخش از کتاب به بررسی الگوریتم‌های گراف می‌پردازد و تمرکز ویژه‌ای بر مباحث مهم در برنامه‌نویسی رقابتی دارد. در این فصل، مفاهیم مربوط به گراف‌ها را مرور کرده و روش‌های مختلف نمایش گراف در الگوریتم‌ها را بررسی می‌کنیم.

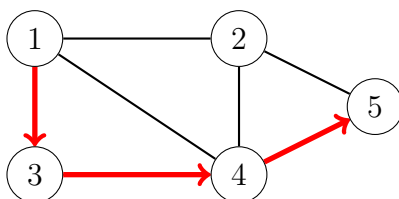
۱.۱۱ اصطلاحات گراف

یک **گراف** شامل **راس‌ها** (گره‌ها) و **یال‌ها** است. در این کتاب، متغیر n بیانگر تعداد راس‌ها در گراف و متغیر m نشان‌دهنده تعداد یال‌ها است. راس‌ها با اعداد صحیح $1, 2, \dots, n$ شماره‌گذاری می‌شوند.

برای نمونه، گراف زیر دارای ۵ راس و ۷ یال است:



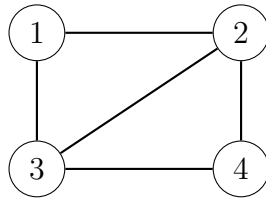
یک **مسیر** از راس a به راس b از طریق یال‌های گراف حرکت می‌کند. **طول** مسیر برابر با تعداد یال‌های موجود در آن است. برای نمونه، گراف بالا شامل یک مسیر $1 \rightarrow 3 \rightarrow 4 \rightarrow 5$ با طول ۳ از راس ۱ به راس ۵ است:



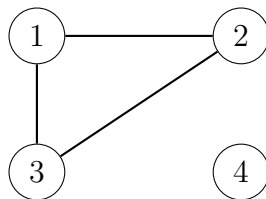
یک مسیر در صورتی یک **دور** است که راس اول و آخر آن یکسان باشد. برای نمونه، گراف بالا شامل دور $1 \rightarrow 3 \rightarrow 4 \rightarrow 1$ است. یک مسیر در صورتی **ساده** است که هر راس حداکثر یک بار در مسیر ظاهر شود.

همبندی

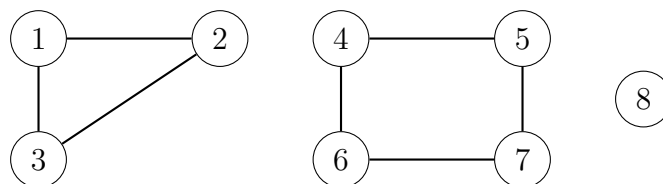
گراف همبند است اگر مسیری بین هر دو رأس آن وجود داشته باشد. برای نمونه، گراف زیر همبند است:



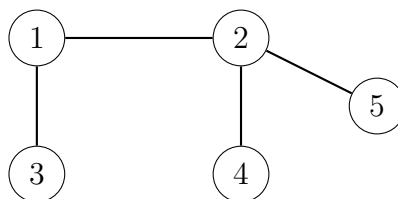
گراف زیر ناهمبند است، زیرا رفتن از رأس ۴ به هر رأس دیگری ممکن نیست:



بخش‌های همبند گراف مؤلفه‌های آن نامیده می‌شوند. برای نمونه، گراف زیر سه مؤلفه دارد: $\{1, 2, 3\}$ ، $\{4, 5, 6, 7\}$ و $\{8\}$.

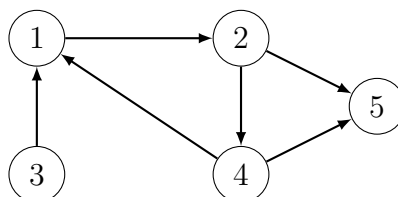


درخت گرافی همبند با n رأس و $n - 1$ یال است. بین هر دو رأس درخت دقیقاً یک مسیر یکتا وجود دارد. برای نمونه، گراف زیر یک درخت است:



جهت‌یال‌ها

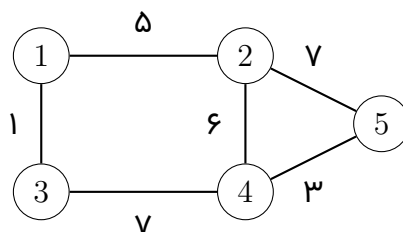
یک گراف جهت‌دار است اگر یال‌ها تنها در یک جهت قابل پیمایش باشند. برای مثال، گراف زیر جهت‌دار است:



گراف بالا شامل یک مسیر $5 \rightarrow 2 \rightarrow 1 \rightarrow 3$ از گره ۵ به گره ۳ است، اما هیچ مسیری از گره ۳ به گره ۵ وجود ندارد.

وزن یال‌ها

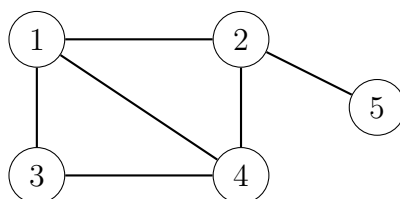
در یک گراف وزن‌دار، به هر یال یک **وزن** نسبت داده می‌شود. وزن‌ها اغلب به عنوان طول یال تعبیر می‌شوند. برای مثال، گراف زیر وزن‌دار است:



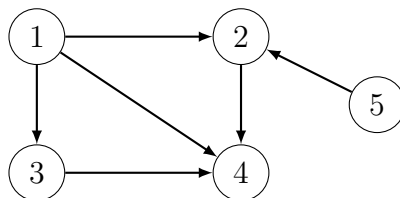
طول یک مسیر در گراف وزن‌دار، مجموع وزن یال‌های آن مسیر است. برای مثال، در گراف بالا، طول مسیر $1 \rightarrow 2 \rightarrow 5$ برابر 12 است، و طول مسیر $1 \rightarrow 3 \rightarrow 4 \rightarrow 5$ برابر 11 است. مسیر دوم، **کوتاه‌ترین** مسیر از گره 1 به گره 5 است.

همسایه‌ها و درجه‌ها

دو گره **همسایه** یا **مجاور** هستند اگر یالی بین آن‌ها وجود داشته باشد. **درجه** یک گره تعداد همسایه‌های آن است. برای مثال، در گراف زیر، همسایه‌های گره ۲ عبارت‌اند از ۱، ۴ و ۵، بنابراین درجه آن ۳ است.



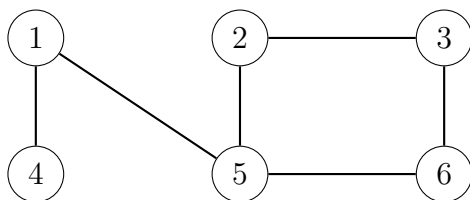
مجموع درجات در گراف همیشه $2m$ است؛ m تعداد یال‌ها است، زیرا هر یال درجه دقیقاً دو رأس را یک واحد افزایش می‌دهد. به همین دلیل، مجموع درجات همیشه زوج است. گراف **منتظم** است اگر درجه هر رأس مقدار ثابت d باشد. گراف **کامل** است اگر درجه هر رأس $n - 1$ باشد؛ یعنی گراف شامل تمام یال‌های ممکن بین رأس‌ها است. در گراف جهت‌دار، **درجه ورودی** یک رأس برابر با تعداد یال‌هایی است که به آن رأس ختم می‌شوند، و **درجه خروجی** یک رأس برابر با تعداد یال‌هایی است که از آن رأس آغاز می‌شوند. برای نمونه، در گراف زیر، درجه ورودی رأس ۲ برابر ۲، و درجه خروجی رأس ۲ برابر ۱ است.



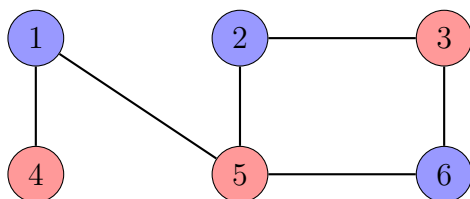
رنگ‌آمیزی‌ها

در **رنگ‌آمیزی** گراف، به هر رأس رنگی تخصیص داده می‌شود به گونه‌ای که هیچ دو رأس مجاوری رنگ یکسان نداشته باشند.

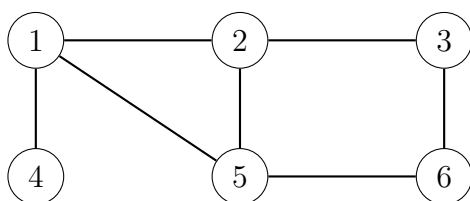
یک گراف **دوبخشی** (یا دوپاره) است اگر بتوان آن را با دو رنگ، رنگ‌آمیزی کرد. مشخص شده است که یک گراف دوبخشی است دقیقاً زمانی که هیچ دوری با تعداد یال‌های فرد نداشته باشد. برای مثال، گراف



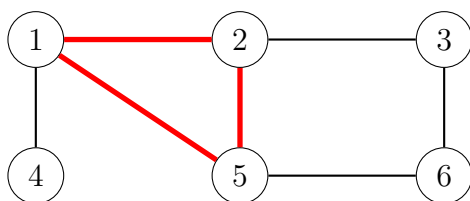
دوبخشی است، زیرا می‌توان آن را به شکل زیر رنگ‌آمیزی کرد:



اما گراف زیر

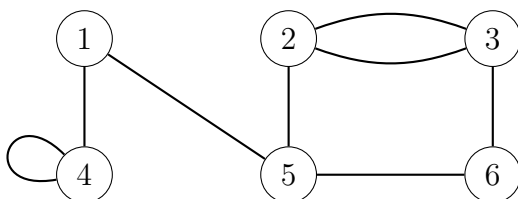


دوبخشی نیست، زیرا نمی‌توان دور سه رأسی زیر را با دو رنگ، رنگ‌آمیزی کرد:



ساده بودن

یک گراف **ساده** است اگر هیچ یالی در یک رأس شروع و پایان نیابد، و هیچ یال چندگانه‌ای بین دو رأس وجود نداشته باشد. اغلب فرض می‌کنیم گراف‌ها ساده هستند. به عنوان مثال، گراف زیر ساده نیست:



۲.۱۱ نمایش گراف

روش‌های متعددی برای نمایش گراف‌ها در الگوریتم‌ها وجود دارد. انتخاب ساختمان داده به اندازه گراف و نحوه پردازش آن توسط الگوریتم بستگی دارد. در ادامه سه نمایش متداول را بررسی می‌کنیم.

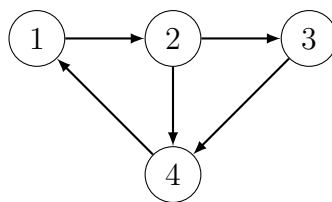
نمایش لیست مجاورت

در نمایش لیست مجاورت، به هر رأس x در گراف یک لیست مجاورت تخصیص داده می‌شود که شامل رأس‌هایی است که یالی از x به آن‌ها وجود دارد. لیست‌های مجاورت محبوب‌ترین روش برای نمایش گراف‌ها هستند و اغلب الگوریتم‌ها را می‌توان به‌طور کارآمد با استفاده از آن‌ها پیاده‌سازی کرد.

روش مناسب برای ذخیره لیست‌های مجاورت، تعریف یک آرایه از بردارها به صورت زیر است:

```
vector<int> adj[N];
```

مقدار ثابت N طوری انتخاب می‌شود که تمام لیست‌های مجاورت قابل ذخیره‌سازی باشند. به‌عنوان مثال، گراف



می‌تواند به این صورت ذخیره شود:

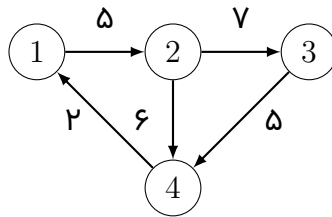
```
adj[1].push_back(2);  
adj[2].push_back(3);  
adj[2].push_back(4);  
adj[3].push_back(4);  
adj[4].push_back(1);
```

اگر گراف بدون جهت باشد، می‌توان آن را به شکل مشابه ذخیره کرد، اما هر یال در هر دو جهت اضافه می‌شود.

برای یک گراف وزن‌دار، ساختار را می‌توان به صورت زیر گسترش داد:

```
vector<pair<int,int>> adj[N];
```

در این حالت، لیست مجاورت رأس a شامل جفت (b, w) است هر زمان که یالی از رأس a به رأس b با وزن w وجود داشته باشد. به‌عنوان مثال، گراف



می‌تواند به صورت زیر ذخیره شود:

```
,5}); ({2push_back[1].adj
,7}); ({3push_back[2].adj
,6}); ({4push_back[2].adj
,5}); ({4push_back[3].adj
,2}); ({1push_back[4].adj
```

مزیت استفاده از لیست‌های مجاورت این است که می‌توان به شکل کارآمد گره‌هایی را یافت که از یک گره مفروض با یک یال قابل دسترسی هستند. برای نمونه، حلقه زیر روی تمام گره‌هایی پیمایش می‌کند که از گره s قابل دسترسی‌اند:

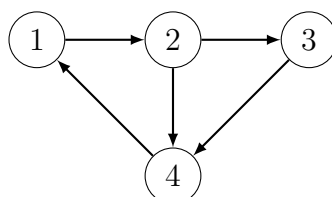
```
{ })s[adj : u auto( for
u //
}
```

نمایش ماتریس مجاورت

یک **ماتریس مجاورت** آرایه‌ای دوبعدی است که نشان می‌دهد گراف شامل چه یال‌هایی است. می‌توان از ماتریس مجاورت وجود یال بین دو گره را به صورت کارآمد بررسی کرد. این ماتریس می‌تواند به شکل آرایه‌ای ذخیره شود:

```
];N] [N[adj int
```

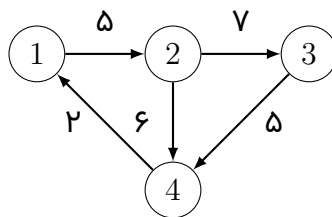
یا است b گره به a گره از یالی شامل گراف آیا می‌دهد نشان $adj[a][b]$ مقدار هر آن در که $adj[a][b] = 0$ صورت این غیر در و $adj[a][b] = 1$ باشد، داشته وجود گراف در یال اگر خیر. گراف نمونه، برای بود. خواهد



می‌تواند به صورت زیر نمایش داده شود:

	۱	۲	۳	۴
۱	۰	۱	۰	۰
۲	۰	۰	۱	۱
۳	۰	۰	۰	۱
۴	۱	۰	۰	۰

اگر گراف وزن دار باشد، نمایش ماتریس مجاورت می‌تواند تعمیم یابد تا در صورت وجود یال، ماتریس حاوی وزن آن یال باشد. با استفاده از این نمایش، گراف



با ماتریس زیر متناظر است:

	۱	۲	۳	۴
۱	۰	۵	۰	۰
۲	۰	۰	۷	۶
۳	۰	۰	۰	۵
۴	۲	۰	۰	۰

عیب نمایش ماتریس مجاورت وجود n^2 عنصر است که اغلب صفر هستند. به همین دلیل گراف بزرگ باشد این نمایش قابل استفاده نیست.

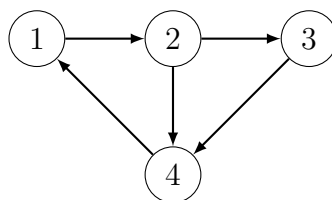
نمایش فهرست یال‌ها

یک **فهرست یال** تمام یال‌های گراف را با ترتیبی دلخواه نگه می‌دارد. روش مناسبی است اگر الگوریتم همه یال‌ها را پیمایش کند و نیازی به یافتن یال‌های خروجی از یک رأس خاص نباشد.

فهرست یال‌ها را می‌توان در یک بردار ذخیره کرد:

```
>>int,int<pair<vector
```

گراف بنابراین، است. a رأس به a رأس از یالی نشان‌دهنده (a, b) جفت هر آن در که



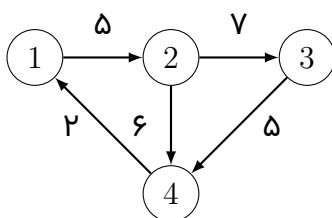
به شکل زیر نمایش داده می‌شود:

```
,2});({1push_back.edges
,3});({2push_back.edges
,4});({2push_back.edges
,4});({3push_back.edges
,1});({4push_back.edges
```

اگر گراف وزن دار باشد، ساختار داده بدین صورت گسترش می یابد:

```
;edges >>int,int,int<tuple<vector
```

وزن با a رأس به b رأس از یالی بیانگر که است (a, b, w) فرم به فهرست این در عضو هر گراف نمونه، برای است. w



می تواند به صورت زیر نمایش داده شود:

```
,5});,2({1push_back.edges
,7});,3({2push_back.edges
,6});,4({2push_back.edges
,5});,4({3push_back.edges
,2});,1({4push_back.edges
```

¹در برخی کامپایلرهای قدیمی تر، باید از تابع `make_tuple` به جای آکولاد استفاده شود (برای مثال، `make_tuple(1,2,5)` به جای `{1,2,5}`).

فصل ۱۲

پیمایش گراف

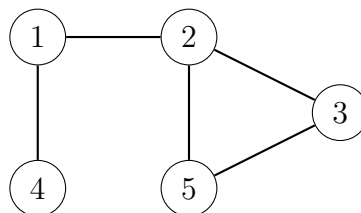
این فصل به بررسی دو الگوریتم پایه‌ای گراف می‌پردازد: جستجوی اول سطح و جستجوی اول عمق. هر دو الگوریتم یک گره آغازین در گراف دریافت می‌کنند و تمام گره‌هایی را که از گره آغازین قابل دسترسی باشند، ملاقات می‌کنند. تفاوت این الگوریتم‌ها در ترتیبی است که گره‌ها را با آن ملاقات می‌کنند.

۱.۱۲ جستجوی اول عمق

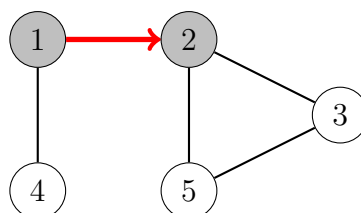
جستجوی اول عمق (DFS) یک روش سرراست برای پیمایش گراف است. الگوریتم از یک گره آغازین شروع می‌شود، و با استفاده از یال‌های گراف به تمام گره‌های دیگری که از گره آغازین قابل دسترسی هستند، پیش می‌رود. جستجوی اول عمق همواره یک مسیر منفرد را در گراف تا زمانی که گره‌های جدید بیابد، دنبال می‌کند. پس از آن، به گره‌های قبلی بازمی‌گردد و شروع به کاوش در بخش‌های دیگر گراف می‌کند. الگوریتم گره‌های ملاقات شده را ثبت می‌کند، به‌طوری که هر گره را تنها یک بار پردازش کند.

مثال

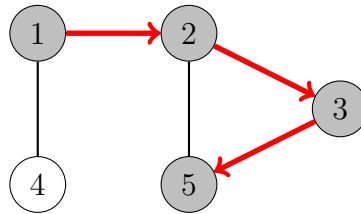
بررسی می‌کنیم که چگونه جستجوی اول عمق گراف زیر را پردازش می‌کند:



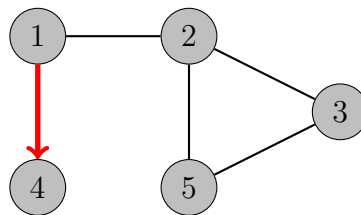
می‌توان جستجو را از هر گره‌ای در گراف آغاز کرد؛ اکنون جستجو را از گره ۱ آغاز می‌کنیم. جستجو ابتدا به گره ۲ پیش می‌رود:



پس از آن، گره‌های ۳ و ۵ ملاقات خواهند شد:



همسایه‌های رأس ۵ گره‌های ۲ و ۳ هستند، ولی جستجو هر دو را قبلاً پیمایش کرده است. زمان بازگشت به گره‌های قبلی فرارسیده است. همسایه‌های گره‌های ۳ و ۲ نیز پیمایش شده‌اند، پس مسیر بعدی از گره ۱ به گره ۴ است:



سپس جستجو پایان می‌پذیرد؛ همه گره‌ها دیده شده‌اند. پیچیدگی زمانی جستجوی اول عمق $O(n + m)$ است که n شمار گره‌ها و m شمار یال‌هاست، زیرا الگوریتم هر گره و یال را یک بار بررسی می‌کند.

پیاده‌سازی

جستجوی اول عمق به سادگی با بازگشت پیاده‌سازی می‌شود. تابع `dfs` زیر جستجوی اول عمق را از گره داده‌شده آغاز می‌کند. فرض تابع: گراف به فرم لیست مجاورت در آرایه نگهداری شده است:

```
];N[adj >int<vector
```

دارد: آرایه‌ای و

```
];N[visited bool
```

رسیدن با هستند؛ `false` آرایه مقادیر همه ابتدا می‌کند. ذخیره را پیمایش شده گره‌های که تابع: پیاده‌سازی می‌شود. `true` برابر `visited[s]` مقدار `s`، گره به جستجو

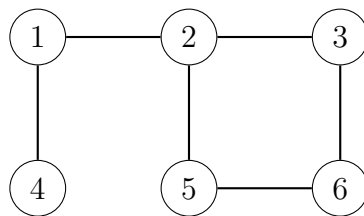
```
{ )s int(dfs void
;return ]s[visited( if
;true = ]s[visited
s node process //
{ ]s[adj :u auto( for
);u(dfs
}
```

۲.۱۲ جستجوی اول سطح

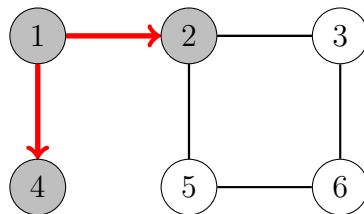
جستجوی اول سطح (BFS) گره‌ها را به ترتیب افزایش فاصله از گره آغازین بررسی می‌کند. از این رو، فاصله از گره آغازین به سایر گره‌ها با جستجوی اول سطح به دست می‌آید. البته پیاده‌سازی جستجوی اول سطح سخت‌تر از جستجوی اول عمق است. جستجوی اول سطح گره‌ها را سطح به سطح پیمایش می‌کند. نخست گره‌های با فاصله ۱ از گره مبدأ، سپس گره‌های با فاصله ۲ و همین ترتیب ادامه می‌یابد. روند تا مشاهده تمام گره‌ها ادامه دارد.

مثال

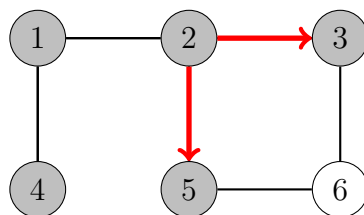
بررسی عملکرد جستجوی اول سطح روی گراف زیر:



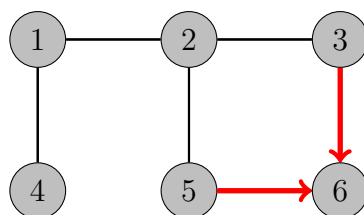
فرض کنید جستجو از گره ۱ آغاز می‌شود. ابتدا همه گره‌هایی را که می‌توان با یک یال از گره ۱ به آن‌ها رسید، پردازش می‌کنیم:



پس از آن، به گره‌های ۳ و ۵ می‌رویم:



در نهایت، گره ۶ را پیمایش می‌کنیم:



اکنون فاصله‌ها را از گره آغازین تا همه گره‌های گراف محاسبه کرده‌ایم. فاصله‌ها به شرح زیر است:

گره	فاصله
۱	۰
۲	۱
۳	۲
۴	۱
۵	۲
۶	۳

مانند جستجوی اول سطح، پیچیدگی زمانی جستجوی اول سطح برابر با $O(n + m)$ است، که در آن n شمار گره‌ها و m شمار یال‌ها است.

پیاده‌سازی

پیاده‌سازی جستجوی اول سطح از جستجوی اول عمق دشوارتر است، زیرا الگوریتم، گره‌ها را در بخش‌های گوناگون گراف پیمایش می‌کند. یک پیاده‌سازی متداول بر پایه یک صف شامل گره‌ها انجام می‌شود. در هر گام، گره بعدی در صف پردازش می‌شود. کد زیر فرض می‌کند گراف به صورت لیست‌های مجاورت ذخیره شده است و داده‌ساختارهای زیر را نگهداری می‌کند:

```
q >int<queue
N[visited bool]
N[distance int]
```

صف q شامل گره‌هایی است که باید به ترتیب صعودی فاصله‌شان پردازش شوند. گره‌های تازه همواره به انتهای صف افزوده می‌شوند و گره ابتدای صف، گره بعدی برای پردازش است. آرایه `visited` نشان می‌دهد کدام گره‌ها پیش‌تر توسط جستجو دیده شده‌اند، و آرایه `distance` شامل فاصله‌ها از گره آغازین به تمام گره‌های گراف خواهد بود. جستجو را می‌توان با شروع از گره x به صورت زیر پیاده‌سازی کرد:

```

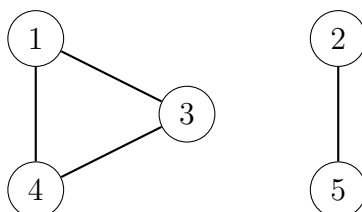
;true = ]x[visited
0; = ]x[distance
);x(push.q
{ ()empty.q(! while
());pop.q ();front.q = s int
s node process //
{ ]s[adj : u auto( for
;continue ]u[visited( if
;true = ]u[visited
]+1;s[distance = ]u[distance
);u(push.q
}
}
```

۳.۱۲ کاربردها

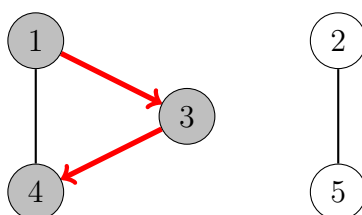
با استفاده از الگوریتم‌های پیمایش گراف، می‌توان ویژگی‌های متعددی از گراف‌ها را بررسی کرد. معمولاً هر دو الگوریتم جستجوی اول عمق و جستجوی اول سطح قابل استفاده هستند، اما در عمل، جستجوی اول عمق گزینه بهتری است زیرا پیاده‌سازی آن ساده‌تر است. در کاربردهای زیر فرض می‌کنیم گراف بدون جهت است.

بررسی هم‌بندی

یک گراف هم‌بند است اگر مسیری میان هر دو گره از گراف وجود داشته باشد. بنابراین، می‌توان هم‌بند بودن گراف را با شروع از یک گره دلخواه و بررسی دسترسی‌پذیری به همه گره‌های دیگر ارزیابی کرد. برای نمونه، در گراف



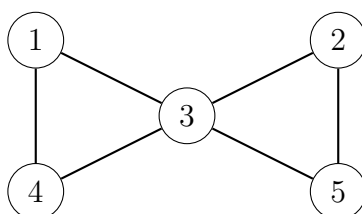
یک جستجوی اول سطح عمق از گره 1 گره‌های زیر را پیمایش می‌کند:



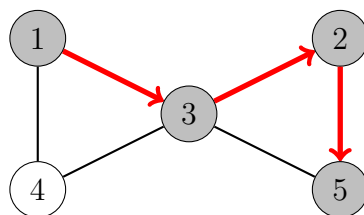
از آنجا که جستجو همه گره‌ها را پیمایش نکرد، می‌توان نتیجه گرفت که گراف هم‌بند نیست. به روش مشابه، می‌توان تمام مؤلفه‌های هم‌بندی یک گراف را نیز با پیمایش روی گره‌ها پیدا کرد؛ به این صورت که اگر گره فعلی هنوز به هیچ مؤلفه‌ای تعلق ندارد، همواره یک جستجوی اول سطح عمق جدید آغاز شود.

یافتن دوره‌ها

یک گراف شامل دور است اگر هنگام پیمایش گراف، گره‌ای پیدا کنیم که همسایه آن (غیر از گره قبلی در مسیر فعلی) قبلاً پیمایش شده باشد. برای مثال، گراف



دارای دو دور است و می‌توان یکی از آنها را به صورت زیر پیدا کرد:

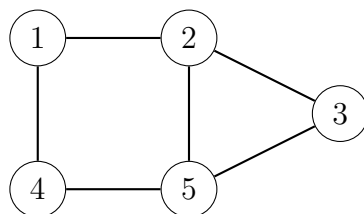


پس از حرکت از گره ۲ به گره ۵ متوجه می‌شویم که همسایه ۳ از گره ۵ قبلاً پیمایش شده است. بنابراین، گراف شامل دوری است که از گره ۳ می‌گذرد، برای مثال، $3 \rightarrow 2 \rightarrow 5 \rightarrow 3$. روش دیگر برای تشخیص وجود دور در گراف، محاسبه ساده تعداد رأس‌ها و یال‌ها در هر مؤلفه است. اگر مؤلفه‌ای شامل c رأس و فاقد دور باشد، دقیقاً شامل $c - 1$ یال است (بنابراین باید یک درخت باشد). اگر c یا تعداد بیشتری یال وجود داشته باشد، آن مؤلفه قطعاً شامل دور است.

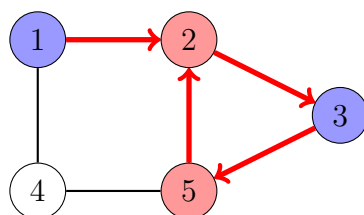
بررسی دوبخشی بودن

یک گراف دوبخشی است اگر بتوان رأس‌های آن را با دو رنگ به‌گونه‌ای رنگ‌آمیزی کرد که هیچ دو رأس مجاوری هم‌رنگ نباشند. بررسی دوبخشی بودن یک گراف با الگوریتم‌های پیمایش گراف بسیار آسان است.

ایده کار رنگ کردن رأس شروع با رنگ آبی، تمام همسایه‌های آن با رنگ قرمز، همسایه‌های آن‌ها با رنگ آبی، و به همین ترتیب است. اگر در نقطه‌ای از جستجو مشخص شود که دو رأس مجاور هم‌رنگ هستند، گراف دوبخشی نیست. در غیر این صورت، گراف دوبخشی است و یک رنگ‌آمیزی یافت شده است. برای مثال، گراف



دوبخشی نیست، زیرا جستجو از رأس ۱ به این صورت پیش می‌رود:



مشاهده می‌شود که رنگ هر دو رأس ۲ و ۵ قرمز است، در حالی که این دو در گراف مجاور یکدیگرند. بنابراین، گراف دوبخشی نیست.

این الگوریتم همواره کار می‌کند؛ زیرا با داشتن تنها دو رنگ، رنگ رأس شروع در یک مؤلفه، رنگ تمام رأس‌های دیگر آن مؤلفه را تعیین می‌کند. قرمز یا آبی بودن رأس شروع هیچ تفاوتی ایجاد نمی‌کند.

در حالت کلی، تشخیص امکان رنگ‌آمیزی رأس‌های گراف با k رنگ به‌طوری که هیچ دو رأس مجاوری هم‌رنگ نباشند، دشوار است. حتی برای $k = 3$ نیز الگوریتم کارآمدی شناخته نشده و مسئله NP-hard است.

فصل ۱۳

کوتاه‌ترین مسیرها

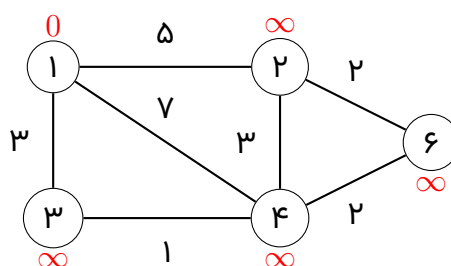
یافتن کوتاه‌ترین مسیر بین دو گره در گراف، مسئله‌ای مهم با کاربردهای عملی فراوان است. به عنوان مثال، مسئله‌ای طبیعی در شبکه جاده‌ای، محاسبه کوتاه‌ترین طول ممکن برای یک مسیر بین دو شهر بر اساس طول جاده‌ها است. در گراف بدون وزن، طول مسیر برابر با تعداد یال‌های آن است و برای یافتن کوتاه‌ترین مسیر، استفاده از جستجوی اول سطح کافی است. با این حال، تمرکز این فصل بر گراف‌های وزن‌دار است که یافتن کوتاه‌ترین مسیر در آن‌ها نیازمند الگوریتم‌های پیشرفته‌تر است.

۱.۱۳ الگوریتم بلمن-فورد

الگوریتم بلمن-فورد^۱ کوتاه‌ترین مسیرها را از یک گره مبدا به تمام گره‌های گراف پیدا می‌کند. این الگوریتم انواع گراف‌ها را پردازش می‌کند، مشروط بر این‌که گراف دوری با طول منفی نداشته باشد. اگر گراف دور منفی داشته باشد، الگوریتم قادر به تشخیص آن است. الگوریتم، فاصله‌ها از گره مبدا به تمام گره‌های گراف را نگهداری می‌کند. در ابتدا، فاصله تا گره مبدا برابر ۰ و فاصله تا سایر گره‌ها بی‌نهایت است. الگوریتم با پیدا کردن یال‌هایی که مسیرها را کوتاه‌تر می‌کنند فاصله‌ها را کاهش می‌دهد تا زمانی که امکان کاهش بیشتر هیچ فاصله‌ای وجود نداشته باشد.

مثال

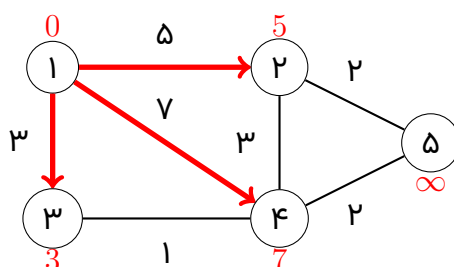
عملکرد الگوریتم بلمن-فورد را در گراف زیر بررسی می‌کنیم:



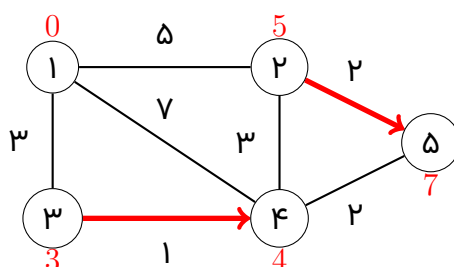
به هر گره گراف یک فاصله نسبت داده می‌شود. در ابتدا، فاصله تا گره مبدا برابر ۰ و فاصله تا سایر گره‌ها بی‌نهایت است.

^۱ نام این الگوریتم برگرفته از ریچارد بلمن و لستر فورد است که آن را به صورت مستقل در سال‌های ۱۹۵۸ و ۱۹۵۶ منتشر کردند [۵، ۲۴].

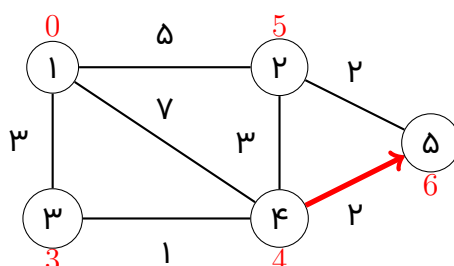
الگوریتم یال‌هایی را جستجو می‌کند که فاصله‌ها را کاهش دهند. ابتدا، تمام یال‌های خروجی از گره ۱ فاصله‌ها را کاهش می‌دهند:



پس از این، یال‌های $2 \rightarrow 5$ و $3 \rightarrow 4$ فاصله‌ها را کاهش می‌دهند:

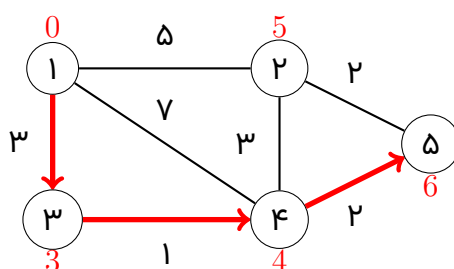


در نهایت یک تغییر دیگر وجود دارد:



پس از این، هیچ یالی نمی‌تواند هیچ فاصله‌ای را کاهش دهد. این یعنی فاصله‌ها نهایی هستند و ما با موفقیت کوتاه‌ترین فاصله‌ها را از گره آغازین به تمام گره‌های گراف محاسبه کرده‌ایم.

برای نمونه، کوتاه‌ترین فاصله ۳ از گره ۱ به گره ۵ متناظر با مسیر زیر است:



پیاده‌سازی

پیاده‌سازی زیر از الگوریتم بلمن-فورد کوتاه‌ترین فاصله‌ها را از یک گره x به تمام گره‌های گراف تعیین می‌کند. کد فرض می‌کند گراف به صورت فهرست یال‌ها edges شامل چندتایی‌هایی به شکل (a, b, w) ذخیره شده است، به این معنی که یالی از گره a به گره b با وزن w وجود دارد.

این الگوریتم از $n - 1$ دور تشکیل شده است و در هر دور، تمام یال‌های گراف را بررسی کرده و تلاش می‌کند فاصله‌ها را کاهش دهد. الگوریتم یک آرایه distance می‌سازد که شامل فاصله‌ها از x به تمام رأس‌های گراف خواهد بود. ثابت INF نشان‌دهنده فاصله بی‌نهایت است.

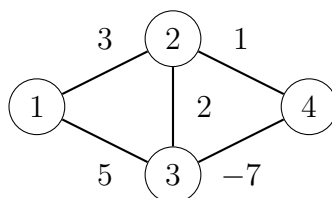
```
int( for i = 1; i <= n; ++i ) {
    x[i] = INF;
    for ( auto e : edges ) {
        int a = e.a, b = e.b, w = e.w;
        if ( x[a] + w < x[b] ) x[b] = x[a] + w;
    }
}
```

پیچیدگی زمانی الگوریتم $O(nm)$ است، زیرا الگوریتم شامل $n - 1$ دور است و در طول یک دور روی تمام m یال پیمایش می‌کند. اگر هیچ دور منفی در گراف وجود نداشته باشد، تمام فاصله‌ها پس از $n - 1$ دور نهایی می‌شوند، زیرا هر کوتاه‌ترین مسیر می‌تواند حداکثر شامل $n - 1$ یال باشد.

در عمل، فاصله‌های نهایی معمولاً سریع‌تر از $n - 1$ دور پیدا می‌شوند. بنابراین، یک روش ممکن برای افزایش کارایی الگوریتم این است که اگر در طول یک دور هیچ فاصله‌ای کاهش نیافت، اجرای الگوریتم متوقف شود.

دوره‌ای منفی

الگوریتم بلمن-فورد همچنین می‌تواند برای بررسی وجود دور با طول منفی در گراف به کار رود. برای نمونه، گراف



شامل یک دور منفی $2 \rightarrow 3 \rightarrow 4 \rightarrow 2$ با طول -4 است. اگر گراف شامل دور منفی باشد، می‌توان هر مسیری را که شامل این دور است، با تکرار چندباره دور به تعداد نامتناهی کوتاه‌تر کرد. بنابراین، مفهوم کوتاه‌ترین مسیر در این وضعیت معنایی ندارد.

یک دور منفی را می‌توان با اجرای الگوریتم بلمن-فورد به مدت n دور شناسایی کرد. اگر دور آخر فاصله‌ای را کاهش دهد، گراف شامل دور منفی است. توجه کنید که این الگوریتم می‌تواند برای یافتن دور منفی در کل گراف بدون توجه به رأس شروع استفاده شود.

الگوریتم SPFA

الگوریتم SPFA (کوتاه‌ترین مسیر سریع‌تر) [۲۰] گونه‌ای از الگوریتم بلمن-فورد است که اغلب کارایی بیشتری نسبت به الگوریتم اصلی دارد. الگوریتم SPFA در هر دور از تمام یال‌ها عبور نمی‌کند، بلکه یال‌ها را برای بررسی به روش هوشمندانه‌تری انتخاب می‌کند.

این الگوریتم صف گره‌هایی که ممکن است برای کاهش فاصله‌ها استفاده شوند نگه می‌دارد. ابتدا الگوریتم گره شروع x را به صف اضافه می‌کند. سپس همیشه اولین گره صف را پردازش می‌کند، و وقتی یال $a \rightarrow b$ فاصله‌ای را کاهش می‌دهد، گره b به صف اضافه می‌شود.

کارایی الگوریتم SPFA وابسته به ساختار گراف است: الگوریتم اغلب کارا است، اما بدترین پیچیدگی زمانی آن همچنان $O(nm)$ است و ساخت ورودی‌هایی که الگوریتم را به کندی الگوریتم اصلی بلمن-فورد کنند ممکن است.

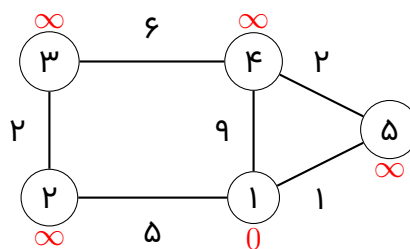
۲.۱۳ الگوریتم دایکسترا

الگوریتم دایکسترا^۲ کوتاه‌ترین مسیرها از گره شروع به تمام گره‌های گراف را مثل الگوریتم بلمن-فورد پیدا می‌کند. مزیت الگوریتم دایکسترا کارایی بالاتر و قابلیت پردازش گراف‌های بزرگ است. با این حال، الگوریتم نبود یال با وزن منفی در گراف را شرط می‌داند.

مانند الگوریتم بلمن-فورد، الگوریتم دایکسترا فاصله‌ها تا گره‌ها را نگه می‌دارد و حین جستجو کاهش می‌دهد. الگوریتم دایکسترا کارا است، چون با تکیه بر نبود یال‌های منفی، هر یال گراف را فقط یک بار پردازش می‌کند.

مثال

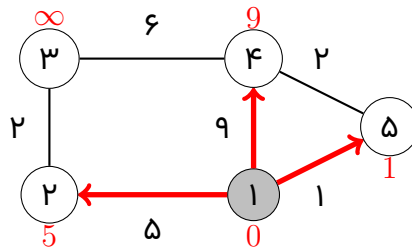
عملکرد الگوریتم دایکسترا در گراف زیر با شروع از گره ۱ را بررسی می‌کنیم:



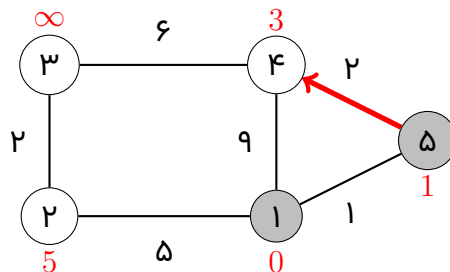
مشابه الگوریتم بلمن-فورد، در ابتدا فاصله تا رأس آغازین ∞ و فاصله تا تمام رأس‌های دیگر بی‌نهایت است.

در هر گام، الگوریتم دایکسترا رأسی را انتخاب می‌کند که هنوز پردازش نشده و فاصله‌اش تا حد امکان کمینه است. نخستین رأس با این مشخصه، رأس ۱ با فاصله ∞ است. وقتی رأسی انتخاب شد، الگوریتم تمام یال‌های خروجی از آن را بررسی می‌کند و فاصله‌ها را به کمک آن‌ها کاهش می‌دهد:

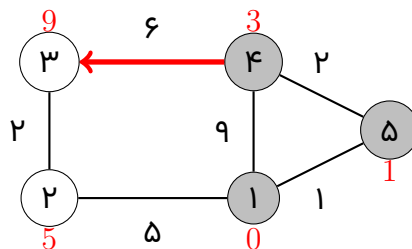
^۲ E. W. Dijkstra published the algorithm in ۱۹۵۹ [۱۴]; however, his original paper does not mention how to implement the algorithm efficiently.



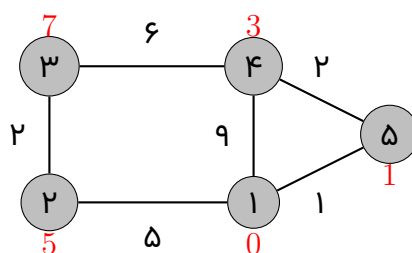
در این حالت، یال‌های رأس ۱ فاصله رأس‌های ۲، ۴ و ۵ را کاهش دادند؛ این فاصله‌ها اکنون ۵، ۹ و ۱ هستند.
رأس بعدی برای پردازش، رأس ۵ با فاصله ۱ است. این انتخاب فاصله تا رأس ۴ را از ۹ به ۳ کاهش می‌دهد:



پس از این، گره بعدی گره ۴ است که فاصله تا گره ۳ را به ۹ کاهش می‌دهد:

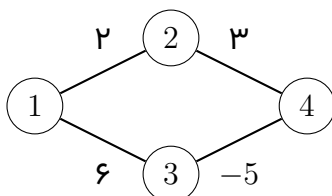


ویژگی قابل توجه در الگوریتم دایکسترا این است که هر زمان گره‌ای انتخاب شود، فاصله آن نهایی است. برای نمونه، در این مرحله از الگوریتم، فواصل ۰، ۱ و ۳ فواصل نهایی تا گره‌های ۱، ۵ و ۴ هستند.
پس از این، الگوریتم دو گره باقی‌مانده را پردازش می‌کند و فواصل نهایی به این صورت خواهند بود:



یال‌های منفی

کارایی الگوریتم دایکسترا مبتنی بر این است که گراف یال منفی نداشته باشد. اگر یال منفی وجود داشته باشد، الگوریتم ممکن است نتایج نادرست بدهد. به عنوان مثال، گراف زیر را در نظر بگیرید:



کوتاه‌ترین مسیر از رأس ۱ به رأس ۴ مسیر $1 \rightarrow 3 \rightarrow 4$ با طول ۱ است. با این حال، الگوریتم دایکسترا با دنبال کردن یال‌های با کمترین وزن، مسیر $1 \rightarrow 2 \rightarrow 4$ را پیدا می‌کند. این الگوریتم در نظر نمی‌گیرد که در مسیر دیگر، وزن -5 وزن بزرگ قبلی یعنی ۶ را جبران می‌کند.

پیاده‌سازی

پیاده‌سازی زیر از الگوریتم دایکسترا، کمترین فاصله از رأس x به سایر رئوس گراف را محاسبه می‌کند. گراف به صورت لیست‌های مجاورت ذخیره می‌شود به طوری که هرگاه یالی از رأس a به رأس b با وزن w وجود داشته باشد، $[adj, w]$ شامل جفت (b, w) است.

پیاده‌سازی کارآمد الگوریتم دایکسترا نیازمند آن است که بتوان رأس با کمترین فاصله را که هنوز پردازش نشده است، به صورت بهینه پیدا کرد. داده‌ساختار مناسب برای این کار یک صف اولویت‌دار است که رئوس را بر اساس فاصله‌شان مرتب نگه می‌دارد. با استفاده از صف اولویت‌دار، رأس بعدی برای پردازش در زمان لگاریتمی قابل بازیابی است.

در کد زیر، صف اولویت‌دار q شامل جفت‌هایی به فرم $(-d, x)$ است؛ به این معنی که فاصله فعلی تا رأس x برابر d است. آرایه `distance` شامل فاصله تا هر رأس است و آرایه `processed` نشان می‌دهد که آیا یک رأس پردازش شده است یا خیر. در ابتدا فاصله تا x برابر ۰ و برای سایر رئوس ∞ است.

```

;INF = ]i[distance ++)i ;n <= i 1; = i int( for
0; = ]x[distance
});x, ({0push.q
{ ()empty.q(! while
());pop.q ;second().top.q = a int
;continue ])a[processed( if
;true = ]a[processed
{ ])a[adj : u auto( for
;second.u = w ,first.u = b int
{ ])b[distance < w]+a[distance( if
;w]+a[distance = ]b[distance
});b,]b[distance-({push.q
}
}
}

```

توجه کنید که صف اولویت‌دار شامل فاصله‌های منفی تا رئوس است. دلیل این موضوع این است که نسخه پیش‌فرض صف اولویت‌دار در C++، بیشترین عنصر را پیدا می‌کند، در حالی که ما به دنبال کمترین عنصر هستیم. با استفاده از فاصله‌های منفی، می‌توان مستقیماً از صف اولویت‌دار پیش‌فرض استفاده کرد^۳. همچنین توجه داشته باشید که ممکن است چندین نمونه از یک رأس درون صف اولویت‌دار قرار گیرد؛ با این حال، فقط نمونه با کمترین فاصله پردازش خواهد شد.

پیچیدگی زمانی پیاده‌سازی فوق $O(n + m \log m)$ است، زیرا الگوریتم تمام رئوس گراف را پیمایش کرده و به ازای هر یال، حداکثر یک فاصله به صف اولویت‌دار اضافه می‌کند.

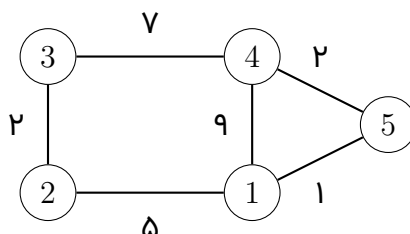
۳.۱۳ الگوریتم فلوید-وارشال

الگوریتم فلوید-وارشال^۴ رویکرد دیگری برای حل مسئله یافتن کوتاه‌ترین مسیرها ارائه می‌دهد. برخلاف سایر الگوریتم‌های این فصل، این الگوریتم تمام کوتاه‌ترین مسیرهای بین گره‌ها را در یک بار اجرا پیدا می‌کند.

این الگوریتم یک آرایه دوبعدی شامل فاصله‌های بین گره‌ها را نگهداری می‌کند. ابتدا، فاصله‌ها فقط با استفاده از یال‌های مستقیم بین گره‌ها محاسبه می‌شوند، و پس از آن، الگوریتم با استفاده از گره‌های میانی در مسیرها، فاصله‌ها را کاهش می‌دهد.

مثال

نحوه کارکرد الگوریتم فلوید-وارشال را در گراف زیر بررسی می‌کنیم:



در ابتدا، فاصله هر گره تا خودش برابر 0 است، و اگر یالی با وزن x بین گره‌های a و b وجود داشته باشد، فاصله بین این دو گره x خواهد بود. سایر فاصله‌ها بی‌نهایت هستند. در این گراف، آرایه اولیه به صورت زیر است:

۵	۴	۳	۲	۱	
۱	۹	∞	۵	۰	۱
∞	∞	۲	۰	۵	۲
∞	۷	۰	۲	∞	۳
۲	۰	۷	∞	۹	۴
۰	۲	∞	∞	۱	۵

^۳البته می‌توانستیم صف اولویت‌دار را مشابه بخش ۵.۴ تعریف کرده و از فاصله‌های مثبت استفاده کنیم، اما پیاده‌سازی کمی طولانی‌تر می‌شد.

^۴این الگوریتم به نام‌های آر. دبلیو. فلوید و اس. وارشال نام‌گذاری شده است که آن را به طور مستقل در سال ۱۹۶۲ منتشر کردند [۷۰، ۲۳].

الگوریتم شامل دوره‌های متوالی است. در هر دور، الگوریتم گره جدیدی را انتخاب می‌کند که از این پس می‌تواند به عنوان گره میانی در مسیرها عمل کند، و فاصله‌ها با استفاده از این گره کاهش می‌یابند.

در دور اول، گره ۱ گره میانی جدید است. یک مسیر جدید بین گره‌های ۲ و ۴ با طول ۱۴ پدید می‌آید، زیرا گره ۱ آن‌ها را به هم وصل می‌کند. همچنین یک مسیر جدید بین گره‌های ۲ و ۵ با طول ۶ ایجاد می‌شود.

۵	۴	۳	۲	۱	
۱	۹	∞	۵	۰	۱
۶	۱۴	۲	۰	۵	۲
∞	۷	۰	۲	∞	۳
۲	۰	۷	۱۴	۹	۴
۰	۲	∞	۶	۱	۵

در دور دوم، گره ۲ گره میانی جدید است. این کار مسیرهای جدیدی بین گره‌های ۱ و ۳ و بین گره‌های ۳ و ۵ ایجاد می‌کند:

۵	۴	۳	۲	۱	
۱	۹	۷	۵	۰	۱
۶	۱۴	۲	۰	۵	۲
۸	۷	۰	۲	۷	۳
۲	۰	۷	۱۴	۹	۴
۰	۲	۸	۶	۱	۵

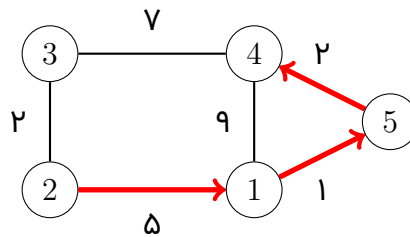
در دور سوم، گره ۳ گره میانی جدید است. مسیر جدیدی بین گره‌های ۲ و ۴ ایجاد می‌شود:

۵	۴	۳	۲	۱	
۱	۹	۷	۵	۰	۱
۶	۹	۲	۰	۵	۲
۸	۷	۰	۲	۷	۳
۲	۰	۷	۹	۹	۴
۰	۲	۸	۶	۱	۵

الگوریتم به همین ترتیب ادامه می‌یابد تا زمانی که تمام گره‌ها به عنوان گره میانی تعیین شوند. پس از پایان الگوریتم، آرایه شامل کوتاه‌ترین فاصله‌ها بین هر دو گره است:

۵	۴	۳	۲	۱	
۱	۳	۷	۵	۰	۱
۶	۸	۲	۰	۵	۲
۸	۷	۰	۲	۷	۳
۲	۰	۷	۸	۳	۴
۰	۲	۸	۶	۱	۵

برای مثال، آرایه نشان می‌دهد که کوتاه‌ترین فاصله بین گره‌های ۲ و ۴ برابر ۸ است. این متناظر با مسیر زیر است:



پیاده‌سازی

مزیت الگوریتم فلوید-وارشال سادگی پیاده‌سازی آن است. کد زیر ماتریس فاصله‌ای می‌سازد که در آن $distance[a][b]$ کوتاه‌ترین فاصله بین گره‌های a و b است. ابتدا، الگوریتم $distance$ را با استفاده از ماتریس مجاورت adj گراف مقاردهی اولیه می‌کند:

```
for (i = 1; i <= n; ++i)
    for (j = 1; j <= n; ++j)
        if (i != j)
            distance[i][j] = adj[i][j];
        else
            distance[i][j] = INF;
```

می‌شوند: محاسبه زیر صورت به فاصله‌ها کوتاه‌ترین سپس،

```
for (k = 1; k <= n; ++k)
    for (i = 1; i <= n; ++i)
        for (j = 1; j <= n; ++j)
            distance[i][j] = min(distance[i][j], distance[i][k] + distance[k][j]);
```

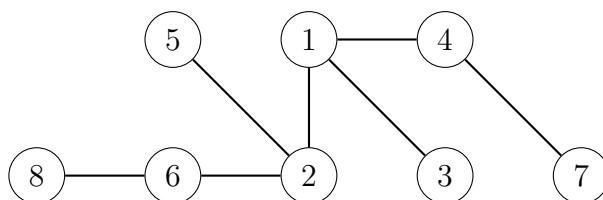
پیچیدگی زمانی الگوریتم $O(n^3)$ است، زیرا شامل سه حلقه تودرتو است که روی رأس‌های گراف پیمایش می‌کنند.

از آنجا که پیاده‌سازی الگوریتم فلوید-وارشال ساده است، حتی اگر فقط پیدا کردن یک کوتاه‌ترین مسیر در گراف نیاز باشد، این الگوریتم می‌تواند انتخاب خوبی باشد. با این حال، الگوریتم تنها زمانی قابل استفاده است که گراف به اندازه‌ای کوچک باشد که پیچیدگی زمانی مکعبی برای آن به اندازه کافی سریع باشد.

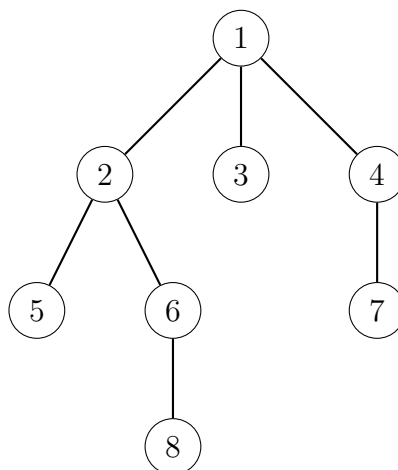
فصل ۱۴

الگوریتم‌های درخت

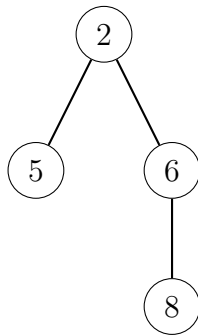
یک **درخت** گرافی همبند و بدون دور است که شامل n رأس و $n - 1$ یال می‌باشد. حذف هر یال از یک درخت، آن را به دو مؤلفه تقسیم می‌کند، و اضافه کردن هر یال به درخت، یک دور ایجاد می‌نماید. علاوه بر این، همواره یک مسیر یکتا بین هر دو رأس از درخت وجود دارد. برای مثال، درخت زیر از ۸ رأس و ۷ یال تشکیل شده است:



برگ‌های یک درخت، رأس‌هایی با درجه‌ی ۱ هستند، یعنی فقط یک همسایه دارند. برای مثال، برگ‌های درخت بالا رأس‌های ۳، ۵، ۷ و ۸ هستند. در یک درخت **ریشه‌دار**، یکی از رأس‌ها به عنوان **ریشه** درخت تعیین می‌شود، و تمام رأس‌های دیگر زیر ریشه قرار می‌گیرند. برای مثال، در درخت زیر، رأس ۱ ریشه است.



در یک درخت ریشه‌دار، **فرزندان** یک رأس همسایه‌های پایینی آن هستند، و **والد** یک رأس همسایه‌ی بالایی آن است. هر رأس دقیقاً یک والد دارد، به جز ریشه که والدی ندارد. برای مثال، در درخت بالا، فرزندان رأس ۲ رأس‌های ۵ و ۶ هستند، و والد آن رأس ۱ است. ساختار درخت ریشه‌دار بازگشتی است: هر گره درخت، ریشه یک **زیردرخت** است که شامل خود گره و تمام گره‌های موجود در زیردرخت‌های فرزندان آن است. به عنوان مثال، در درخت بالا، زیردرخت گره ۲ شامل گره‌های ۲، ۵، ۶ و ۸ است:



۱.۱۴ پیمایش درخت

الگوریتم‌های عمومی پیمایش گراف می‌توانند برای پیمایش گره‌های یک درخت استفاده شوند. با این حال، پیاده‌سازی پیمایش درخت ساده‌تر از گراف عمومی است، زیرا هیچ دوری در درخت وجود ندارد و رسیدن به یک گره از چندین جهت ممکن نیست. روش معمول پیمایش درخت، شروع جستجوی اول-عمق از یک گره دلخواه است. تابع بازگشتی زیر می‌تواند استفاده شود:

```

void dfs(int s, int e) {
    // s
    for (u : adj[s]) {
        if (u != e) {
            dfs(u, s);
        }
    }
}

```

تابع دو پارامتر می‌گیرد: گره فعلی s و گره قبلی e . هدف پارامتر e اطمینان از این است که جستجو فقط به گره‌هایی حرکت کند که هنوز بازدید نشده‌اند. فراخوانی زیر جستجو را از گره x آغاز می‌کند:

```
dfs(x, 0);
```

در فراخوانی اول $e = 0$ است، زیرا هیچ گره قبلی وجود ندارد و حرکت به هر جهتی در درخت مجاز است.

برنامه‌نویسی پویا

برنامه‌نویسی پویا می‌تواند برای محاسبه اطلاعات حین پیمایش درخت استفاده شود. با استفاده از برنامه‌نویسی پویا می‌توان، برای مثال، در زمان $O(n)$ برای هر گره درخت ریشه‌دار، تعداد گره‌های زیردرخت آن یا طول طولانی‌ترین مسیر از آن گره تا یک برگ را محاسبه کرد. به عنوان مثال، برای هر گره s مقدار $count[s]$ را محاسبه می‌کنیم: تعداد گره‌های زیردرخت آن. زیردرخت شامل خود گره و تمام گره‌های زیردرخت‌های فرزندان آن است، بنابراین می‌توان تعداد گره‌ها را به صورت بازگشتی با کد زیر محاسبه کرد:

```

void dfs(int s, int e) {
    count[s] = 1;
    for (u : adj[s]) {
        if (u != e) {
            dfs(u, s);
        }
    }
}

```

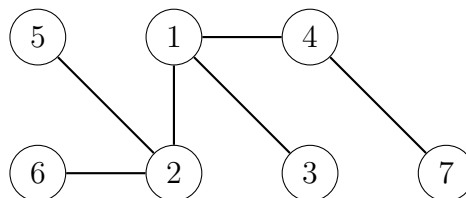
```

    { }s[adj : u auto( for
;continue )e == u( if
                );s ,u(dfs
                ];u[count += ]s[count
                }
    }

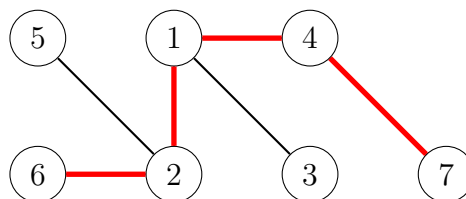
```

۲.۱۴ قطر

قطر یک درخت بیشینه طول یک مسیر بین دو رأس است. برای مثال، درخت زیر را در نظر بگیرید:



قطر این درخت برابر ۴ است که متناظر با مسیر زیر است:



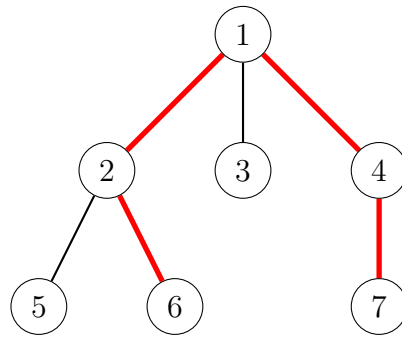
توجه کنید ممکن است چندین مسیر با بیشینه طول وجود داشته باشد. در مسیر بالا، می‌توان رأس ۶ را با رأس ۵ جایگزین کرد تا مسیر دیگری با طول ۴ به دست آید. در ادامه دو الگوریتم با زمان اجرای $O(n)$ برای محاسبه قطر درخت بررسی می‌شود. الگوریتم نخست بر پایه برنامه‌نویسی پویا است و الگوریتم دوم از دو پیمایش اول-عمق استفاده می‌کند.

الگوریتم ۱

یک روش کلی برای حل مسائل درخت این است که ابتدا درخت را دلخواه ریشه‌دار کنیم. پس از آن، می‌توان مسئله را به‌طور جداگانه برای هر زیردرخت حل کرد. الگوریتم نخست برای محاسبه قطر بر پایه همین ایده است.

مشاهده مهم این است که هر مسیر در یک درخت ریشه‌دار، یک بالاترین نقطه دارد: بالاترین رأسی که متعلق به مسیر است. بنابراین، می‌توان برای هر رأس، طول طولانی‌ترین مسیری را که بالاترین نقطه‌اش آن رأس است محاسبه کرد. یکی از این مسیرها متناظر با قطر درخت خواهد بود.

برای مثال، در درخت زیر، گره ۱ بالاترین نقطه در مسیری است که متناظر با قطر است:



برای هر گره x دو مقدار محاسبه می‌کنیم:

• $\text{toLeaf}(x)$: بیشینه طول مسیر از x به هر برگ

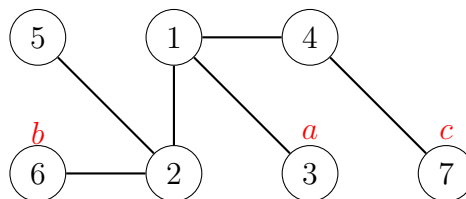
• $\text{maxLength}(x)$: بیشینه طول مسیری که بالاترین نقطه آن x است

برای مثال، در درخت بالا، $\text{toLeaf}(1) = 2$ ، زیرا مسیر $1 \rightarrow 2 \rightarrow 6$ وجود دارد، و $\text{maxLength}(1) = 4$ ، زیرا مسیر $1 \rightarrow 4 \rightarrow 7$ وجود دارد. در این حالت، $\text{maxLength}(1)$ برابر با قطر است.

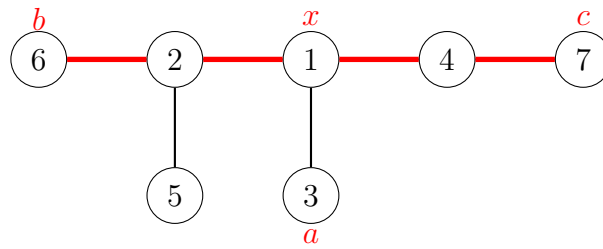
می‌توان از برنامه‌نویسی پویا برای محاسبه مقادیر بالا برای همه گره‌ها در زمان $O(n)$ استفاده کرد. ابتدا برای محاسبه $\text{toLeaf}(x)$ ، فرزندان x را بررسی می‌کنیم، فرزند c با بیشینه $\text{toLeaf}(c)$ را انتخاب کرده و یک واحد به این مقدار اضافه می‌کنیم. سپس برای محاسبه $\text{maxLength}(x)$ ، دو فرزند متمایز a و b را طوری انتخاب می‌کنیم که مجموع $\text{toLeaf}(a) + \text{toLeaf}(b)$ بیشینه باشد و دو واحد به این مجموع اضافه می‌کنیم.

الگوریتم ۲

روش کارآمد دیگر برای محاسبه قطر درخت مبتنی بر دو بار جستجوی اول عمق است. ابتدا یک گره دلخواه a در درخت انتخاب می‌کنیم و دورترین گره b از a را می‌یابیم. سپس دورترین گره c از b را پیدا می‌کنیم. قطر درخت فاصله بین b و c است. در گراف زیر، a ، b و c می‌توانند به این صورت باشند:



```
draw.thick,-,color=red,linedraw,thick,-,color=red,linewidth=2pt] (7)--(3); \path
draw.thick,-draw,thick,-,color=red,linewidth=2pt] (1)--(2); \path!(1)-(3) width=2pt]
```

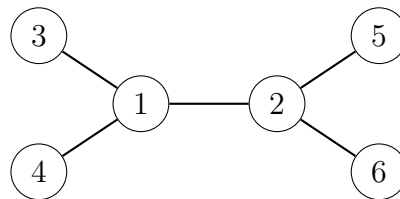


رأس x محل اتصال مسیر مبدأ a به مسیر قطر است. دورترین رأس از a ، رأس b ، رأس c یا رأسی دیگر با دست کم همین فاصله از x است. بنابراین، این رأس همیشه انتخابی معتبر برای یک سر مسیر قطر است.

۳.۱۴ همه طولانی‌ترین مسیرها

مسئله بعدی: محاسبه بیشینه طول مسیر شروع شده از هر رأس درخت. این تعمیم مسئله قطر درخت است، چون بیشینه این مقادیر برابر قطر درخت است. حل در زمان $O(n)$ ممکن است.

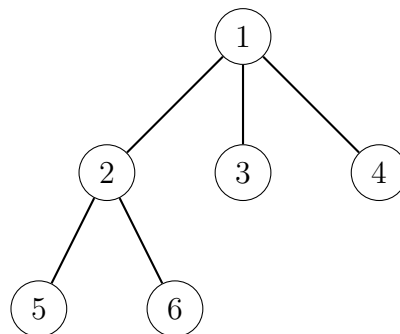
مثال، درخت زیر:



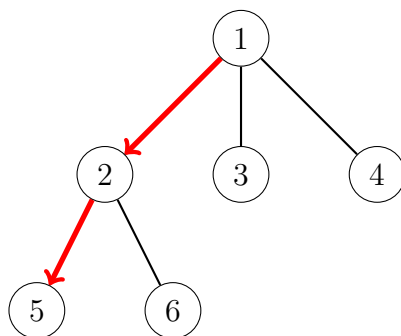
فرض کنید $\text{maxLength}(x)$ بیشینه طول مسیر با شروع از رأس x باشد. در درخت بالا، $\text{maxLength}(4) = 3$ است، چون مسیر $4 \rightarrow 1 \rightarrow 2 \rightarrow 6$ وجود دارد. جدول کامل مقادیر:

۶	۵	۴	۳	۲	۱	رأس x
۳	۳	۳	۳	۲	۲	$\text{maxLength}(x)$

همچنین در این مسئله، یک نقطه شروع خوب برای حل، ریشه‌دار کردن دلخواه درخت است:

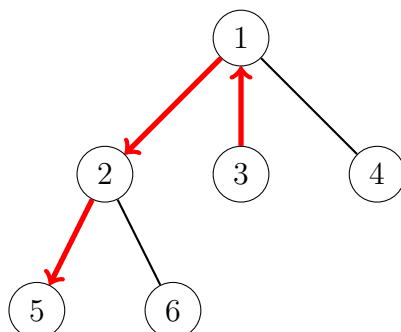


بخش اول مسئله، محاسبه بیشترین طول مسیری است که به ازای هر گره x ، از یکی از فرزندان x عبور می‌کند. برای نمونه، طولانی‌ترین مسیر از گره ۱ از فرزندان آن، یعنی ۲، می‌گذرد:

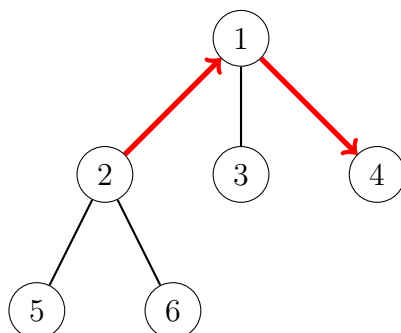


حل این بخش در زمان $O(n)$ ساده است، زیرا می‌توان مانند گذشته از برنامه‌نویسی پویا استفاده کرد.

سپس، بخش دوم مسئله محاسبه بیشترین طول مسیر از طریق والد p به ازای هر گره x است. برای نمونه، طولانی‌ترین مسیر از گره ۳ از والد آن، یعنی ۱، می‌گذرد:



در نگاه اول، به نظر می‌رسد باید طولانی‌ترین مسیر از p را انتخاب کنیم. اما این روش همیشه کار نمی‌کند، زیرا ممکن است طولانی‌ترین مسیر از p از x عبور کند. نمونه‌ای از این وضعیت:



با این حال، می‌توان بخش دوم را در زمان $O(n)$ با ذخیره دو بیشینه طول برای هر رأس x حل کرد:

• $\text{maxLength}_1(x)$: بیشینه طول یک مسیر از x

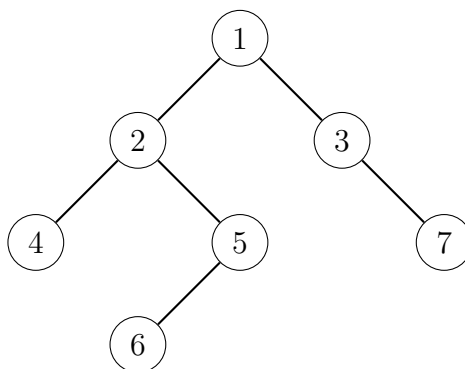
• $\text{maxLength}_2(x)$: بیشینه طول یک مسیر از x در جهتی متفاوت از مسیر اول

به عنوان مثال، در گراف بالا، $\text{maxLength}_1(1) = 2$ با استفاده از مسیر $1 \rightarrow 2 \rightarrow 5$ ، و $\text{maxLength}_2(1) = 1$ با استفاده از مسیر $1 \rightarrow 3$.

در نهایت، اگر مسیر متناظر با $\text{maxLength}_1(p)$ از x بگذرد، نتیجه می‌گیریم بیشینه طول برابر است با $\text{maxLength}_2(p) + 1$ ، و در غیر این صورت بیشینه طول برابر است با $\text{maxLength}_1(p) + 1$.

۴.۱۴ درخت‌های دودویی

یک **درخت دودویی** درختی ریشه‌دار است که هر رأس آن یک زیردرخت چپ و راست دارد. امکان تهی بودن زیردرخت یک رأس وجود دارد. بنابراین، هر رأس در درخت دودویی صفر، یک یا دو فرزند دارد. برای نمونه، درخت زیر یک درخت دودویی است:



رأس‌های درخت دودویی دارای سه ترتیب طبیعی هستند که متناظر با روش‌های مختلف پیمایش بازگشتی درخت است:

- **پیش‌ترتیب:** ابتدا پردازش ریشه، سپس پیمایش زیردرخت چپ، سپس پیمایش زیردرخت راست
- **میان‌ترتیب:** ابتدا پیمایش زیردرخت چپ، سپس پردازش ریشه، سپس پیمایش زیردرخت راست
- **پس‌ترتیب:** ابتدا پیمایش زیردرخت چپ، سپس پیمایش زیردرخت راست، سپس پردازش ریشه

برای درخت بالا، گره‌ها در پیمایش پیش‌ترتیب $[1, 2, 4, 5, 6, 3, 7]$ ، در میان‌ترتیب $[4, 2, 6, 5, 1, 3, 7]$ و در پس‌ترتیب $[4, 6, 5, 2, 7, 3, 1]$ هستند.

اگر پیش‌ترتیب و میان‌ترتیب درختی معلوم باشد، می‌توان ساختار دقیق درخت را بازسازی کرد. برای نمونه، درخت بالا تنها درخت ممکن با پیش‌ترتیب $[1, 2, 4, 5, 6, 3, 7]$ و میان‌ترتیب $[4, 2, 6, 5, 1, 3, 7]$ است. به شیوه مشابه، پس‌ترتیب و میان‌ترتیب نیز ساختار درخت را تعیین می‌کنند.

با این حال، اگر تنها پیش‌ترتیب و پس‌ترتیب درخت معلوم باشد، وضعیت متفاوت است. در این حالت، ممکن است بیش از یک درخت با این ترتیب‌ها همخوانی داشته باشد. برای نمونه، در هر دوی درخت‌های

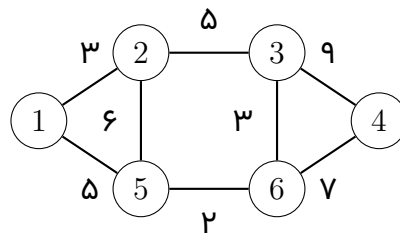


پیش‌ترتیب $[1, 2]$ و پس‌ترتیب $[2, 1]$ است، اما ساختار درخت‌ها با یکدیگر تفاوت دارد.

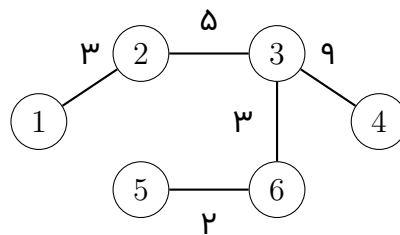
فصل ۱۵

درخت‌های فراگیر

یک **درخت فراگیر** از یک گراف شامل تمام رأس‌های گراف و تعدادی از یال‌های آن است، به طوری که بین هر دو رأس یک مسیر وجود داشته باشد. مانند درخت‌ها به طور کلی، درخت‌های فراگیر همبند و بدون دور هستند. معمولاً چندین روش برای ساخت یک درخت فراگیر وجود دارد. برای مثال، گراف زیر را در نظر بگیرید:

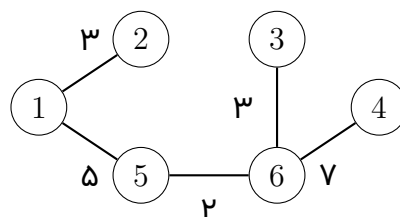


یک درخت فراگیر برای این گراف به صورت زیر است:

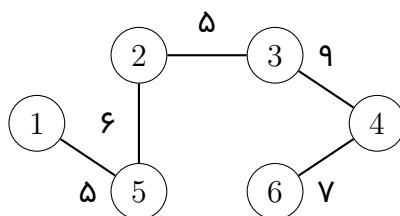


وزن یک درخت فراگیر برابر مجموع وزن یال‌های آن است. برای مثال، وزن درخت فراگیر بالا $3 + 5 + 9 + 3 + 2 = 22$ است.

یک **درخت فراگیر کمینه** درخت فراگیری است که وزن آن تا حد امکان کمینه باشد. وزن درخت فراگیر کمینه برای گراف مثال برابر ۲۰ است، و چنین درختی می‌تواند به صورت زیر ساخته شود:



به روشی مشابه، یک **درخت فراگیر بیشینه** درختی فراگیر است که وزن آن تا حد امکان بیشینه باشد. وزن یک درخت فراگیر بیشینه برای گراف نمونه برابر ۳۲ است:



توجه داشته باشید یک گراف ممکن است چندین درخت فراگیر کمینه و بیشینه داشته باشد؛ بنابراین درخت‌ها یکتا نیستند.

مشخص شده روش‌های حریصانه متعددی برای ساخت درخت‌های فراگیر کمینه و بیشینه وجود دارد. در این فصل، دو الگوریتم را بررسی می‌کنیم که یال‌های گراف را بر اساس وزن پردازش می‌کنند. تمرکز ما روی یافتن درخت‌های فراگیر کمینه است، اما همین الگوریتم‌ها با پردازش معکوس یال‌ها قادر به یافتن درخت‌های فراگیر بیشینه هستند.

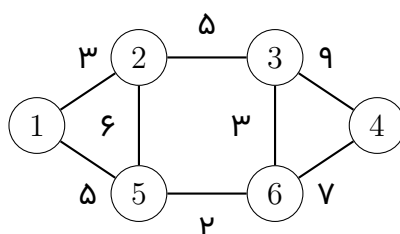
۱.۱۵ الگوریتم کروسکال

در **الگوریتم کروسکال**^۱، درخت فراگیر اولیه فقط شامل رأس‌های گراف است و هیچ یالی ندارد. سپس الگوریتم یال‌ها را بر اساس وزن بررسی می‌کند و هر بار یالی را که دوری نسازد به درخت اضافه می‌کند.

الگوریتم مؤلفه‌های درخت را نگهداری می‌کند. در ابتدا، هر رأس گراف متعلق به یک مؤلفه جداگانه است. هرگاه یالی به درخت اضافه شود، دو مؤلفه ترکیب می‌شوند. در نهایت، همه رأس‌ها عضو یک مؤلفه یکسان می‌شوند و درخت فراگیر کمینه پیدا می‌شود.

مثال

نحوه پردازش گراف زیر توسط الگوریتم کروسکال را بررسی می‌کنیم:

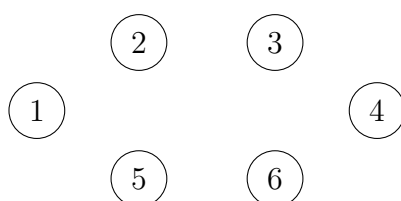


گام نخست الگوریتم، مرتب‌سازی یال‌ها به ترتیب صعودی وزن آن‌ها است. نتیجه، فهرست زیر است:

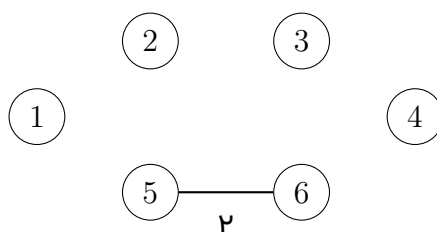
^۱ این الگوریتم در سال ۱۹۵۶ توسط جی. بی. کروسکال منتشر شد [۴۸].

یال	وزن
۶-۵	۲
۲-۱	۳
۶-۳	۳
۵-۱	۵
۳-۲	۵
۵-۲	۶
۶-۴	۷
۴-۳	۹

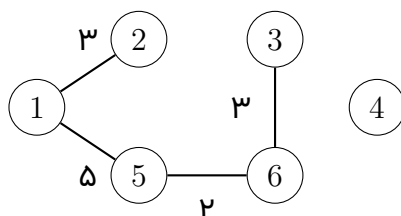
سپس الگوریتم فهرست را پیمایش کرده و هر یال را در صورتی که دو مؤلفه مجزا را به هم متصل کند، به درخت اضافه می‌کند.
در ابتدا، هر رأس در مؤلفه اختصاصی خود قرار دارد:



نخستین یالی که به درخت افزوده می‌شود یال ۶-۵ است که با ادغام دو مؤلفه $\{5\}$ و $\{6\}$ ، مؤلفه $\{5, 6\}$ را می‌سازد:



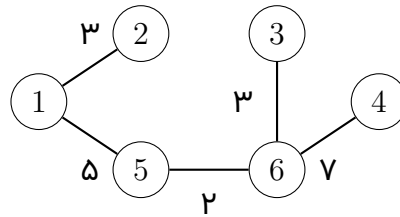
پس از آن، یال‌های ۲-۱، ۶-۳ و ۵-۱ به شکل مشابه اضافه می‌شوند:



پس از این مراحل، بیشتر مولفه‌ها ادغام شده‌اند و دو مولفه در درخت وجود دارد: $\{1, 2, 3, 5, 6\}$ و $\{4\}$.

یال بعدی فهرست یال ۳-۲ است، اما به درخت اضافه نمی‌شود، چون رأس‌های ۲ و ۳ از قبل در یک مولفه‌اند. به همین دلیل، یال ۵-۲ نیز به درخت اضافه نخواهد شد.

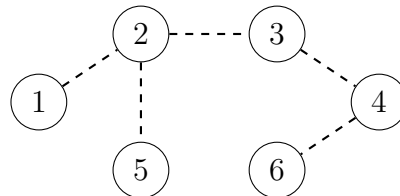
در نهایت، یال ۴-۶ به درخت اضافه می‌شود:



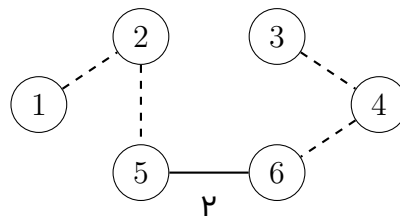
پس از این، الگوریتم یال جدیدی اضافه نخواهد کرد، زیرا گراف همبند است و بین هر دو راس یک مسیر وجود دارد. گراف حاصل یک درخت فراگیر کمینه با وزن $2+3+3+5+7 = 20$ است.

چرا این روش کار می‌کند؟

این پرسش خوبی است که چرا الگوریتم کروسکال به درستی کار می‌کند. چرا راهبرد حریصانه تضمین می‌کند که یک درخت فراگیر کمینه پیدا خواهیم کرد؟
بیایید بررسی کنیم چه اتفاقی می‌افتد اگر یال با کمترین وزن گراف، در درخت فراگیر قرار نگیرد. برای نمونه، فرض کنید یک درخت فراگیر برای گراف پیشین شامل یال با کمترین وزن یعنی ۵-۶ نباشد. ساختار دقیق چنین درخت فراگیری را نمی‌دانیم، اما در هر صورت باید یال‌هایی داشته باشد. فرض کنید درخت به شکل زیر باشد:



اما درخت فوق نمی‌تواند درخت فراگیر کمینه برای گراف باشد. دلیل این است که می‌توانیم یالی را از درخت حذف کرده و یال با کمینه وزن یعنی ۵-۶ را جایگزین آن کنیم. این کار درختی فراگیر با وزن کمتر تولید می‌کند:



به همین دلیل، همواره بهینه است که یال با کمترین وزن برای ساخت درخت فراگیر کمینه به درخت اضافه شود. با استدلالی مشابه می‌توان نشان داد اضافه کردن یال بعدی بر اساس ترتیب وزن به درخت نیز بهینه است، و به همین ترتیب تا انتها ادامه می‌یابد. بنابراین الگوریتم کروسکال به درستی کار می‌کند و همواره یک درخت فراگیر کمینه می‌سازد.

پیاده‌سازی

در پیاده‌سازی الگوریتم کروسکال، استفاده از نمایش فهرست یال‌ها برای گراف مناسب است. گام اول الگوریتم، یال‌های فهرست را در زمان $O(m \log m)$ مرتب می‌کند. سپس گام دوم الگوریتم، درخت فراگیر کمینه را به شکل زیر می‌سازد:

```
for (...) {
    if (!same(a,b)) unite(a,b);
}
```

حلقه روی یال‌های فهرست پیمایش کرده و هر بار یال $b-a$ را پردازش می‌کند که در آن a و b دو رأس هستند. دو تابع مورد نیاز است: تابع `same` بررسی می‌کند آیا a و b در یک مولفه هستند، و تابع `unite` مولفه‌های شامل a و b را ادغام می‌کند. مسئله، پیاده‌سازی کارآمد توابع `same` و `unite` است. یک روش، پیاده‌سازی `same` با پیمایش گراف و بررسی وجود مسیر بین a و b است. ولی پیچیدگی زمانی چنین تابعی $O(n+m)$ خواهد بود و الگوریتم کند می‌شود، زیرا `same` به ازای هر یال در گراف فراخوانی خواهد شد.

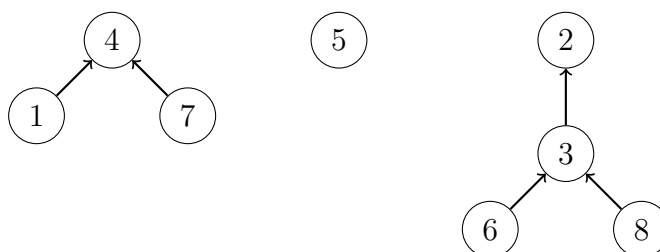
این مسئله را با ساختار مجموعه‌های مجزا (`union-find`) حل می‌کنیم که هر دو تابع را در زمان $O(\log n)$ اجرا می‌کند. بدین ترتیب پیچیدگی زمانی الگوریتم کروسکال پس از مرتب‌سازی فهرست یال‌ها، $O(m \log n)$ خواهد بود.

۲.۱۵ ساختار مجموعه‌های مجزا

یک ساختمان داده مجموعه‌های مجزا (`union-find`) مجموعه‌ای از دسته‌ها را نگهداری می‌کند. مجموعه‌ها مجزا هستند، بنابراین هیچ عنصری در بیش از یک مجموعه وجود ندارد. دو عملیات با زمان $O(\log n)$ پشتیبانی می‌شوند: عملیات `unite` دو مجموعه را ادغام می‌کند، و عملیات `find` نماینده‌ی مجموعه‌ای شامل عنصر داده‌شده را می‌یابد.^۲

ساختار

در ساختار مجموعه‌های مجزا، یک عضو در هر مجموعه نماینده‌ی مجموعه است، و زنجیره‌ای از هر عضو دیگر مجموعه به سمت نماینده وجود دارد. برای مثال فرض کنید مجموعه‌ها $\{1, 4, 7\}$ ، $\{5\}$ و $\{2, 3, 6, 8\}$ باشند:

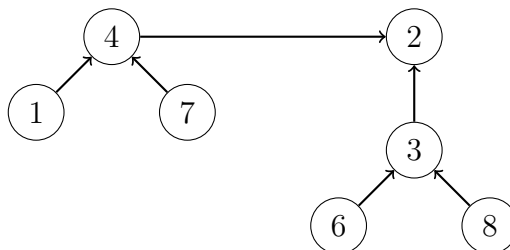


در این حالت، نمایندگان مجموعه‌ها ۴، ۵ و ۲ هستند. می‌توان نماینده‌ی هر عضو را با پیمایش زنجیره‌ی شروع‌شده از آن عضو پیدا کرد. برای نمونه، عنصر ۲ نماینده‌ی عنصر ۶

^۲ ساختار ارائه‌شده در اینجا در سال ۱۹۷۱ توسط جی. دی. هاپکرافت و جی. دی. اولمن معرفی شد [۳۸]. سپس، در ۱۹۷۵، آر. ای. تارجان نسخه‌ای پیشرفته‌تر از این ساختار را بررسی کرد [۶۴] که امروزه در کتاب‌های مرجع الگوریتم بررسی می‌شود.

است، زیرا زنجیره‌ی $2 \rightarrow 3 \rightarrow 6$ را دنبال می‌کنیم. دو عنصر متعلق به یک مجموعه‌اند دقیقاً اگر نمایندگان یکسانی داشته باشند.

دو مجموعه را می‌توان با متصل کردن نماینده‌ی یک مجموعه به نماینده‌ی مجموعه‌ی دیگر ادغام کرد. برای مثال، مجموعه‌های $\{1, 4, 7\}$ و $\{2, 3, 6, 8\}$ به این شکل ادغام می‌شوند:



مجموعه حاصل شامل اعضای $\{1, 2, 3, 4, 6, 7, 8\}$ است. از این پس، عنصر ۲ نماینده‌ی کل مجموعه است و نماینده‌ی پیشین ۴ به عنصر ۲ اشاره دارد. کارایی ساختار مجموعه‌های مجزا (union-find) به نحوه ادغام مجموعه‌ها بستگی دارد. می‌توان از یک استراتژی ساده پیروی کرد: همیشه نماینده مجموعه کوچک‌تر را به نماینده مجموعه بزرگ‌تر متصل کنید (یا اگر مجموعه‌ها هم‌اندازه باشند، می‌توان انتخابی دلخواه داشت). با این روش، طول هر زنجیره $O(\log n)$ خواهد بود، بنابراین می‌توان نماینده هر عنصر را با دنبال کردن زنجیره مربوطه به طور کارآمد پیدا کرد.

پیاده‌سازی

ساختار مجموعه‌های مجزا را می‌توان با آرایه‌ها پیاده‌سازی کرد. در پیاده‌سازی زیر، آرایه `link` برای هر عنصر، عنصر بعدی در زنجیره یا در صورت نماینده بودن، خود آن عنصر را نگه‌داری می‌کند، و آرایه `size` برای هر نماینده، اندازه مجموعه متناظر را نشان می‌دهد. در ابتدا، هر عنصر به یک مجموعه مجزا تعلق دارد:

```
;i = ]i[link ++)i ;n <= i 1; = i int( for
1; = ]i[size ++)i ;n <= i 1; = i int( for
```

تابع `find` نماینده عنصر x را برمی‌گرداند. نماینده را می‌توان با دنبال کردن زنجیره‌ای که از x شروع می‌شود پیدا کرد.

```
{ )x int(find int
];x[link = x ])x[link != x( while
;x return
}
```

تابع `same` بررسی می‌کند که آیا عناصر a و b به یک مجموعه تعلق دارند یا خیر. این کار را می‌توان به سادگی با استفاده از تابع `find` انجام داد:

```
{ )b int ,a int(same bool
);b(find == )a(find return
}
```

تابع unite مجموعه‌های شامل عناصر a و b را ادغام می‌کند (عناصر باید در مجموعه‌های متفاوتی باشند). تابع ابتدا نمایندگان مجموعه‌ها را پیدا می‌کند و سپس مجموعه کوچک‌تر را به مجموعه بزرگ‌تر متصل می‌کند.

```
void unite(int a, int b)
{
    a=find(a);
    b=find(b);
    if(a!=b)
    {
        if(a>b)
            a=b;
        else
            b=a;
    }
    b[link]=a;
    b[size]=b[size]+a[size];
}
```

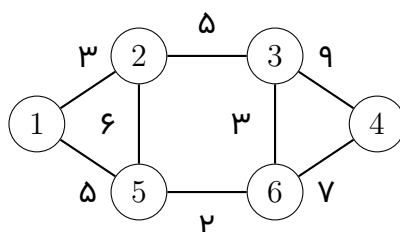
پیچیدگی زمانی تابع find با فرض اینکه طول هر زنجیره $O(\log n)$ است، برابر $O(\log n)$ خواهد بود. در این حالت، توابع unite و same نیز در زمان $O(\log n)$ اجرا می‌شوند. تابع unite تضمین می‌کند که با اتصال مجموعه کوچک‌تر به مجموعه بزرگ‌تر، طول هر زنجیره $O(\log n)$ باقی بماند.

۳.۱۵ الگوریتم پریم

الگوریتم پریم^۳ روشی جایگزین برای یافتن درخت پوشای کمینه است. الگوریتم ابتدا یک رأس دلخواه را به درخت اضافه می‌کند. پس از آن، الگوریتم همواره یالی با کمترین وزن را انتخاب می‌کند که یک رأس جدید را به درخت اضافه کند. در نهایت، همه رئوس به درخت اضافه می‌شوند و یک درخت پوشای کمینه حاصل می‌شود. الگوریتم پریم شبیه الگوریتم دایکسترا است. تفاوت این است که الگوریتم دایکسترا همیشه یالی را برمی‌گزیند که فاصله‌اش از گره آغازین کمینه باشد، ولی الگوریتم پریم کم‌وزن‌ترین یالی را برمی‌گزیند که گرهی تازه به درخت می‌افزاید.

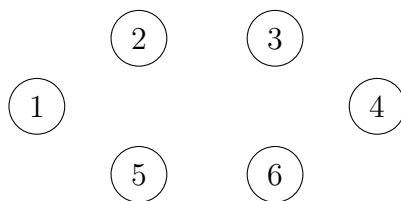
مثال

چگونگی کارکرد الگوریتم پریم در گراف زیر را بررسی می‌کنیم:

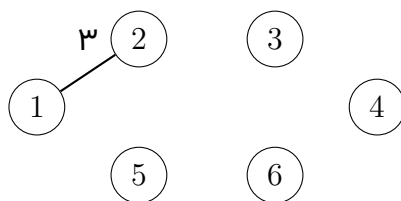


در ابتدا، هیچ یالی بین رأس‌ها وجود ندارد:

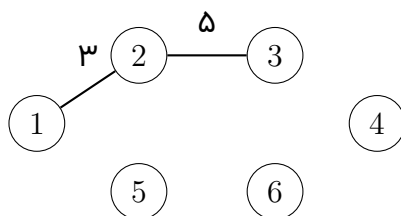
^۳این الگوریتم به نام آر. سی. پریم (R. C. Prim) که آن را در سال ۱۹۵۷ منتشر کرد [۵۴] نام‌گذاری شده است. با این حال، همین الگوریتم پیش‌تر در سال ۱۹۳۰ توسط وی. یارنیک (V. Jarník) کشف شده بود.



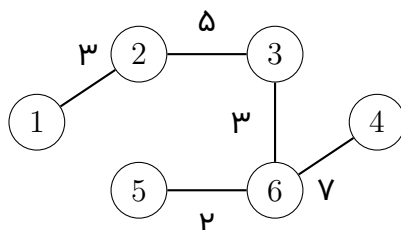
یک رأس دلخواه می‌تواند رأس آغازین باشد، پس رأس ۱ را انتخاب می‌کنیم. ابتدا، رأس ۲ را که با یالی به وزن ۳ متصل است اضافه می‌کنیم:



پس از این، دو یال با وزن ۵ وجود دارد، بنابراین می‌توانیم رأس ۳ یا رأس ۵ را به درخت اضافه کنیم. ابتدا رأس ۳ را اضافه می‌کنیم:



فرایند تا زمانی که تمام رأس‌ها به درخت اضافه شوند ادامه می‌یابد:



پی‌اده‌سازی

مشابه الگوریتم دایکسترا، الگوریتم پریم را می‌توان با استفاده از یک صف اولویت‌دار بهینه‌سازی و پیاده‌سازی کرد. صف اولویت‌دار باید شامل تمام رأس‌هایی باشد که می‌توان آن‌ها را با یک یال به مؤلفه فعلی متصل کرد، مرتب‌شده به صورت صعودی بر اساس وزن یال‌های متناظر. پیچیدگی زمانی الگوریتم پریم $O(n + m \log m)$ است که با پیچیدگی زمانی الگوریتم دایکسترا برابر است. در عمل، هر دو الگوریتم پریم و کروسکال کارآمد هستند و انتخاب الگوریتم سلیقه‌ای است. با این حال، اکثر برنامه‌نویسان رقابتی از الگوریتم کروسکال استفاده می‌کنند.

فصل ۱۶

گراف‌های جهت‌دار

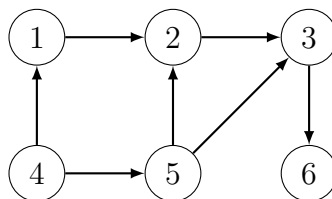
در این فصل، روی دو دسته از گراف‌های جهت‌دار تمرکز می‌کنیم:

- **گراف‌های بدون دور:** هیچ دوری در گراف وجود ندارد، بنابراین هیچ مسیری از هیچ گرهی به خودش وجود ندارد^۱.
- **گراف‌های جانشین:** درجه خروجی هر گره ۱ است، بنابراین هر گره یک جانشین یکتا دارد.

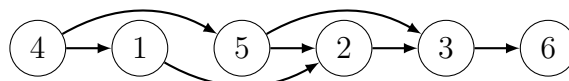
مشخص می‌شود که در هر دو حالت، می‌توانیم الگوریتم‌های کارآمدی طراحی کنیم که بر پایه ویژگی‌های خاص این گراف‌ها هستند.

۱.۱۶ مرتب‌سازی توپولوژیکی

یک **مرتب‌سازی توپولوژیکی** ترتیبی از گره‌های یک گراف جهت‌دار است به طوری که اگر مسیری از گره a به گره b وجود داشته باشد، گره a قبل از گره b در این ترتیب ظاهر شود. به عنوان مثال، برای گراف



یک مرتب‌سازی توپولوژیکی $[4, 1, 5, 2, 3, 6]$ است:



یک گراف بدون دور همیشه دارای مرتب‌سازی توپولوژیکی است. با این حال، اگر گراف شامل دور باشد، تشکیل یک مرتب‌سازی توپولوژیکی ممکن نیست، زیرا هیچ گرهی از دور نمی‌تواند قبل از گره‌های دیگر دور در ترتیب ظاهر شود. مشخص می‌شود که می‌توان از جستجوی اول عمق هم برای بررسی وجود دور در گراف جهت‌دار استفاده کرد و هم در صورت نداشتن دور، برای ساخت یک مرتب‌سازی توپولوژیکی.

^۱گراف‌های جهت‌دار بدون دور گاهی DAG نامیده می‌شوند.

الگوریتم

ایده این است که گره‌های گراف را پیمایش کنیم و همیشه یک جستجوی اول عمق را از گره فعلی شروع کنیم اگر تا به حال پردازش نشده باشد. در طول جستجوها، گره‌ها سه وضعیت ممکن دارند:

- وضعیت ۰: گره پردازش نشده است (سفید)
- وضعیت ۱: گره در حال پردازش است (خاکستری روشن)
- وضعیت ۲: گره پردازش شده است (خاکستری تیره)

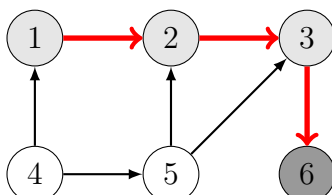
در ابتدا، وضعیت هر رأس ۰ است. وقتی جستجو برای نخستین بار به رأسی می‌رسد، وضعیت آن ۱ می‌شود. در نهایت، پس از پردازش تمام جانشین‌های رأسی، وضعیت آن ۲ می‌شود.

اگر گراف دارای دور باشد، در طول جستجو متوجه این موضوع خواهیم شد؛ چرا که دیر یا زود به رأسی با وضعیت ۱ می‌رسیم. در این حالت، ساخت مرتب‌سازی توپولوژیکی ناممکن است.

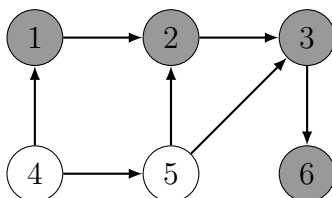
اگر گراف فاقد دور باشد، می‌توانیم یک مرتب‌سازی توپولوژیکی با افزودن هر رأس به یک لیست به محض ۲ شدن وضعیت آن، بسازیم. این لیست به ترتیب معکوس، یک مرتب‌سازی توپولوژیکی است.

مثال ۱

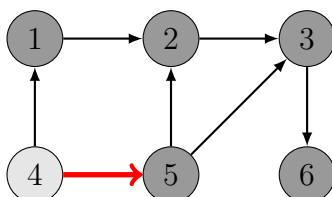
در گراف نمونه، جستجو ابتدا از رأس ۱ به رأس ۶ پیش می‌رود:



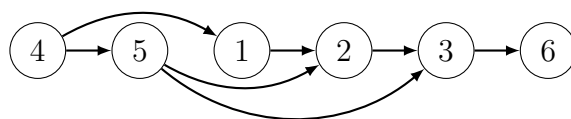
اکنون رأس ۶ پردازش شده است، بنابراین به لیست اضافه می‌شود. پس از آن، رأس‌های ۳، ۲ و ۱ نیز به لیست اضافه می‌شوند:



در این نقطه، لیست برابر با $[6, 3, 2, 1]$ است. جستجوی بعدی از رأس ۴ آغاز می‌شود:



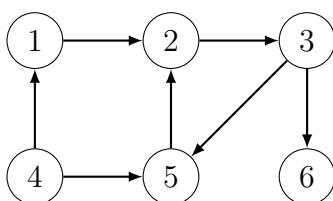
بنابراین لیست نهایی $[6, 3, 2, 1, 5, 4]$ است. همه گره‌ها پردازش شدند، پس مرتب‌سازی توپولوژیک پیدا شد. مرتب‌سازی توپولوژیک معکوس این لیست است: $[4, 5, 1, 2, 3, 6]$:



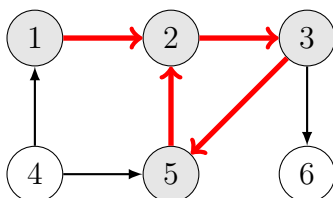
مرتب‌سازی توپولوژیک یکتا نیست. گراف می‌تواند چند مرتب‌سازی توپولوژیک داشته باشد.

مثال ۲

گرافی بدون مرتب‌سازی توپولوژیک بررسی می‌شود، چون گراف دور دارد:



روند جستجو:



جستجو به گره ۲ با وضعیت ۱ می‌رسد. یعنی گراف دور دارد. در این مثال، دور برابر است با: $2 \rightarrow 3 \rightarrow 5 \rightarrow 2$.

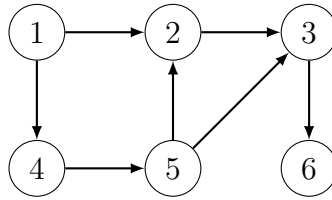
۲.۱۶ برنامه‌نویسی پویا

اگر گراف جهت‌دار بدون دور باشد، برنامه‌نویسی پویا روی آن قابل اعمال است. مسائل زیر برای مسیرهای بین گره آغازین و پایانی سریع حل می‌شوند:

- چند مسیر متمایز وجود دارد؟
- کوتاه‌ترین/طولانی‌ترین مسیر کدام است؟
- کمینه/بیشینه تعداد یال‌ها در یک مسیر چقدر است؟
- کدام رأس‌ها قطعاً در هر مسیری ظاهر می‌شوند؟

شمارش تعداد مسیرها

به عنوان مثال، بیایید تعداد مسیرها از رأس ۱ به رأس ۶ را در گراف زیر محاسبه کنیم:



در مجموع سه مسیر این چنینی وجود دارد:

$$1 \rightarrow 2 \rightarrow 3 \rightarrow 6 \bullet$$

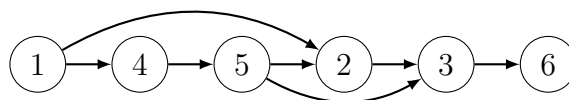
$$1 \rightarrow 4 \rightarrow 5 \rightarrow 2 \rightarrow 3 \rightarrow 6 \bullet$$

$$1 \rightarrow 4 \rightarrow 5 \rightarrow 3 \rightarrow 6 \bullet$$

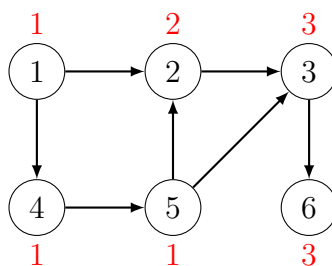
فرض کنید $\text{paths}(x)$ بیانگر تعداد مسیرها از رأس ۱ به رأس x باشد. به عنوان حالت پایه، $\text{paths}(1) = 1$. سپس، برای محاسبه سایر مقادیر $\text{paths}(x)$ ، می‌توان از رابطه بازگشتی زیر استفاده کرد:

$$\text{paths}(x) = \text{paths}(a_1) + \text{paths}(a_2) + \dots + \text{paths}(a_k)$$

که در آن $a_1, a_2, \dots, a_k$ رأس‌هایی هستند که از آن‌ها یالی به x وجود دارد. از آنجا که گراف بدون دور است، مقادیر $\text{paths}(x)$ را می‌توان به ترتیب مرتب‌سازی توپولوژیکی محاسبه کرد. یک مرتب‌سازی توپولوژیکی برای گراف بالا به صورت زیر است:



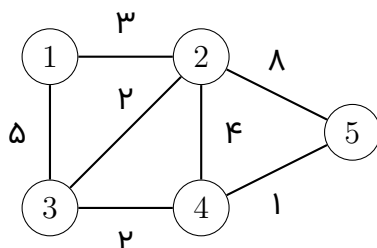
بنابراین، تعداد مسیرها به شرح زیر است:



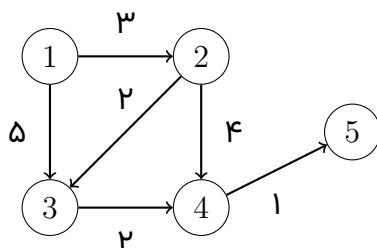
برای مثال، جهت محاسبه مقدار $\text{paths}(3)$ ، می‌توان از فرمول $\text{paths}(2) + \text{paths}(5)$ استفاده کرد، زیرا یال‌هایی از گره‌های ۲ و ۵ به گره ۳ وجود دارد. از آنجا که $\text{paths}(2) = 2$ و $\text{paths}(5) = 1$ ، نتیجه می‌گیریم $\text{paths}(3) = 3$.

گسترش الگوریتم دایکسترا

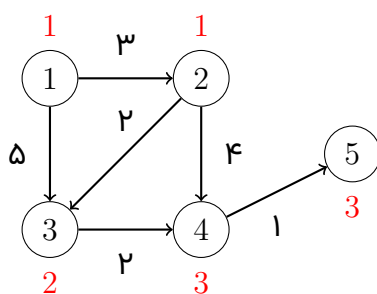
یک نتیجه جانبی الگوریتم دایکسترا، گراف جهت‌دار بدون دور است که برای هر گره از گراف اصلی، مسیرهای ممکن برای رسیدن به آن گره با استفاده از کوتاه‌ترین مسیر از گره مبدا را مشخص می‌کند. می‌توان برنامه‌نویسی پویا را روی این گراف اعمال کرد. به عنوان مثال، در گراف



کوتاه‌ترین مسیرها از گره ۱ ممکن است از یال‌های زیر استفاده کنند:



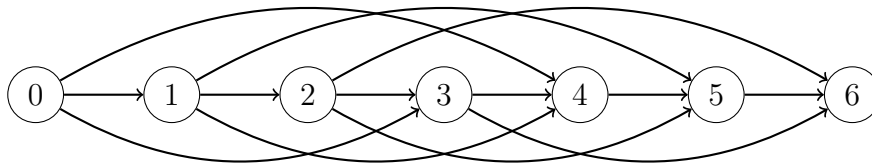
اکنون می‌توانیم به عنوان نمونه، تعداد کوتاه‌ترین مسیرها از گره ۱ به گره ۵ را با استفاده از برنامه‌نویسی پویا محاسبه کنیم:



نمایش مسائل به صورت گراف

در واقع، هر مسئله برنامه‌نویسی پویا می‌تواند به صورت یک گراف جهت‌دار بدون دور نمایش داده شود. در چنین گرافی، هر گره متناظر با یک وضعیت برنامه‌نویسی پویا است و یال‌ها نشان می‌دهند که وضعیت‌ها چگونه به یکدیگر وابسته‌اند.

به عنوان مثال، مسئله ساختن مقدار پول n با استفاده از سکه‌های $\{c_1, c_2, \dots, c_k\}$ را در نظر بگیرید. در این مسئله، می‌توان گرافی ساخت که در آن هر گره متناظر با یک مقدار پول است، و یال‌ها نحوه انتخاب سکه‌ها را نشان می‌دهند. برای نمونه، به ازای سکه‌های $\{1, 3, 4\}$ و $n = 6$ ، گراف به صورت زیر است:



با استفاده از این نمایش، کوتاه‌ترین مسیر از گره $\circ$ به گره n متناظر با پاسخی با کمترین تعداد سکه است، و تعداد کل مسیرها از گره $\circ$ به گره n برابر با تعداد کل راه‌حل‌ها است.

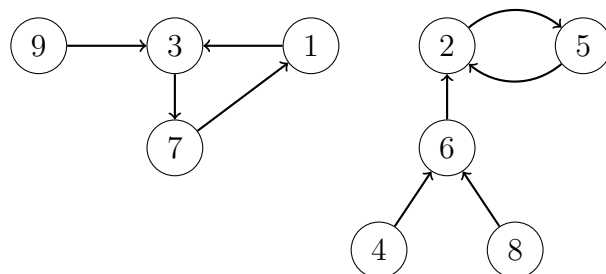
۳.۱۶ مسیرهای جانشین

در ادامه این فصل، بر روی **گراف‌های جانشین** تمرکز خواهیم کرد. در این گراف‌ها، درجه خروجی هر گره ۱ است، یعنی دقیقاً یک یال از هر گره شروع می‌شود. یک گراف جانشین شامل یک یا چند مؤلفه است که هر کدام شامل یک دور و مسیرهایی است که به آن منتهی می‌شوند.

گراف‌های جانشین گاهی اوقات **گراف‌های تابعی** نامیده می‌شوند. دلیل این نام‌گذاری این است که هر گراف جانشین متناظر با تابعی است که یال‌های گراف را تعریف می‌کند. پارامتر ورودی تابع یک گره از گراف است، و خروجی تابع گره جانشین آن خواهد بود. برای مثال، تابع

۹	۸	۷	۶	۵	۴	۳	۲	۱	x
۳	۶	۱	۲	۲	۶	۷	۵	۳	$\text{succ}(x)$

گراف زیر را تعریف می‌کند:



از آنجا که هر راس در گراف جانشین دارای یک جانشین یکتا است، می‌توانیم تابع $\text{succ}(x, k)$ را نیز تعریف کنیم که راسی را مشخص می‌کند که با شروع از راس x و پیمودن k گام به سمت جلو به آن می‌رسیم. برای نمونه، در گراف بالا $\text{succ}(4, 6) = 2$ است، زیرا با طی کردن ۶ گام از راس ۴ به راس ۲ می‌رسیم:



یک روش سرراست برای محاسبه مقدار $\text{succ}(x, k)$ شروع از راس x و پیمودن k گام رو به جلو است که به زمان $O(k)$ نیاز دارد. با این حال، با استفاده از پیش‌پردازش، هر مقدار از $\text{succ}(x, k)$ تنها در زمان $O(\log k)$ قابل محاسبه است.

ایده کار، پیش‌محاسبه تمام مقادیر $\text{succ}(x, k)$ است که در آن k توانی از دو و حداکثر برابر با u باشد؛ به طوری که حداکثر تعداد گام‌هایی است که پیمایش خواهیم کرد. این کار با بازگشت زیر به صورت کارا انجام‌پذیر است:

$$\text{succ}(x, k) = \begin{cases} \text{succ}(x) & k = 1 \\ \text{succ}(\text{succ}(x, k/2), k/2) & k > 1 \end{cases}$$

پیش‌محاسبه مقادیر، زمان $O(n \log u)$ می‌برد، زیرا برای هر راس $O(\log u)$ مقدار محاسبه می‌شود. در گراف بالا، مقادیر اولیه به صورت زیر هستند:

۹	۸	۷	۶	۵	۴	۳	۲	۱	x
۳	۶	۱	۲	۲	۶	۷	۵	۳	$\text{succ}(x, 1)$
۷	۲	۳	۵	۵	۲	۱	۲	۷	$\text{succ}(x, 2)$
۳	۲	۱	۵	۵	۲	۷	۲	۳	$\text{succ}(x, 4)$
۷	۲	۳	۵	۵	۲	۱	۲	۷	$\text{succ}(x, 8)$
									...

پس از این، هر مقدار $\text{succ}(x, k)$ را می‌توان با نمایش تعداد گام‌های k به صورت مجموع توان‌های دو محاسبه کرد. به عنوان مثال، برای محاسبه مقدار $\text{succ}(x, 11)$ ، ابتدا نمایش $11 = 8 + 2 + 1$ را تشکیل می‌دهیم. با استفاده از آن،

$$\text{succ}(x, 11) = \text{succ}(\text{succ}(\text{succ}(x, 8), 2), 1).$$

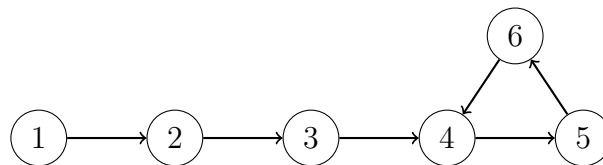
به عنوان مثال، در گراف قبلی

$$\text{succ}(4, 11) = \text{succ}(\text{succ}(\text{succ}(4, 8), 2), 1) = 5.$$

چنین نمایشی همیشه شامل $O(\log k)$ بخش است، بنابراین محاسبه مقدار $\text{succ}(x, k)$ زمان $O(\log k)$ می‌برد.

۴.۱۶ تشخیص دور

یک گراف جانشین را در نظر بگیرید که فقط شامل یک مسیر منتهی به دور است. می‌توان پرسش‌های زیر را مطرح کرد: اگر پیمایش را از راس آغازین شروع کنیم، اولین راس در دور کدام است و دور شامل چند راس است؟ به عنوان مثال، در گراف



پیمایش را از راس ۱ شروع می‌کنیم، اولین راسی که متعلق به دور است راس ۴ بوده و دور متشکل از سه راس (۴، ۵ و ۶) است.

یک روش ساده برای تشخیص دور، پیمایش گراف و ردگیری همه راس‌های دیده‌شده است. به محض این‌که راسی بار دوم دیده شود، می‌توان نتیجه گرفت که این راس، اولین راس در دور است. این روش در زمان $O(n)$ اجرا می‌شود و از $O(n)$ حافظه استفاده می‌کند.

با این حال، الگوریتم‌های بهتری برای تشخیص دور وجود دارند. پیچیدگی زمانی چنین الگوریتم‌هایی هنوز $O(n)$ است، اما تنها از $O(1)$ حافظه استفاده می‌کنند. این یک بهینه‌سازی مهم برای مقادیر بزرگ n است. در ادامه به الگوریتم فلوید می‌پردازیم که این ویژگی‌ها را داراست.

الگوریتم فلوی د

الگوریتم فلوی^۲ با دو اشاره گر a و b در گراف جلو می‌رود. هر دو اشاره گر از گره x ، گره شروع گراف، آغاز می‌کنند. سپس هر نوبت، اشاره گر a یک گام و اشاره گر b دو گام جلو می‌روند. فرآیند تا زمان برخورد اشاره گرها ادامه دارد:

```
);x(succ = a
));x(succ(succ = b
{ )b != a( while
);a(succ = a
));b(succ(succ = b
}
```

این نقطه، اشاره گر a به اندازه k گام و اشاره گر b به اندازه $2k$ گام رفته‌اند، پس طول دور شمارنده k است. بنابراین اولین گره دور با بردن اشاره گر a به گره x و حرکت گام به گام اشاره گرها تا برخورد دوباره پیدا می‌شود.

```
;x = a
{ )b != a( while
);a(succ = a
);b(succ = b
}
;a = first
```

سپس طول دور این‌طور حساب می‌شود:

```
);a(succ = b
1; = length
{ )b != a( while
);b(succ = b
++;length
}
```

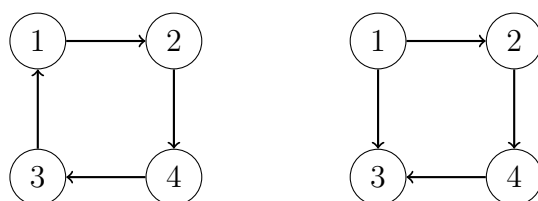
^۲ایده الگوریتم در [۴۶] ذکر شده و به آر. دبلیو. فلویید نسبت داده شده؛ ولی معلوم نیست فلویید واقعا کشف کرد یا نه.

فصل ۱۷

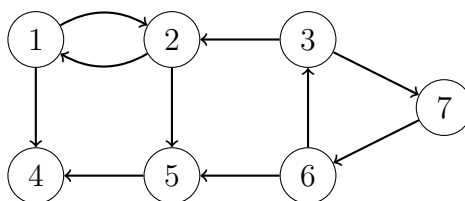
همبندی قوی

در گراف جهت‌دار، یال‌ها فقط در یک جهت پیمایش می‌شوند؛ بنابراین حتی اگر گراف همبند باشد، وجود مسیر بین هر دو گره تضمین نمی‌شود. به همین دلیل، تعریف مفهومی قوی‌تر از همبندی نیاز است.

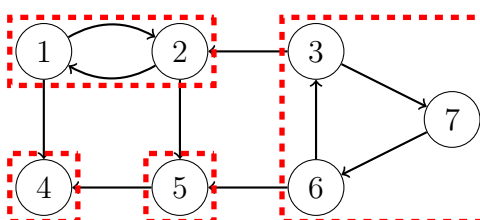
گراف **قویاً همبند** است اگر بین هر گره و تمام گره‌های دیگر مسیر وجود داشته باشد. برای نمونه، در تصویر زیر، گراف سمت چپ قویاً همبند است ولی گراف سمت راست نیست.



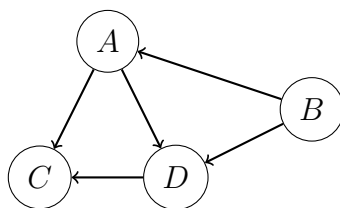
گراف سمت راست قویاً همبند نیست چون مثلاً مسیری از گره ۲ به گره ۱ وجود ندارد. **مؤلفه‌های قویاً همبند** یک گراف، آن را به بزرگ‌ترین بخش‌های قویاً همبند افراز می‌کنند. این مؤلفه‌ها یک **گراف مؤلفه‌های** بدون دور تشکیل می‌دهند که ساختار عمیق گراف اصلی را نشان می‌دهد. برای نمونه، در گراف



مؤلفه‌های قویاً همبند این‌گونه هستند:



گراف مؤلفه‌ای متناظر به صورت زیر است:



مؤلفه‌ها عبارتند از $A = \{1, 2\}$ ، $B = \{3, 6, 7\}$ ، $C = \{4\}$ و $D = \{5\}$.
یک گراف مؤلفه‌ای، گرافی جهت‌دار و بدون دور است، بنابراین پردازش آن نسبت به گراف اصلی آسان‌تر است. از آنجا که گراف فاقد دور است، همواره می‌توان یک مرتب‌سازی توپولوژیکی ایجاد کرد و روش‌های برنامه‌نویسی پویا مانند روش‌های ارائه‌شده در فصل ۱۶ را به کار برد.

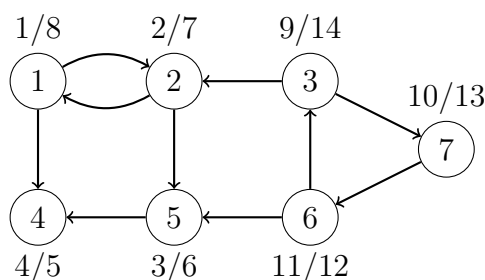
۱.۱۷ الگوریتم کوساراجو

الگوریتم کوساراجو^۱ روشی کارآمد برای یافتن مؤلفه‌های قویاً همبند یک گراف جهت‌دار است. الگوریتم دو جستجوی اول‌سطح‌عمق انجام می‌دهد: جستجوی اول فهرستی از گره‌ها را با توجه به ساختار گراف می‌سازد، و جستجوی دوم مؤلفه‌های قویاً همبند را تشکیل می‌دهد.

جستجوی ۱

مرحله اول الگوریتم کوساراجو، فهرستی از گره‌ها را به ترتیبی که جستجوی اول‌سطح‌عمق آن‌ها را پردازش می‌کند، می‌سازد. الگوریتم روی گره‌ها پیمایش می‌کند، و در هر گره پردازش‌نشده یک جستجوی اول‌سطح‌عمق را آغاز می‌کند. هر گره پس از پردازش، به فهرست اضافه می‌شود.

در گراف نمونه، گره‌ها به ترتیب زیر پردازش می‌شوند:



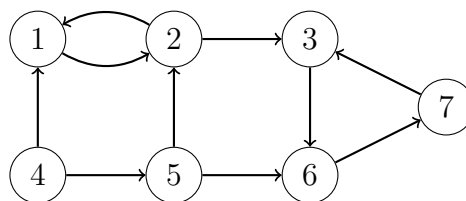
نمادگذاری x/y یعنی پردازش رأس در زمان x شروع و در زمان y پایان یافت. بنابراین لیست متناظر بدین صورت است:

^۱طبق [۱]، اس. آر. کوساراجو این الگوریتم را در سال ۱۹۷۸ ابداع کرد ولی آن را منتشر نکرد. در سال ۱۹۸۱، همین الگوریتم مجدداً توسط ام. شریر کشف و منتشر شد [۵۷].

رأس	زمان پردازش
۴	۵
۵	۶
۲	۷
۱	۸
۶	۱۲
۷	۱۳
۳	۱۴

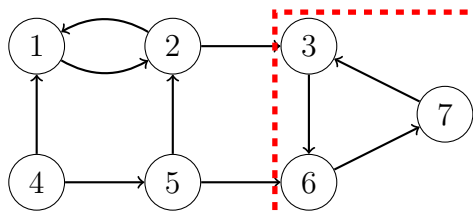
جستجوی ۲

مرحله دوم الگوریتم، مؤلفه‌های قویاً همبند گراف را تشکیل می‌دهد. ابتدا الگوریتم تمام یال‌های گراف را معکوس می‌کند. این کار تضمین می‌کند که طی جستجوی دوم، همواره مؤلفه‌های قویاً همبندی بدون رأس‌های اضافه پیدا کنیم.
پس از معکوس کردن یال‌ها، گراف نمونه بدین صورت درمی‌آید:



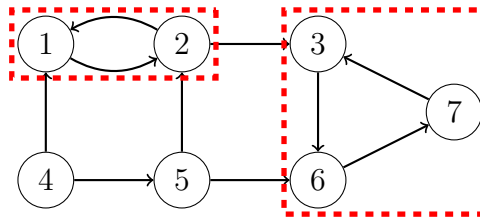
سپس الگوریتم لیست رأس‌های حاصل از جستجوی اول را به ترتیب معکوس پیمایش می‌کند. اگر رأسی متعلق به مؤلفه‌ای نباشد، الگوریتم مؤلفه جدیدی ایجاد می‌کند و یک جستجوی اول سطح عمیق آغاز می‌کند که تمام رأس‌های جدید یافته شده طی جستجو را به مؤلفه جدید اضافه می‌کند.

در گراف نمونه، مؤلفه نخست از رأس ۳ آغاز می‌شود:

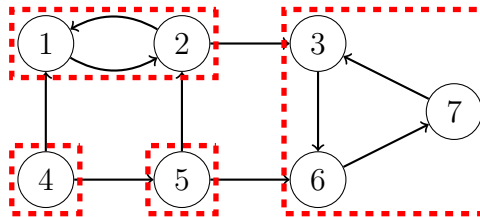


توجه کنید چون تمام یال‌ها معکوس شده‌اند، مؤلفه به بخش‌های دیگر گراف نشت نمی‌کند.

راس‌های بعدی در لیست راس‌های ۷ و ۶ هستند، اما این راس‌ها از قبل به یک مؤلفه تعلق دارند، بنابراین مؤلفه جدید بعدی از راس ۱ آغاز می‌شود:



در نهایت، الگوریتم راس‌های ۵ و ۴ را پردازش می‌کند که مؤلفه‌های قویا همبند باقی‌مانده را تشکیل می‌دهند:



پیچیدگی زمانی الگوریتم $O(n + m)$ است، چرا که الگوریتم دو جستجوی اول عمق انجام می‌دهد.

۲.۱۷ مسئله ۲SAT

همبندی قوی با مسئله ۲SAT مرتبط است. در این مسئله، فرمول منطقی زیر داده می‌شود:

$$(a_1 \vee b_1) \wedge (a_2 \vee b_2) \wedge \dots \wedge (a_m \vee b_m),$$

که هر a_i و b_i متغیر منطقی $(x_1, x_2, \dots, x_n)$ یا نقیض متغیر منطقی $(\neg x_1, \neg x_2, \dots, \neg x_n)$ است. نمادهای "∧" و "∨" نشان‌دهنده عملگرهای منطقی «و» و «یا» هستند. هدف، مقداردهی به متغیرها برای صدق فرمول، یا اعلام عدم امکان آن است.
مثال: فرمول

$$L_1 = (x_2 \vee \neg x_1) \wedge (\neg x_1 \vee \neg x_2) \wedge (x_1 \vee x_3) \wedge (\neg x_2 \vee \neg x_3) \wedge (x_1 \vee x_4)$$

با مقداردهی زیر درست است:

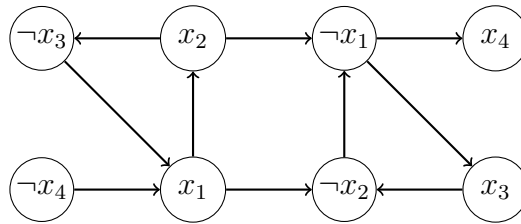
$$\begin{cases} x_1 = \text{false} \\ x_2 = \text{false} \\ x_3 = \text{true} \\ x_4 = \text{true} \end{cases}$$

اما فرمول

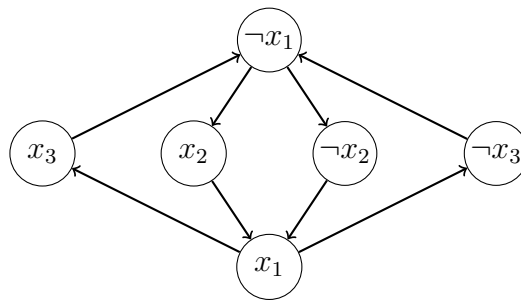
$$L_2 = (x_1 \vee x_2) \wedge (x_1 \vee \neg x_2) \wedge (\neg x_1 \vee x_3) \wedge (\neg x_1 \vee \neg x_3)$$

الگوریتم ارائه‌شده در اینجا در [۴] معرفی شد. الگوریتم زمان خطی شناخته‌شده دیگری نیز بر پایه عقب‌گرد (backtracking) وجود دارد [۱۹].

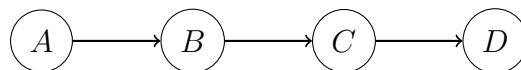
با هر مقداردهی نادرست است. دلیل: انتخاب مقدار برای x_1 بدون تناقض ممکن نیست. اگر x_1 نادرست باشد، هر دو x_2 و $\neg x_2$ باید درست باشند که ناممکن است. اگر x_1 درست باشد، هر دو x_3 و $\neg x_3$ باید درست باشند که این نیز ناممکن است. مسئله ۲SAT با گرافی مدل می‌شود که رأس‌های آن متغیرهای x_i و نقیض‌هایشان $\neg x_i$ هستند، و یال‌ها روابط بین متغیرها را تعیین می‌کنند. هر مؤلفه $(a_i \vee b_i)$ دو یال ایجاد می‌کند: $\neg a_i \rightarrow b_i$ و $\neg b_i \rightarrow a_i$. یعنی اگر a_i برقرار نباشد، b_i باید برقرار باشد، و برعکس. گراف فرمول L_1 :



گراف فرمول L_2 :



ساختار گراف امکان مقداردهی درست به متغیرها را مشخص می‌کند. شرط لازم و کافی: هیچ دو گره x_i و $\neg x_i$ داخل یک مؤلفه قویا همبند مشترک نباشند. اگر باشند، مسیر بین x_i به $\neg x_i$ و برعکس هست؛ یعنی هر دو باید درست باشند، که تناقض است. گراف L_1 : هیچ x_i و $\neg x_i$ در یک مؤلفه قویا همبند نیستند؛ جواب دارد. گراف L_2 : همه گره‌ها در یک مؤلفه قویا همبند مشترک هستند؛ جواب ندارد. یافتن مقادیر متغیرها در صورت وجود جواب: پیمایش گره‌های گراف مؤلفه‌ها با مرتب‌سازی توپولوژیکی معکوس. هر مرحله: پردازش مؤلفه‌ای بدون یال خروجی به مؤلفه پردازش نشده. متغیرهای مؤلفه اگر بیست مقدار باشند، طبق مؤلفه مقدار می‌گیرند. اگر مقدار دارند، بدون تغییر می‌مانند. تکرار تا مقداردهی به همه متغیرها. گراف مؤلفه‌ها برای فرمول L_1 بدین صورت است:



مؤلفه‌ها عبارتند از: $A = \{\neg x_4\}$, $B = \{x_1, x_2, \neg x_3\}$, $C = \{\neg x_1, \neg x_2, x_3\}$ و $D = \{x_4\}$. هنگام ساخت پاسخ، ابتدا مؤلفه D پردازش می‌شود که در آن مقدار درست (true) می‌گیرد. سپس مؤلفه C پردازش می‌شود که در آن x_1 و x_2 نادرست (false) و x_3 درست می‌شوند. تمام متغیرها مقداردهی شده‌اند، بنابراین مؤلفه‌های باقی‌مانده A و B مقدار متغیرها را تغییر نمی‌دهند.

توجه شود این روش کار می‌کند، زیرا گراف ساختار ویژه‌ای دارد: اگر مسیری از گره x_i به گره x_j و از گره x_j به گره $\neg x_j$ وجود داشته باشد، گره x_i هرگز درست نمی‌شود. دلیل

این است که مسیری نیز از گره x_j به گره x_i وجود دارد، و هر دو متغیر x_i و x_j نادرست می‌شوند.

مسئله دشوارتر **۳SAT** است، که در آن هر بخش فرمول به شکل $(a_i \vee b_i \vee c_i)$ است. این مسئله NP-hard است، بنابراین الگوریتم کارآمدی برای حل آن شناخته نشده است.

فصل ۱۸

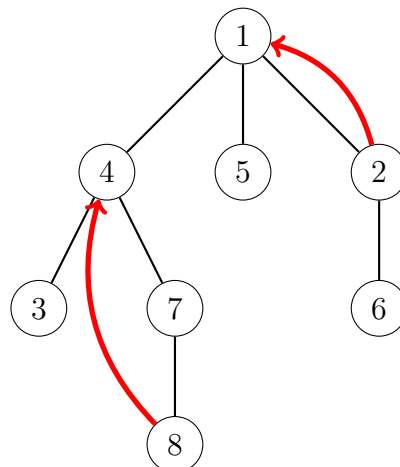
پرسش‌های درختی

این فصل روش‌های پردازش پرسش‌ها روی زیردرخت‌ها و مسیرهای یک درخت ریشه‌دار را بررسی می‌کند. برای نمونه، چنین پرسش‌هایی عبارتند از:

- k امین نیاکان یک گره چیست؟
- مجموع مقادیر در زیردرخت یک گره چقدر است؟
- مجموع مقادیر روی مسیر بین دو گره چقدر است؟
- کمترین جد مشترک دو گره کدام است؟

۱.۱۸ یافتن نیاکان

k امین نیاکان گره x در درخت ریشه‌دار، گره‌ای است که با پیمودن k سطح به سمت بالا از x به آن می‌رسیم. فرض کنید $\text{ancestor}(x, k)$ نشان‌دهنده k امین نیاکان گره x باشد (یا ۰ اگر چنین نیاکانی وجود نداشته باشد). برای نمونه، در درخت زیر، $\text{ancestor}(2, 1) = 1$ و $\text{ancestor}(8, 2) = 4$ است.



روشی ساده برای محاسبه هر مقدار $\text{ancestor}(x, k)$ ، پیمودن k گام متوالی در درخت است. پیچیدگی زمانی این روش $O(k)$ است و امکان دارد کند باشد، زیرا درختی با n گره ممکن است شامل زنجیری از n گره باشد.

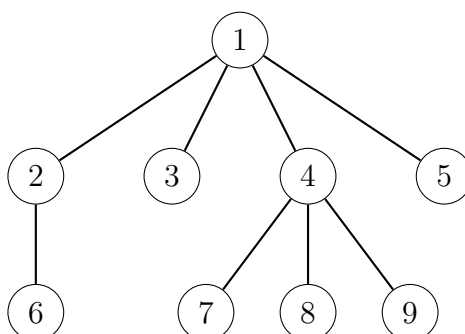
با تکنیکی مشابه روش فصل ۳.۱۶، پس از پیش‌پردازش، هر مقدار $\text{ancestor}(x, k)$ در زمان $O(\log k)$ قابل محاسبه است. ایده کار، پیش‌محاسبه تمام مقادیر $\text{ancestor}(x, k)$ است که در آن $k \leq n$ از دو باشد. برای نمونه، مقادیر درخت بالا چنین است:

۸	۷	۶	۵	۴	۳	۲	۱	x
۷	۴	۲	۱	۱	۴	۱	۰	$\text{ancestor}(x, 1)$
۴	۱	۱	۰	۰	۱	۰	۰	$\text{ancestor}(x, 2)$
۰	۰	۰	۰	۰	۰	۰	۰	$\text{ancestor}(x, 4)$
								...

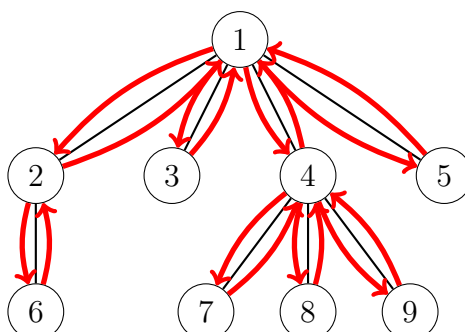
پیش‌پردازش نیازمند زمان $O(n \log n)$ است، زیرا برای هر گره $O(\log n)$ مقدار محاسبه می‌شود. سپس، هر مقدار $\text{ancestor}(x, k)$ با نمایش k به عنوان مجموع توان‌هایی از دو، در زمان $O(\log k)$ محاسبه می‌شود.

۲.۱۸ زیردرخت‌ها و مسیرها

یک آرایه پیمایش درخت گره‌های یک درخت ریشه‌دار را به ترتیبی نگه می‌دارد که جستجوی اول عمق از گره ریشه آن‌ها را ملاقات می‌کند. برای مثال، در درخت



یک جستجوی اول عمق به این صورت پیش می‌رود:



بنابراین، آرایه پیمایش درخت متناظر به شرح زیر است:

1	2	6	3	4	7	8	9	5
---	---	---	---	---	---	---	---	---

پرسش‌های زیردرخت

هر زیردرخت از یک درخت متناظر با زیرآرایه‌ای از آرایه پیمایش درخت است به طوری که عنصر اول زیرآرایه، گره ریشه است. برای مثال، زیرآرایه زیر شامل گره‌های زیردرخت گره 4 است:

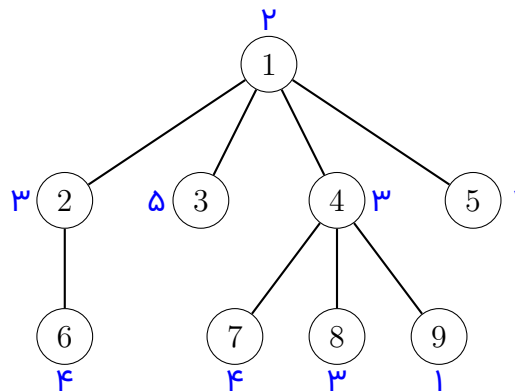
1	2	6	3	4	7	8	9	5
---	---	---	---	---	---	---	---	---

با استفاده از این ویژگی، می‌توان پرسش‌های مرتبط با زیردرخت‌های یک درخت را بهینه‌تر پردازش کرد. به عنوان مثال، مسئله‌ای را در نظر بگیرید که در آن به هر گره مقداری اختصاص داده شده است و وظیفه ما پشتیبانی از پرسش‌های زیر است:

• به‌روزرسانی مقدار یک گره

• محاسبه مجموع مقادیر در زیردرخت یک گره

درخت زیر را در نظر بگیرید که اعداد آبی‌رنگ مقادیر گره‌ها هستند. برای مثال، مجموع مقادیر زیردرخت گره 4 برابر با $11 = 3 + 4 + 3 + 1$ است.



ایده ساخت آرایه پیمایش درخت است شامل سه مقدار برای هر رأس: شناسه رأس، اندازه زیردرخت و مقدار رأس. برای نمونه آرایه درخت بالا چنین است:

رأس شناسه	1	2	6	3	4	7	8	9	5
زیردرخت اندازه	9	2	1	1	4	1	1	1	1
رأس مقدار	2	3	4	5	3	4	3	1	1

با استفاده از این آرایه، محاسبه مجموع مقادیر در هر زیردرخت با یافتن اندازه زیردرخت و سپس مقادیر رئوس متناظر امکان‌پذیر است. برای نمونه مقادیر زیردرخت رأس 4 به این صورت یافت می‌شوند:

رأس شناسه	1	2	6	3	4	7	8	9	5
زیردرخت اندازه	9	2	1	1	4	1	1	1	1
رأس مقدار	2	3	4	5	3	4	3	1	1

برای پاسخ‌دهی به پرسش‌ها به صورت کارا، ذخیره مقادیر گره‌ها در درخت ایندکس‌شده باینری یا درخت بازه‌ای کافی است. سپس، هم به‌روزرسانی مقدار و هم محاسبه مجموع مقادیر در زمان $O(\log n)$ انجام می‌شود.

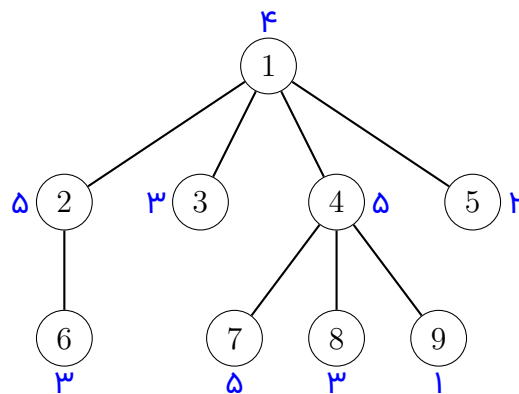
پرس وجوه ای مسیّر

با استفاده از آرایه پیمایش درخت، محاسبه کارای مجموع مقادیر روی مسیرهای از ریشه به هر گره دلخواه درخت نیز ممکن است. مسئله‌ای را در نظر بگیرید که هدف پشتیبانی از این پرس وجوها است:

• تغییر مقدار یک گره

• محاسبه مجموع مقادیر روی مسیر از ریشه تا یک گره

برای نمونه، در درخت زیر، مجموع مقادیر از ریشه تا گره ۷ برابر است با $4 + 5 + 5 = 14$:



حل مسئله مانند قبل ممکن است، اما اکنون هر مقدار سطر آخر آرایه نشان‌دهنده مجموع مقادیر روی مسیر از ریشه تا گره است. برای نمونه، آرایه زیر متناظر با درخت بالا است:

گره شناسه	1	2	6	3	4	7	8	9	5
زیردرخت اندازه	9	2	1	1	4	1	1	1	1
مسیر مجموع	4	9	12	7	9	14	12	10	6

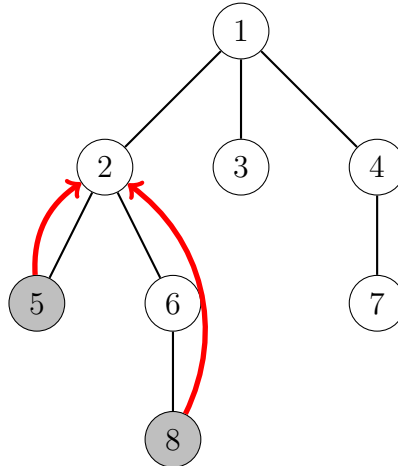
وقتی مقدار یک رأس به اندازه x افزایش می‌یابد، مجموع‌های تمام رؤس در زیردرخت آن نیز به اندازه x افزایش می‌یابند. به عنوان مثال، اگر مقدار رأس ۴ به اندازه ۱ افزایش یابد، آرایه به صورت زیر تغییر می‌کند:

رأس شناسه	1	2	6	3	4	7	8	9	5
زیردرخت اندازه	9	2	1	1	4	1	1	1	1
مسیر مجموع	4	9	12	7	10	15	13	11	6

بنابراین، برای پشتیبانی از هر دو عملیات، باید بتوانیم مقادیر یک بازه را افزایش دهیم و مقدار یک عضو منفرد را بازیابی کنیم. این کار در زمان $O(\log n)$ با استفاده از درخت پدیده‌آورده دودویی (فنوئیک) یا درخت بازه‌ای امکان‌پذیر است (بخش ۴.۹ را ببینید).

۳.۱۸ پایین‌ترین جد مشترک

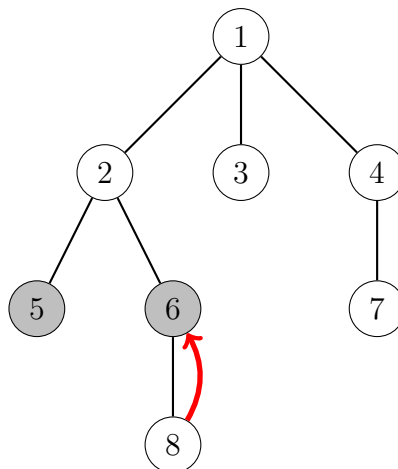
پایین‌ترین جد مشترک دو رأس در یک درخت ریشه‌دار، پایین‌ترین رأسی است که زیردرخت آن شامل هر دو رأس باشد. یک مسئله معمول، پردازش کارآمد پرسش‌هایی است که پایین‌ترین جد مشترک دو رأس را می‌خواهند.
به عنوان مثال، در درخت زیر، پایین‌ترین جد مشترک رئوس ۵ و ۸، رأس ۲ است:



در ادامه دو روش کارآمد برای یافتن کمترین جد مشترک دو رأس بررسی می‌شود.

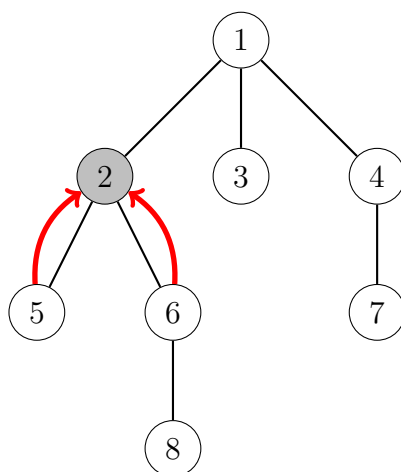
روش ۱

یک روش حل مسئله، استفاده از امکان یافتن سریع جد k ام هر رأس در درخت است. با این ایده، مسئله یافتن کمترین جد مشترک به دو بخش تقسیم می‌شود.
دو اشاره‌گر به کار می‌رود که ابتدا به دو رأس هدف اشاره دارند. نخست، یکی از اشاره‌گرها به سمت بالا برده می‌شود تا هر دو اشاره‌گر روی یک سطح قرار گیرند.
در مثال فرضی، اشاره‌گر دوم یک سطح به بالا منتقل می‌شود تا به رأس ۶ برسد؛ رأسی که هم‌سطح با رأس ۵ است:



سپس حداقل تعداد گام‌های لازم برای انتقال هم‌زمان هر دو اشاره‌گر به بالا، تا رسیدن به رأسی یکسان، تعیین می‌شود. رأسی که اشاره‌گرها در پایان به آن اشاره دارند، کمترین جد مشترک است.

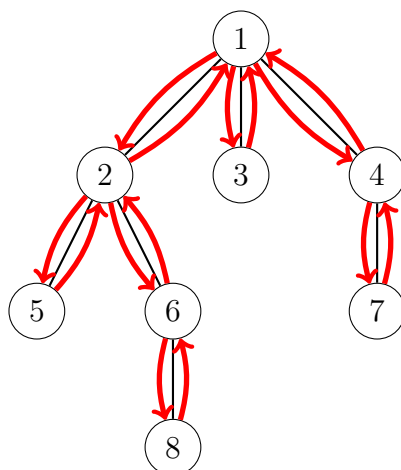
در مثال فرضی، کافی است هر دو اشاره‌گر یک گام به بالا بروند و به رأس ۲ برسند که همان کمترین جد مشترک است:



از آنجا که هر دو بخش الگوریتم با داده‌های پیش‌محاسبه‌شده در زمان $O(\log n)$ انجام می‌شوند، کمترین جد مشترک هر دو رأس در زمان $O(\log n)$ به دست می‌آید.

روش ۲

روش دیگر برای حل مسئله بر پایه آرایه پیمایش درخت است^۱. باز هم ایده، پیمایش گره‌ها با جستجوی اول عمق است:



اما از آرایه پیمایش درختی متفاوتی نسبت به قبل استفاده می‌کنیم: هر بار که جستجوی اول عمق از گره عبور می‌کند، گره را همواره به آرایه اضافه می‌کنیم، نه فقط در اولین بازدید. بنابراین گره با k فرزند، $k + 1$ بار در آرایه ظاهر می‌شود و در کل $2n - 1$ گره در آرایه وجود دارد.

دو مقدار در آرایه ذخیره می‌کنیم: شناسه گره و عمق گره در درخت. آرایه زیر متناظر با درخت بالا است:

^۱ این الگوریتم کمترین جد مشترک در $[V]$ ارائه شد. این روش گاهی **تکنیک تور اویلری** نامیده می‌شود [۶۶].

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14
گره شناسه	1	2	5	2	6	8	6	2	1	3	1	4	7	4	1
عمق	1	2	3	2	3	4	3	2	1	2	1	2	3	2	1

اکنون می‌توان کمترین نیاکان مشترک گره‌های a و b را با یافتن گره دارای کمینه عمق بین گره‌های a و b در آرایه پیدا کرد. برای نمونه، کمترین نیاکان مشترک گره‌های 5 و 8 به صورت زیر یافت می‌شود:

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14
گره شناسه	1	2	5	2	6	8	6	2	1	3	1	4	7	4	1
عمق	1	2	3	2	3	4	3	2	1	2	1	2	3	2	1

↑

گره 5 در موقعیت 2، گره 8 در موقعیت 5 قرار دارد و گره با کمینه عمق بین موقعیت‌های 2...5 گره 2 در موقعیت 3 است که عمق آن 2 می‌باشد. بنابراین، کمترین نیاکان مشترک گره‌های 5 و 8، گره 2 است.

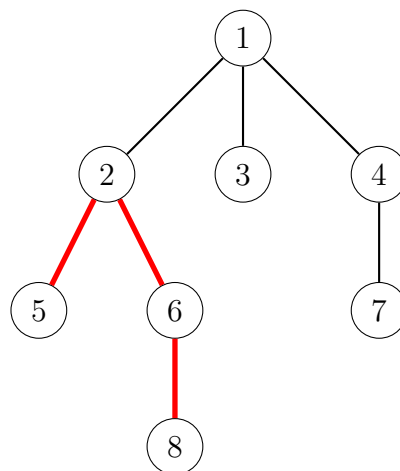
بنابراین، برای یافتن کمترین نیاکان مشترک دو رأس، پردازش یک پرس‌وجوی کمینه بازه کافی است. از آنجا که آرایه ایستا است، می‌توان این پرس‌وجوها را در زمان $O(1)$ پس از پیش‌پردازشی با زمان $O(n \log n)$ پاسخ داد.

فاصله رأس‌ها

فاصله بین دو رأس a و b برابر طول مسیر از a به b است. مشخص می‌شود که مسئله محاسبه فاصله بین رأس‌ها به یافتن کمترین نیاکان مشترک آن‌ها کاهش می‌یابد. ابتدا، درخت را به دلخواه ریشه‌دار می‌کنیم. سپس، فاصله رأس‌های a و b با استفاده از این فرمول محاسبه می‌شود:

$$\text{depth}(a) + \text{depth}(b) - 2 \cdot \text{depth}(c),$$

که در آن c کمترین نیاکان مشترک a و b است و $\text{depth}(s)$ عمق رأس s را نشان می‌دهد. برای مثال، فاصله رأس‌های 5 و 8 را در نظر بگیرید:



کمترین نیاکان مشترک رأس‌های 5 و 8 رأس 2 است. عمق این رأس‌ها عبارت است از: $\text{depth}(5) = 3$ ، $\text{depth}(8) = 4$ و $\text{depth}(2) = 2$ ، بنابراین فاصله بین رأس‌های 5 و 8 برابر است با: $3 + 4 - 2 \cdot 2 = 3$.

۴.۱۸ الگوریتم‌های برون خط

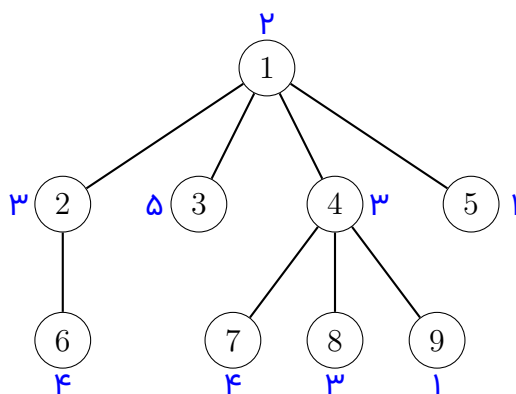
تا اینجا، الگوریتم‌های برخط برای پرس‌وجوهای درخت را بررسی کردیم. این الگوریتم‌ها پرس‌وجوها را پی‌درپی پردازش می‌کنند، به طوری که پاسخ هر پرس‌وجو پیش از دریافت پرس‌وجوی بعدی داده می‌شود.

با این حال، در بسیاری از مسائل، ویژگی برخط بودن الزامی نیست. در این بخش، بر الگوریتم‌های برون خط تمرکز می‌کنیم. به این الگوریتم‌ها مجموعه‌ای از پرس‌وجوها داده می‌شود که می‌توان آن‌ها را به هر ترتیبی پاسخ داد. طراحی یک الگوریتم برون خط معمولاً آسان‌تر از یک الگوریتم برخط است.

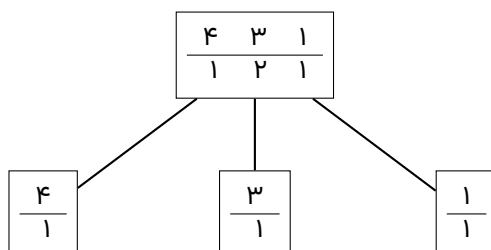
ادغام ساختمان داده‌ها

یک روش برای ساخت الگوریتم برون خط، پیمایش اول سطح یا اول عمق درخت و نگهداری ساختمان داده‌ها در رأس‌ها است. در هر رأس s ، ساختمان داده $d[s]$ بر پایه ساختمان داده‌های فرزندان s ایجاد می‌شود. سپس با استفاده از این ساختمان داده، تمام پرس‌وجوهای مربوط به s پردازش می‌شوند.

به عنوان مثال، مسئله زیر را در نظر بگیرید: درختی داده شده است که هر رأس مقداری دارد. هدف، پاسخ به پرسش‌هایی به این صورت است: «تعداد راس‌های با مقدار x در زیردرخت راس s را محاسبه کنید». برای نمونه، در درخت زیر، زیردرخت راس ۴ شامل دو راس با مقدار ۳ است.



در این مسئله، می‌توان از ساختارهای نگاشت (map) برای پاسخ به پرسش‌ها استفاده کرد. به عنوان مثال، نگاشت‌های مربوط به راس ۴ و فرزندان آن بدین صورت است:



اگر چنین داده‌ساختاری برای هر راس بسازیم، پردازش تمام پرسش‌های داده‌شده آسان است، زیرا می‌توان همه پرسش‌های مربوط به یک راس را بلافاصله پس از ساخت داده‌ساختار آن پاسخ داد. برای مثال، ساختار نگاشت بالا برای راس ۴ نشان می‌دهد زیردرخت آن شامل دو راس با مقدار ۳ است.

اما ساخت تمام داده‌ساختارها از ابتدا بسیار کند خواهد بود. در عوض، در هر راس s ، یک داده‌ساختار اولیه $d[s]$ می‌سازیم که فقط شامل مقدار s است. سپس فرزندان s را پیمایش کرده و $d[s]$ را با تمام داده‌ساختارهای $d[u]$ که u فرزند s است، ادغام می‌کنیم. برای مثال، در درخت بالا، نگاشت راس 4 با ادغام نگاشت‌های زیر ساخته می‌شود:

$$\begin{array}{|c|} \hline \frac{3}{1} \\ \hline \end{array} \quad \begin{array}{|c|} \hline \frac{4}{1} \\ \hline \end{array} \quad \begin{array}{|c|} \hline \frac{3}{1} \\ \hline \end{array} \quad \begin{array}{|c|} \hline \frac{1}{1} \\ \hline \end{array}$$

در اینجا، مپ اول ساختمان داده اولیه برای رأس 4 است، و سه مپ دیگر مربوط به رؤوس 7، 8 و 9 هستند.

ادغام در رأس s می‌تواند به شکل زیر انجام شود: فرزندان s را پیمایش می‌کنیم و در هر فرزند u ، مقادیر $d[s]$ و $d[u]$ را ادغام می‌کنیم. همیشه محتویات را از $d[u]$ به $d[s]$ کپی می‌کنیم. با این حال، پیش از این کار، در صورتی که $d[s]$ کوچک‌تر از $d[u]$ باشد، محتویات $d[s]$ و $d[u]$ را جابه‌جا (swap) می‌کنیم. با این روش، هر مقدار در طول پیمایش درخت تنها $O(\log n)$ بار کپی می‌شود که کارایی الگوریتم را تضمین می‌کند. برای جابه‌جایی بهینه محتویات دو ساختمان داده a و b ، کافی است از کد زیر استفاده کنیم:

```
); b, a (swap
```

استاندارد کتابخانه داده‌های ساختمان از a و b که صورتی در بالا کد که می‌شود تضمین می‌شود. اجرا ثابت زمان در باشند، C++

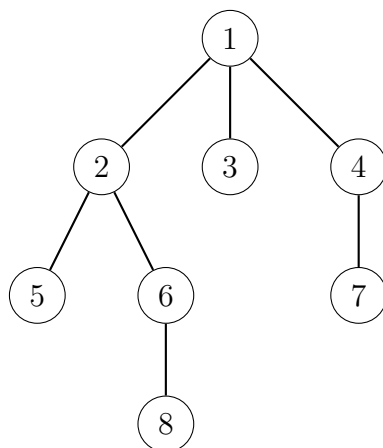
کمترین نیاکان مشترک

یک الگوریتم برون‌خط (offline) نیز برای پردازش مجموعه‌ای از پرسش‌های کمترین نیاکان مشترک وجود دارد.^۲ این الگوریتم بر پایه ساختمان داده مجموعه‌های مجزا (union-find) کار می‌کند (بخش ۲.۱۵ را ببینید)، و مزیت آن پیاده‌سازی ساده‌تر نسبت به الگوریتم‌های بررسی‌شده قبلی در این فصل است.

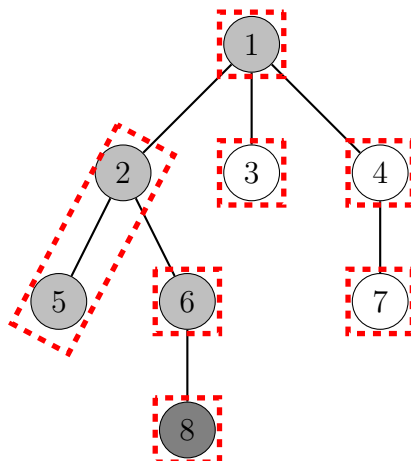
الگوریتم مجموعه‌ای از جفت‌های رؤوس را به عنوان ورودی دریافت می‌کند و برای هر جفت، کمترین نیاکان مشترک آن‌ها را مشخص می‌سازد. الگوریتم یک پیمایش اول عمق روی درخت انجام می‌دهد و مجموعه‌های مجزایی از رؤوس را نگهداری می‌کند. در ابتدا، هر رأس متعلق به یک مجموعه مجزاست. برای هر مجموعه، بالاترین رأس درخت که به آن مجموعه تعلق دارد را نیز ذخیره می‌کنیم.

هنگامی که الگوریتم یک رأس x را ملاقات می‌کند، تمامی رؤوس y را بررسی می‌کند که کمترین نیاکان مشترک x و y باید برای آن‌ها پیدا شود. اگر y قبلاً ملاقات شده باشد، الگوریتم گزارش می‌دهد که کمترین نیاکان مشترک x و y همان بالاترین رأس در مجموعه y است. سپس، پس از پردازش رأس x ، الگوریتم مجموعه‌های x و والد آن را ادغام می‌کند. برای مثال، فرض کنید می‌خواهیم کمترین جد مشترک جفت‌راس‌های (5, 8) و (2, 7) را در درخت زیر پیدا کنیم:

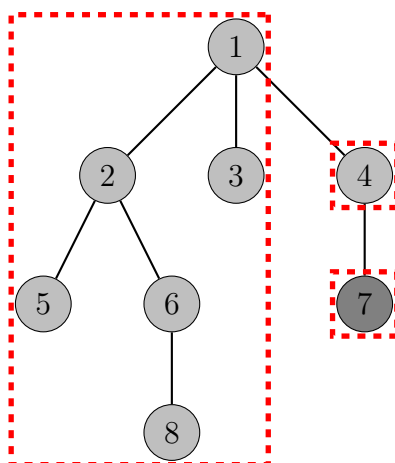
^۲این الگوریتم توسط آر. ای. تارجان (R. Tarjan) در سال ۱۹۷۹ منتشر شد [۶۵].



در درخت‌های زیر، رأس‌های خاکستری نشان‌دهنده رأس‌های پیمایش‌شده هستند و دسته‌های خط‌چین رأس‌های متعلق به یک مجموعه را مشخص می‌کنند. وقتی الگوریتم به رأس ۸ می‌رسد، متوجه می‌شود رأس ۵ قبلاً پیمایش شده و بالاترین رأس در مجموعه آن، رأس ۲ است. بنابراین، کمترین جد مشترک رأس‌های ۵ و ۸ برابر ۲ است:



سپس، هنگام پیمایش گره ۷، الگوریتم تعیین می‌کند که کمترین جد مشترک گره‌های ۷ و ۲ برابر ۱ است:



فصل ۱۹

مسیرها و مدارها

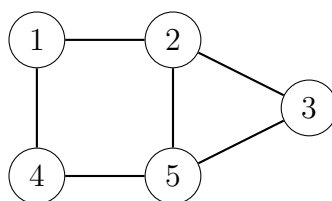
این فصل روی دو نوع مسیر در گراف تمرکز دارد:

- یک **مسیر اویلری** مسیری است که از هر یال دقیقاً یک بار عبور می‌کند.
- یک **مسیر هامیلتونی** مسیری است که از هر رأس دقیقاً یک بار بازدید می‌کند.

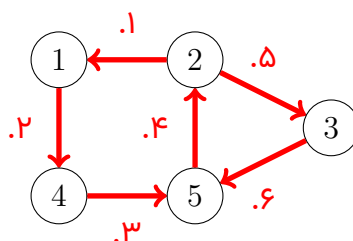
اگرچه مسیرهای اویلری و هامیلتونی در نگاه اول مفاهیم مشابهی به نظر می‌رسند، مسائل محاسباتی مربوط به آن‌ها بسیار متفاوت است. مشخص شده قاعده‌ای وجود دارد که تعیین می‌کند آیا یک گراف شامل مسیر اویلری است یا خیر، و همچنین الگوریتم کارآمدی برای یافتن چنین مسیری در صورت وجود در دست است. در مقابل، بررسی وجود مسیر هامیلتونی یک مسئله NP-hard است و هیچ الگوریتم کارآمدی برای حل آن شناخته نشده است.

۱.۱۹ مسیرهای اویلری

یک **مسیر اویلری**^۱ مسیری است که دقیقاً یک بار از هر یال گراف عبور می‌کند. برای مثال، گراف

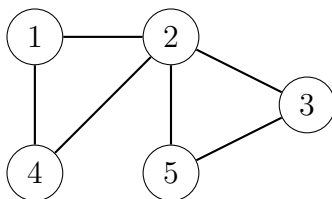


دارای یک مسیر اویلری از رأس ۲ به رأس ۵ است:

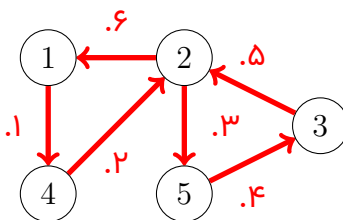


^۱ل. اویلر چنین مسیرهایی را در سال ۱۷۳۶ هنگامی که مسئله مشهور پل‌های کونیگسبرگ را حل کرد، بررسی نمود. این رویداد نقطه پیدایش نظریه گراف بود.

یک مدار اویلری مسیر اویلری است که در همان رأس شروع، پایان می‌یابد. برای مثال، گراف



دارای یک مدار اویلری است که از رأس ۱ شروع شده و در رأس ۱ پایان می‌یابد:



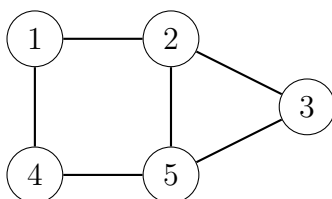
شرایط وجود

وجود مسیرها و مدارهای اویلری به درجه راس‌ها بستگی دارد. نخست، یک گراف بدون جهت دارای مسیر اویلری است دقیقاً زمانی که تمام یال‌ها به یک مؤلفه همبندی یکسان تعلق داشته باشند و

- درجه هر راس زوج باشد یا

- درجه دقیقاً دو راس فرد باشد، و درجه تمام راس‌های دیگر زوج باشد.

در حالت اول، هر مسیر اویلری یک مدار اویلری نیز هست. در حالت دوم، راس‌های با درجه فرد، راس‌های شروع و پایان یک مسیر اویلری هستند که مدار اویلری نیست. به عنوان مثال، در گراف



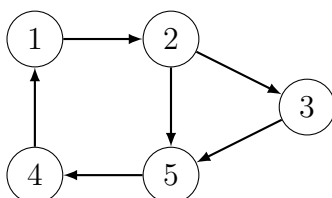
راس‌های ۱، ۳ و ۴ دارای درجه ۲ هستند، و راس‌های ۲ و ۵ دارای درجه ۳ هستند. دقیقاً دو راس درجه فرد دارند، بنابراین یک مسیر اویلری بین راس‌های ۲ و ۵ وجود دارد، اما گراف شامل هیچ مدار اویلری نیست.

در گراف جهت‌دار، تمرکز روی درجه‌های ورودی و خروجی راس‌ها است. گراف جهت‌دار دقیقاً زمانی شامل مسیر اویلری است که همه یال‌ها متعلق به یک مؤلفه همبند باشند و

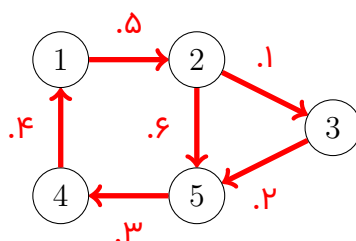
- در هر راس، درجه ورودی برابر با درجه خروجی باشد، یا

- در یک راس، درجه ورودی یکی بیشتر از درجه خروجی باشد، در راسی دیگر، درجه خروجی یکی بیشتر از درجه ورودی باشد، و در تمام راس‌های دیگر، درجه ورودی برابر با درجه خروجی باشد.

در حالت اول، هر مسیر اویلری یک دور اویلری نیز هست، و در حالت دوم، گراف شامل مسیر اویلری است که از راسی با درجه خروجی بزرگتر شروع می‌شود و در راسی با درجه ورودی بزرگتر پایان می‌یابد.
برای مثال، در گراف



راس‌های ۱، ۳ و ۴ همگی دارای درجه ورودی ۱ و درجه خروجی ۱ هستند، راس ۲ دارای درجه ورودی ۱ و درجه خروجی ۲ است، و راس ۵ دارای درجه ورودی ۲ و درجه خروجی ۱ است. بنابراین، گراف شامل یک مسیر اویلری از راس ۲ به راس ۵ است:



الگوریتم هیروولتزر

الگوریتم هیروولتزر^۲ روشی کارآمد برای ساخت دور اویلری است. الگوریتم از چند مرحله تشکیل شده است که هر مرحله یال‌های جدیدی به دور اضافه می‌کند. البته فرض بر این است که گراف شامل دور اویلری باشد؛ در غیر این صورت الگوریتم هیروولتزر نمی‌تواند آن را پیدا کند.

ابتدا، الگوریتم مداری می‌سازد که شامل برخی از یال‌های گراف است (نه لزوماً همه آن‌ها). سپس الگوریتم مدار را گام‌به‌گام با افزودن زیرمدارها گسترش می‌دهد. این فرایند تا افزوده شدن تمام یال‌ها به مدار ادامه می‌یابد.

الگوریتم با یافتن گره x که متعلق به مدار است ولی یال خروجی خارج از مدار دارد، مدار را گسترش می‌دهد. الگوریتم مسیر جدیدی از گره x می‌سازد که فقط شامل یال‌های خارج از مدار است. در نهایت، مسیر به گره x بازمی‌گردد و یک زیرمدار ایجاد می‌کند.

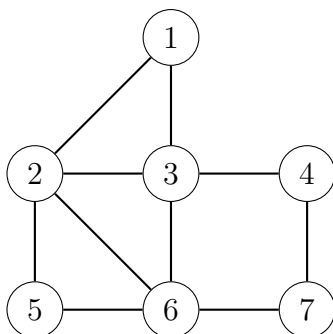
اگر گراف فقط شامل مسیر اویلری باشد، همچنان می‌توان از الگوریتم هیروولتزر استفاده کرد؛ کافی است یال اضافی به گراف اضافه شود و پس از ساخت مدار، آن یال حذف گردد. مثلاً در گراف غیرجهت‌دار، یال اضافی بین دو گره با درجه فرد اضافه می‌شود.

در ادامه، نحوه ساخت مدار اویلری در گراف غیرجهت‌دار با الگوریتم هیروولتزر نشان داده می‌شود.

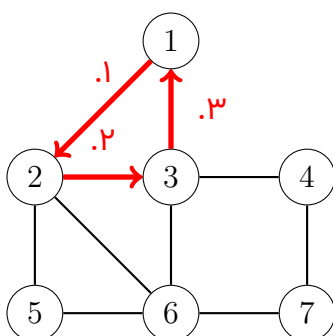
^۲این الگوریتم در سال ۱۸۷۳ پس از مرگ هیروولتزر منتشر شد [۳۵].

مثال

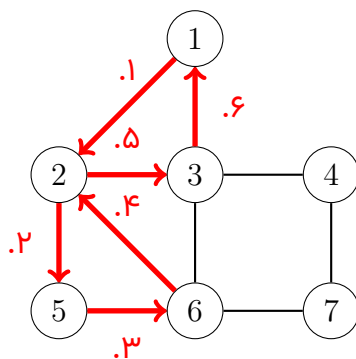
گراف زیر را در نظر بگیرید:



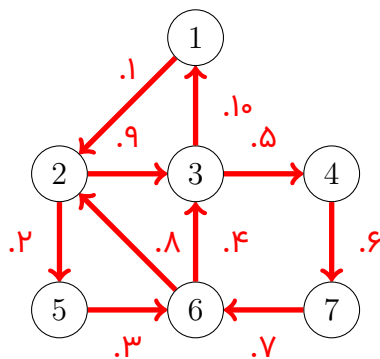
فرض کنید الگوریتم ابتدا مداری از گره ۱ بسازد. یک مدار ممکن $1 \rightarrow 2 \rightarrow 3 \rightarrow 1$ است:



پس از این، الگوریتم زیرمدار $2 \rightarrow 5 \rightarrow 6 \rightarrow 2$ را به مدار اضافه می‌کند:



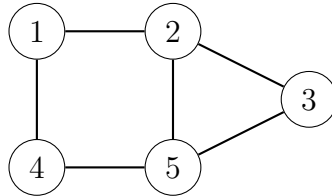
در نهایت، الگوریتم زیرمدار $6 \rightarrow 3 \rightarrow 4 \rightarrow 7 \rightarrow 6$ را به مدار اضافه می‌کند:



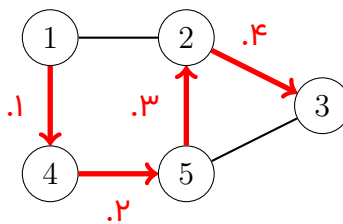
اکنون تمام یال‌ها در مدار گنجانده شده‌اند، بنابراین یک مدار اویلری با موفقیت ساخته شد.

۲.۱۹ مسیرهای همیلتونی

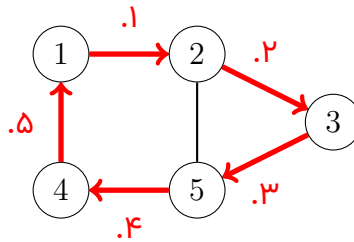
یک **مسیر همیلتونی** مسیری است که دقیقاً یک بار از هر رأس گراف عبور می‌کند. به عنوان مثال، گراف



شامل یک مسیر همیلتونی از رأس ۱ به رأس ۳ است:



اگر یک مسیر همیلتونی در همان رأس شروع، پایان یابد، **دور همیلتونی** نامیده می‌شود. گراف بالا دارای یک دور همیلتونی نیز هست که از رأس ۱ شروع و به آن ختم می‌شود:



وجود

هیچ روش کارآمدی برای بررسی وجود مسیر همیلتونی در یک گراف شناخته نشده است و این مسئله NP-hard است. با این حال، در برخی حالت‌های خاص می‌توان مطمئن بود که گراف حاوی مسیر همیلتونی است.

یک مشاهده ساده این است که اگر گراف کامل باشد، یعنی بین هر جفت رأس یک یال وجود داشته باشد، حاوی مسیر همیلتونی نیز هست. همچنین نتایج قوی‌تری به دست آمده است:

• **قضیه دیراک:** اگر درجه هر رأس حداقل $n/2$ باشد، گراف شامل مسیر همیلتونی است.

• **قضیه اور:** اگر مجموع درجات هر جفت رأس غیرمجاور حداقل n باشد، گراف شامل مسیر همیلتونی است.

ویژگی مشترک این قضایا و نتایج دیگر این است که اگر گراف دارای تعداد زیادی یال باشد، وجود مسیر همیلتونی را تضمین می‌کنند. این امر منطقی است، زیرا هرچه گراف یال‌های بیشتری داشته باشد، امکان‌های بیشتری برای ساخت یک مسیر همیلتونی وجود دارد.

ساخت

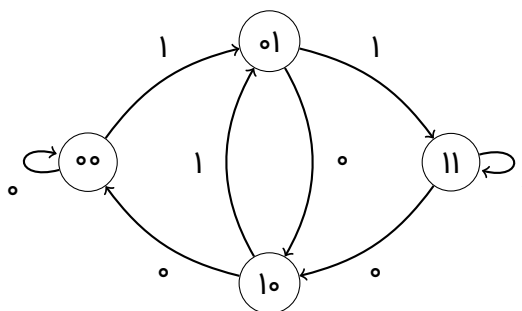
از آنجا که روش کارآمدی برای بررسی وجود مسیر هامیلتونی وجود ندارد، مشخص است که روشی نیز برای ساخت کارآمد این مسیر وجود ندارد، زیرا در غیر این صورت می‌توانستیم تلاش کنیم مسیر را بسازیم و ببینیم آیا وجود دارد یا خیر. یک روش ساده برای جستجوی مسیر هامیلتونی استفاده از الگوریتم عقب‌گرد است که تمام حالت‌های ممکن برای ساخت مسیر را بررسی می‌کند. پیچیدگی زمانی چنین الگوریتمی حداقل $O(n!)$ است، زیرا $n!$ روش مختلف برای انتخاب ترتیب n رأس وجود دارد. یک راهکار کارآمدتر مبتنی بر برنامه‌نویسی پویا است (فصل ۵.۱۰ را ببینید). ایده محاسبه مقادیر تابع $\text{possible}(S, x)$ است، که در آن S زیرمجموعه‌ای از رأس‌ها و x یکی از رأس‌ها است. تابع مشخص می‌کند آیا مسیر همیلتونی وجود دارد که رأس‌های S را پیمایش کند و در رأس x پایان یابد. پیاده‌سازی این راهکار در زمان $O(2^n n^2)$ ممکن است.

۳.۱۹ دنباله‌های دی بروین

یک **دنباله دی بروین** رشته‌ای است که شامل هر رشته به طول n دقیقاً یک بار به عنوان زیررشته است، برای یک الفبای ثابت با k نویسه. طول چنین رشته‌ای $k^n + n - 1$ نویسه است. به عنوان مثال، وقتی $n = 3$ و $k = 2$ باشد، یک نمونه از دنباله دی بروین برابر است با

0001011100.

زیررشته‌های این رشته تمام ترکیبات سه بیتی هستند: ۰۰۰، ۰۰۱، ۰۱۰، ۰۱۱، ۱۰۰، ۱۰۱، ۱۱۰ و ۱۱۱. مشخص می‌شود هر دنباله دی بروین متناظر با یک مسیر اویلری در یک گراف است. ایده ساخت گرافی است که در آن هر رأس شامل یک رشته از $n - 1$ نویسه باشد و هر یال یک نویسه به رشته اضافه کند. گراف زیر متناظر با سناریوی بالا است:



یک مسیر اویلری در این گراف متناظر با رشته‌ای است که شامل تمام رشته‌های به طول n باشد. رشته شامل نویسه‌های رأس شروع و تمام نویسه‌های یال‌ها است. رأس شروع دارای $n - 1$ نویسه است و k^n نویسه در یال‌ها وجود دارد، بنابراین طول رشته $k^n + n - 1$ است.

۴.۱۹ گشت‌های اسب

یک **گشت اسب** دنباله‌ای از حرکات یک اسب روی صفحه شطرنج $n \times n$ طبق قوانین شطرنج است به طوری که اسب هر خانه را دقیقاً یک بار ملاقات کند. یک گشت اسب بسته نامیده می‌شود اگر اسب در نهایت به خانه شروع برگردد و در غیر این صورت یک گشت باز نامیده می‌شود.

به عنوان مثال، در اینجا یک گشت اسب باز روی صفحه 5×5 آمده است:

1	4	11	16	25
12	17	2	5	10
3	20	7	24	15
18	13	22	9	6
21	8	19	14	23

گشت اسب معادل مسیر همیلتونی در گرافی است که رأس‌های آن خانه‌های صفحه شطرنج هستند، و دو رأس با یال به هم متصل‌اند اگر اسب طبق قوانین شطرنج بتواند بین آن خانه‌ها حرکت کند. یک روش طبیعی برای ساخت گشت اسب، روش عقب‌گرد است. می‌توان با استفاده از روش‌های ابتکاری (heuristics) که اسب را برای یافتن سریع گشت کامل هدایت می‌کنند، کارایی جستجو را افزایش داد.

قاعده وارنسدورف

قاعده وارنسدورف روشی ابتکاری، ساده و کارآمد برای یافتن گشت اسب است^۳. با استفاده از این قاعده، ساخت کارآمد گشت حتی در صفحه‌های بزرگ نیز ممکن است. ایده این است: اسب همواره به خانه‌ای برود که تعداد حرکات ممکن بعدی از آن خانه تا حد امکان کمترین باشد. برای مثال، در وضعیت زیر، پنج خانه ممکن وجود دارد که اسب می‌تواند به آن‌ها برود (خانه‌های $a \dots e$):

1				a
		2		
b				e
	c		d	

در این وضعیت، قاعده وارنسدورف اسب را به خانه a می‌برد، زیرا پس از این انتخاب، تنها یک حرکت ممکن باقی می‌ماند. انتخاب‌های دیگر اسب را به خانه‌هایی می‌برند که از آن‌ها سه حرکت ممکن وجود دارد.

^۳ این روش ابتکاری در کتاب وارنسدورف [۶۹] در سال ۱۸۲۳ پیشنهاد شد. همچنین الگوریتم‌های چندجمله‌ای برای یافتن گشت اسب وجود دارند [۵۲]، اما پیچیده‌ترند.

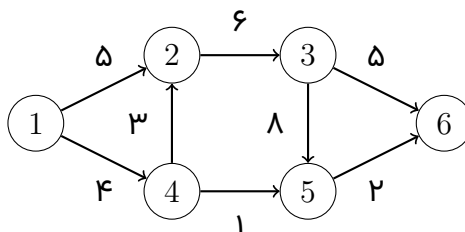
فصل ۲۰

شاره‌ها و برش‌ها

در این فصل، روی دو مسئله زیر تمرکز می‌کنیم:

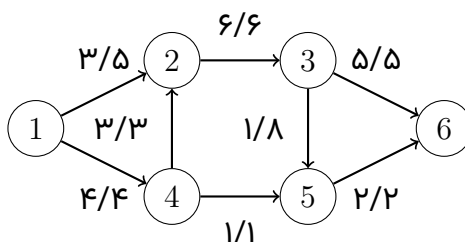
- **یافتن بیشینه شار:** بیشترین مقدار شار که می‌توان از یک رأس به رأس دیگر فرستاد چقدر است؟
- **یافتن کمینه برش:** مجموعه‌ای از یال‌ها با کمینه وزن که دو رأس گراف را از هم جدا کند چیست؟

ورودی هر دو مسئله یک گراف جهت‌دار و وزن‌دار شامل دو رأس خاص است: مبدأ رأسی بدون یال ورودی است، و مقصد رأسی بدون یال خروجی است. به عنوان مثال، از گراف زیر استفاده می‌کنیم که رأس ۱ مبدأ و رأس ۶ مقصد است:



بیشینه شار

در مسئله **بیشینه شار**، هدف فرستادن بیشترین مقدار ممکن شار از مبدأ به مقصد است. وزن هر یال ظرفیتی است که شار عبوری از آن یال را محدود می‌کند. در هر رأس میانی، مقدار شار ورودی و خروجی باید برابر باشد. برای نمونه، بیشینه مقدار شار در گراف مثال برابر ۷ است. تصویر زیر نحوه هدایت شار را نشان می‌دهد:

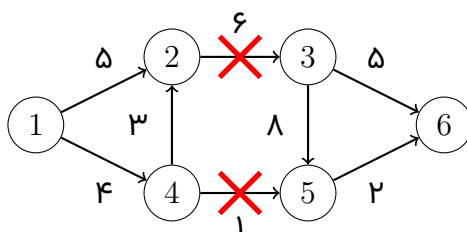


نمادگذاری v/k به این معنی است که جریانی به اندازه v واحد از یالی با ظرفیت k واحد عبور داده شده است. اندازه جریان 7 است، زیرا مبدأ $3 + 4$ واحد جریان می‌فرستد و مقصد $5 + 2$ واحد جریان دریافت می‌کند. به سادگی می‌توان دریافت که این جریان بیشینه است، زیرا ظرفیت کل یال‌های منتهی به مقصد برابر 7 است.

برش کمینه

در مسئله **برش کمینه** (minimum cut)، هدف ما حذف مجموعه‌ای از یال‌ها از گراف است به طوری که پس از حذف هیچ مسیری از مبدأ به مقصد وجود نداشته باشد و مجموع وزن یال‌های حذف شده کمینه باشد.

اندازه کمینه برش در گراف نمونه برابر 7 است. کافی است یال‌های $2 \rightarrow 3$ و $4 \rightarrow 5$ را حذف کنیم:



پس از حذف این یال‌ها، هیچ مسیری از مبدأ به مقصد وجود نخواهد داشت. اندازه برش برابر 7 است، زیرا وزن‌های یال‌های حذف شده 6 و 1 هستند. این برش کمینه است، زیرا هیچ روش معتبری برای حذف یال‌ها از گراف وجود ندارد به شکلی که مجموع وزن آن‌ها کمتر از 7 شود.

تصادفی نیست که اندازه بیشینه یک جریان و اندازه کمینه یک برش در مثال بالا یکسان است. مشخص می‌شود که یک جریان بیشینه و یک برش کمینه همیشه هم‌اندازه‌اند؛ بنابراین این مفاهیم دو روی یک سکه‌اند.

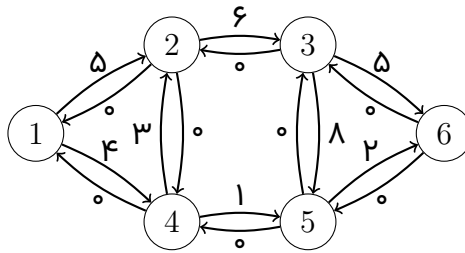
در ادامه الگوریتم فورد-فالکرسون را بررسی می‌کنیم که می‌توان از آن برای یافتن جریان بیشینه و برش کمینه در یک گراف استفاده کرد. این الگوریتم همچنین به ما کمک می‌کند تا بفهمیم چرا این دو مقدار همواره برابر هستند.

۱.۲۰ الگوریتم فورد-فالکرسون

الگوریتم فورد-فالکرسون [۲۵] جریان بیشینه را در یک گراف می‌یابد. الگوریتم با جریان تهی آغاز می‌شود، و در هر گام مسیری از مبدأ به مقصد پیدا می‌کند که جریان بیشتری تولید می‌نماید. در نهایت، هنگامی که الگوریتم نتواند جریان را بیش از این افزایش دهد، جریان بیشینه یافته شده است.

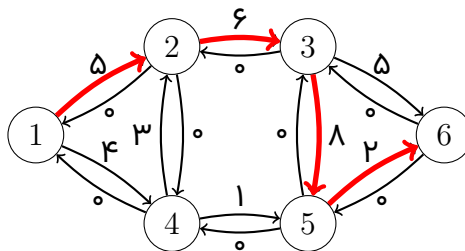
این الگوریتم از نمایش ویژه‌ای از گراف استفاده می‌کند که در آن هر یال اصلی یک یال معکوس در جهت مخالف دارد. وزن هر یال نشان می‌دهد چه مقدار شار دیگر می‌توان از آن عبور داد. در آغاز الگوریتم، وزن هر یال اصلی برابر با ظرفیت آن یال و وزن هر یال معکوس برابر با صفر است.

نمایش جدید برای گراف نمونه به شکل زیر است:

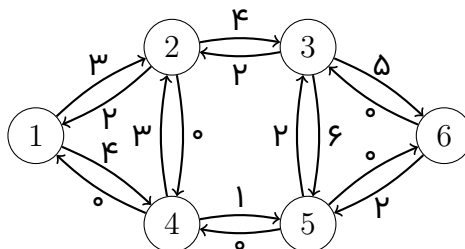


توصیف الگوریتم

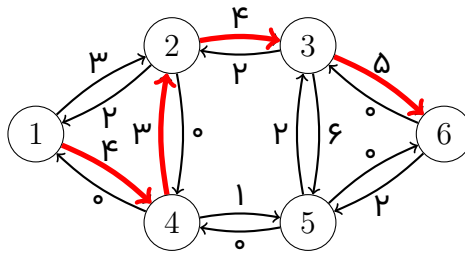
الگوریتم فورد-فالکرسون از چندین مرحله تشکیل شده است. در هر مرحله، الگوریتم مسیری از مبدأ به مقصد پیدا می‌کند به طوری که هر یال در این مسیر دارای وزنی مثبت باشد. اگر بیش از یک مسیر ممکن وجود داشته باشد، می‌توانیم هر یک از آنها را انتخاب کنیم. برای مثال، فرض کنید مسیر زیر را انتخاب کنیم:



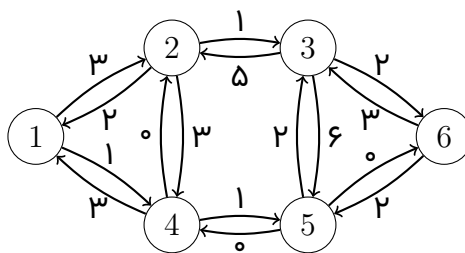
پس از انتخاب مسیر، شار به اندازه x واحد افزایش می‌یابد که در آن x کمترین وزن یال روی مسیر است. علاوه بر این، وزن هر یال روی مسیر به اندازه x کاهش و وزن هر یال معکوس به اندازه x افزایش می‌یابد. در مسیر بالا، وزن یال‌ها برابر ۵، ۶، ۸ و ۲ است. کمترین وزن ۲ است، بنابراین شار به اندازه ۲ افزایش می‌یابد و گراف جدید به شکل زیر خواهد بود:



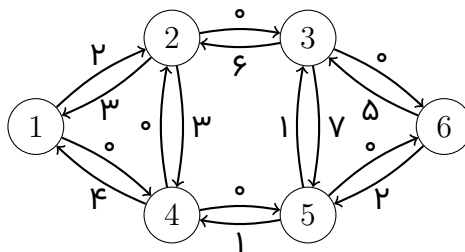
ایده این است که افزایش شار، مقدار شاری را که می‌تواند در آینده از یال‌ها عبور کند کاهش می‌دهد. از سوی دیگر، می‌توان بعداً با استفاده از یال‌های معکوس گراف، شار را لغو کرد اگر مشخص شود که هدایت شار از مسیری دیگر سودمندتر است. الگوریتم تا زمانی که مسیری با یال‌های دارای وزن مثبت از مبدأ به مقصد وجود داشته باشد، شار را افزایش می‌دهد. در این مثال، مسیر بعدی می‌تواند به صورت زیر باشد:



کمینه وزن یال در این مسیر ۳ است، بنابراین این مسیر شار را به اندازه ۳ واحد افزایش می‌دهد، و شار کل پس از پردازش مسیر برابر ۵ می‌شود. گراف جدید به صورت زیر خواهد بود:



هنوز دو مرحله دیگر تا رسیدن به بیشینه شار نیاز است. برای نمونه می‌توان مسیرهای $1 \rightarrow 2 \rightarrow 3 \rightarrow 6$ و $1 \rightarrow 4 \rightarrow 5 \rightarrow 3 \rightarrow 6$ را برگزید. هر دو مسیر شار را به اندازه ۱ واحد افزایش می‌دهند و گراف نهایی به صورت زیر خواهد بود:



دیگر امکان افزایش شار وجود ندارد، زیرا مسیری با وزن یال‌های مثبت از مبدأ به مقصد موجود نیست. بنابراین، الگوریتم پایان می‌یابد و بیشینه شار برابر ۷ است.

یافتن مسیره‌ها

الگوریتم فورد-فالکرسون مشخص نمی‌کند که مسیرهای افزایشده شار چگونه باید انتخاب شوند. در هر حال، الگوریتم دیر یا زود متوقف خواهد شد و بیشینه شار را به‌درستی پیدا می‌کند. با این وجود، کارایی الگوریتم به نحوه انتخاب مسیرها وابسته است. یک روش ساده برای یافتن مسیرها استفاده از جستجوی اول‌سطح عمق است. معمولاً این روش به‌خوبی کار می‌کند، اما در بدترین حالت، هر مسیر شار را تنها به اندازه ۱ واحد افزایش می‌دهد و الگوریتم کند عمل خواهد کرد. خوشبختانه می‌توان با استفاده از یکی از فنون زیر از این وضعیت جلوگیری کرد:

الگوریتم ادموندز-کارپ [۱۸] هر مسیر را طوری انتخاب می‌کند که تعداد یال‌های آن مسیر کمترین مقدار ممکن باشد. این کار با به‌کارگیری جستجوی اول‌سطح به‌جای جستجوی

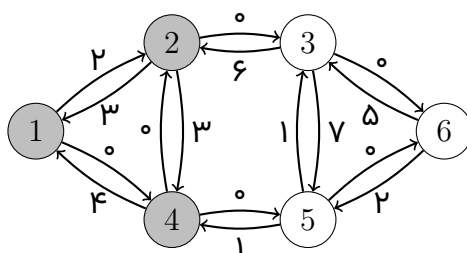
اول عمق برای یافتن مسیرها انجام پذیر است. می توان اثبات کرد این امر متضمن افزایش سریع شار است و پیچیدگی زمانی الگوریتم برابر $O(m^2n)$ خواهد بود.

الگوریتم مقیاس بندی [۲] از جستجوی اول عمق جهت یافتن مسیرهایی استفاده می کند که در آن ها وزن هر یال دست کم برابر با یک مقدار آستانه است. در آغاز، مقدار آستانه عدد بزرگی در نظر گرفته می شود، مثلاً مجموع تمام وزن های یال های گراف. هر بار که یافتن مسیر ناممکن گردد، مقدار آستانه بر ۲ تقسیم می شود. پیچیدگی زمانی الگوریتم $O(m^2 \log c)$ است که در آن c نمایانگر مقدار آستانه اولیه است.

در عمل، پیاده سازی الگوریتم مقیاس بندی آسان تر است، زیرا می توان از جستجوی اول عمق برای یافتن مسیرها استفاده کرد. هر دو الگوریتم برای مسائلی که معمولاً در مسابقات برنامه نویسی مطرح می شوند، به اندازه کافی کارآمد هستند.

برش های کمی نه

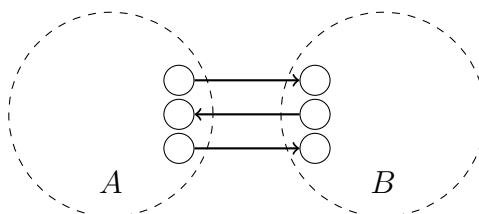
مشخص می شود به محض اینکه الگوریتم فورد-فالکرسون شار بیشینه را پیدا کند، یک برش کمینه را نیز مشخص کرده است. فرض کنید A مجموعه ای از رأس ها باشد که می توان با استفاده از یال های با وزن مثبت از مبدأ به آن ها دسترسی پیدا کرد. در گراف نمونه، A شامل رأس های ۱، ۲ و ۴ است:



اکنون برش کمینه شامل یال هایی از گراف اصلی است که از رأسی در A شروع شده، به رأسی خارج از A ختم می شوند، و ظرفیت آن ها در شار بیشینه به طور کامل استفاده شده است. در گراف فوق، این یال ها $2 \rightarrow 3$ و $4 \rightarrow 5$ هستند که متناظر با برش کمینه $6 + 1 = 7$ می باشند.

چرا شار تولید شده توسط الگوریتم بیشینه است و چرا برش کمینه است؟ دلیل آن این است که یک گراف نمی تواند شامل شاری باشد که اندازه آن از وزن هر برشی از گراف بزرگتر باشد. بنابراین، همواره هنگامی که یک شار و یک برش هم اندازه باشند، آن ها شار بیشینه و برش کمینه هستند.

برش دلخواهی از گراف را در نظر بگیرید به طوری که مبدأ متعلق به A باشد، مقصد متعلق به B باشد و یال هایی بین این دو مجموعه وجود داشته باشد:



اندازه برش مجموع یال هایی است که از A به B می روند. این مقدار کران بالایی برای جریان در گراف است، زیرا جریان باید از A به B عبور کند. بنابراین اندازه بیشینه شار کوچک تر یا مساوی اندازه هر برشی در گراف است.

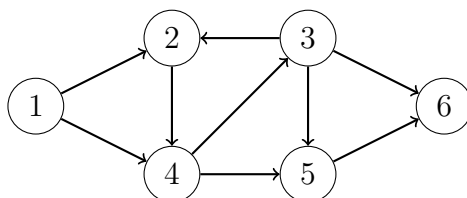
از طرف دیگر، الگوریتم فورد-فالکرسون جریانی تولید می‌کند که اندازه‌اش دقیقاً برابر با اندازه یک برش در گراف است. بنابراین، این جریان باید بیشینه شار و آن برش باید کمینه برش باشد.

۲.۲۰ مسیرهای مجزا

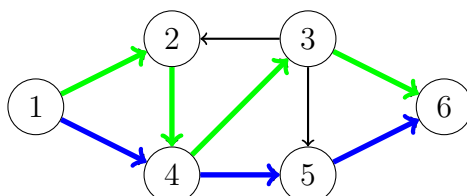
بسیاری از مسائل گراف را می‌توان با کاهش آن‌ها به مسئله بیشینه شار حل کرد. اولین نمونه از چنین مسئله‌ای به این صورت است: یک گراف جهت‌دار با یک مبدأ و یک مقصد داده شده است، و وظیفه ما یافتن بیشینه تعداد مسیرهای مجزا از مبدأ به مقصد است.

مسیرهای مجزا از یال

ابتدا بر روی مسئله یافتن بیشینه تعداد مسیرهای مجزا از یال از مبدأ به مقصد تمرکز می‌کنیم. این بدان معناست که باید مجموعه‌ای از مسیرها را تشکیل دهیم به طوری که هر یال حداکثر در یک مسیر ظاهر شود. به عنوان مثال، گراف زیر را در نظر بگیرید:



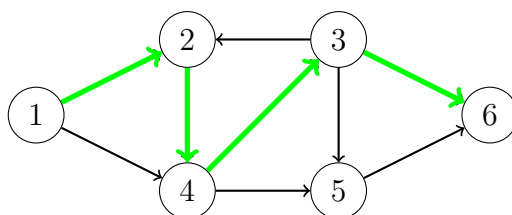
در این گراف، بیشینه تعداد مسیرهای مجزا از یال برابر ۲ است. می‌توانیم مسیرهای $1 \rightarrow 2 \rightarrow 4 \rightarrow 3 \rightarrow 6$ و $1 \rightarrow 4 \rightarrow 5 \rightarrow 6$ را به صورت زیر انتخاب کنیم:



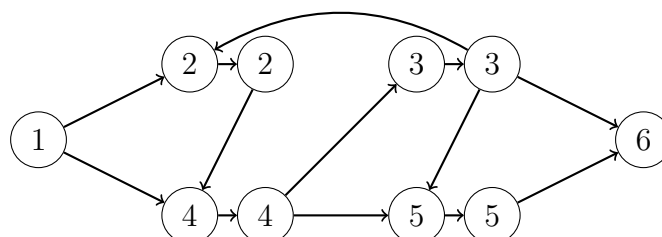
مشخص می‌شود که حداکثر تعداد مسیرهای یال‌مجزا برابر با بیشینه شار گراف است، با این فرض که ظرفیت هر یال برابر یک باشد. پس از ساخت بیشینه شار، مسیرهای یال‌مجزا را می‌توان به صورت حریصانه با دنبال کردن مسیرها از مبدأ تا مقصد یافت.

مسیرهای رأس‌مجزا

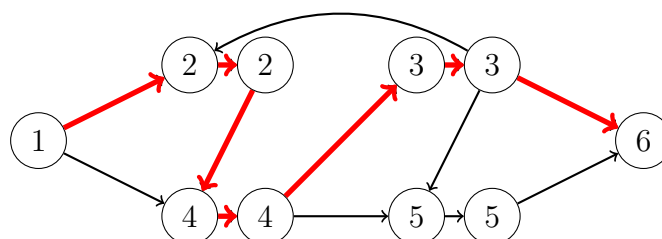
حال مسئله دیگری را بررسی می‌کنیم: یافتن بیشینه تعداد مسیرهای رأس‌مجزا از مبدأ به مقصد. در این مسئله، هر رأس، به جز مبدأ و مقصد، می‌تواند حداکثر در یک مسیر ظاهر شود. تعداد مسیرهای رأس‌مجزا ممکن است کمتر از تعداد مسیرهای یال‌مجزا باشد. برای نمونه، در گراف قبلی، بیشینه تعداد مسیرهای رأس‌مجزا برابر ۱ است:



این مسئله را نیز می‌توان به مسئله بیشینه شار کاهش داد. از آنجا که هر رأس می‌تواند حداکثر در یک مسیر ظاهر شود، باید شاری را که از رأس‌ها عبور می‌کند محدود کنیم. یک روش استاندارد برای این کار، تقسیم هر رأس به دو رأس است به طوری که رأس نخست یال‌های ورودی رأس اصلی را داشته باشد، رأس دوم یال‌های خروجی رأس اصلی را شامل شود، و یک یال جدید از رأس نخست به رأس دوم وجود داشته باشد. در مثال ما، گراف به صورت زیر تبدیل می‌شود:



شار بیشینه برای گراف به صورت زیر است:



بنابراین بیشینه تعداد مسیرهای مجزا از راس ۱ به مقصد برابر با ۱ است.

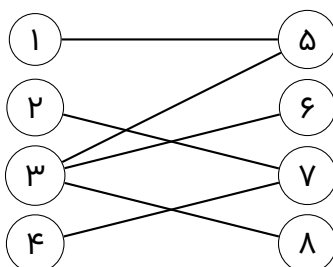
۳.۲۰ تطابق‌های بیشینه

مسئله **تطابق بیشینه** به دنبال یافتن مجموعه‌ای با بیشترین اندازه از جفت‌راس‌ها در گراف بدون جهت است به طوری که هر جفت با یک یال متصل باشد و هر راس حداکثر به یک جفت تعلق داشته باشد.

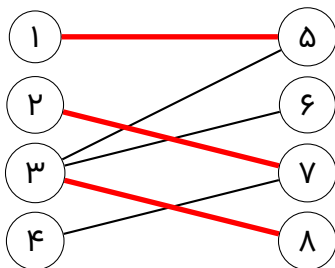
الگوریتم‌های چندجمله‌ای برای یافتن تطابق بیشینه در گراف‌های کلی وجود دارند [۱۷]، اما این الگوریتم‌ها پیچیده‌اند و به‌ندرت در مسابقات برنامه‌نویسی دیده می‌شوند. با این حال، در گراف‌های دوبخشی، مسئله تطابق بیشینه بسیار ساده‌تر حل می‌شود، زیرا می‌توانیم آن را به مسئله شار بیشینه تبدیل کنیم.

یافتن تطابق‌های بیشینه

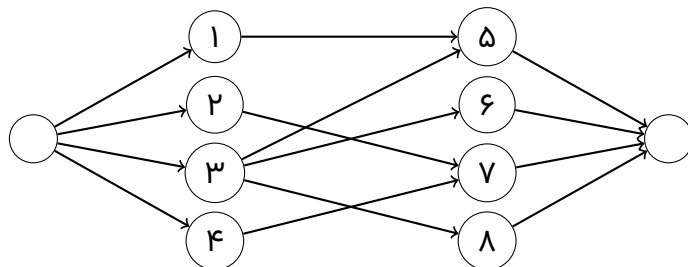
گره‌های یک گراف دوبخشی همواره می‌توانند به دو گروه تقسیم شوند، به طوری که تمام یال‌های گراف از گروه سمت چپ به گروه سمت راست بروند. به عنوان مثال، در گراف دوبخشی زیر، گروه‌ها $\{1, 2, 3, 4\}$ و $\{5, 6, 7, 8\}$ هستند.



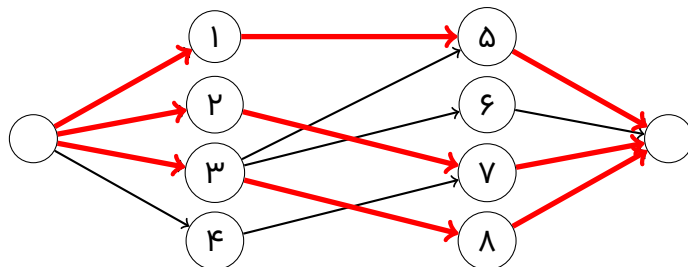
اندازه تطابق بیشینه این گراف برابر ۳ است:



می‌توانیم مسئله تطابق بیشینه دوبخشی را با افزودن دو گره جدید به گراف به مسئله شار بیشینه تقلیل دهیم: یک مبدأ و یک مقصد. همچنین یال‌هایی از مبدأ به هر گره سمت چپ، و از هر گره سمت راست به مقصد اضافه می‌کنیم. پس از این کار، اندازه شار بیشینه در گراف برابر با اندازه تطابق بیشینه در گراف اولیه خواهد بود. برای نمونه، تقلیل گراف بالا به شکل زیر است:



شار بیشینه این گراف به صورت زیر است:

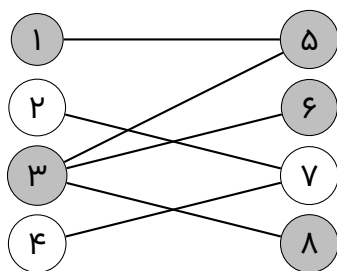


قضیه هال

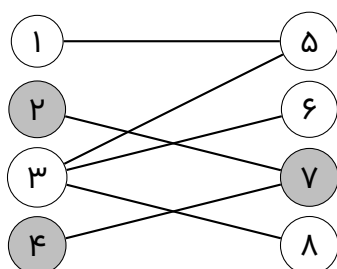
از **قضیه هال** می‌توان برای تشخیص وجود تطابقی در یک گراف دوبخشی استفاده کرد که تمام رأس‌های سمت چپ یا راست را شامل شود. اگر تعداد رأس‌های چپ و راست برابر باشد، قضیه هال مشخص می‌کند آیا امکان ساخت یک **تطابق کامل** شامل تمام رأس‌های گراف وجود دارد یا خیر.

فرض کنید می‌خواهیم تطابقی شامل تمام رأس‌های سمت چپ بیابیم. فرض کنید X زیرمجموعه‌ای دلخواه از رأس‌های سمت چپ و $f(X)$ مجموعه همسایه‌های آن‌ها باشد. طبق قضیه هال، تطابقی شامل تمام رأس‌های سمت چپ وجود دارد اگر و تنها اگر به ازای هر X ، شرط $|X| \leq |f(X)|$ برقرار باشد.

قضیه هال را در گراف مثال بررسی می‌کنیم. ابتدا فرض کنید $X = \{1, 3\}$ باشد که نتیجه می‌دهد $f(X) = \{5, 6, 8\}$:



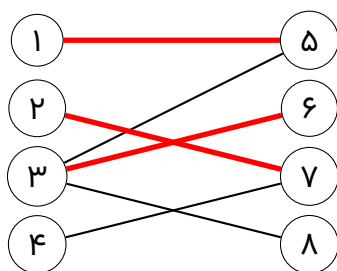
شرط قضیه هال برقرار است، زیرا $|X| = 2$ و $|f(X)| = 3$. سپس فرض کنید $X = \{2, 4\}$ باشد که نتیجه می‌دهد $f(X) = \{7\}$:



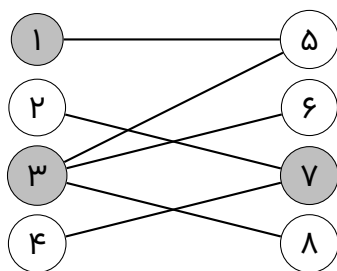
در این حالت، $|X| = 2$ و $|f(X)| = 1$ است، بنابراین شرط قضیه هال برقرار نیست. یعنی تشکیل تطابق کامل برای این گراف ممکن نیست. این نتیجه دور از انتظار نیست، چون از قبل می‌دانیم تطابق بیشینه گراف ۳ است نه ۴. اگر شرط قضیه هال برقرار نباشد، مجموعه X نشان می‌دهد چرا نمی‌توان چنین تطابقی تشکیل داد. چون X گره‌های بیشتری نسبت به $f(X)$ دارد، جفت کافی برای همه گره‌های X وجود ندارد. مثلاً در گراف بالا، هر دو گره ۲ و ۴ باید به گره ۷ متصل شوند که شدنی نیست.

قضیه کونینگ

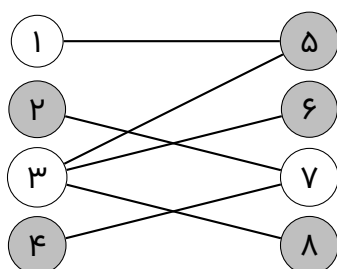
یک پوشش راس کمینه برای گراف، مجموعه‌ای کمینه از گره‌هاست که هر یال گراف حداقل یک سرش در آن مجموعه باشد. در گراف کلی، پیدا کردن پوشش راس کمینه یک مسئله NP-hard است. اما اگر گراف دوبخشی باشد، قضیه کونینگ می‌گوید اندازه پوشش راس کمینه و اندازه تطابق بیشینه همیشه برابرند. بنابراین می‌توان اندازه پوشش راس کمینه را با الگوریتم شار بیشینه محاسبه کرد. گراف زیر با تطابق بیشینه به اندازه ۳ را در نظر بگیرید:



قضیه کونینگ بیان می‌کند اندازه پوشش راسی کمینه نیز برابر ۳ است. چنین پوششی را می‌توان به صورت زیر ساخت:



راس‌هایی که به پوشش راسی کمینه تعلق ندارند، تشکیل یک **مجموعه مستقل بیشینه** می‌دهند. این بزرگ‌ترین مجموعه ممکن از راس‌ها است به‌طوری‌که هیچ دو راسی در مجموعه با یک یال به هم متصل نباشند. یافتن مجموعه مستقل بیشینه در گراف عمومی مسئله‌ای NP-hard است، اما در گراف دوبخشی می‌توان از قضیه کونینگ برای حل کارآمد مسئله استفاده کرد. در گراف نمونه، مجموعه مستقل بیشینه به صورت زیر است:

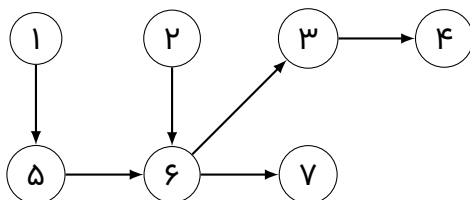


۴.۲۰ پوشش‌های مسیری

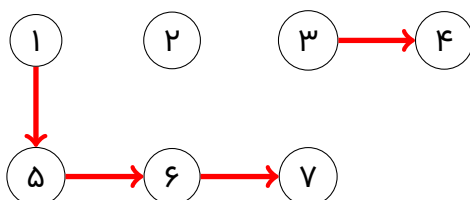
یک **پوشش مسیری** (path cover) (cover مجموعه‌ای از مسیرها در یک گراف است به‌طوری‌که هر رأس گراف متعلق به حداقل یک مسیر باشد. مشخص می‌شود که در گراف‌های جهت‌دار بدون دور (DAG)، می‌توان مسئله‌ی یافتن کمینه‌پوشش مسیری را به مسئله‌ی یافتن بیشینه‌شار در گرافی دیگر کاهش داد.

پوشش مسیری رأس‌جدا

در یک **پوشش مسیری رأس‌جدا** (node-disjoint path cover)، هر رأس دقیقاً متعلق به یک مسیر است. به عنوان مثال، گراف زیر را در نظر بگیرید:

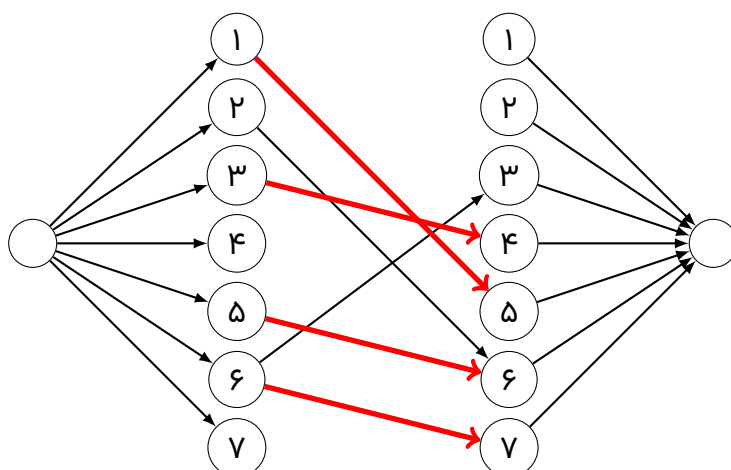


یک پوشش مسیری رأس‌جدا کمینه برای این گراف شامل سه مسیر است. به عنوان مثال، می‌توانیم مسیرهای زیر را انتخاب کنیم:



توجه کنید یکی از مسیرها فقط شامل رأس ۲ است، بنابراین ممکن است یک مسیر اصلاً هیچ یالی نداشته باشد.

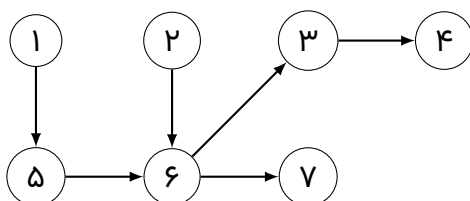
می‌توانیم با ساخت یک گراف تطابق، پوشش مسیری رأس‌جدای کمینه را بیابیم که در آن هر رأس گراف اصلی با دو رأس نمایش داده می‌شود: یک رأس چپ و یک رأس راست. یالی از رأس چپ به رأس راست وجود دارد اگر چنین یالی در گراف اصلی موجود باشد. علاوه بر این، گراف تطابق شامل یک مبدأ و یک مقصد است، و یال‌هایی از مبدأ به همه ی رؤسای چپ و از همه ی رؤسای راست به مقصد وجود دارد. یک بیشینه‌تطابق در گراف حاصل متناظر با یک کمینه‌پوشش مسیری رأس‌جدا در گراف اصلی است. به عنوان مثال، گراف تطابق زیر برای گراف بالا شامل یک بیشینه‌تطابق با اندازه‌ی ۴ است:



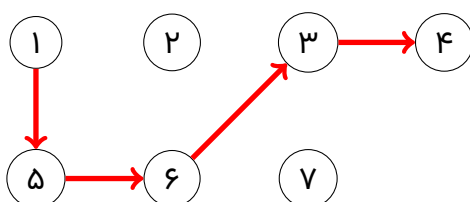
هر یال در تطابق بیشینه گراف تطابق، متناظر با یک یال در پوشش مسیری گره‌مجزای کمینه گراف اصلی است. بنابراین، اندازه پوشش مسیری گره‌مجزای کمینه برابر با $n - c$ است، که در آن n تعداد گره‌های گراف اصلی و c اندازه تطابق بیشینه است.

پوشش مسیری عمومی

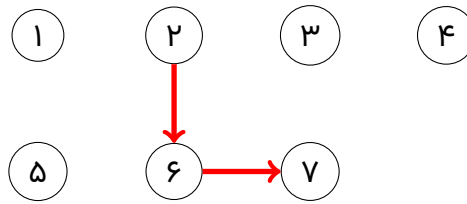
یک پوشش مسیری عمومی پوششی از مسیرها است که در آن یک گره می‌تواند به بیش از یک مسیر تعلق داشته باشد. یک پوشش مسیری عمومی کمینه ممکن است از یک پوشش مسیری گره‌مجزای کمینه کوچک‌تر باشد، زیرا یک گره می‌تواند چندین بار در مسیرها استفاده شود. دوباره گراف زیر را در نظر بگیرید:



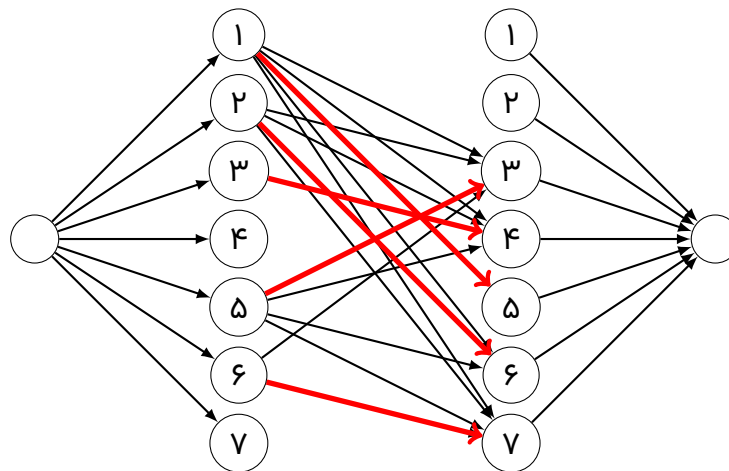
پوشش مسیر عمومی کمینه این گراف شامل دو مسیر است. برای نمونه، مسیر نخست می‌تواند به این صورت باشد:



و مسیر دوم می‌تواند چنین باشد:

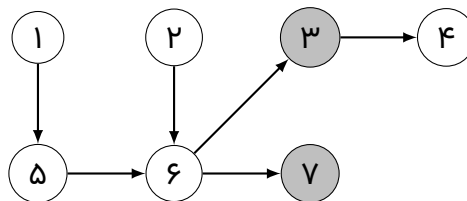


یافتن پوشش مسیر عمومی کمینه تقریباً مشابه پوشش مسیر با راس‌های مجزا کمینه است. تنها افزودن یال‌های تازه به گراف تطابق کافی است به‌طوری که یال $a \rightarrow b$ همیشه برقرار باشد هرگاه مسیری از a به b در گراف اولیه وجود دارد (احتمالاً شامل چندین یال). گراف تطابق برای گراف بالا بدین گونه است:



قضیه دیلوورث

یک پادزنجیر مجموعه‌ای از راس‌های گراف است به‌طوری که هیچ مسیری بین هیچ دو راسی با استفاده از یال‌های گراف وجود نداشته باشد. **قضیه دیلوورث** بیان می‌کند در یک گراف جهت‌دار بدون دور، اندازه حداقل پوشش مسیری عمومی برابر اندازه حداکثر پادزنجیر است. برای نمونه، راس‌های ۳ و ۷ یک پادزنجیر در گراف زیر تشکیل می‌دهند:



این یک پادزنجیر بیشینه است، زیرا ساخت هیچ پادزنجیری شامل سه رأس ممکن نیست. پیش‌تر دیدیم که اندازه یک پوشش مسیری عمومی کمینه برای این گراف برابر با دو مسیر است.

بخش سوم

مباحث پیشرفته

فصل ۲۱

نظریه اعداد

نظریه اعداد شاخه‌ای از ریاضیات است که به مطالعه اعداد صحیح می‌پردازد. نظریه اعداد حوزه‌ای شگفت‌انگیز است، چرا که بسیاری از پرسش‌های مربوط به اعداد صحیح، حتی اگر در نگاه اول ساده به نظر برسند، حل بسیار دشواری دارند. به عنوان مثال، معادله زیر را در نظر بگیرید:

$$x^3 + y^3 + z^3 = 33$$

یافتن سه عدد حقیقی x ، y و z که در این معادله صدق کنند آسان است. برای نمونه می‌توان انتخاب کرد:

$$\begin{aligned}x &= 3, \\y &= \sqrt[3]{3}, \\z &= \sqrt[3]{3}.\end{aligned}$$

با این حال، در نظریه اعداد این مسئله‌ای باز است که آیا سه عدد صحیح x ، y و z وجود دارند که در این معادله صدق کنند یا خیر [۶]. در این فصل، روی مفاهیم پایه و الگوریتم‌های نظریه اعداد تمرکز خواهیم کرد. در سرتاسر این فصل، فرض بر این است که همه اعداد صحیح هستند، مگر خلاف آن بیان شود.

۱.۲۱ اعداد اول و مقسوم‌علیه‌ها

عدد a را یک **عامل** یا **مقسوم‌علیه** عدد b می‌نامند هرگاه a عدد b را عاد کند (تقسیم کند). اگر a مقسوم‌علیه b باشد، می‌نویسیم $a \mid b$ ، و در غیر این صورت می‌نویسیم $a \nmid b$. برای مثال، مقسوم‌علیه‌های ۲۴ عبارتند از: ۱، ۲، ۳، ۴، ۶، ۸، ۱۲ و ۲۴. عدد $n > 1$ یک **عدد اول** است اگر تنها مقسوم‌علیه‌های مثبت آن ۱ و n باشند. برای نمونه، ۷، ۱۹ و ۴۱ اول هستند، اما ۳۵ اول نیست، زیرا $35 = 5 \cdot 7$. به ازای هر عدد $n > 1$ ، یک **تجزیه به عوامل اول** یکتا وجود دارد:

$$n = p_1^{\alpha_1} p_2^{\alpha_2} \cdots p_k^{\alpha_k},$$

که در آن $p_1, p_2, \dots, p_k$ اعداد اول متمایز و $\alpha_1, \alpha_2, \dots, \alpha_k$ اعداد مثبت هستند. برای مثال، تجزیه به عوامل اول برای ۸۴ به صورت زیر است:

$$84 = 2^2 \cdot 3^1 \cdot 7^1.$$

تعداد مقسوم‌علیه‌های عدد n برابر است با:

$$\tau(n) = \prod_{i=1}^k (\alpha_i + 1),$$

زیرا برای هر عامل اول p_i ، به تعداد $\alpha_i + 1$ روش برای انتخاب تعداد دفعات حضور آن در مقسوم‌علیه وجود دارد. برای مثال، تعداد مقسوم‌علیه‌های ۸۴ برابر است با: $\tau(84) = 12 = 2 \cdot 2 \cdot 3$. این مقسوم‌علیه‌ها عبارتند از: ۱، ۲، ۳، ۴، ۶، ۷، ۱۲، ۱۴، ۲۱، ۲۸، ۴۲ و ۸۴. **مجموع مقسوم‌علیه‌های n برابر است با:**

$$\sigma(n) = \prod_{i=1}^k (1 + p_i + \dots + p_i^{\alpha_i}) = \prod_{i=1}^k \frac{p_i^{\alpha_i+1} - 1}{p_i - 1},$$

که در آن، فرمول دوم بر اساس فرمول تصاعد هندسی به دست آمده است. برای مثال، مجموع مقسوم‌علیه‌های ۸۴ برابر است با:

$$\sigma(84) = \frac{2^3 - 1}{2 - 1} \cdot \frac{3^2 - 1}{3 - 1} \cdot \frac{7^2 - 1}{7 - 1} = 7 \cdot 4 \cdot 8 = 224.$$

حاصل‌ضرب مقسوم‌علیه‌های n برابر است با:

$$\mu(n) = n^{\tau(n)/2},$$

زیرا می‌توان از میان مقسوم‌علیه‌ها $\tau(n)/2$ جفت تشکیل داد که حاصل‌ضرب هر جفت برابر با n است. برای نمونه، مقسوم‌علیه‌های ۸۴ این جفت‌ها را می‌سازند: $1 \cdot 84$ ، $2 \cdot 42$ ، $3 \cdot 28$ و غیره، و حاصل‌ضرب مقسوم‌علیه‌ها برابر است با: $\mu(84) = 84^6 = 351298031616$. عدد n یک **عدد کامل** نامیده می‌شود اگر $n = \sigma(n) - n$ باشد؛ یعنی n برابر با مجموع مقسوم‌علیه‌های خود بین ۱ و $n - 1$ باشد. برای مثال، ۲۸ یک عدد کامل است، زیرا $28 = 1 + 2 + 4 + 7 + 14$.

تعداد اعداد اول

نشان دادن نامتناهی بودن اعداد اول ساده است. اگر تعداد اعداد اول متناهی بود، مجموعه $P = \{p_1, p_2, \dots, p_n\}$ شامل همه اعداد اول تشکیل می‌شد. مثال: $p_1 = 2$ ، $p_2 = 3$ ، $p_3 = 5$ و الی آخر. ولی با اعضای P ، عدد اول جدید

$$p_1 p_2 \cdots p_n + 1$$

ساخته می‌شود که از تمام اعضای P بزرگ‌تر است. این تناقض است. تعداد اعداد اول باید نامتناهی باشد.

تراکم اعداد اول

تراکم اعداد اول یعنی بسامد رخداد اعداد اول میان اعداد. فرض کنید $\pi(n)$ نماد تعداد اعداد اول بین ۱ و n باشد. مثال: $\pi(10) = 4$ ، چون ۴ عدد اول بین ۱ و ۱۰ هست: ۲، ۳، ۵ و ۷. می‌توان نشان داد

$$\pi(n) \approx \frac{n}{\ln n},$$

یعنی اعداد اول متداول هستند. مثال: تعداد اعداد اول بین ۱ و 10^6 برابر $\pi(10^6) = 78498$ است، و $10^6 / \ln 10^6 \approx 72382$.

حدس‌ها

حدس‌های زیادی درباره اعداد اول وجود دارد. اکثر افراد حدس‌ها را درست می‌دانند، اما اثباتی برای آن‌ها نیست. حدس‌های مشهور:

- **حدس گلدباخ:** هر عدد صحیح زوج $n > 2$ به شکل مجموع $n = a + b$ نوشته می‌شود که هر دو عدد a و b اول هستند.
- **حدس اعداد اول دوقلو:** تعداد نامتناهی جفت به فرم $\{p, p + 2\}$ وجود دارد که در آن هم p و هم $p + 2$ اول هستند.
- **حدس لژاندر:** همیشه یک عدد اول بین n^2 و $(n + 1)^2$ هست، که n هر عدد صحیح مثبت است.

الگوریتم‌های پای‌ه

اگر عدد n اول نباشد، به صورت حاصل ضرب $a \cdot b$ نوشته می‌شود، که $a \leq \sqrt{n}$ یا $b \leq \sqrt{n}$ است، پس قطعاً عاملی بین 2 و $\lfloor \sqrt{n} \rfloor$ دارد. با این نکته، تست اول بودن عدد و تجزیه به عوامل اول در زمان $O(\sqrt{n})$ انجام می‌شود. تابع `prime` بررسی می‌کند آیا عدد ورودی n اول است یا خیر. تابع تقسیم n بر تمام اعداد بین 2 و $\lfloor \sqrt{n} \rfloor$ را امتحان می‌کند، و اگر هیچ‌کدام n را عاد نکند، n اول است.

```
bool prime(int n) {
    if (n < 2) return false;
    for (int x = 2; x * x <= n; ++x)
        if (n % x == 0) return false;
    return true;
}
```

تابع `factors` یک بردار شامل عوامل اول n می‌سازد. تابع n را بر عوامل اولش تقسیم می‌کند، و آن‌ها را به بردار می‌افزاید. فرایند زمانی پایان می‌یابد که عدد باقی‌مانده n عاملی بین 2 و $\lfloor \sqrt{n} \rfloor$ نداشته باشد. اگر $n > 1$ ، اول است و آخرین عامل خواهد بود.

```
vector<int> factors(int n) {
    vector<int> f;
    for (int x = 2; x * x <= n; ++x)
        while (n % x == 0) {
            f.push_back(x);
            n /= x;
        }
    if (n > 1) f.push_back(n);
    return f;
}
```

توجه کنید هر عامل اول به تعداد دفعاتی که عدد را عاد می‌کند در بردار ظاهر می‌شود. برای مثال، $24 = 2^3 \cdot 3$ ، بنابراین خروجی تابع `factors` `[2, 2, 2, 3]` است.

غربال اراتستن

غربال اراتستن الگوریتم پیش‌پردازشی است که آرایه‌ای می‌سازد. با این آرایه می‌توان اول بودن هر عدد دلخواه بین $2 \dots n$ را به‌طور کارآمد بررسی کرد و در صورت اول نبودن، یکی از عامل‌های اول آن را یافت.

این الگوریتم آرایه‌ای به نام sieve می‌سازد که خانه‌های $2, 3, \dots, n$ آن استفاده می‌شوند. مقدار $\text{sieve}[k] = 0$ یعنی k اول است، و مقدار $\text{sieve}[k] \neq 0$ یعنی k مرکب است و یکی از عامل‌های اول آن $\text{sieve}[k]$ است.

الگوریتم اعداد $2 \dots n$ را تک‌به‌تک پیمایش می‌کند. هر بار با یافتن عدد اول جدید x ، مضرب‌های آن $(2x, 3x, 4x, \dots)$ را به عنوان عدد غیر اول علامت‌گذاری می‌کند، چون عدد x آنها را عاد می‌کند.

برای مثال، اگر $n = 20$ باشد، آرایه به این صورت است:

2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20
0	0	2	0	3	0	2	3	5	0	3	0	7	5	2	0	3	0	5

کد زیر غربال اراتستن را پیاده‌سازی می‌کند. کد فرض می‌کند تمام عناصر sieve در ابتدا صفر هستند.

```

for (int x = 2; x <= n; ++x)
    if (sieve[x] == 0)
    {
        for (int u = x; u <= n; u += x)
            sieve[u] = x;
    }

```

حلقه داخلی الگوریتم به ازای هر مقدار x ، به تعداد n/x بار اجرا می‌شود. بنابراین، یک کران بالا برای زمان اجرای الگوریتم، مجموع هارمونیک زیر است:

$$\sum_{x=2}^n n/x = n/2 + n/3 + n/4 + \dots + n/n = O(n \log n).$$

در واقع، الگوریتم کارآمدتر است، زیرا حلقه داخلی تنها زمانی اجرا می‌شود که عدد x اول باشد. می‌توان نشان داد که زمان اجرای الگوریتم تنها $O(n \log \log n)$ است؛ پیچیدگی زمانی بسیار نزدیکی به $O(n)$.

الگوریتم اقل‌ی‌دس

بزرگ‌ترین مقسوم‌علیه مشترک دو عدد a و b ، یعنی $\text{gcd}(a, b)$ ، بزرگ‌ترین عددی است که هر دو عدد a و b را عاد می‌کند، و **کوچک‌ترین مضرب مشترک** a و b ، یعنی $\text{lcm}(a, b)$ ، کوچک‌ترین عددی است که بر هر دوی a و b بخش‌پذیر است. به عنوان مثال، $\text{gcd}(24, 36) = 12$ و $\text{lcm}(24, 36) = 72$.

بزرگ‌ترین مقسوم‌علیه مشترک و کوچک‌ترین مضرب مشترک به شکل زیر به هم مرتبط هستند:

$$\text{lcm}(a, b) = \frac{ab}{\text{gcd}(a, b)}$$

الگوریتم اقلیدس^۱ روشی کارآمد برای یافتن بزرگ‌ترین مقسوم‌علیه مشترک دو عدد فراهم می‌کند. این الگوریتم مبتنی بر فرمول زیر است:

$$\gcd(a, b) = \begin{cases} a & b = 0 \\ \gcd(b, a \bmod b) & b \neq 0 \end{cases}$$

برای نمونه،

$$\gcd(24, 36) = \gcd(36, 24) = \gcd(24, 12) = \gcd(12, 0) = 12.$$

الگوریتم را می‌توان به صورت زیر پیاده‌سازی کرد:

```
int gcd(int a, int b) {
    if (b == 0) return a;
    return gcd(b, a % b);
}
```

می‌توان نشان داد که الگوریتم اقلیدس در زمان $O(\log n)$ اجرا می‌شود که در آن $n = \min(a, b)$ است. بدترین حالت برای این الگوریتم، زمانی رخ می‌دهد که a و b دو عدد متوالی فیبوناچی باشند. برای نمونه،

$$\gcd(13, 8) = \gcd(8, 5) = \gcd(5, 3) = \gcd(3, 2) = \gcd(2, 1) = \gcd(1, 0) = 1.$$

تابع فی‌اولر

دو عدد a و b را **متباین** (نسبت به هم اول) می‌نامند اگر $\gcd(a, b) = 1$ باشد. **تابع فی‌اولر** $\varphi(n)$ تعداد اعداد متباین با n را در بازه ۱ تا n به دست می‌دهد. برای نمونه، $\varphi(12) = 4$ ، زیرا اعداد ۱، ۵، ۷ و ۱۱ نسبت به ۱۲ اول هستند. مقدار $\varphi(n)$ را می‌توان از روی تجزیه n به عوامل اول با استفاده از فرمول زیر محاسبه کرد:

$$\varphi(n) = \prod_{i=1}^k p_i^{\alpha_i-1} (p_i - 1).$$

برای نمونه، $\varphi(12) = 2^1 \cdot (2 - 1) \cdot 3^0 \cdot (3 - 1) = 4$. دقت کنید که اگر n اول باشد، $\varphi(n) = n - 1$ خواهد بود.

۲.۲۱ حساب پیمانه‌ای

در **حساب پیمانه‌ای**، مجموعه اعداد به گونه‌ای محدود می‌شود که تنها اعداد $0, 1, 2, \dots, m-1$ به کار می‌روند؛ جایی که m یک عدد ثابت است. هر عدد x با عدد $x \bmod m$ نشان داده می‌شود: باقی‌مانده تقسیم x بر m . برای نمونه، اگر $m = 17$ باشد، عدد ۷۵ به صورت $75 \bmod 17 = 7$ نشان داده می‌شود.

^۱اقلیدس ریاضی‌دانی یونانی بود که حدود ۳۰۰ سال پیش از میلاد می‌زیست. این احتمالاً نخستین الگوریتم شناخته‌شده در تاریخ است.

اغلب می‌توان پیش از انجام محاسبات، باقی‌مانده را محاسبه کرد. به ویژه، روابط زیر برقرار هستند:

$$\begin{aligned}(x + y) \bmod m &= (x \bmod m + y \bmod m) \bmod m \\(x - y) \bmod m &= (x \bmod m - y \bmod m) \bmod m \\(x \cdot y) \bmod m &= (x \bmod m \cdot y \bmod m) \bmod m \\x^n \bmod m &= (x \bmod m)^n \bmod m\end{aligned}$$

توان‌رسانی پی‌مان‌های

اغلب نیاز است مقدار $x^n \bmod m$ به شکل کارآمد محاسبه شود. این کار را می‌توان در زمان $O(\log n)$ و با استفاده از رابطه بازگشتی زیر انجام داد:

$$x^n = \begin{cases} 1 & n = 0 \\ x^{n/2} \cdot x^{n/2} & n \text{ زوج است} \\ x^{n-1} \cdot x & n \text{ فرد است} \end{cases}$$

مهم است که در حالت n زوج، مقدار $x^{n/2}$ تنها یک بار محاسبه شود. این امر تضمین می‌کند که پیچیدگی زمانی الگوریتم برابر $O(\log n)$ باشد، چرا که n در صورت زوج بودن همواره نصف می‌شود.

تابع زیر مقدار $x^n \bmod m$ را محاسبه می‌کند:

```
{ m int , n int , x int(modpow int
    ; m1% return 0) == n( if
    ); m, /2n, x(modpow = u long long
    ; m)%u*u( = u
    ; m)%x*u( = u 1) == %2n( if
    ; u return
}
```

قضیه فرما و قضیه اوایلر

قضیه فرما بیان می‌کند که

$$x^{m-1} \bmod m = 1$$

وقتی که m اول باشد و x و m نسبت به هم اول باشند. همچنین نتیجه می‌دهد:

$$x^k \bmod m = x^{k \bmod (m-1)} \bmod m.$$

در حالتی کلی‌تر، **قضیه اوایلر** بیان می‌کند که

$$x^{\varphi(m)} \bmod m = 1$$

وقتی که x و m نسبت به هم اول باشند. قضیه فرما از قضیه اوایلر به دست می‌آید، زیرا اگر m عددی اول باشد، $\varphi(m) = m - 1$ خواهد بود.

معکوس پی‌مان‌های

معکوس x به پیمانۀ m عددی مانند x^{-1} است به طوری که:

$$xx^{-1} \bmod m = 1.$$

برای نمونه، اگر $x = 6$ و $m = 17$ باشد، آنگاه $x^{-1} = 3$ است، زیرا $6 \cdot 3 \bmod 17 = 1$.
با استفاده از معکوس‌های پیمانۀ m می‌توان اعداد را به پیمانۀ m تقسیم کرد، چرا که تقسیم بر x معادل ضرب در x^{-1} است. برای نمونه، جهت محاسبه مقدار $36/6 \bmod 17$ ، می‌توان از فرمول $2 \cdot 3 \bmod 17$ استفاده کرد، زیرا $36 \bmod 17 = 2$ و $6^{-1} \bmod 17 = 3$ است. با این حال، معکوس پیمانۀ همواره وجود ندارد. برای نمونه، اگر $x = 2$ و $m = 4$ باشد، معادله

$$xx^{-1} \bmod m = 1$$

پاسخی ندارد، زیرا همه مضارب ۲ زوج هستند و باقی‌مانده آن‌ها در تقسیم بر $m = 4$ هرگز نمی‌تواند ۱ باشد. مشخص می‌شود که مقدار $x^{-1} \bmod m$ دقیقاً زمانی قابل محاسبه است که x و m نسبت به هم اول باشند. اگر معکوس ضربی به هنگ وجود داشته باشد، با فرمول زیر به دست می‌آید:

$$x^{-1} = x^{\varphi(m)-1}.$$

اگر m اول باشد، فرمول به صورت زیر درمی‌آید:

$$x^{-1} = x^{m-2}.$$

به عنوان مثال،

$$6^{-1} \bmod 17 = 6^{17-2} \bmod 17 = 3.$$

این فرمول امکان محاسبه کارآمد معکوس ضربی پیمانۀ m با الگوریتم توان‌رسانی پیمانۀ m را فراهم می‌سازد. فرمول با قضیه اوایلر اثبات می‌شود. نخست، معکوس ضربی پیمانۀ m باید در رابطه زیر صدق کند:

$$xx^{-1} \bmod m = 1.$$

از سوی دیگر، بر اساس قضیه اوایلر:

$$x^{\varphi(m)} \bmod m = xx^{\varphi(m)-1} \bmod m = 1,$$

بنابراین دو مقدار x^{-1} و $x^{\varphi(m)-1}$ برابر هستند.

حساب کام‌پیوتری

در برنامه‌نویسی، اعداد صحیح بدون علامت در پیمانۀ 2^k نمایش داده می‌شوند که در آن k تعداد بیت‌های نوع داده است. پیامد معمول این شیوه سرریز شدن عدد و بازگشت به صفر هنگام بزرگ شدن بیش از حد مقدار است.
برای نمونه، در ++C اعداد از نوع `int unsigned` در پیمانۀ 2^{32} نشان داده می‌شوند. کد زیر متغیری از نوع `int unsigned` با مقدار 123456789 تعریف می‌کند. سپس مقدار در خودش ضرب می‌شود و نتیجه برابر است با: $123456789^2 \bmod 2^{32} = 2537071545$.

```
123456789; = x int unsigned
2537071545 // ; "n\" << x*x << cout
```

۳.۲۱ حل معادلات

معادلات سیال‌های

یک معادله سیال‌های معادله‌ای است به شکل

$$ax + by = c,$$

که در آن a و b و c مقادیر ثابت هستند و باید مقادیر x و y تعیین شوند. تمامی اعداد در معادله باید صحیح باشند. برای مثال، یک جواب برای معادله $5x + 2y = 11$ عبارت است از $x = 3$ و $y = -2$.

حل کارآمد معادله سیال‌های با استفاده از الگوریتم اقلیدس امکان‌پذیر است. می‌توان الگوریتم اقلیدس را گسترش داد به طوری که مقادیر x و y را برای برقراری معادله زیر به دست آورد:

$$ax + by = \gcd(a, b)$$

یک معادله سیال‌های در صورتی جواب دارد که c بر $\gcd(a, b)$ بخش‌پذیر باشد، در غیر این صورت قابل حل نیست. به عنوان نمونه، مقادیر x و y را برای رابطه زیر بیابیم:

$$39x + 15y = 12$$

معادله قابل حل است، چون $\gcd(39, 15) = 3$ و $3 \mid 12$. هنگامی که الگوریتم اقلیدس بزرگ‌ترین مقسوم‌علیه مشترک ۳۹ و ۱۵ را محاسبه می‌کند، دنباله زیر از فراخوانی توابع تولید می‌شود:

$$\gcd(39, 15) = \gcd(15, 9) = \gcd(9, 6) = \gcd(6, 3) = \gcd(3, 0) = 3$$

این روند متناظر با معادلات زیر است:

$$\begin{aligned} 39 - 2 \cdot 15 &= 9 \\ 15 - 1 \cdot 9 &= 6 \\ 9 - 1 \cdot 6 &= 3 \end{aligned}$$

با استفاده از این معادلات نتیجه می‌شود:

$$39 \cdot 2 + 15 \cdot (-5) = 3$$

و با ضرب آن در ۴، مقدار حاصل برابر است با:

$$39 \cdot 8 + 15 \cdot (-20) = 12,$$

بنابراین یک پاسخ برای معادله $x = 8$ و $y = -20$ است. پاسخ یک معادله سیال‌های یکتا نیست، زیرا در صورت دانستن یک پاسخ، می‌توان بی‌شمار پاسخ دیگر ساخت. اگر جفت (x, y) یک پاسخ باشد، تمامی جفت‌های

$$\left(x + \frac{kb}{\gcd(a, b)}, y - \frac{ka}{\gcd(a, b)}\right)$$

نیز پاسخ خواهند بود، به طوری که k هر عدد صحیحی است.

قضیه باقی‌مانده چینی

قضیه باقیمانده چینی دسته‌ای از معادلات به شکل

$$\begin{aligned}x &= a_1 \bmod m_1 \\x &= a_2 \bmod m_2 \\&\dots \\x &= a_n \bmod m_n\end{aligned}$$

را حل می‌کند که در آن تمام جفت‌های $m_1, m_2, \dots, m_n$ نسبت به هم اول هستند. فرض کنید x_m^{-1} معکوس ضربی x به پیمانه m باشد، و

$$X_k = \frac{m_1 m_2 \cdots m_n}{m_k}.$$

با استفاده از این نمادگذاری، یک پاسخ برای معادلات برابر است با

$$x = a_1 X_1 X_{1m_1}^{-1} + a_2 X_2 X_{2m_2}^{-1} + \cdots + a_n X_n X_{nm_n}^{-1}.$$

در این پاسخ، به ازای هر $k = 1, 2, \dots, n$ داریم:

$$a_k X_k X_{km_k}^{-1} \bmod m_k = a_k,$$

زیرا

$$X_k X_{km_k}^{-1} \bmod m_k = 1.$$

از آنجا که تمام جملات دیگر در مجموع بر m_k بخش‌پذیرند، هیچ تأثیری روی باقیمانده ندارند، و $x \bmod m_k = a_k$ برقرار است. به عنوان مثال، یک پاسخ برای

$$\begin{aligned}x &= 3 \bmod 5 \\x &= 4 \bmod 7 \\x &= 2 \bmod 3\end{aligned}$$

برابر است با

$$3 \cdot 21 \cdot 1 + 4 \cdot 15 \cdot 1 + 2 \cdot 35 \cdot 2 = 263.$$

پس از یافتن یک پاسخ x ، می‌توان بی‌شمار پاسخ دیگر تولید کرد، زیرا تمام اعداد به شکل

$$x + m_1 m_2 \cdots m_n$$

نیز پاسخ هستند.

۴.۲۱ سایر نتایج

قضیه لاگرانژ

قضیه لاگرانژ بیان می‌کند که هر عدد صحیح مثبت می‌تواند به صورت مجموع چهار مربع کامل، یعنی $a^2 + b^2 + c^2 + d^2$ نمایش داده شود. برای مثال، عدد ۱۲۳ می‌تواند به صورت مجموع $8^2 + 5^2 + 5^2 + 3^2$ نمایش یابد.

قضیه زکن دورف

قضیه زکندورف بیان می‌کند که هر عدد صحیح مثبت نمایش یکتایی به صورت مجموع اعداد فیبوناچی دارد، به طوری که هیچ دو عددی برابر نبوده و دو عدد متوالی از دنباله فیبوناچی نیز نباشند. برای مثال، عدد ۷۴ می‌تواند به صورت مجموع $55 + 13 + 5 + 1$ نمایش یابد.

سه‌تایی‌های فیثاغورسی

یک **سه‌تایی فیثاغورسی**، سه‌تایی (a, b, c) است که در قضیه فیثاغورس $a^2 + b^2 = c^2$ صدق می‌کند؛ به این معنی که یک مثلث قائم‌الزاویه با طول اضلاع a, b و c وجود دارد. برای مثال، $(3, 4, 5)$ یک سه‌تایی فیثاغورسی است.

اگر (a, b, c) یک سه‌تایی فیثاغورسی باشد، تمام سه‌تایی‌ها به شکل (ka, kb, kc) که در آن $k > 1$ است نیز سه‌تایی‌های فیثاغورسی هستند. یک سه‌تایی فیثاغورسی ابتدایی نامیده می‌شود اگر a, b و c نسبت به هم اول باشند، و تمام سه‌تایی‌های فیثاغورسی را می‌توان از سه‌تایی‌های ابتدایی با استفاده از یک ضریب k ساخت.

فرمول اقلیدس می‌تواند برای تولید تمام سه‌تایی‌های فیثاغورسی ابتدایی استفاده شود. هر یک از این سه‌تایی‌ها به شکل زیر است:

$$(n^2 - m^2, 2nm, n^2 + m^2),$$

که در آن $0 < m < n$ است، m و n نسبت به هم اول هستند و حداقل یکی از بین n و m زوج است. به عنوان مثال، وقتی $m = 1$ و $n = 2$ باشد، فرمول کوچک‌ترین سه‌تایی فیثاغورسی را تولید می‌کند:

$$(2^2 - 1^2, 2 \cdot 2 \cdot 1, 2^2 + 1^2) = (3, 4, 5).$$

قضیه ویلسون

قضیه ویلسون بیان می‌کند که یک عدد n اول است دقیقاً اگر

$$(n-1)! \bmod n = n-1.$$

برای مثال، عدد ۱۱ اول است، زیرا

$$10! \bmod 11 = 10,$$

و عدد ۱۲ اول نیست، زیرا

$$11! \bmod 12 = 0 \neq 11.$$

بنابراین، از قضیه ویلسون می‌توان برای تشخیص اول بودن یک عدد استفاده کرد. با این حال، در عمل، این قضیه نمی‌تواند برای مقادیر بزرگ n به کار رود، زیرا محاسبه مقادیر $(n-1)!$ در شرایطی که n بزرگ باشد دشوار است.

فصل ۲۲

ترکیبیات

ترکیبیات روش‌های شمارش ترکیب‌های اشیاء را بررسی می‌کند. معمولاً هدف یافتن روشی برای شمارش کارآمد ترکیب‌ها بدون تولید مجزای هر ترکیب است. به عنوان مثال، مسئله شمارش تعداد روش‌های نمایش یک عدد صحیح n به صورت مجموعی از اعداد صحیح مثبت را در نظر بگیرید. به عنوان مثال، ۸ روش نمایش برای عدد ۴ وجود دارد:

$$\begin{array}{ll} 2 + 2 \cdot & 1 + 1 + 1 + 1 \cdot \\ 3 + 1 \cdot & 1 + 1 + 2 \cdot \\ 1 + 3 \cdot & 1 + 2 + 1 \cdot \\ 4 \cdot & 2 + 1 + 1 \cdot \end{array}$$

یک مسئله ترکیبیاتی اغلب می‌تواند با استفاده از یک تابع بازگشتی حل شود. در این مسئله، می‌توانیم تابع $f(n)$ را تعریف کنیم که تعداد روش‌های نمایش n را به دست می‌دهد. به عنوان مثال، مطابق با مثال بالا $f(4) = 8$ است. مقادیر این تابع را می‌توان به صورت بازگشتی به روش زیر محاسبه کرد:

$$f(n) = \begin{cases} 1 & n = 0 \\ f(0) + f(1) + \cdots + f(n-1) & n > 0 \end{cases}$$

حالت پایه $f(0) = 1$ است، زیرا مجموع تهی بیانگر عدد ۰ است. سپس، اگر $n > 0$ باشد، تمام روش‌های انتخاب عدد اول مجموع را در نظر می‌گیریم. اگر عدد اول k باشد، $f(n-k)$ روش نمایش برای بخش باقیمانده مجموع وجود دارد. بنابراین، مجموع تمام مقادیر به فرم $f(n-k)$ را محاسبه می‌کنیم که در آن $k < n$ است. نخستین مقادیر این تابع عبارتند از:

$$\begin{array}{ll} f(0) & = 1 \\ f(1) & = 1 \\ f(2) & = 2 \\ f(3) & = 4 \\ f(4) & = 8 \end{array}$$

گاهی می‌توان فرمول بازگشتی را با یک فرمول بسته جایگزین کرد. در این مسئله،

$$f(n) = 2^{n-1},$$

که بر اساس این واقعیت است که $n - 1$ موقعیت ممکن برای علامت‌های $+$ در مجموع وجود دارد و ما می‌توانیم هر زیرمجموعه‌ای از آن‌ها را انتخاب کنیم.

۱.۲۲ ضرایب دوجمله‌ای

ضریب دوجمله‌ای $\binom{n}{k}$ برابر با تعداد روش‌هایی است که می‌توان یک زیرمجموعه k عضوی را از یک مجموعه n عضوی انتخاب کرد. برای مثال، $\binom{5}{3} = 10$ ، زیرا مجموعه $\{1, 2, 3, 4, 5\}$ دارای ۱۰ زیرمجموعه ۳ عضوی است:

$\{1, 2, 3\}, \{1, 2, 4\}, \{1, 2, 5\}, \{1, 3, 4\}, \{1, 3, 5\}, \{1, 4, 5\}, \{2, 3, 4\}, \{2, 3, 5\}, \{2, 4, 5\}, \{3, 4, 5\}$

فرمول ۱

ضرایب دوجمله‌ای را می‌توان به صورت بازگشتی به صورت زیر محاسبه کرد:

$$\binom{n}{k} = \binom{n-1}{k-1} + \binom{n-1}{k}$$

ایده این است که یک عنصر x را در مجموعه تثبیت کنیم. اگر x در زیرمجموعه گنجانده شود، باید $k - 1$ عنصر را از $n - 1$ عنصر انتخاب کنیم، و اگر x در زیرمجموعه گنجانده نشود، باید k عنصر را از $n - 1$ عنصر انتخاب کنیم. حالت‌های پایه برای بازگشت عبارتند از:

$$\binom{n}{0} = \binom{n}{n} = 1,$$

زیرا همواره دقیقاً یک روش برای ساخت زیرمجموعه تهی و یک زیرمجموعه شامل تمام عناصر وجود دارد.

فرمول ۲

روش دیگر برای محاسبه ضرایب دوجمله‌ای به صورت زیر است:

$$\binom{n}{k} = \frac{n!}{k!(n-k)!}.$$

$n!$ جایگشت برای n عنصر وجود دارد. تمام جایگشت‌ها را بررسی می‌کنیم و همواره k عنصر اول جایگشت را در زیرمجموعه قرار می‌دهیم. از آنجا که ترتیب عناصر در داخل زیرمجموعه و خارج از آن اهمیتی ندارد، نتیجه بر $k!$ و $(n-k)!$ تقسیم می‌شود.

ویژگی‌ها

برای ضرایب دوجمله‌ای،

$$\binom{n}{k} = \binom{n}{n-k},$$

چرا که در واقع یک مجموعه از n عنصر را به دو زیرمجموعه تقسیم می‌کنیم: اولی شامل k عنصر و دومی شامل $n - k$ عنصر است.

مجموع ضرایب دوجمله‌ای برابر است با:

$$\binom{n}{0} + \binom{n}{1} + \binom{n}{2} + \dots + \binom{n}{n} = 2^n.$$

علت نام‌گذاری «ضریب دوجمله‌ای» هنگام به توان n رساندن دوجمله‌ای $(a+b)$ مشخص می‌شود:

$$(a+b)^n = \binom{n}{0}a^n b^0 + \binom{n}{1}a^{n-1}b^1 + \dots + \binom{n}{n-1}a^1 b^{n-1} + \binom{n}{n}a^0 b^n.$$

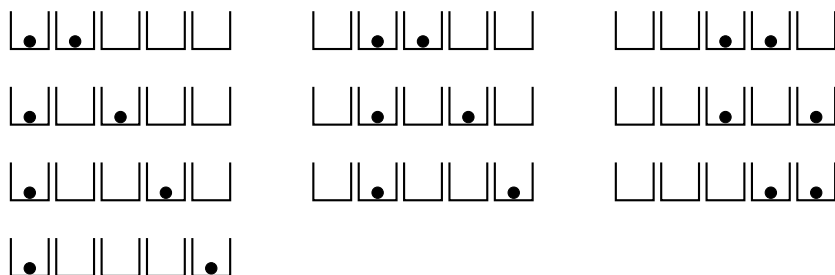
ضرایب دوجمله‌ای در **مثلث پاسکال** نیز ظاهر می‌شوند، جایی که هر مقدار برابر با مجموع دو مقدار بالای خود است:

$$\begin{array}{ccccccc} & & & 1 & & & \\ & & 1 & & 1 & & \\ & 1 & & 2 & & 1 & \\ 1 & & 1 & & 3 & & 3 & & 1 \\ & 1 & & 4 & & 6 & & 4 & & 1 \\ & \dots & & \dots & & \dots & & \dots & & \dots \end{array}$$

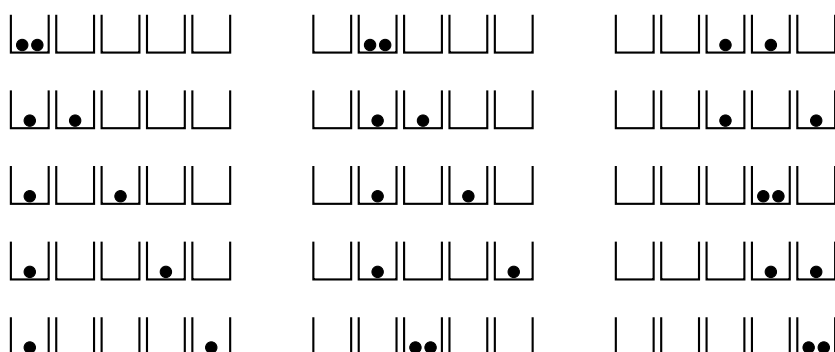
جعبه‌ها و توپ‌ها

«جعبه‌ها و توپ‌ها» یک مدل کاربردی است که در آن تعداد روش‌های قرار دادن k توپ در n جعبه را می‌شماریم. سه حالت را در نظر می‌گیریم:

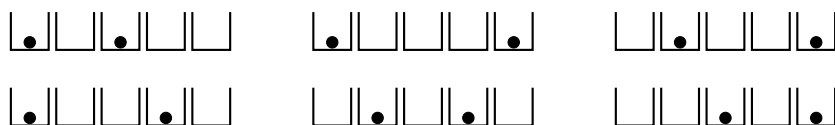
حالت ۱: هر جعبه می‌تواند حداکثر شامل یک توپ باشد. به عنوان مثال، وقتی $n = 5$ و $k = 2$ باشد، ۱۰ روش وجود دارد:



در این حالت، پاسخ مستقیماً ضریب دوجمله‌ای $\binom{n}{k}$ است. حالت ۲: یک جعبه می‌تواند شامل چند توپ باشد. به عنوان مثال، وقتی $n = 5$ و $k = 2$ باشد، ۱۵ روش وجود دارد:



فرایند قرار دادن توپ‌ها در جعبه‌ها را می‌توان با رشته‌ای شامل نمادهای «0» و «→» نمایش داد. ابتدا فرض کنید در چپ‌ترین جعبه ایستاده‌ایم. نماد «0» یعنی یک توپ در جعبه فعلی بگذار، و نماد «→» یعنی به جعبه بعدی در سمت راست برو. با این نمایش، هر جواب رشته‌ای است شامل k بار نماد «0» و $n - 1$ بار نماد «→». برای نمونه، جواب بالا-راست در تصویر بالا معادل رشته «→ 0 → 0 → 0 → 0» است. بنابراین، تعداد جواب‌ها برابر است با: $\binom{k+n-1}{k}$. سناریوی ۳: هر جعبه حداکثر یک توپ دارد، بعلاوه هیچ دو جعبه مجاوری نباید همزمان توپ داشته باشند. برای نمونه، به ازای $n = 5$ و $k = 2$ ، تعداد ۶ جواب وجود دارد:



در این سناریو فرض کنید k توپ ابتدا درون جعبه‌ها قرار دارند و بین هر دو جعبه مجاور یک جعبه خالی قرار گرفته است. کار باقی‌مانده انتخاب جایگاه برای جعبه‌های خالی باقی‌مانده است. تعداد $n - 2k + 1$ جعبه خالی و $k + 1$ موقعیت ممکن برای قرار دادن آن‌ها داریم. بنابراین با فرمول سناریوی ۲، تعداد جواب‌ها برابر است با: $\binom{n-k+1}{n-2k+1}$.

ضرایب چندجمله‌ای

ضریب چندجمله‌ای

$$\binom{n}{k_1, k_2, \dots, k_m} = \frac{n!}{k_1! k_2! \dots k_m!},$$

برابر با تعداد روش‌هایی است که می‌توان n عنصر را به زیرمجموعه‌هایی با اندازه‌های $k_1, k_2, \dots, k_m$ تقسیم کرد، به طوری که $k_1 + k_2 + \dots + k_m = n$ باشد. ضرایب چندجمله‌ای را می‌توان تعمیم ضرایب دوجمله‌ای دانست؛ اگر $m = 2$ باشد، فرمول بالا با فرمول ضریب دوجمله‌ای مطابقت دارد.

۲.۲۲ اعداد کاتالان

عدد کاتالان C_n برابر با تعداد عبارت‌های پرانتزگذاری معتبری است که از n پرانتز باز و n پرانتز بسته تشکیل شده‌اند.

به عنوان مثال، $C_3 = 5$ است، زیرا می‌توان عبارت‌های پرانتزی زیر را با استفاده از سه پرانتز باز و بسته ساخت:

- () () ()
- (()) ()
- () (())
- ((()))
- (() ())

عبارت‌های پرانتزی

یک عبارت پرانتزی معتبر دقیقا چیست؟ قوانین زیر تمام عبارت‌های پرانتزی معتبر را به طور دقیق تعریف می‌کنند:

- یک عبارت پرانتزی خالی معتبر است.
- اگر عبارت A معتبر باشد، آنگاه عبارت (A) نیز معتبر است.
- اگر عبارت‌های A و B معتبر باشند، آنگاه عبارت AB نیز معتبر است.

روش دیگر برای توصیف عبارت‌های پرانتزی معتبر این است که اگر هر پیشوندی از چنین عبارتی را انتخاب کنیم، باید حداقل به تعداد پرانتزهای بسته، پرانتز باز داشته باشد. علاوه بر این، کل عبارت باید دارای تعداد برابری از پرانتزهای باز و بسته باشد.

فرمول ۱

اعداد کاتالان را می‌توان با استفاده از این فرمول محاسبه کرد:

$$C_n = \sum_{i=0}^{n-1} C_i C_{n-i-1}.$$

این مجموع، حالات تقسیم عبارت به دو بخش را بررسی می‌کند به طوری که هر دو بخش، عبارت‌های معتبری باشند و بخش اول تا حد امکان کوتاه اما غیرخالی باشد. به ازای هر i ، بخش اول شامل $i + 1$ جفت پرانتز است و تعداد عبارت‌ها حاصل ضرب مقادیر زیر است:

- C_i : تعداد روش‌های ساخت یک عبارت با استفاده از پرانتزهای بخش اول، بدون در نظر گرفتن بیرونی‌ترین پرانتزها

- C_{n-i-1} : تعداد روش‌های ساخت یک عبارت با استفاده از پرانتزهای بخش دوم

حالت پایه $C_0 = 1$ است، زیرا می‌توان با استفاده از صفر جفت پرانتز، یک عبارت پرانتزی خالی ساخت.

فرمول ۲

اعداد کاتالان را می‌توان با استفاده از ضرایب دوجمله‌ای نیز محاسبه کرد:

$$C_n = \frac{1}{n+1} \binom{2n}{n}$$

این فرمول را می‌توان به صورت زیر توضیح داد:

در مجموع $\binom{2n}{n}$ روش برای ساخت یک عبارت پرانتزی (نه لزوماً معتبر) شامل n پرانتز باز و n پرانتز بسته وجود دارد. بیاپید تعداد عباراتی از این دست را که معتبر نیستند محاسبه کنیم.

اگر یک عبارت پرانتزی معتبر نباشد، باید دارای پیشوندی باشد که در آن تعداد پرانتزهای بسته از تعداد پرانتزهای باز بیشتر شود. ایده این است که جهت هر پرانتز متعلق به چنین پیشوندی را معکوس کنیم. برای نمونه، عبارت $(())$ شامل پیشوند $(())$ است، و پس از معکوس کردن این پیشوند، عبارت به شکل $(())$ درمی‌آید.

عبارت حاصل از $n + 1$ پرانتز باز و $n - 1$ پرانتز بسته تشکیل شده است. تعداد چنین عباراتی برابر با $\binom{2n}{n+1}$ است، که معادل تعداد عبارات پرانتزی نامعتبر می‌باشد. بنابراین، تعداد عبارات پرانتزی معتبر را می‌توان با فرمول زیر محاسبه کرد:

$$\binom{2n}{n} - \binom{2n}{n+1} = \binom{2n}{n} - \frac{n}{n+1} \binom{2n}{n} = \frac{1}{n+1} \binom{2n}{n}.$$

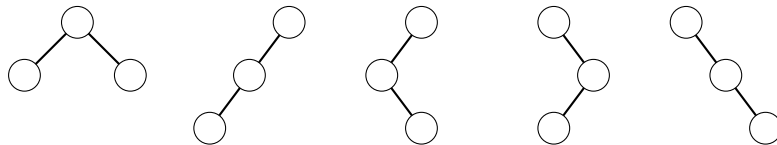
شم‌ارش درخت‌ها

اعداد کاتالان با درخت‌ها نیز مرتبط هستند:

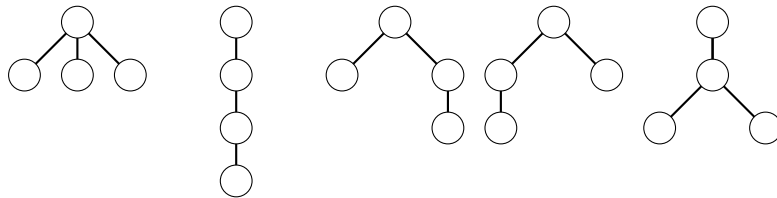
• C_n درخت دودویی با n رأس وجود دارد

• C_{n-1} درخت ریشه‌دار با n رأس وجود دارد

به عنوان مثال، برای $C_3 = 5$ ، درخت‌های دودویی عبارتند از:



و درخت‌های ریشه‌دار عبارتند از:

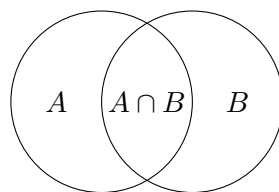


۳.۲۲ اصل شمول و عدم شمول

اصل شمول و عدم شمول روشی برای شمارش اندازه اجتماع مجموعه‌هاست، وقتی اندازه اشتراک‌ها معلوم باشد (و برعکس). نمونه ساده فرمول:

$$|A \cup B| = |A| + |B| - |A \cap B|,$$

که A و B مجموعه هستند و $|X|$ اندازه X را نشان می‌دهد. نمایش فرمول:

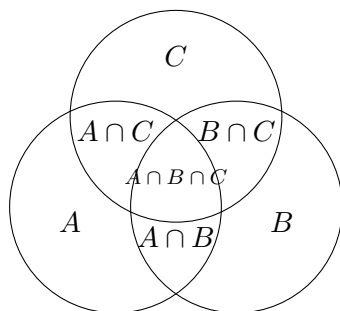


هدف: محاسبه اندازه اجتماع $A \cup B$ ؛ هم‌ارز مساحت ناحیه درون حداقل یک دایره. تصویر نشان می‌دهد: مساحت $A \cup B$ برابر مجموع مساحت‌های A و B منهای مساحت $A \cap B$.

ایده برای تعداد بیشتر مجموعه‌ها هم کار می‌کند. برای سه مجموعه، فرمول شمول و عدم شمول:

$$|A \cup B \cup C| = |A| + |B| + |C| - |A \cap B| - |A \cap C| - |B \cap C| + |A \cap B \cap C|$$

تصویر متناظر:



حالت کلی: محاسبه اندازه اجتماع $X_1 \cup X_2 \cup \dots \cup X_n$ با پیمایش تمام اشتراک‌های ممکن از زیرمجموعه‌های $X_1, X_2, \dots, X_n$. اگر اشتراک شامل تعداد فردی مجموعه باشد، اندازه‌اش به جواب اضافه می‌شود؛ در غیر این صورت، از جواب کم می‌شود. توجه کنید که فرمول‌های مشابهی نیز برای محاسبه اندازه اشتراک بر اساس اندازه اجتماع‌ها وجود دارد. به عنوان مثال،

$$|A \cap B| = |A| + |B| - |A \cup B|$$

9

$$|A \cap B \cap C| = |A| + |B| + |C| - |A \cup B| - |A \cup C| - |B \cup C| + |A \cup B \cup C|.$$

پری‌ش‌ها

به عنوان مثال، بیایید تعداد **پری‌ش‌های** (derangements) اعضای مجموعه $\{1, 2, \dots, n\}$ را بشماریم؛ یعنی جایگشت‌هایی که در آن‌ها هیچ عنصری در جایگاه اصلی خود باقی نمی‌ماند. برای نمونه، وقتی $n = 3$ باشد، دو پری‌ش وجود دارد: $(2, 3, 1)$ و $(3, 1, 2)$. یک روش برای حل مسئله، استفاده از اصل شمول و عدم شمول است. فرض کنید X_k مجموعه جایگشت‌هایی باشد که عنصر k در موقعیت k قرار دارد. برای مثال، وقتی $n = 3$ باشد، این مجموعه‌ها به صورت زیر هستند:

$$\begin{aligned} X_1 &= \{(1, 2, 3), (1, 3, 2)\} \\ X_2 &= \{(1, 2, 3), (3, 2, 1)\} \\ X_3 &= \{(1, 2, 3), (2, 1, 3)\} \end{aligned}$$

با استفاده از این مجموعه‌ها، تعداد پری‌ش‌ها برابر است با

$$n! - |X_1 \cup X_2 \cup \dots \cup X_n|,$$

بنابراین کافی است اندازه اجتماع محاسبه شود. با استفاده از اصل شمول و عدم شمول، مسئله به محاسبه اندازه اشتراک‌ها تبدیل می‌شود که به صورت کارآمد قابل انجام است. برای نمونه، زمانی که $n = 3$ باشد، اندازه $|X_1 \cup X_2 \cup X_3|$ برابر است با

$$\begin{aligned} &|X_1| + |X_2| + |X_3| - |X_1 \cap X_2| - |X_1 \cap X_3| - |X_2 \cap X_3| + |X_1 \cap X_2 \cap X_3| \\ &= 2 + 2 + 2 - 1 - 1 - 1 + 1 \\ &= 4, \end{aligned}$$

بنابراین تعداد جواب‌ها $3! - 4 = 2$ است.

مشخص می‌شود که این مسئله را می‌توان بدون استفاده از اصل شمول و عدم شمول نیز حل کرد. فرض کنید $f(n)$ نشان‌دهنده تعداد پری‌ش‌ها برای $\{1, 2, \dots, n\}$ باشد. می‌توان از رابطه بازگشتی زیر استفاده کرد:

$$f(n) = \begin{cases} 0 & n = 1 \\ 1 & n = 2 \\ (n-1)(f(n-2) + f(n-1)) & n > 2 \end{cases}$$

این رابطه را می‌توان با بررسی حالت‌های ممکن برای تغییر جایگاه عنصر ۱ به دست آورد. تعداد $n-1$ روش برای انتخاب عنصر x که جایگزین عنصر ۱ می‌شود وجود دارد. در هر یک از این انتخاب‌ها، دو حالت ممکن است رخ دهد:

حالت ۱: عنصر x نیز با عنصر ۱ جایگزین شود. پس از این، وظیفه باقی‌مانده ساخت یک پریش از $n-2$ عنصر است.

حالت ۲: عنصر x با عنصری غیر از ۱ جایگزین شود. اکنون باید یک پریش از $n-1$ عنصر بسازیم، زیرا نمی‌توانیم عنصر x را با عنصر ۱ جایگزین کنیم، و تمام عناصر دیگر نیز باید تغییر موقعیت دهند.

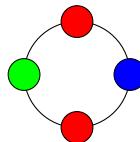
۴.۲۲ لم برنساید

لم برنساید می‌تواند برای شمارش تعداد ترکیب‌ها به کار رود، به‌گونه‌ای که برای هر گروه از ترکیب‌های متقارن فقط یک نماینده شمرده شود. لم برنساید بیان می‌کند که تعداد ترکیب‌ها برابر است با:

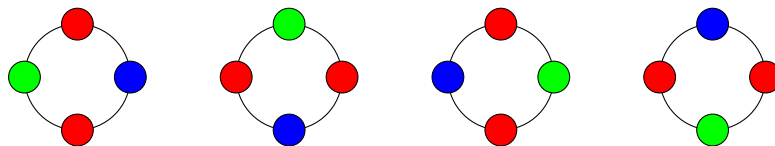
$$\sum_{k=1}^n \frac{c(k)}{n},$$

که در آن n روش برای تغییر وضعیت یک ترکیب وجود دارد، و $c(k)$ تعداد ترکیب‌هایی است که با اعمال روش k ام بدون تغییر باقی می‌مانند.

به عنوان مثال، تعداد گردنبندهای با n مروارید را محاسبه می‌کنیم، که در آن هر مروارید m رنگ ممکن دارد. دو گردنبند متقارن‌اند اگر با دوران مشابه یکدیگر شوند. برای مثال، گردنبند



گردنبندهای متقارن زیر را دارد:



n روش برای تغییر وضعیت گردنبند وجود دارد، زیرا می‌توان آن را $0, 1, \dots, n-1$ گام در جهت عقربه‌های ساعت چرخاند. اگر تعداد گام‌ها m^n باشد، همه m^n گردنبند یکسان می‌مانند، و اگر تعداد گام‌ها ۱ باشد، تنها m گردنبندی که در آن‌ها همه مرواریدها یک‌رنگ هستند بدون تغییر باقی می‌مانند.

به طور کلی‌تر، وقتی تعداد گام‌ها k است، در مجموع

$$m^{\gcd(k,n)}$$

گردنبند بدون تغییر باقی می‌ماند، که $\gcd(k, n)$ ب.م.م k و n است. دلیل این است که بلوک‌هایی از مرواریدها با اندازه $\gcd(k, n)$ جایگزین یکدیگر می‌شوند. بنابراین، بر طبق لم برنساید، تعداد گردنبندها برابر است با:

$$\sum_{i=0}^{n-1} \frac{m^{\gcd(i, n)}}{n}.$$

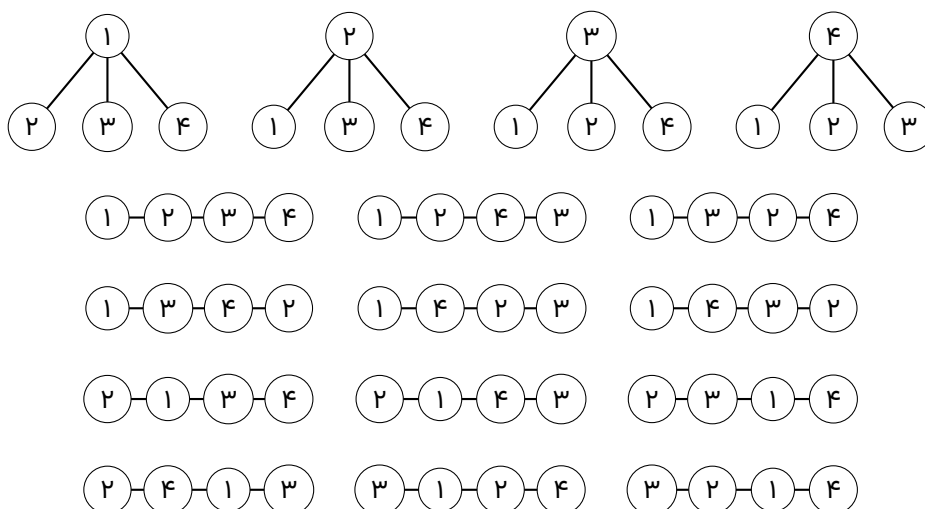
برای مثال، تعداد گردنبندهای به طول ۴ با ۳ رنگ برابر است با:

$$\frac{3^4 + 3 + 3^2 + 3}{4} = 24.$$

۵.۲۲ فرمول کیلی

فرمول کیلی بیان می‌کند که n^{n-2} درخت برچسب‌دار شامل n رأس وجود دارد. رأس‌ها با $1, 2, \dots, n$ برچسب‌گذاری شده‌اند، و دو درخت متفاوت در نظر گرفته می‌شوند اگر ساختار یا برچسب‌گذاری آن‌ها متفاوت باشد.

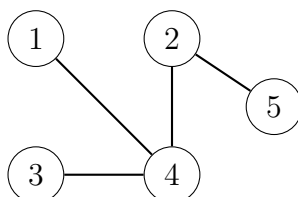
برای مثال، وقتی $n = 4$ باشد، تعداد درخت‌های برچسب‌دار برابر $4^{4-2} = 16$ است:



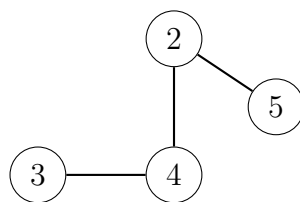
در ادامه بررسی می‌شود چگونه فرمول کیلی با استفاده از کدهای پروف‌ر اثبات می‌شود.

کد پروف‌ر

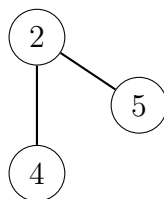
یک **کد پروف‌ر** دنباله‌ای از $n - 2$ عدد است که یک درخت برچسب‌دار را توصیف می‌کند. این کد طی فرایندی ساخته می‌شود که در آن $n - 2$ برگ از درخت حذف می‌شوند. در هر مرحله، برگ با کوچک‌ترین برچسب حذف می‌شود، و برچسب تنها همسایه آن به کد اضافه می‌گردد. برای مثال، کد پروف‌ر گراف زیر محاسبه می‌شود:



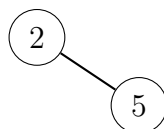
ابتدا رأس ۱ حذف شده و رأس ۴ به کد اضافه می‌شود:



سپس رأس ۳ را حذف و رأس ۴ را به کد اضافه می‌کنیم:



در نهایت رأس ۴ را حذف و رأس ۲ را به کد اضافه می‌کنیم:



بنابراین، کد پروفور گراف برابر $[4, 4, 2]$ است. برای هر درخت می‌توان یک کد پروفور ساخت و مهم‌تر آنکه، درخت اصلی را می‌توان از روی کد پروفور بازسازی کرد. از این رو، تعداد درخت‌های برچسب‌دار با n رأس برابر است با n^{n-2} ، یعنی تعداد کدهای پروفور به طول $n - 2$ برای n رأس.

فصل ۲۳

ماتریس‌ها

یک **ماتریس** مفهومی ریاضی است که در برنامه‌نویسی متناظر با یک آرایه دوبعدی است. برای مثال،

$$A = \begin{bmatrix} 6 & 13 & 7 & 4 \\ 7 & 0 & 8 & 2 \\ 9 & 5 & 4 & 18 \end{bmatrix}$$

یک ماتریس با ابعاد 3×4 است، یعنی دارای ۳ سطر و ۴ ستون است. نماد $[i, j]$ به عنصر واقع در سطر i و ستون j در یک ماتریس اشاره دارد. برای نمونه، در ماتریس بالا، $A[2, 3] = 8$ و $A[3, 1] = 9$ است.

حالت خاصی از ماتریس، یک **بردار** است که ماتریسی یک‌بعدی با ابعاد $n \times 1$ است. برای مثال،

$$V = \begin{bmatrix} 4 \\ 7 \\ 5 \end{bmatrix}$$

یک بردار است که سه عنصر دارد.

ترانزپوز A^T از ماتریس A زمانی به دست می‌آید که سطرها و ستون‌های A جابه‌جا شوند، یعنی $A^T[i, j] = A[j, i]$.

$$A^T = \begin{bmatrix} 6 & 7 & 9 \\ 13 & 0 & 5 \\ 7 & 8 & 4 \\ 4 & 2 & 18 \end{bmatrix}$$

یک ماتریس را **ماتریس مربعی** گویند اگر تعداد سطرها و ستون‌های آن برابر باشد. برای نمونه، ماتریس زیر یک ماتریس مربعی است:

$$S = \begin{bmatrix} 3 & 12 & 4 \\ 5 & 9 & 15 \\ 0 & 2 & 4 \end{bmatrix}$$

۱.۲۳ عملگرها

مجموع $A + B$ برای ماتریس‌های A و B زمانی تعریف می‌شود که ابعاد آن‌ها یکسان باشد. نتیجه ماتریسی است که هر عنصر آن برابر با مجموع عناصر متناظر در A و B است.

برای مثال،

$$\begin{bmatrix} 6 & 1 & 4 \\ 3 & 9 & 2 \end{bmatrix} + \begin{bmatrix} 4 & 9 & 3 \\ 8 & 1 & 3 \end{bmatrix} = \begin{bmatrix} 6+4 & 1+9 & 4+3 \\ 3+8 & 9+1 & 2+3 \end{bmatrix} = \begin{bmatrix} 10 & 10 & 7 \\ 11 & 10 & 5 \end{bmatrix}.$$

ضرب ماتریس A در مقدار x به این معنی است که هر عنصر از A در x ضرب می‌شود. برای مثال،

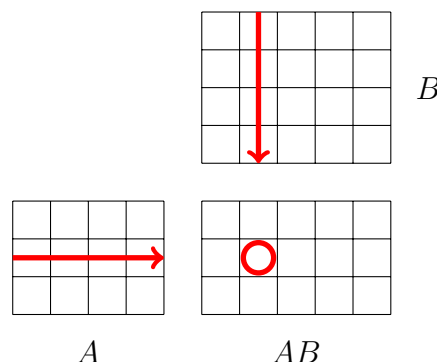
$$2 \cdot \begin{bmatrix} 6 & 1 & 4 \\ 3 & 9 & 2 \end{bmatrix} = \begin{bmatrix} 2 \cdot 6 & 2 \cdot 1 & 2 \cdot 4 \\ 2 \cdot 3 & 2 \cdot 9 & 2 \cdot 2 \end{bmatrix} = \begin{bmatrix} 12 & 2 & 8 \\ 6 & 18 & 4 \end{bmatrix}.$$

ضرب ماتریس

حاصل ضرب AB برای ماتریس‌های A و B زمانی تعریف می‌شود که اندازه A برابر $a \times n$ و اندازه B برابر $n \times b$ باشد، یعنی تعداد ستون‌های A برابر با تعداد سطرهای B باشد. نتیجه ماتریسی به ابعاد $a \times b$ است که عناصر آن با رابطه زیر محاسبه می‌شوند:

$$AB[i, j] = \sum_{k=1}^n A[i, k] \cdot B[k, j].$$

ایده اصلی این است که هر عنصر از AB برابر با مجموع حاصل ضرب‌های عناصر A و B مطابق تصویر زیر است:



برای مثال،

$$\begin{bmatrix} 1 & 4 \\ 3 & 9 \\ 8 & 6 \end{bmatrix} \cdot \begin{bmatrix} 1 & 6 \\ 2 & 9 \end{bmatrix} = \begin{bmatrix} 1 \cdot 1 + 4 \cdot 2 & 1 \cdot 6 + 4 \cdot 9 \\ 3 \cdot 1 + 9 \cdot 2 & 3 \cdot 6 + 9 \cdot 9 \\ 8 \cdot 1 + 6 \cdot 2 & 8 \cdot 6 + 6 \cdot 9 \end{bmatrix} = \begin{bmatrix} 9 & 42 \\ 21 & 99 \\ 20 & 102 \end{bmatrix}.$$

ضرب ماتریس‌ها شرکت‌پذیر است، بنابراین $A(BC) = (AB)C$ برقرار است، اما جابه‌جایی‌پذیر نیست، بنابراین $AB = BA$ معمولاً برقرار نیست.

یک **ماتریس همانی** ماتریسی مربعی است که در آن هر عنصر روی قطر اصلی ۱ و تمام عناصر دیگر ۰ هستند. برای مثال، ماتریس زیر ماتریس همانی 3×3 است:

$$I = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

ضرب یک ماتریس در ماتریس همانی آن را تغییر نمی‌دهد. برای مثال،

$$\begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} \cdot \begin{bmatrix} 1 & 4 \\ 3 & 9 \\ 8 & 6 \end{bmatrix} = \begin{bmatrix} 1 & 4 \\ 3 & 9 \\ 8 & 6 \end{bmatrix} \quad \text{و} \quad \begin{bmatrix} 1 & 4 \\ 3 & 9 \\ 8 & 6 \end{bmatrix} \cdot \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} = \begin{bmatrix} 1 & 4 \\ 3 & 9 \\ 8 & 6 \end{bmatrix}.$$

با استفاده از یک الگوریتم سرراست، می‌توان حاصل ضرب دو ماتریس $n \times n$ را در زمان $O(n^3)$ محاسبه کرد. الگوریتم‌های کارآمدتری نیز برای ضرب ماتریس‌ها وجود دارند^۱، اما آنها عمدتاً جنبه نظری دارند و چنین الگوریتم‌هایی در برنامه‌نویسی رقابتی لازم نیستند.

توان ماتریس

توان A^k ماتریس A در صورتی تعریف می‌شود که A یک ماتریس مربعی باشد. این تعریف مبتنی بر ضرب ماتریس‌ها است:

$$A^k = \underbrace{A \cdot A \cdot A \cdots A}_{k \text{ بار}}$$

برای مثال،

$$\begin{bmatrix} 2 & 5 \\ 1 & 4 \end{bmatrix}^3 = \begin{bmatrix} 2 & 5 \\ 1 & 4 \end{bmatrix} \cdot \begin{bmatrix} 2 & 5 \\ 1 & 4 \end{bmatrix} \cdot \begin{bmatrix} 2 & 5 \\ 1 & 4 \end{bmatrix} = \begin{bmatrix} 48 & 165 \\ 33 & 114 \end{bmatrix}.$$

علاوه بر این، A^0 ماتریس همانی است. برای مثال،

$$\begin{bmatrix} 2 & 5 \\ 1 & 4 \end{bmatrix}^0 = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}.$$

ماتریس A^k را می‌توان با استفاده از الگوریتم فصل ۲.۲۱ به‌طور کارآمد در زمان $O(n^3 \log k)$ محاسبه کرد. برای مثال،

$$\begin{bmatrix} 2 & 5 \\ 1 & 4 \end{bmatrix}^8 = \begin{bmatrix} 2 & 5 \\ 1 & 4 \end{bmatrix}^4 \cdot \begin{bmatrix} 2 & 5 \\ 1 & 4 \end{bmatrix}^4.$$

دترمینان

دترمینان $\det(A)$ ماتریس A زمانی تعریف می‌شود که A ماتریسی مربعی باشد. اگر اندازه A برابر 1×1 باشد، آن‌گاه $\det(A) = A[1, 1]$. دترمینان ماتریس‌های بزرگ‌تر به‌صورت بازگشتی با این رابطه محاسبه می‌شود:

$$\det(A) = \sum_{j=1}^n A[1, j] C[1, j],$$

که در آن $C[i, j]$ **همساز** ماتریس A در سطر i و ستون j است. همساز از رابطه زیر محاسبه می‌شود:

$$C[i, j] = (-1)^{i+j} \det(M[i, j]),$$

^۱نخستین مورد از این الگوریتم‌ها الگوریتم استراسن بود، منتشر شده در سال ۱۹۶۹ [۶۳]، که پیچیدگی زمانی آن $O(n^{2.80735})$ است؛ بهترین الگوریتم فعلی [۲۷] در زمان $O(n^{2.37286})$ کار می‌کند.

که در آن $M[i, j]$ از حذف سطر i و ستون j ماتریس A به دست می آید. به دلیل وجود ضریب $(-1)^{i+j}$ در همساز، علامت دترمینان ها یک در میان مثبت و منفی می شود. برای نمونه،

$$\det\left(\begin{bmatrix} 3 & 4 \\ 1 & 6 \end{bmatrix}\right) = 3 \cdot 6 - 4 \cdot 1 = 14$$

9

$$\det\left(\begin{bmatrix} 2 & 4 & 3 \\ 5 & 1 & 6 \\ 7 & 2 & 4 \end{bmatrix}\right) = 2 \cdot \det\left(\begin{bmatrix} 1 & 6 \\ 2 & 4 \end{bmatrix}\right) - 4 \cdot \det\left(\begin{bmatrix} 5 & 6 \\ 7 & 4 \end{bmatrix}\right) + 3 \cdot \det\left(\begin{bmatrix} 5 & 1 \\ 7 & 2 \end{bmatrix}\right) = 81.$$

دترمینان A نشان می دهد آیا **ماتریس معکوس** A^{-1} وجود دارد که در رابطه $A \cdot A^{-1} = I$ صدق کند یا خیر؛ که در آن I ماتریس همانی است. معلوم شده است که A^{-1} دقیقاً زمانی وجود دارد که $\det(A) \neq 0$ باشد، و می توان آن را با فرمول زیر محاسبه کرد:

$$A^{-1}[i, j] = \frac{C[j, i]}{\det(A)}.$$

برای نمونه،

$$\underbrace{\begin{bmatrix} 2 & 4 & 3 \\ 5 & 1 & 6 \\ 7 & 2 & 4 \end{bmatrix}}_A \cdot \underbrace{\frac{1}{81} \begin{bmatrix} -8 & -10 & 21 \\ 22 & -13 & 3 \\ 3 & 24 & -18 \end{bmatrix}}_{A^{-1}} = \underbrace{\begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}}_I.$$

۲.۲۳ رابطه های بازگشتی خطی

یک **رابطه بازگشتی خطی** تابعی مانند $f(n)$ است که مقادیر آغازین آن $f(0), f(1), \dots, f(k-1)$ بوده و مقادیر بزرگتر آن به صورت بازگشتی با این رابطه محاسبه می شوند:

$$f(n) = c_1 f(n-1) + c_2 f(n-2) + \dots + c_k f(n-k),$$

که در آن $c_1, c_2, \dots, c_k$ ضرایب ثابت هستند. با برنامه نویسی پویا می توان هر مقداری از $f(n)$ را در زمان $O(kn)$ با محاسبه پشت سرهم مقادیر $f(0), f(1), \dots, f(n)$ به دست آورد. اما اگر k کوچک باشد، می توان $f(n)$ را با عملیات ماتریسی بسیار کارآمدتر در زمان $O(k^3 \log n)$ محاسبه کرد.

اعداد فی بوناچی

یک نمونه ساده از رابطه بازگشتی خطی، تابع زیر است که اعداد فیبوناچی را تعریف می کند:

$$\begin{aligned} f(0) &= 0 \\ f(1) &= 1 \\ f(n) &= f(n-1) + f(n-2) \end{aligned}$$

در این حالت، $k=2$ و $c_1=c_2=1$ است.

برای محاسبه کارآمد اعداد فیبوناچی، رابطه فیبوناچی را به صورت یک ماتریس مربعی X با اندازه 2×2 نمایش می‌دهیم که رابطه زیر برای آن برقرار است:

$$X \cdot \begin{bmatrix} f(i) \\ f(i+1) \end{bmatrix} = \begin{bmatrix} f(i+1) \\ f(i+2) \end{bmatrix}$$

بنابراین، مقادیر $f(i)$ و $f(i+1)$ به عنوان «ورودی» به X داده می‌شوند و X مقادیر $f(i+1)$ و $f(i+2)$ را از روی آن‌ها محاسبه می‌کند. چنین ماتریسی برابر است با:

$$X = \begin{bmatrix} 0 & 1 \\ 1 & 1 \end{bmatrix}.$$

برای مثال،

$$\begin{bmatrix} 0 & 1 \\ 1 & 1 \end{bmatrix} \cdot \begin{bmatrix} f(5) \\ f(6) \end{bmatrix} = \begin{bmatrix} 0 & 1 \\ 1 & 1 \end{bmatrix} \cdot \begin{bmatrix} 5 \\ 8 \end{bmatrix} = \begin{bmatrix} 8 \\ 13 \end{bmatrix} = \begin{bmatrix} f(6) \\ f(7) \end{bmatrix}.$$

از این رو، می‌توانیم $f(n)$ را با استفاده از فرمول زیر محاسبه کنیم:

$$\begin{bmatrix} f(n) \\ f(n+1) \end{bmatrix} = X^n \cdot \begin{bmatrix} f(0) \\ f(1) \end{bmatrix} = \begin{bmatrix} 0 & 1 \\ 1 & 1 \end{bmatrix}^n \cdot \begin{bmatrix} 0 \\ 1 \end{bmatrix}.$$

مقدار X^n در زمان $O(\log n)$ قابل محاسبه است، بنابراین مقدار $f(n)$ نیز در زمان $O(\log n)$ به دست می‌آید.

حالت کلی

اکنون حالت کلی را در نظر می‌گیریم که در آن $f(n)$ یک رابطه بازگشتی خطی دلخواه است. هدف دوباره ساخت ماتریس X است به گونه‌ای که:

$$X \cdot \begin{bmatrix} f(i) \\ f(i+1) \\ \vdots \\ f(i+k-1) \end{bmatrix} = \begin{bmatrix} f(i+1) \\ f(i+2) \\ \vdots \\ f(i+k) \end{bmatrix}.$$

چنین ماتریسی عبارت است از:

$$X = \begin{bmatrix} 0 & 1 & 0 & 0 & \cdots & 0 \\ 0 & 0 & 1 & 0 & \cdots & 0 \\ 0 & 0 & 0 & 1 & \cdots & 0 \\ \vdots & \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & 0 & 0 & \cdots & 1 \\ c_k & c_{k-1} & c_{k-2} & c_{k-3} & \cdots & c_1 \end{bmatrix}.$$

در $k-1$ سطر اول، هر عنصر برابر ۰ است به جز یک عنصر که ۱ است. این سطرها $f(i)$ را با $f(i+1)$ ، $f(i+1)$ را با $f(i+2)$ و به همین ترتیب جایگزین می‌کنند. سطر آخر شامل ضرایب رابطه بازگشتی برای محاسبه مقدار جدید $f(i+k)$ است.

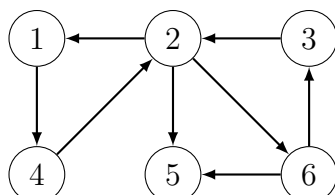
اکنون، $f(n)$ در زمان $O(k^3 \log n)$ با استفاده از فرمول زیر قابل محاسبه است:

$$\begin{bmatrix} f(n) \\ f(n+1) \\ \vdots \\ f(n+k-1) \end{bmatrix} = X^n \cdot \begin{bmatrix} f(0) \\ f(1) \\ \vdots \\ f(k-1) \end{bmatrix}.$$

۳.۲۳ گراف‌ها و ماتریس‌ها

شم‌ارش مسیره‌ا

توان‌های ماتریس مجاورت یک گراف ویژگی جالبی دارند. وقتی V ماتریس مجاورت یک گراف بدون وزن باشد، ماتریس V^n شامل تعداد مسیرهای به طول n یال بین رأس‌های گراف است. برای مثال، برای گراف



ماتریس مجاورت به صورت زیر است:

$$V = \begin{bmatrix} 0 & 0 & 0 & 1 & 0 & 0 \\ 1 & 0 & 0 & 0 & 1 & 1 \\ 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 1 & 0 \end{bmatrix}.$$

اکنون، برای مثال، ماتریس

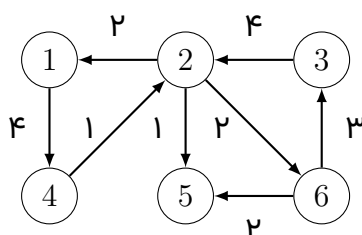
$$V^4 = \begin{bmatrix} 0 & 0 & 1 & 1 & 1 & 0 \\ 2 & 0 & 0 & 0 & 2 & 2 \\ 0 & 2 & 0 & 0 & 0 & 0 \\ 0 & 2 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 1 & 1 & 0 \end{bmatrix}$$

شامل تعداد مسیرهای با ۴ یال بین رأس‌ها است. برای مثال، $V^4[2, 5] = 2$ ، زیرا دو مسیر با ۴ یال از رأس ۲ به رأس ۵ وجود دارد: $2 \rightarrow 1 \rightarrow 4 \rightarrow 2 \rightarrow 5$ و $2 \rightarrow 6 \rightarrow 3 \rightarrow 2 \rightarrow 5$.

کوتاه‌ترین مسیره‌ا

با استفاده از ایده‌ای مشابه در یک گراف وزن‌دار، می‌توان برای هر جفت از رأس‌ها کمینه طول مسیری بین آن‌ها را که دقیقاً شامل n یال است محاسبه کرد. برای محاسبه این مقدار، باید ضرب ماتریسی را به روشی جدید تعریف کنیم تا به جای محاسبه تعداد مسیرها، طول مسیرها را کمینه کنیم.

به عنوان مثال، گراف زیر را در نظر بگیرید:



ماتریس مجاورتی تشکیل می‌دهیم که در آن ∞ به معنای نبود یال است و سایر مقادیر

نشان‌دهنده وزن یال‌ها هستند. ماتریس عبارت است از:

$$V = \begin{bmatrix} \infty & \infty & \infty & 4 & \infty & \infty \\ 2 & \infty & \infty & \infty & 1 & 2 \\ \infty & 4 & \infty & \infty & \infty & \infty \\ \infty & 1 & \infty & \infty & \infty & \infty \\ \infty & \infty & \infty & \infty & \infty & \infty \\ \infty & \infty & 3 & \infty & 2 & \infty \end{bmatrix}.$$

به جای فرمول

$$AB[i, j] = \sum_{k=1}^n A[i, k] \cdot B[k, j]$$

اکنون از فرمول

$$AB[i, j] = \min_{k=1}^n A[i, k] + B[k, j]$$

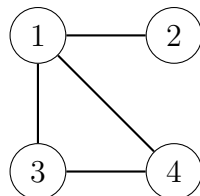
برای ضرب ماتریس‌ها استفاده می‌کنیم؛ یعنی به جای جمع کمینه را حساب می‌کنیم و به جای ضرب مقادیر، مجموع آن‌ها را به دست می‌آوریم. پس از این تغییر، توان‌های ماتریس متناظر با کوتاه‌ترین مسیرها در گراف خواهند بود.
برای نمونه، با توجه به اینکه

$$V^4 = \begin{bmatrix} \infty & \infty & 10 & 11 & 9 & \infty \\ 9 & \infty & \infty & \infty & 8 & 9 \\ \infty & 11 & \infty & \infty & \infty & \infty \\ \infty & 8 & \infty & \infty & \infty & \infty \\ \infty & \infty & \infty & \infty & \infty & \infty \\ \infty & \infty & 12 & 13 & 11 & \infty \end{bmatrix},$$

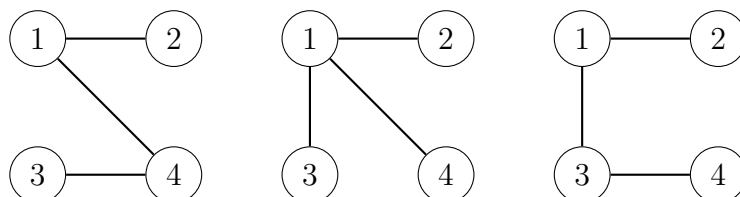
می‌توان نتیجه گرفت که کمینه طول یک مسیر با ۴ یال از رأس ۲ به رأس ۵ برابر ۸ است.
یک نمونه از چنین مسیری $2 \rightarrow 1 \rightarrow 4 \rightarrow 2 \rightarrow 5$ است.

قضیه کیرشهف

قضیه کیرشهف روشی ارائه می‌دهد تا تعداد درخت‌های پوشای یک گراف را با استفاده از دترمینان یک ماتریس خاص محاسبه کنیم. به عنوان مثال، گراف



دارای سه درخت پوشا است:



برای محاسبه تعداد درخت‌های فراگیر، یک ماتریس لاپلاس L می‌سازیم، به طوری که $L[i, i]$ درجه رأس i است و اگر یالی بین رؤس i و j وجود داشته باشد $L[i, j] = -1$ ، و در غیر این صورت $L[i, j] = 0$ است. ماتریس لاپلاس برای گراف بالا به شکل زیر است:

$$L = \begin{bmatrix} 3 & -1 & -1 & -1 \\ -1 & 1 & 0 & 0 \\ -1 & 0 & 2 & -1 \\ -1 & 0 & -1 & 2 \end{bmatrix}$$

می‌توان نشان داد که تعداد درخت‌های فراگیر برابر است با دترمینان ماتریسی که از حذف هر سطر و هر ستون دلخواه از L به دست می‌آید. برای مثال، اگر سطر و ستون اول را حذف کنیم، نتیجه برابر است با:

$$\det \begin{pmatrix} 1 & 0 & 0 \\ 0 & 2 & -1 \\ 0 & -1 & 2 \end{pmatrix} = 3.$$

مقدار دترمینان همواره یکسان است، صرف‌نظر از اینکه کدام سطر و ستون از L حذف شود. توجه کنید که فرمول کیلی در بخش ۵.۲۲ یک حالت خاص از قضیه کیرشهف است، زیرا در یک گراف کامل با n رأس داریم:

$$\det \begin{pmatrix} n-1 & -1 & \cdots & -1 \\ -1 & n-1 & \cdots & -1 \\ \vdots & \vdots & \ddots & \vdots \\ -1 & -1 & \cdots & n-1 \end{pmatrix} = n^{n-2}.$$

فصل ۲۴

احتمال

یک احتمال عددی حقیقی بین 0 و 1 است که میزان محتمل بودن یک پیشامد را نشان می‌دهد. اگر رخ دادن پیشامدی قطعی باشد، احتمال آن 1 است، و اگر پیشامدی غیرممکن باشد، احتمال آن 0 است. احتمال یک پیشامد با $P(\cdot)$ نشان داده می‌شود که در آن سه نقطه پیشامد را توصیف می‌کند.

برای مثال، هنگام پرتاب یک تاس، نتیجه یک عدد صحیح بین 1 و 6 است، و احتمال هر نتیجه $1/6$ است. برای مثال، می‌توانیم احتمال‌های زیر را محاسبه کنیم:

$$\begin{aligned}P(\text{"نتیجه ۴ باشد"}) &= 1/6 \cdot \\P(\text{"نتیجه ۶ نباشد"}) &= 5/6 \cdot \\P(\text{"نتیجه زوج باشد"}) &= 1/2 \cdot\end{aligned}$$

۱.۲۴ محاسبه

برای محاسبه احتمال یک پیشامد، می‌توانیم از ترکیبیات استفاده کنیم یا فرآیندی را که پیشامد را تولید می‌کند شبیه‌سازی نماییم. به عنوان مثال، بیایید احتمال کشیدن سه کارت با ارزش یکسان از یک دسته کارت بر خورده را محاسبه کنیم (برای مثال، $8\spadesuit$ ، $8\clubsuit$ و $8\diamondsuit$).

روش ۱

می‌توانیم احتمال را با استفاده از فرمول زیر محاسبه کنیم:

$$\frac{\text{تعداد پیشامدهای مطلوب}}{\text{تعداد کل پیشامدها}}.$$

در این مسئله، پیشامدهای مطلوب مواردی هستند که در آن‌ها ارزش هر کارت یکسان است. تعداد $13\binom{4}{3}$ از این پیشامدها وجود دارد، زیرا 13 حالت ممکن برای ارزش کارت‌ها و $\binom{4}{3}$ روش برای انتخاب 3 خال از 4 خال ممکن وجود دارد. در مجموع $\binom{52}{3}$ برآمد وجود دارد، زیرا ما 3 کارت از 52 کارت را انتخاب می‌کنیم. بنابراین، احتمال پیشامد برابر است با:

$$\frac{13\binom{4}{3}}{\binom{52}{3}} = \frac{1}{425}.$$

روش ۲

روش دیگر برای محاسبه احتمال، شبیه‌سازی فرآیندی است که پیشامد را تولید می‌کند. در این مثال، سه کارت می‌کشیم، بنابراین فرآیند شامل سه مرحله است. لازم است که هر مرحله از فرآیند موفقیت‌آمیز باشد.

کشیدن اولین کارت قطعاً موفقیت‌آمیز است، زیرا هیچ محدودیتی وجود ندارد. مرحله دوم با احتمال $3/51$ موفق می‌شود، زیرا ۵۱ کارت باقی مانده و ۳ تا از آن‌ها ارزشی یکسان با کارت اول دارند. به طور مشابه، مرحله سوم با احتمال $2/50$ موفق می‌شود. احتمال اینکه کل فرآیند موفقیت‌آمیز باشد برابر است با:

$$1 \cdot \frac{3}{51} \cdot \frac{2}{50} = \frac{1}{425}.$$

۲.۲۴ پیشامدها

یک پیشامد در نظریه احتمال می‌تواند به عنوان یک مجموعه نمایش داده شود:

$$A \subset X,$$

که در آن X شامل همه برآمدهای ممکن است و A زیرمجموعه‌ای از برآمدهاست. برای مثال، هنگام پرتاب یک تاس، برآمدها عبارتند از:

$$X = \{1, 2, 3, 4, 5, 6\}.$$

حال، به عنوان مثال، پیشامد "نتیجه زوج باشد" متناظر با مجموعه زیر است:

$$A = \{2, 4, 6\}.$$

به هر برآمد x یک احتمال $p(x)$ نسبت داده می‌شود. سپس، احتمال $P(A)$ برای یک پیشامد A را می‌توان به صورت مجموع احتمالات برآمدها با استفاده از فرمول زیر محاسبه کرد:

$$P(A) = \sum_{x \in A} p(x).$$

به عنوان مثال، هنگام پرتاب یک تاس، برای هر برآمد x داریم $p(x) = 1/6$ ، بنابراین احتمال پیشامد «برآمد زوج است» برابر است با:

$$p(2) + p(4) + p(6) = 1/2.$$

مجموع احتمال برآمدهای موجود در X باید برابر با ۱ باشد، یعنی $P(X) = 1$. از آنجا که پیشامدها در نظریه احتمالات مجموعه هستند، می‌توانیم با استفاده از عملگرهای استاندارد مجموعه‌ها روی آن‌ها عملیات انجام دهیم:

• **متمم** $\bar{A}$ یعنی « A رخ ندهد». به عنوان مثال، هنگام پرتاب تاس، متمم $A = \{2, 4, 6\}$ برابر است با $\bar{A} = \{1, 3, 5\}$.

• **اجتماع** $A \cup B$ یعنی « A یا B رخ دهد». به عنوان مثال، اجتماع $A = \{2, 5\}$ و $B = \{4, 5, 6\}$ برابر است با $A \cup B = \{2, 4, 5, 6\}$.

• **اشتراک** $A \cap B$ یعنی « A و B رخ دهند». به عنوان مثال، اشتراک $A = \{2, 5\}$ و $B = \{4, 5, 6\}$ برابر است با $A \cap B = \{5\}$.

متمم

احتمال متمم $\bar{A}$ با فرمول زیر محاسبه می‌شود:

$$P(\bar{A}) = 1 - P(A).$$

گاهی می‌توان یک مسئله را به سادگی با کمک متمم و حل مسئله معکوس آن حل کرد. به عنوان مثال، احتمال به دست آوردن حداقل یک ۶ در ده بار پرتاب تاس برابر است با:

$$1 - (5/6)^{10}.$$

در اینجا $5/6$ احتمال آن است که برآمد یک بار پرتاب، ۶ نباشد، و $(5/6)^{10}$ احتمال آن است که هیچ‌کدام از ده پرتاب، ۶ نشوند. متمم این پیشامد پاسخ مسئله است.

اجتماع

احتمال اجتماع $A \cup B$ با فرمول زیر محاسبه می‌شود:

$$P(A \cup B) = P(A) + P(B) - P(A \cap B).$$

به عنوان مثال، هنگام پرتاب تاس، اجتماع پیشامدهای

$$A = \text{«برآمد زوج باشد»}$$

9

$$B = \text{«برآمد کمتر از ۴ باشد»}$$

برابر است با

$$A \cup B = \text{«برآمد زوج باشد یا کمتر از ۴ باشد»}$$

و احتمال آن برابر است با:

$$P(A \cup B) = P(A) + P(B) - P(A \cap B) = 1/2 + 1/2 - 1/6 = 5/6.$$

اگر پیشامدهای A و B ناسازگار باشند، یعنی $A \cap B$ تهی باشد، احتمال پیشامد $A \cup B$ به سادگی برابر است با:

$$P(A \cup B) = P(A) + P(B).$$

احتمال شرطی

احتمال شرطی

$$P(A|B) = \frac{P(A \cap B)}{P(B)}$$

احتمال رخ دادن A است به فرض این که B رخ داده باشد. بنابراین، هنگام محاسبه احتمال A ، تنها برآمدهایی را بررسی می‌کنیم که متعلق به B نیز باشند. با استفاده از مجموعه‌های قبلی،

$$P(A|B) = 1/3,$$

زیرا برآمدهای B برابر هستند با $\{1, 2, 3\}$ ، و یکی از آن‌ها زوج است. این مقدار، احتمال زوج بودن برآمد است اگر بدانیم برآمد بین $1 \dots 3$ است.

اشتراک

با استفاده از احتمال شرطی، احتمال اشتراک $A \cap B$ با رابطه زیر محاسبه می‌شود:

$$P(A \cap B) = P(A)P(B|A).$$

پیشامدهای A و B **مستقل** هستند اگر

$$P(A|B) = P(A) \text{ و } P(B|A) = P(B)$$

به این معنی که رخ دادن B احتمال وقوع A را تغییر نمی‌دهد و بالعکس. در این حالت، احتمال اشتراک برابر است با:

$$P(A \cap B) = P(A)P(B).$$

برای نمونه، هنگام کشیدن یک کارت از دسته ورق، پیشامدهای

$$A = \text{"خال کارت گشنیز است"}$$

9

$$B = \text{"ارزش کارت چهار است"}$$

مستقل هستند. بنابراین پیشامد

$$A \cap B = \text{"کارت، چهار گشنیز است"}$$

با احتمال زیر رخ می‌دهد:

$$P(A \cap B) = P(A)P(B) = 1/4 \cdot 1/13 = 1/52.$$

۳.۲۴ متغیرهای تصادفی

یک **متغیر تصادفی** مقداری است که توسط یک فرآیند تصادفی تولید می‌شود. برای نمونه، هنگام پرتاب دو تاس، یک متغیر تصادفی ممکن عبارت است از:

$$X = \text{"مجموع برآمدها"}.$$

برای مثال، اگر برآمدها $[4, 6]$ باشند (یعنی ابتدا چهار و سپس شش بیاید)، آنگاه مقدار X برابر ۱۰ است.

احتمال اینکه مقدار متغیر تصادفی X برابر x باشد را با $P(X = x)$ نشان می‌دهیم. برای نمونه، هنگام پرتاب دو تاس، $P(X = 10) = 3/36$ ، زیرا کل حالات ممکن ۳۶ است و سه روش ممکن برای به دست آوردن مجموع ۱۰ وجود دارد: $[4, 6]$ ، $[5, 5]$ و $[6, 4]$.

امید ریاضی

امید ریاضی $E[X]$ میانگین مقدار یک متغیر تصادفی X را نشان می‌دهد. امید ریاضی به صورت مجموع زیر محاسبه می‌شود:

$$\sum_x P(X = x)x$$

که در آن x تمام مقادیر ممکن X را طی می‌کند.
برای نمونه، هنگام پرتاب یک تاس، برآمد مورد انتظار برابر است با:

$$1/6 \cdot 1 + 1/6 \cdot 2 + 1/6 \cdot 3 + 1/6 \cdot 4 + 1/6 \cdot 5 + 1/6 \cdot 6 = 7/2.$$

یک ویژگی مفید امید ریاضی، **خطی بودن** آن است. این ویژگی نشان می‌دهد مجموع $E[X_1 + X_2 + \dots + X_n]$ همواره برابر است با مجموع $E[X_1] + E[X_2] + \dots + E[X_n]$.
این رابطه حتی در صورتی که متغیرهای تصادفی به یکدیگر وابسته باشند نیز برقرار است.
برای نمونه، هنگام پرتاب دو تاس، مجموع مورد انتظار برابر است با:

$$E[X_1 + X_2] = E[X_1] + E[X_2] = 7/2 + 7/2 = 7.$$

اکنون مسئله‌ای را در نظر بگیرید که در آن n توپ به صورت تصادفی درون n جعبه قرار داده می‌شوند، و هدف محاسبه امید ریاضی تعداد جعبه‌های خالی است. هر توپ احتمال برابری برای قرار گرفتن در هر یک از جعبه‌ها دارد. برای نمونه، اگر $n = 2$ باشد، حالت‌های ممکن به صورت زیر خواهند بود:



در این حالت، امید ریاضی تعداد جعبه‌های خالی برابر است با:

$$\frac{0 + 0 + 1 + 1}{4} = \frac{1}{2}.$$

در حالت کلی، احتمال خالی ماندن یک جعبه خاص برابر است با:

$$\left(\frac{n-1}{n}\right)^n,$$

زیرا هیچ توپی نباید درون آن قرار گیرد. بنابراین با استفاده از خطی بودن امید ریاضی، امید ریاضی تعداد جعبه‌های خالی برابر است با:

$$n \cdot \left(\frac{n-1}{n}\right)^n.$$

توزیع‌ها

توزیع یک متغیر تصادفی X ، احتمال هر مقداری را که X می‌تواند داشته باشد نشان می‌دهد. توزیع از مقادیر $P(X = x)$ تشکیل می‌شود. برای مثال هنگام پرتاب دو تاس، توزیع مجموع آن‌ها برابر است با:

۱۲	۱۱	۱۰	۹	۸	۷	۶	۵	۴	۳	۲	x
1/36	2/36	3/36	4/36	5/36	6/36	5/36	4/36	3/36	2/36	1/36	$P(X = x)$

در یک **توزیع یکنواخت**، متغیر تصادفی X دارای n مقدار ممکن $a, a+1, \dots, b$ است و احتمال هر مقدار برابر با $1/n$ می‌باشد. به عنوان مثال، هنگام پرتاب یک تاس، $a = 1$ و $b = 6$ است و برای هر مقدار x داریم $P(X = x) = 1/6$.
امید ریاضی X در یک توزیع یکنواخت برابر است با:

$$E[X] = \frac{a+b}{2}.$$

در یک توزیع دوجمله‌ای، n آزمایش انجام می‌شود و احتمال موفقیت در هر آزمایش برابر با p است. متغیر تصادفی X تعداد آزمایش‌های موفق را می‌شمارد و احتمال مقدار x برابر است با:

$$P(X = x) = p^x (1 - p)^{n-x} \binom{n}{x},$$

که در آن p^x و $(1 - p)^{n-x}$ متناظر با آزمایش‌های موفق و ناموفق هستند، و $\binom{n}{x}$ بیانگر تعداد روش‌های انتخاب ترتیب آزمایش‌ها است.

برای مثال در ده بار پرتاب یک تاس، احتمال این‌که دقیقاً سه بار عدد شش بیاید برابر است با $(1/6)^3 (5/6)^7 \binom{10}{3}$.

امید ریاضی X در توزیع دوجمله‌ای برابر است با:

$$E[X] = pn.$$

در یک توزیع هندسی، احتمال موفقیت در هر آزمایش برابر با p است و تا زمان رخ دادن اولین موفقیت ادامه می‌دهیم. متغیر تصادفی X تعداد آزمایش‌های لازم را می‌شمارد و احتمال مقدار x برابر است با:

$$P(X = x) = (1 - p)^{x-1} p,$$

که در آن $(1 - p)^{x-1}$ متناظر با آزمایش‌های ناموفق و p متناظر با اولین آزمایش موفق است. به عنوان مثال، اگر یک تاس را تا زمانی که عدد شش ظاهر شود پرتاب کنیم، احتمال

این‌که تعداد پرتاب‌ها دقیقاً ۴ بار باشد برابر است با $(5/6)^3 1/6$.

امید ریاضی X در یک توزیع هندسی برابر است با:

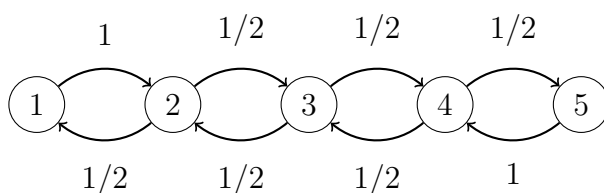
$$E[X] = \frac{1}{p}.$$

۴.۲۴ زنجیره‌های مارکوف

یک زنجیره مارکوف فرآیندی تصادفی شامل مجموعه‌ای از وضعیت‌ها و گذارهای بین آن‌ها است. به ازای هر وضعیت، احتمال انتقال به وضعیت‌های دیگر معلوم است. زنجیره مارکوف با گرافی نمایش داده می‌شود که رأس‌های آن وضعیت‌ها و یال‌های آن گذارها هستند.

به عنوان مثال، مسئله‌ای را در نظر بگیرید که در طبقه ۱ ساختمانی با n طبقه قرار داریم. در هر گام، به صورت تصادفی یک طبقه به بالا یا یک طبقه به پایین می‌رویم، به جز اینکه از طبقه ۱ همیشه یک طبقه بالا رفته و از طبقه n همیشه یک طبقه پایین می‌آییم. احتمال حضور در طبقه m پس از k گام چقدر است؟

در این مسئله، هر طبقه از ساختمان متناظر با وضعیتی در زنجیره مارکوف است. مثلاً اگر $n = 5$ باشد، گراف به شکل زیر خواهد بود:



توزیع احتمال زنجیره مارکوف، برداری به صورت $[p_1, p_2, \dots, p_n]$ است که در آن p_k بیانگر احتمال قرار گرفتن در وضعیت k است. همواره رابطه $p_1 + p_2 + \dots + p_n = 1$ برقرار است. در سناریوی بالا، توزیع اولیه برابر با $[1, 0, 0, 0, 0]$ است، چون همیشه از طبقه ۱ شروع می‌کنیم. توزیع بعدی $[0, 1, 0, 0, 0]$ است، چرا که از طبقه ۱ فقط به طبقه ۲ می‌توان رفت. پس از آن، می‌توان یا یک طبقه بالا رفت یا یک طبقه پایین، بنابراین توزیع بعدی برابر با $[1/2, 0, 1/2, 0, 0]$ است و به همین ترتیب ادامه می‌یابد.

روشی کارآمد برای شبیه‌سازی حرکت در زنجیره مارکوف، استفاده از برنامه‌نویسی پویا است. ایده کار نگهداری توزیع احتمال و در هر گام، بررسی تمام حالت‌های ممکن برای جابه‌جایی است. با این روش، می‌توان یک مسیر به طول m گام را در زمان $O(n^2 m)$ شبیه‌سازی کرد.

گذارهای زنجیره مارکوف را می‌توان به صورت ماتریسی نیز نمایش داد که توزیع احتمال را به‌روزرسانی می‌کند. در سناریوی بالا، این ماتریس به شکل زیر است:

$$\begin{bmatrix} 0 & 1/2 & 0 & 0 & 0 \\ 1 & 0 & 1/2 & 0 & 0 \\ 0 & 1/2 & 0 & 1/2 & 0 \\ 0 & 0 & 1/2 & 0 & 1 \\ 0 & 0 & 0 & 1/2 & 0 \end{bmatrix}.$$

وقتی یک توزیع احتمال را در این ماتریس ضرب کنیم، توزیع جدید پس از یک گام حرکت به دست می‌آید. برای نمونه، می‌توان از توزیع $[1, 0, 0, 0, 0]$ به توزیع $[0, 1, 0, 0, 0]$ به صورت زیر رفت:

$$\begin{bmatrix} 0 & 1/2 & 0 & 0 & 0 \\ 1 & 0 & 1/2 & 0 & 0 \\ 0 & 1/2 & 0 & 1/2 & 0 \\ 0 & 0 & 1/2 & 0 & 1 \\ 0 & 0 & 0 & 1/2 & 0 \end{bmatrix} \begin{bmatrix} 1 \\ 0 \\ 0 \\ 0 \\ 0 \end{bmatrix} = \begin{bmatrix} 0 \\ 1 \\ 0 \\ 0 \\ 0 \end{bmatrix}.$$

با محاسبه توان‌های ماتریس به شکل بهینه، می‌توان توزیع پس از m گام را در زمان $O(n^3 \log m)$ محاسبه کرد.

۵.۲۴ الگوریتم‌های تصادفی

گاهی می‌توان از تصادف برای حل مسئله استفاده کرد، حتی اگر مسئله ربطی به احتمال نداشته باشد. یک **الگوریتم تصادفی** الگوریتمی است که بر پایه تصادف کار می‌کند. یک **الگوریتم مونت‌کارلو** الگوریتمی تصادفی است که ممکن است گاهی پاسخ نادرست بدهد. برای کاربردی بودن چنین الگوریتمی، احتمال پاسخ نادرست باید کم باشد. یک **الگوریتم لاس‌وگاس** الگوریتمی تصادفی است که همیشه پاسخ درست می‌دهد، ولی زمان اجرای آن به صورت تصادفی تغییر می‌کند. هدف طراحی الگوریتمی است که با احتمال بالا کارآمد باشد.

در ادامه به سه مسئله نمونه می‌پردازیم که با تصادف حل می‌شوند.

آماره‌های ترتیبی

k -امین آماره ترتیبی در یک آرایه، عنصر واقع در موقعیت k پس از مرتب‌سازی صعودی آرایه است. محاسبه هر آماره ترتیبی با مرتب‌سازی اولیه آرایه در زمان $O(n \log n)$ آسان است،

ولی آیا واقعاً مرتب‌سازی کل آرایه فقط برای یافتن یک عنصر لازم است؟ مشخص می‌شود می‌توان آماره‌های ترتیبی را با الگوریتم تصادفی بدون مرتب‌سازی آرایه یافت. این الگوریتم، معروف به **انتخاب سریع**^۱، یک الگوریتم لاس‌وگاس است: زمان اجرای آن معمولاً $O(n)$ است ولی در بدترین حالت $O(n^2)$ می‌شود. الگوریتم یک عنصر تصادفی x از آرایه انتخاب می‌کند، و عناصر کوچک‌تر از x را به بخش چپ آرایه منتقل می‌کند، و سایر عناصر را به بخش راست آرایه می‌برد. این کار در حضور n عنصر، زمان $O(n)$ می‌برد. فرض کنید بخش چپ شامل a عنصر و بخش راست شامل b عنصر باشد. اگر $a = k$ باشد، عنصر x همان k -امین آماره ترتیبی است. در غیر این صورت، اگر $a > k$ باشد، به شکل بازگشتی k -امین آماره ترتیبی را برای بخش چپ پیدا می‌کنیم، و اگر $a < k$ باشد، به شکل بازگشتی r -امین آماره ترتیبی را برای بخش راست می‌یابیم که در آن $r = k - a$ است. جستجو به همین ترتیب ادامه می‌یابد تا عنصر پیدا شود. وقتی هر عنصر x تصادفی انتخاب شود، اندازه آرایه در هر مرحله تقریباً نصف می‌شود، بنابراین پیچیدگی زمانی برای یافتن k -امین آماره ترتیبی تقریباً برابر است با

$$n + n/2 + n/4 + n/8 + \dots < 2n = O(n).$$

بدترین حالت الگوریتم همچنان به زمان $O(n^2)$ نیاز دارد، زیرا ممکن است x همواره به گونه‌ای انتخاب شود که یکی از کوچک‌ترین یا بزرگ‌ترین عناصر آرایه باشد و به $O(n)$ مرحله نیاز شود. با این حال، احتمال وقوع این حالت آن قدر کم است که در عمل هرگز رخ نمی‌دهد.

بررسی درستی ضرب ماتریس‌ها

مسئله بعدی ما بررسی این است که آیا رابطه $AB = C$ برقرار است یا خیر، در حالی که A ، B و C ماتریس‌هایی با اندازه $n \times n$ هستند. البته می‌توان مسئله را با محاسبه دوباره حاصل ضرب AB حل کرد (در زمان $O(n^3)$ با استفاده از الگوریتم پایه)، اما انتظار می‌رود که بررسی درستی پاسخ آسان‌تر از محاسبه آن از ابتدا باشد.

مشخص می‌شود که می‌توان مسئله را با یک الگوریتم مونت کارلو^۲ حل کرد که پیچیدگی زمانی آن تنها $O(n^2)$ است. ایده ساده است: بردار تصادفی X شامل n عنصر را انتخاب می‌کنیم و ماتریس‌های ABX و CX را محاسبه می‌نماییم. اگر $ABX = CX$ باشد، اعلام می‌کنیم $AB = C$ ، و در غیر این صورت گزارش می‌دهیم $AB \neq C$.

پیچیدگی زمانی الگوریتم $O(n^2)$ است، زیرا می‌توان ماتریس‌های ABX و CX را در زمان $O(n^2)$ محاسبه کرد. می‌توان ماتریس ABX را با استفاده از نمایش $A(BX)$ بهینه محاسبه کرد، بنابراین تنها به دو ضرب ماتریس با اندازه‌های $n \times n$ و $n \times 1$ نیاز است.

نقطه ضعف الگوریتم این است که احتمال کمی وجود دارد که هنگام اعلام $AB = C$ دچار خطا شود. برای مثال،

$$\begin{bmatrix} 6 & 8 \\ 1 & 3 \end{bmatrix} \neq \begin{bmatrix} 8 & 7 \\ 3 & 2 \end{bmatrix},$$

اما

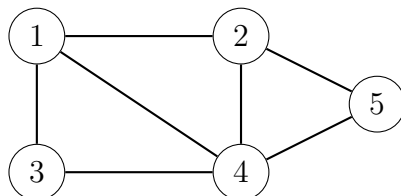
$$\begin{bmatrix} 6 & 8 \\ 1 & 3 \end{bmatrix} \begin{bmatrix} 3 \\ 6 \end{bmatrix} = \begin{bmatrix} 8 & 7 \\ 3 & 2 \end{bmatrix} \begin{bmatrix} 3 \\ 6 \end{bmatrix}.$$

با این حال در عمل، احتمال خطای الگوریتم اندک است، و می‌توان این احتمال را با بررسی نتیجه با چندین بردار تصادفی X پیش از اعلام $AB = C$ ، کاهش داد.

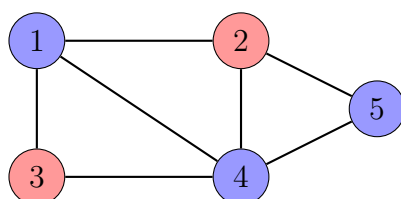
^۱در سال ۱۹۶۱، سی. ای. آر. هاور دو الگوریتم معرفی کرد که در حالت میانگین کارآمد هستند: **مرتب‌سازی سریع** [۳۶] برای مرتب‌سازی آرایه‌ها و **انتخاب سریع** [۳۷] برای یافتن آماره‌های ترتیبی.
^۲آر. ام. فرایوالدز این الگوریتم را در سال ۱۹۷۷ منتشر کرد [۲۶]، و گاهی **الگوریتم فرایوالدز** نامیده می‌شود.

رنگ‌آمیزی گراف

با فرض داشتن گرافی شامل n رأس و m یال، وظیفه ما یافتن روشی برای رنگ‌آمیزی رأس‌های گراف با دو رنگ است به طوری که حداقل برای $m/2$ یال، دو سر یال دارای رنگ‌های متفاوتی باشند. برای مثال، در گراف



یک رنگ‌آمیزی معتبر به صورت زیر است:



گراف بالا دارای ۷ یال است و برای ۵ تای آن‌ها، دو سر یال رنگ‌های متفاوتی دارند، بنابراین رنگ‌آمیزی معتبر است.

مسئله را می‌توان با الگوریتم لاس‌وگاس حل کرد که رنگ‌آمیزی‌های تصادفی تولید می‌کند تا زمانی که رنگ‌آمیزی معتبر یافت شود. در رنگ‌آمیزی تصادفی، رنگ هر رأس به طور مستقل انتخاب می‌شود به طوری که احتمال هر دو رنگ برابر $1/2$ است. در رنگ‌آمیزی تصادفی، احتمال اینکه دو سر یک یال رنگ‌های متفاوت داشته باشند $1/2$ است. بنابراین، امید ریاضی تعداد یال‌هایی که دو سر آن‌ها رنگ‌های متفاوت دارند $m/2$ است. از آنجا که انتظار می‌رود رنگ‌آمیزی تصادفی معتبر باشد، در عمل خیلی سریع رنگ‌آمیزی معتبر پیدا می‌کنیم.

فصل ۲۵

نظریه بازی‌ها

در این فصل، روی بازی‌های دو نفره بدون عامل شانس تمرکز می‌کنیم. هدف یافتن راهبردی است که در صورت وجود، پیروزی ما را مستقل از حرکت‌های حریف تضمین کند. یک راهبرد کلی برای چنین بازی‌هایی وجود دارد و تحلیل آن‌ها با استفاده از **نظریه نیم (nim theory)** امکان‌پذیر است. ابتدا بازی‌های ساده‌ای بررسی می‌شوند که در آن‌ها بازیکنان چوب‌ها را از کپه‌ها برمی‌دارند؛ سپس این راهبرد به بازی‌های دیگر تعمیم داده می‌شود.

۱.۲۵ وضعیت‌های بازی

بازی با یک کپه شامل n چوب را در نظر بگیرید. بازیکنان A و B به نوبت بازی می‌کنند و شروع با بازیکن A است. در هر نوبت، بازیکن باید ۱، ۲ یا ۳ چوب از کپه بردارد. بازیکنی که آخرین چوب را بردارد برنده است.

به عنوان مثال، اگر $n = 10$ باشد، روند بازی می‌تواند چنین باشد:

- بازیکن A تعداد ۲ چوب برمی‌دارد (۸ چوب باقی می‌ماند).
- بازیکن B تعداد ۳ چوب برمی‌دارد (۵ چوب باقی می‌ماند).
- بازیکن A تعداد ۱ چوب برمی‌دارد (۴ چوب باقی می‌ماند).
- بازیکن B تعداد ۲ چوب برمی‌دارد (۲ چوب باقی می‌ماند).
- بازیکن A تعداد ۲ چوب برمی‌دارد و برنده می‌شود.

این بازی از وضعیت‌های $0, 1, 2, \dots, n$ تشکیل شده است، که در آن شماره وضعیت نشان‌دهنده تعداد چوب‌های باقی‌مانده است.

وضعیت‌های برنده و بازنده

یک **وضعیت برنده** وضعیتی است که اگر بازیکن بهینه بازی کند، برنده خواهد شد. یک **وضعیت بازنده** وضعیتی است که در صورت بازی بهینه حریف، بازیکن بازنده خواهد شد. می‌توان تمام وضعیت‌های بازی را به‌گونه‌ای دسته‌بندی کرد که هر وضعیت یا برنده باشد یا بازنده.

در بازی بالا، وضعیت ۰ مشخصاً یک وضعیت بازنده است، چون بازیکن نمی‌تواند حرکتی انجام دهد. وضعیت‌های ۱، ۲ و ۳ وضعیت‌های برنده هستند، زیرا می‌توان ۱، ۲ یا ۳ چوب برداشت و بازی را برد. وضعیت ۴ به نوبه خود یک وضعیت بازنده است، زیرا هر حرکتی به وضعیتی منجر می‌شود که برای حریف یک وضعیت برنده است.

به‌طور کلی، اگر حرکتی از وضعیت فعلی به یک وضعیت بازنده وجود داشته باشد، وضعیت فعلی برنده است؛ در غیر این صورت، وضعیت فعلی بازنده است. با استفاده از این

قاعده، می‌توان تمام وضعیت‌های یک بازی را با شروع از وضعیت‌های بازنده‌ای که حرکتی در آن‌ها ممکن نیست دسته‌بندی کرد.

وضعیت‌های $0 \dots 15$ بازی بالا را می‌توان به شکل زیر دسته‌بندی کرد (W نشان‌دهنده وضعیت برنده و L نشان‌دهنده وضعیت بازنده است):

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
L	W	W	W	L	W	W	W	L	W	W	W	L	W	W	W

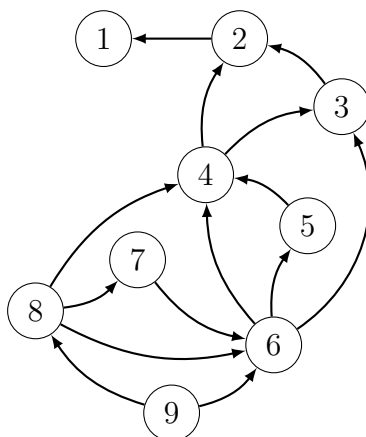
تحلیل این بازی ساده است: یک وضعیت k وضعیت بازنده است اگر k بر ۴ بخش‌پذیر باشد، وگرنه یک وضعیت برنده است. روش بهینه بازی کردن این است که همواره حرکتی انتخاب شود که پس از آن تعداد چوب‌ها در دسته بر ۴ بخش‌پذیر باشد. در نهایت، هیچ چوبی باقی نمی‌ماند و حریف باخته است.

البته این راهبرد نیازمند آن است که هنگام نوبت ما، تعداد چوب‌ها بر ۴ بخش‌پذیر نباشد. اگر باشد، کاری از دست ما برنمی‌آید، و در صورت بازی بهینه حریف، او برنده خواهد شد.

گراف وضعیت

حال بازی چوب دیگری را بررسی می‌کنیم، که در آن در هر وضعیت k ، مجاز به حذف هر تعداد x چوب هستیم به طوری که x کوچک‌تر از k بوده و k را عاد کند (بر آن بخش‌پذیر باشد). برای مثال، در وضعیت ۸ می‌توانیم ۱، ۲ یا ۴ چوب حذف کنیم، اما در وضعیت ۷ تنها حرکت مجاز حذف ۱ چوب است.

تصویر زیر وضعیت‌های $1 \dots 9$ بازی را به عنوان یک **گراف وضعیت** نشان می‌دهد، که گره‌های آن وضعیت‌ها و یال‌ها حرکات بین آن‌ها هستند:



وضعیت پایانی در این بازی همواره وضعیت ۱ است، که وضعیتی بازنده است، زیرا هیچ حرکت معتبری وجود ندارد. دسته‌بندی وضعیت‌های $1 \dots 9$ به صورت زیر است:

1	2	3	4	5	6	7	8	9
L	W	L	W	L	W	L	W	L

جالب توجه است که در این بازی، تمام وضعیت‌های زوج وضعیت‌های برد و تمام وضعیت‌های فرد وضعیت‌های باخت هستند.

۲.۲۵ بازی نيم

بازی نيم (nim game) بازی ساده‌ای است که نقش مهمی در نظریه بازی‌ها دارد، زیرا بازی‌های بسیار دیگری را می‌توان با استفاده از همین راهبرد انجام داد. ابتدا روی نيم تمرکز می‌کنیم و سپس این راهبرد را به بازی‌های دیگر تعمیم می‌دهیم.

در نيم n دسته وجود دارد و هر دسته شامل تعدادی چوب است. بازیکنان به نوبت حرکت می‌کنند و در هر نوبت، بازیکن دسته‌ای را انتخاب می‌کند که هنوز چوب دارد و هر تعداد چوب دلخواه از آن برمی‌دارد. برنده بازیکنی است که آخرین چوب را بردارد.

وضعیت‌ها در نيم به شکل $[x_1, x_2, \dots, x_n]$ هستند که در آن x_k نشان‌دهنده تعداد چوب‌های دسته k است. برای نمونه، $[10, 12, 5]$ یک بازی است که در آن سه دسته با ۱۰، ۱۲ و ۵ چوب وجود دارد. وضعیت $[0, 0, \dots, 0]$ یک وضعیت باخت است، زیرا امکان برداشتن هیچ چوبی وجود ندارد، و این همواره وضعیت پایانی است.

تحلیل

مشخص می‌شود که می‌توانیم به سادگی هر وضعیت نيم را با محاسبه **مجموع نيم** $s = x_1 \oplus x_2 \oplus \dots \oplus x_n$ دسته‌بندی کنیم، که در آن $\oplus$ عملیات XOR است.^۱ وضعیت‌هایی که مجموع نيم آن‌ها برابر ۰ است وضعیت باخت و بقیه وضعیت‌ها وضعیت برد هستند. برای نمونه، مجموع نيم $[10, 12, 5]$ برابر $10 \oplus 12 \oplus 5 = 3$ است، بنابراین این وضعیت یک وضعیت برد است.

اما مجموع نيم چه ارتباطی با بازی نيم دارد؟ می‌توانیم این موضوع را با بررسی چگونگی تغییر مجموع نيم هنگام تغییر وضعیت نيم توضیح دهیم.

وضعیت‌های باخت: وضعیت پایانی $[0, 0, \dots, 0]$ یک وضعیت باخت است و مجموع نيم آن برابر ۰ است، همان‌طور که انتظار می‌رفت. در دیگر وضعیت‌های باخت، هر حرکتی به یک وضعیت برد می‌انجامد، زیرا با تغییر یک مقدار x_k ، مجموع نيم نیز تغییر می‌کند، بنابراین مجموع نيم پس از حرکت با ۰ متفاوت خواهد بود.

وضعیت‌های برد: می‌توانیم به یک وضعیت باخت حرکت کنیم اگر دسته‌ای مانند k وجود داشته باشد که $x_k \oplus s < x_k$ باشد. در این حالت، می‌توانیم چوب‌هایی از دسته k برداریم به طوری که شامل $x_k \oplus s$ چوب شود، که این به یک وضعیت باخت می‌انجامد. همواره چنین دسته‌ای وجود دارد؛ جایی که x_k در موقعیت چپ‌ترین بیت یک s ، دارای بیت یک باشد.

به عنوان نمونه، وضعیت $[10, 12, 5]$ را در نظر بگیرید. این وضعیت یک وضعیت برد است، زیرا مجموع نيم آن برابر ۳ است. بنابراین، باید حرکتی وجود داشته باشد که به یک وضعیت باخت منتهی شود. در ادامه چنین حرکتی را پیدا خواهیم کرد.

مجموع نيم (nim sum) این وضعیت بدین صورت است:

1010	۱۰
1100	۱۲
0101	۵
0011	۳

در این حالت، کپه با ۱۰ چوب تنها کپه‌ای است که در جایگاه چپ‌ترین بیت یک در مجموع نيم، بیت یک دارد:

^۱ راهبرد بهینه برای نيم در سال ۱۹۰۱ توسط Bouton L. C. منتشر شد [۱۰].

0110	۱۰
1100	۱۲
0101	۵
1100	۳

اندازه جدید کپه باید $10 \oplus 3 = 9$ باشد، بنابراین فقط یک چوب حذف می‌کنیم. پس از این، وضعیت به $[9, 12, 5]$ تبدیل می‌شود که وضعیتی بازنده است:

1001	۹
1100	۱۲
0101	۵
0000	۰

بازی مزی (Misère game)

در یک بازی مزی، هدف بازی برعکس است؛ یعنی بازیکنی که آخرین چوب را بردارد، بازی را می‌بازد. مشخص شده که بازی نیم مزی را می‌توان به صورت بهینه تقریباً مشابه بازی استاندارد نیم انجام داد.

ایده این است که ابتدا بازی مزی مانند بازی استاندارد انجام شود، اما در انتهای بازی راهبرد تغییر کند. راهبرد جدید در موقعیتی اعمال می‌شود که پس از حرکت بعدی، هر کپه حداکثر شامل یک چوب باشد.

در بازی استاندارد، باید حرکتی انتخاب شود که پس از آن تعداد زوجی از کپه‌های تک‌چوبی باقی بماند. اما در بازی مزی، حرکتی انتخاب می‌شود که تعداد فردی از کپه‌های تک‌چوبی بر جای بماند.

این راهبرد کار می‌کند زیرا وضعیتی که راهبرد در آن تغییر می‌کند همواره در بازی رخ می‌دهد، و این وضعیت یک وضعیت برنده است، زیرا دقیقاً یک کپه دارد که بیش از یک چوب دارد، پس مجموع نیم آن صفر نیست.

۳.۲۵ قضیه اسپرگ-گراندی (Sprague-Grundy theorem)

قضیه اسپرگ-گراندی^۲ راهبرد استفاده‌شده در بازی نیم را برای تمام بازی‌هایی که شرایط زیر را برآورده کنند، تعمیم می‌دهد:

- دو بازیکن به نوبت حرکت می‌کنند.
- بازی شامل وضعیت‌ها است و حرکات ممکن در هر وضعیت به این که نوبت چه کسی است بستگی ندارد.
- بازی زمانی پایان می‌یابد که یک بازیکن نتواند حرکتی انجام دهد.
- بازی قطعاً دیر یا زود پایان می‌یابد.
- بازیکنان اطلاعات کامل درباره وضعیت‌ها و حرکات مجاز دارند و هیچ تصادفی در بازی وجود ندارد.

ایده این است که برای هر وضعیت بازی یک عدد گراندی محاسبه شود که متناظر با تعداد چوب‌ها در یک کپه بازی نیم باشد. وقتی اعداد گراندی تمام وضعیت‌ها را بدانیم، می‌توانیم بازی را مانند بازی نیم انجام دهیم.

^۲ این قضیه به طور مستقل توسط آر. اسپرگ [۶۱] و پی. ام. گراندی [۳۱] کشف شد.

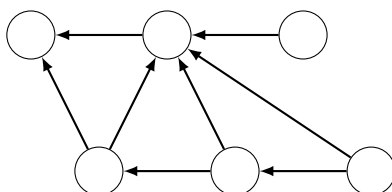
اعداد گراندی

عدد گراندی یک وضعیت بازی برابر است با

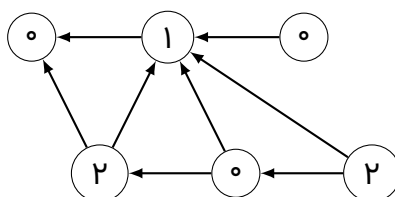
$$\text{mex}(\{g_1, g_2, \dots, g_n\}),$$

که در آن $g_1, g_2, \dots, g_n$ اعداد گراندی وضعیت‌هایی هستند که می‌توان به آن‌ها حرکت کرد، و تابع mex کوچک‌ترین عدد نامنفی را می‌دهد که در مجموعه وجود ندارد. به عنوان مثال، $\text{mex}(\{0, 1, 3\}) = 2$. اگر حرکتی از یک وضعیت امکان‌پذیر نباشد، عدد گراندی آن ۰ است، زیرا $\text{mex}(\emptyset) = 0$.

برای نمونه، در گراف وضعیت



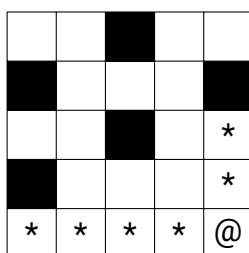
اعداد گراندی به صورت زیر هستند:



عدد گراندی یک وضعیت بازنده برابر ۰ است، و عدد گراندی یک وضعیت برنده عددی مثبت است.

عدد گراندی یک وضعیت متناظر است با تعداد چوب‌ها در یک دسته نیم. اگر عدد گراندی ۰ باشد، فقط می‌توان به وضعیت‌هایی رفت که عدد گراندی آن‌ها مثبت است، و اگر عدد گراندی $x > 0$ باشد، می‌توان به وضعیت‌هایی رفت که اعداد گراندی آن‌ها شامل همه اعداد $0, 1, \dots, x-1$ است.

به عنوان مثال، بازی‌ای را در نظر بگیرید که در آن بازیکنان مهره‌ای را درون یک ماز حرکت می‌دهند. هر خانه ماز یا کف است یا دیوار. در هر نوبت، بازیکن باید مهره را تعدادی گام به چپ یا بالا حرکت دهد. برنده بازی بازیکنی است که آخرین حرکت را انجام دهد. تصویر زیر یک وضعیت آغازین ممکن از بازی را نشان می‌دهد، که در آن @ مهره را مشخص می‌کند و * خانه‌ای را نشان می‌دهد که مهره می‌تواند به آن حرکت کند.



وضعیت‌های بازی، تمام خانه‌های خالی مارپیچ هستند. در مارپیچ بالا، اعداد گراندی به این صورت است:

$$\text{mex}(\{g_1, g_2, \dots, g_n\}),$$

که n تعداد حرکت‌های ممکن است و

$$g_k = a_{k,1} \oplus a_{k,2} \oplus \dots \oplus a_{k,m},$$

به طوری که حرکت k زیربازی‌هایی با اعداد گراندی $a_{k,1}, a_{k,2}, \dots, a_{k,m}$ ایجاد می‌کند. نمونه‌ای از چنین بازی‌هایی، **بازی گراندی** است. در ابتدا، یک دسته شامل n چوب وجود دارد. در هر نوبت، بازیکن یک دسته را انتخاب کرده و آن را به دو دسته ناتهی با اندازه‌های متفاوت تقسیم می‌کند. بازیکنی که آخرین حرکت را انجام دهد برنده بازی است. فرض کنید $f(n)$ عدد گراندی دسته‌ای با n چوب باشد. عدد گراندی با بررسی تمام روش‌های تقسیم دسته به دو دسته محاسبه می‌شود. برای مثال، وقتی $n = 8$ است، حالت‌های ممکن $1 + 7$ ، $2 + 6$ و $3 + 5$ هستند، پس:

$$f(8) = \text{mex}(\{f(1) \oplus f(7), f(2) \oplus f(6), f(3) \oplus f(5)\}).$$

در این بازی، مقدار $f(n)$ بر اساس مقادیر $f(1), \dots, f(n-1)$ به دست می‌آید. حالت‌های پایه $f(1) = f(2) = 0$ هستند، زیرا تقسیم دسته‌های ۱ و ۲ چوبی ممکن نیست. نخستین اعداد گراندی عبارتند از:

$$\begin{aligned} f(1) &= 0 \\ f(2) &= 0 \\ f(3) &= 1 \\ f(4) &= 0 \\ f(5) &= 2 \\ f(6) &= 1 \\ f(7) &= 0 \\ f(8) &= 2 \end{aligned}$$

عدد گراندی برای $n = 8$ برابر ۲ است، بنابراین بردن بازی ممکن است. حرکت برنده، ساخت دسته‌های $1 + 7$ است، زیرا $f(1) \oplus f(7) = 0$.

فصل ۲۶

الگوریتم‌های رشته

این فصل به الگوریتم‌های کارآمد برای پردازش رشته می‌پردازد. بسیاری از مسائل رشته به سادگی در زمان $O(n^2)$ حل می‌شوند، اما چالش اصلی یافتن الگوریتم‌هایی با زمان اجرای $O(n)$ یا $O(n \log n)$ است.

برای نمونه، یک مسئله بنیادین پردازش رشته مسئله **تطابق الگو** است: به ازای یک رشته به طول n و یک الگو به طول m ، هدف یافتن تمام رخدادهای الگو در رشته است. مثلاً، الگوی ABC دو بار در رشته ABABCBABC ظاهر می‌شود.

مسئله تطابق الگو به سادگی با یک الگوریتم جستجوی کامل (brute force) در زمان $O(nm)$ حل می‌شود؛ الگوریتمی که تمام جایگاه‌های ممکن رخداد الگو را بررسی می‌کند. با این حال، در این فصل خواهیم دید الگوریتم‌های کارآمدتری وجود دارند که تنها به زمان $O(n + m)$ نیاز دارند.

۱.۲۶ اصطلاحات رشته

در سراسر این فصل، فرض می‌شود اندیس‌گذاری رشته‌ها از صفر آغاز می‌شود. بنابراین، رشته s به طول n شامل نویسه‌های $s[0], s[1], \dots, s[n-1]$ است. مجموعه نویسه‌های مجاز در رشته‌ها **الفبا** نامیده می‌شود. برای نمونه، الفبای $\{A, B, \dots, Z\}$ شامل حروف بزرگ انگلیسی است.

یک **زیررشته** دنباله‌ای از نویسه‌های متوالی در یک رشته است. از نماد $s[a \dots b]$ برای اشاره به زیررشته‌ای از s استفاده می‌کنیم که در جایگاه a آغاز شده و در جایگاه b پایان می‌یابد. رشته‌ای به طول n دارای $n(n+1)/2$ زیررشته است. برای نمونه، زیررشته‌های ABCD عبارتند از: A, B, C, D, AB, BC, CD, ABC, BCD, ABCD.

یک **زیردنباله** دنباله‌ای از نویسه‌های رشته با حفظ ترتیب اولیه است (نیازی به متوالی بودن نویسه‌ها نیست). رشته‌ای به طول n دارای $2^n - 1$ زیردنباله است. برای نمونه، زیردنباله‌های ABCD عبارتند از: A, B, C, D, AC, AD, BC, BD, ABC, ABD, ACD, BCD, ABCD.

یک **پیشوند** زیررشته‌ای است که از ابتدای رشته آغاز می‌شود، و یک **پسوندد** زیررشته‌ای است که در انتهای رشته پایان می‌پذیرد. برای نمونه، پیشوندهای ABCD عبارتند از: A, AB, ABC و پسوندهای ABCD عبارتند از: D, CD, BCD و ABCD.

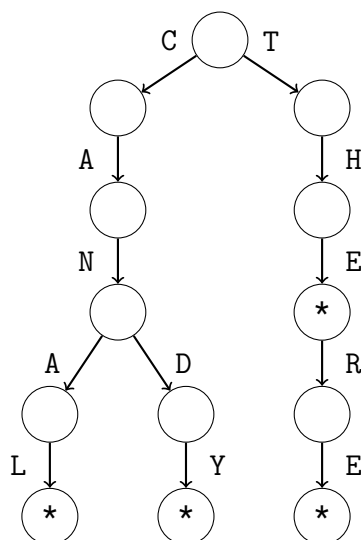
یک **دوران** را می‌توان با انتقال تک‌تک نویسه‌های رشته از ابتدا به انتها (یا برعکس) ایجاد کرد. برای نمونه، دوران‌های ABCD عبارتند از: ABCD, BCDA, CDAB و DABC.

یک **دوره** (تناوب) پیشوندی از رشته است به طوری که رشته با تکرار آن دوره ساخته شود. آخرین تکرار می‌تواند ناقص باشد و فقط پیشوندی از دوره را شامل شود. برای نمونه،

کوتاه‌ترین دوره رشته ABC برای ABC است.
 یک **مرز** رشته‌ای است که هم پیشوند و هم پسوند یک رشته باشد. برای نمونه، مرزهای رشته ABACABA عبارتند از A، ABA و ABACABA.
 رشته‌ها با استفاده از **ترتیب الفبایی** (معادل ترتیب فرهنگ لغت) مقایسه می‌شوند.
 بدین معنا که $x < y$ برقرار است اگر $x \neq y$ بوده و x پیشوندی از y باشد، یا موقعیتی مانند k وجود داشته باشد که به ازای $i < k$ داشته باشیم $x[i] = y[i]$ و همچنین $x[k] < y[k]$.

۲.۲۶ ساختار ترای

یک **ترای** درختی ریشه‌دار است که مجموعه‌ای از رشته‌ها را نگهداری می‌کند. هر رشته در این مجموعه به صورت زنجیره‌ای از نویسه‌ها ذخیره می‌شود که از ریشه آغاز می‌گردد. اگر دو رشته دارای پیشوند مشترک باشند، زنجیره مشترکی نیز در درخت دارند.
 برای نمونه، ترای زیر را در نظر بگیرید:



این درخت ترای متناظر با مجموعه {CANAL, CANDY, THE, THERE} است. نویسه * در یک گره نشان می‌دهد که یک رشته از مجموعه در آن گره پایان یافته است. چنین نویسه‌ای لازم است، زیرا یک رشته ممکن است پیشوندی از رشته دیگر باشد. برای نمونه، در درخت ترای بالا، THE پیشوندی از THERE است.

می‌توان در زمان $O(n)$ بررسی کرد که آیا ترای شامل رشته‌ای به طول n است یا خیر، زیرا می‌توان زنجیره‌ای را که از گره ریشه آغاز می‌شود پیمایش کرد. همچنین می‌توان رشته‌ای به طول n را در زمان $O(n)$ به ترای اضافه کرد؛ نخست با پیمایش زنجیره و سپس در صورت لزوم، افزودن گره‌های جدید به ترای.

با استفاده از ترای، می‌توان طولانی‌ترین پیشوند از یک رشته داده‌شده را یافت به‌طوری که آن پیشوند متعلق به مجموعه باشد. افزون بر این، با ذخیره اطلاعات اضافه در هر گره، می‌توان تعداد رشته‌هایی را که به مجموعه تعلق دارند و رشته داده‌شده پیشوند آن‌هاست محاسبه کرد.

یک ترای را می‌توان در یک آرایه ذخیره کرد:

```
];A][N[trie int
```

مجموعه در موجود رشته‌های طول مجموع (حداکثر گره‌ها تعداد حداکثر N آن در که شماره که طوری به می‌شوند شماره‌گذاری $0, 1, 2, \dots$ از ترای گره‌های است. الفبا اندازه A و

از استفاده با s گره از که هنگامی است زنجیره در بعدی گره $\text{trie}[s][c]$ و است،^۵ ریشه می‌کنیم. حرکت c نویسه

۳.۲۶ هشینگ رشته

هشینگ رشته روشی است که امکان بررسی کارآمد برابری دو رشته را فراهم می‌کند.^۱ ایده در هشینگ رشته، مقایسه مقادیر هش رشته‌ها به جای مقایسه تک‌تک نویسه‌های آن‌ها است.

محاسبه مقادیر هش

مقدار هش یک رشته، عددی است که از روی نویسه‌های آن رشته محاسبه می‌شود. اگر دو رشته یکسان باشند، مقادیر هش آن‌ها نیز یکسان است؛ این ویژگی امکان مقایسه رشته‌ها بر پایه مقادیر هش آن‌ها را فراهم می‌کند. روش متداول برای پیاده‌سازی هشینگ رشته، **هشینگ چندجمله‌ای** است؛ یعنی مقدار هش رشته s به طول n برابر است با:

$$(s[0]A^{n-1} + s[1]A^{n-2} + \dots + s[n-1]A^0) \bmod B,$$

که در آن $s[0], s[1], \dots, s[n-1]$ به عنوان کد نویسه‌های s در نظر گرفته می‌شوند، و A و B ثابت‌های از پیش انتخاب‌شده هستند. برای نمونه، کدهای نویسه‌های ALLEY عبارتند از:

A	L	L	E	Y
۶۵	۷۶	۷۶	۶۹	۸۹

بنابراین، اگر $A = 3$ و $B = 97$ باشد، مقدار هش ALLEY برابر است با:

$$(65 \cdot 3^4 + 76 \cdot 3^3 + 76 \cdot 3^2 + 69 \cdot 3^1 + 89 \cdot 3^0) \bmod 97 = 52.$$

پیش‌پردازش

با استفاده از هش چندجمله‌ای، می‌توان مقدار هش هر زیررشته از یک رشته s را پس از یک پیش‌پردازش با زمان $O(n)$ ، در زمان $O(1)$ محاسبه کرد. ایده این است که آرایه h به گونه‌ای ساخته شود که $h[k]$ حاوی مقدار هش پیشوند $s[0 \dots k]$ باشد. مقادیر این آرایه را می‌توان به صورت بازگشتی مطابق زیر محاسبه کرد:

$$\begin{aligned} h[0] &= s[0] \\ h[k] &= (h[k-1]A + s[k]) \bmod B \end{aligned}$$

علاوه بر این، آرایه p را می‌سازیم که در آن $p[k] = A^k \bmod B$ است:

$$\begin{aligned} p[0] &= 1 \\ p[k] &= (p[k-1]A) \bmod B. \end{aligned}$$

^۱ این روش توسط الگوریتم تطابق الگوی کارپ-رابین محبوبیت یافت [۴۲].

ساخت این آرایه‌ها به زمان $O(n)$ نیاز دارد. پس از آن، مقدار هش هر زیررشته $s[a \dots b]$ با فرض $a > 0$ می‌تواند در زمان $O(1)$ با استفاده از فرمول زیر محاسبه شود:

$$(h[b] - h[a-1]p[b-a+1]) \bmod B$$

اگر $a = 0$ باشد، مقدار هش همان $h[b]$ خواهد بود.

استفاده از مقادیر هش

می‌توان رشته‌ها را با استفاده از مقادیر هش به شکلی کارا مقایسه کرد. به جای مقایسه تک‌تک نویسه‌های رشته‌ها، ایده این است که مقادیر هش آن‌ها مقایسه شود. اگر مقادیر هش برابر باشند، رشته‌ها احتمالاً برابرند، و اگر مقادیر هش متفاوت باشند، رشته‌ها قطعاً متفاوت هستند.

با استفاده از هشینگ، اغلب می‌توان یک الگوریتم جستجوی کامل (brute force) را کارا کرد. به عنوان مثال، مسئله تطابق الگو را در نظر بگیرید: به ازای یک رشته s و یک الگو p ، موقعیت‌هایی را بیابید که p در s رخ می‌دهد. یک الگوریتم جستجوی کامل تمام موقعیت‌هایی را که p ممکن است در آن‌ها رخ دهد پیمایش کرده و رشته‌ها را نویسه‌به‌نویسه مقایسه می‌کند. پیچیدگی زمانی چنین الگوریتمی $O(n^2)$ است.

می‌توان با استفاده از هشینگ، الگوریتم جستجوی کامل را کارا تر کرد، زیرا این الگوریتم زیررشته‌هایی از رشته‌ها را مقایسه می‌کند. با استفاده از هشینگ، هر مقایسه تنها $O(1)$ زمان می‌برد، زیرا فقط مقادیر هش زیررشته‌ها مقایسه می‌شوند. این کار منجر به الگوریتمی با پیچیدگی زمانی $O(n)$ می‌شود که بهترین پیچیدگی زمانی ممکن برای این مسئله است.

با ترکیب هشینگ و جستجوی دودویی، یافتن ترتیب لغت‌نگاشتی (lexicographic) دو رشته در زمان لگاریتمی نیز امکان‌پذیر است. این کار با محاسبه طول طولانی‌ترین پیشوند مشترک دو رشته با استفاده از جستجوی دودویی انجام می‌شود. پس از یافتن طول پیشوند مشترک، کافی است نویسه بعدی پس از پیشوند را بررسی کنیم، زیرا این نویسه ترتیب رشته‌ها را مشخص می‌کند.

تصادم‌ها و پارامترها

یک خطر آشکار هنگام مقایسه مقادیر هش، وقوع یک تصادم (collision) است؛ بدین معنی که دو رشته محتوای متفاوتی دارند اما مقدار هش آن‌ها یکسان است. در این حالت، الگوریتمی که به مقادیر هش اتکا دارد نتیجه می‌گیرد که رشته‌ها برابر هستند، اما در واقعیت این‌گونه نیست و الگوریتم ممکن است نتایج نادرستی ارائه دهد.

برخوردها همواره ممکن هستند، زیرا تعداد رشته‌های متمایز بیشتر از تعداد مقادیر هش متمایز است. با این حال، در صورت انتخاب دقیق ثوابت A و B ، احتمال برخورد کم است. رویکرد متداول، انتخاب ثوابت تصادفی نزدیک به 10^9 است، به عنوان نمونه:

$$A = 911382323$$

$$B = 972663749$$

با چنین ثوابتی، هنگام محاسبه مقادیر هش می‌توان از نوع داده long long استفاده کرد، زیرا حاصل‌ضرب‌های AB و BB در long long جا می‌گیرند. اما آیا وجود حدود 10^9 مقدار هش متمایز کافی است؟

سه سناریو کاربرد هش را در نظر بگیرید:

سناریو ۱: رشته‌های x و y با یکدیگر مقایسه می‌شوند. با فرض هم‌احتمال بودن همه مقادیر هش، احتمال وقوع برخورد $1/B$ است.

سناریو ۲: رشته x با رشته‌های $y_1, y_2, \dots, y_n$ مقایسه می‌شود. احتمال وقوع یک یا چند برخورد برابر است با:

$$1 - \left(1 - \frac{1}{B}\right)^n.$$

سناریو ۳: همه جفت‌های رشته‌های $x_1, x_2, \dots, x_n$ با یکدیگر مقایسه می‌شوند. احتمال وقوع یک یا چند برخورد برابر است با:

$$1 - \frac{B \cdot (B-1) \cdot (B-2) \cdots (B-n+1)}{B^n}.$$

جدول زیر احتمال برخورد را به ازای $n = 10^6$ و مقادیر مختلف B نشان می‌دهد:

ثابت B	سناریو ۱	سناریو ۲	سناریو ۳
10^3	0.001000	1.000000	1.000000
10^6	0.000001	0.632121	1.000000
10^9	0.000000	0.001000	1.000000
10^{12}	0.000000	0.000000	0.393469
10^{15}	0.000000	0.000000	0.000500
10^{18}	0.000000	0.000000	0.000001

جدول نشان می‌دهد در سناریو ۱، هنگام $10^9 \approx B$ ، احتمال برخورد ناچیز است. در سناریو ۲، برخورد ممکن است ولی احتمال آن همچنان بسیار کم است. با این حال در سناریو ۳ وضعیت کاملاً متفاوت است: هنگام $10^9 \approx B$ برخورد تقریباً همواره رخ می‌دهد. پدیده موجود در سناریو ۳ تحت عنوان **پارادوکس روز تولد** شناخته می‌شود: اگر n نفر در یک اتاق باشند، احتمال اینکه حداقل دو نفر روز تولد یکسانی داشته باشند بالاست، حتی اگر n بسیار کوچک باشد. در هشینگ نیز به همین ترتیب، با مقایسه همه مقادیر هش با یکدیگر، احتمال برابر شدن حداقل دو مقدار هش بالاست.

می‌توان با محاسبه چندین مقدار هش با پارامترهای مختلف، احتمال برخورد را کاهش داد. وقوع برخورد همزمان در همه مقادیر هش بعید است. برای نمونه، دو مقدار هش با پارامتر $10^9 \approx B$ معادل یک مقدار هش با پارامتر $10^{18} \approx B$ عمل می‌کند، که احتمال برخورد را بسیار ناچیز می‌سازد.

برخی افراد از ثوابت $B = 2^{32}$ و $B = 2^{64}$ استفاده می‌کنند که کار را راحت می‌کند، زیرا محاسبات با اعداد صحیح ۳۲ و ۶۴ بیتی به پیمانه 2^{32} و 2^{64} انجام می‌شود. با این حال، این انتخاب خوبی نیست، زیرا می‌توان ورودی‌هایی ساخت که با ثوابت به فرم 2^x همیشه برخورد هش ایجاد کنند [۵۱].

۴.۲۶ الگوریتم Z

آرایه Z با نام z برای یک رشته s به طول n ، به ازای هر $k = 0, 1, \dots, n-1$ شامل طول طولانی‌ترین زیررشته‌ای از s است که از مکان k شروع می‌شود و پیشوندی از s است. بنابراین، $z[k] = p$ نشان می‌دهد که $s[0 \dots p-1]$ برابر با $s[k \dots k+p-1]$ است. مسائل متعددی در پردازش رشته‌ها را می‌توان با استفاده از آرایه Z به طور کارآمد حل کرد. به عنوان مثال، آرایه Z رشته ACBACDACBACBACDA به صورت زیر است:

۰	۱	۲	۳	۴	۵	۶	۷	۸	۹	۱۰	۱۱	۱۲	۱۳	۱۴	۱۵
A	C	B	A	C	D	A	C	B	A	C	B	A	C	D	A
—	۰	۰	۲	۰	۰	۵	۰	۰	۷	۰	۰	۲	۰	۰	۱

در این مورد، برای مثال $z[6] = 5$ است، زیرا زیررشته ACBAC با طول ۵ پیشوندی از s است، اما زیررشته ACBACB با طول ۶ پیشوندی از s نیست.

شرح الگوریتم

در ادامه الگوریتمی موسوم به **الگوریتم زد**^۲ را شرح می‌دهیم که آرایه زد را به شکل کارآمد در زمان $O(n)$ می‌سازد. این الگوریتم مقادیر آرایه زد را از چپ به راست محاسبه می‌کند؛ هم با استفاده از اطلاعاتی که از قبل در آرایه زد ذخیره شده‌اند و هم از طریق مقایسه کاراکتر به کاراکتر زیررشته‌ها.

برای محاسبه بهینه مقادیر آرایه زد، الگوریتم بازه‌ای مانند $[x, y]$ را نگهداری می‌کند به طوری که $s[x \dots y]$ پیشوندی از s باشد و y تا حد امکان بزرگ باشد. از آنجا که می‌دانیم $s[0 \dots y-x]$ و $s[x \dots y]$ برابر هستند، می‌توانیم هنگام محاسبه مقادیر زد برای موقعیت‌های $x+1, x+2, \dots, y$ از این اطلاعات بهره ببریم.

در هر موقعیت k ، ابتدا مقدار $z[k-x]$ را بررسی می‌کنیم. اگر $k + z[k-x] < y$ باشد، می‌دانیم که $z[k] = z[k-x]$ است. اما اگر $k + z[k-x] \geq y$ باشد، $s[0 \dots y-k]$ برابر با $s[k \dots y]$ است، و برای تعیین مقدار $z[k]$ باید زیررشته‌ها را کاراکتر به کاراکتر مقایسه کنیم. با این وجود، الگوریتم همچنان در زمان $O(n)$ اجرا می‌شود، زیرا مقایسه را از موقعیت‌های $y-k+1$ و $y+1$ آغاز می‌کنیم.

به عنوان مثال، فرض کنید می‌خواهیم آرایه زد زیر را بسازیم:

۰	۱	۲	۳	۴	۵	۶	۷	۸	۹	۱۰	۱۱	۱۲	۱۳	۱۴	۱۵
A	C	B	A	C	D	A	C	B	A	C	B	A	C	D	A
-	؟	؟	؟	؟	؟	؟	؟	؟	؟	؟	؟	؟	؟	؟	؟

پس از محاسبه مقدار $z[6] = 5$ ، بازه فعلی $[x, y]$ برابر با $[6, 10]$ خواهد بود:

						$\overbrace{\hspace{10em}}^y$									
۰	۱	۲	۳	۴	۵	۶	۷	۸	۹	۱۰	۱۱	۱۲	۱۳	۱۴	۱۵
A	C	B	A	C	D	A	C	B	A	C	B	A	C	D	A
-	۰	۰	۲	۰	۰	۵	؟	؟	؟	؟	؟	؟	؟	؟	؟

اکنون می‌توان مقادیر بعدی آرایه Z را به شکل کارآمد محاسبه کرد، زیرا می‌دانیم $s[0 \dots 4]$ و $s[6 \dots 10]$ برابرند. نخست، چون $z[1] = z[2] = 0$ است، بلافاصله نتیجه می‌شود $z[7] = z[8] = 0$:

						$\overbrace{\hspace{10em}}^y$									
۰	۱	۲	۳	۴	۵	۶	۷	۸	۹	۱۰	۱۱	۱۲	۱۳	۱۴	۱۵
A	C	B	A	C	D	A	C	B	A	C	B	A	C	D	A
-	۰	۰	۲	۰	۰	۵	۰	۰	؟	؟	؟	؟	؟	؟	؟

^۲الگوریتم Z در [۳۲] به عنوان ساده‌ترین روش شناخته‌شده برای تطابق الگو در زمان خطی معرفی شد و ایده اصلی آن به [۵۰] نسبت داده شده است.

سپس، چون $z[3] = 2$ است، می‌دانیم $z[9] \geq 2$:

x						y									
۰	۱	۲	۳	۴	۵	۶	۷	۸	۹	۱۰	۱۱	۱۲	۱۳	۱۴	۱۵
A	C	B	A	C	D	A	C	B	A	C	B	A	C	D	A
-	۰	۰	۲	۰	۰	۵	۰	۰	؟	؟	؟	؟	؟	؟	؟

با این حال، اطلاعاتی از رشته بعد از موقعیت ۱۰ نداریم؛ باید زیررشته‌ها نویسه‌به‌نویسه مقایسه شوند:

x						y									
۰	۱	۲	۳	۴	۵	۶	۷	۸	۹	۱۰	۱۱	۱۲	۱۳	۱۴	۱۵
A	C	B	A	C	D	A	C	B	A	C	B	A	C	D	A
-	۰	۰	۲	۰	۰	۵	۰	۰	؟	؟	؟	؟	؟	؟	؟

مشخص می‌شود که $z[9] = 7$ است، بنابراین بازه جدید $[x, y]$ برابر $[9, 15]$ می‌شود:

x									y						
۰	۱	۲	۳	۴	۵	۶	۷	۸	۹	۱۰	۱۱	۱۲	۱۳	۱۴	۱۵
A	C	B	A	C	D	A	C	B	A	C	B	A	C	D	A
-	۰	۰	۲	۰	۰	۵	۰	۰	۷	؟	؟	؟	؟	؟	؟

پس از این، تمام مقادیر باقی‌مانده آرایه Z را می‌توان با استفاده از اطلاعاتی که تاکنون درون آرایه Z ذخیره شده است، تعیین کرد:

x									y						
۰	۱	۲	۳	۴	۵	۶	۷	۸	۹	۱۰	۱۱	۱۲	۱۳	۱۴	۱۵
A	C	B	A	C	D	A	C	B	A	C	B	A	C	D	A
-	۰	۰	۲	۰	۰	۵	۰	۰	۷	۰	۰	۲	۰	۰	۱

استفاده از آرایه Z

انتخاب بین درهم‌سازی رشته یا الگوریتم Z اغلب سلیقه‌ای است. برخلاف درهم‌سازی، الگوریتم Z همواره کار می‌کند و خطر تصادم ندارد. از طرف دیگر، پیاده‌سازی الگوریتم Z دشوارتر است و برخی مسائل فقط با درهم‌سازی حل می‌شوند.

به عنوان مثال، مسئله تطابق الگو را دوباره در نظر بگیرید که هدف در آن، یافتن رخداد های الگوی p در رشته s است. این مسئله پیش‌تر با درهم‌سازی رشته به‌طور کارا حل شد، ولی الگوریتم Z روش دیگری برای حل آن ارائه می‌دهد.

یک ایده معمول در پردازش رشته ساخت رشته‌ای متشکل از چند رشته با جداکننده نویسه خاص است. در این مسئله می‌توان رشته $s\#p$ را ساخت، جایی که p و s با نویسه

خاص # جدا می‌شوند که درون رشته‌ها نیست. آرایه Z رشته $s\#p$ موقعیت‌های وقوع p در s را مشخص می‌کند، زیرا این موقعیت‌ها حاوی طول p هستند. برای مثال، اگر $HATTIVATTI\#s = ATT\#p$ باشد، آرایه Z بدین صورت است:

۰	۱	۲	۳	۴	۵	۶	۷	۸	۹	۱۰	۱۱	۱۲	۱۳
A	T	T	#	H	A	T	T	I	V	A	T	T	I
-	۰	۰	۰	۰	۳	۰	۰	۰	۰	۳	۰	۰	۰

موقعیت‌های ۵ و ۱۰ مقدار ۳ دارند، یعنی الگوی ATT در موقعیت‌های متناظر HATTIVATTI رخ می‌دهد. پیچیدگی زمانی الگوریتم حاصل خطی است، ساخت آرایه Z و پیمایش مقادیر آن کفایت می‌کند.

پیاده‌سازی

این پیاده‌سازی کوتاه الگوریتم Z است که بردار متناظر با آرایه Z را برمی‌گرداند.

```

{ )s string(z >int<vector
    ();size.s = n int
    );n(z >int<vector
        0; = y ,0 = x int
        { ++i ;n < i 1; = i int( for
            +1));i-y,]x-i[z(min,(0max = ]i[z
            { ]]i[z+i[s == ]]i[z[s && n < ]i[z+i( while
                ]++;i[z ];i[z+i = y ;i = x
            }
        }
    ;z return
}
```

فصل ۲۷

الگوریتم‌های ریشه دوم

یک الگوریتم ریشه دوم الگوریتمی است که ریشه دوم در پیچیدگی زمانی آن وجود دارد. ریشه دوم را می‌توان به عنوان «لگاریتم مرد فقیر» دید: پیچیدگی $O(\sqrt{n})$ بهتر از $O(n)$ اما بدتر از $O(\log n)$ است. در هر حال، بسیاری از الگوریتم‌های ریشه دوم سریع و در عمل کاربردی هستند.

به عنوان مثال، مسئله ساخت یک داده‌ساختار را در نظر بگیرید که از دو عملیات روی یک آرایه پشتیبانی می‌کند: تغییر مقدار یک عنصر در موقعیت داده‌شده و محاسبه مجموع عناصر در بازه داده‌شده. پیش از این، مسئله را با استفاده از درخت‌های بازه‌ای و درخت‌های پدیدار دودویی حل کرده‌ایم که هر دو عملیات را در زمان $O(\log n)$ پشتیبانی می‌کنند. اما اکنون مسئله را به روش دیگری با استفاده از یک ساختار ریشه دوم حل خواهیم کرد که امکان تغییر عناصر در زمان $O(1)$ و محاسبه مجموع در زمان $O(\sqrt{n})$ را فراهم می‌کند. ایده کار، تقسیم آرایه به بلوک‌هایی به اندازه $\sqrt{n}$ است، به طوری که هر بلوک شامل مجموع عناصر درون خود باشد. به عنوان مثال، یک آرایه ۱۶ عنصری به بلوک‌های ۴ عنصری به شکل زیر تقسیم می‌شود:

۲۱				۱۷				۲۰				۱۳			
۵	۸	۶	۳	۲	۷	۲	۶	۷	۱	۷	۵	۶	۲	۳	۲

در این ساختار، تغییر عناصر آرایه ساده است، زیرا پس از هر تغییر، تنها به‌روزرسانی مجموع یک بلوک منفرد لازم است که در زمان $O(1)$ انجام‌پذیر است. به عنوان مثال، تصویر زیر نشان می‌دهد که چگونه مقدار یک عنصر و مجموع بلوک متناظر آن تغییر می‌کنند:

۲۱				۱۵				۲۰				۱۳			
۵	۸	۶	۳	۲	۵	۲	۶	۷	۱	۷	۵	۶	۲	۳	۲

سپس، برای محاسبه مجموع عناصر در یک بازه، بازه را به سه بخش تقسیم می‌کنیم، به طوری که مجموع شامل مقادیر عناصر تکی و مجموع بلوک‌های بین آن‌ها باشد:

۲۱				۱۵				۲۰				۱۳			
۵	۸	۶	۳	۲	۵	۲	۶	۷	۱	۷	۵	۶	۲	۳	۲

از آنجا که تعداد عناصر منفرد $O(\sqrt{n})$ و تعداد بلوک‌ها نیز $O(\sqrt{n})$ است، پرس‌وجوی مجموع به زمان $O(\sqrt{n})$ نیاز دارد. هدف از انتخاب اندازه بلوک $\sqrt{n}$ این است که بین دو چیز تعادل برقرار می‌کند: آرایه به $\sqrt{n}$ بلوک تقسیم می‌شود، که هر کدام شامل $\sqrt{n}$ عنصر است. در عمل، نیازی نیست که از مقدار دقیق $\sqrt{n}$ به‌عنوان پارامتر استفاده شود، و در عوض می‌توانیم از پارامترهای k و n/k بهره ببریم، جایی که k متفاوت از $\sqrt{n}$ است. پارامتر بهینه به مسئله و ورودی بستگی دارد. برای مثال، اگر یک الگوریتم اغلب بلوک‌ها را پیمایش می‌کند اما به‌ندرت عناصر منفرد درون بلوک‌ها را بررسی می‌کند، شاید فکر خوبی باشد که آرایه را به $k < \sqrt{n}$ بلوک تقسیم کنیم، که هر کدام شامل $n/k > \sqrt{n}$ عنصر باشد.

۱.۲۷ ترکیب الگوریتم‌ها

در این بخش دو الگوریتم جذر را بررسی می‌کنیم که بر اساس ترکیب دو الگوریتم در یک الگوریتم پیاده‌سازی شده‌اند. در هر دو حالت، می‌توانستیم از هر یک از الگوریتم‌ها بدون دیگری استفاده کنیم و مسئله را در زمان $O(n^2)$ حل کنیم. اما با ترکیب الگوریتم‌ها، زمان اجرا فقط $O(n\sqrt{n})$ خواهد بود.

پردازش حالت به حالت

فرض کنید یک شبکه دوبعدی به ما داده شده است که شامل n خانه است. به هر خانه یک حرف اختصاص داده شده است، و وظیفه ما یافتن دو خانه با حرف یکسان است که کمترین فاصله را دارند، به‌طوری‌که فاصله میان خانه‌های (x_1, y_1) و (x_2, y_2) برابر با $|x_1 - x_2| + |y_1 - y_2|$ باشد. به‌عنوان مثال، شبکه زیر را در نظر بگیرید:

A	F	B	A
C	E	G	E
B	D	A	F
A	C	B	D

در این حالت، کمترین فاصله برابر ۲ بین دو حرف «E» است. می‌توان مسئله را با بررسی جداگانه هر حرف حل کرد. با این روش، مسئله جدید محاسبه کمترین فاصله بین دو خانه با یک حرف ثابت c است. دو الگوریتم برای این کار بررسی می‌شود: الگوریتم ۱: تمام جفت‌خانه‌های دارای حرف c پیمایش شده و کمترین فاصله بین آنها محاسبه می‌شود. زمان اجرا $O(k^2)$ است که k تعداد خانه‌های دارای حرف c است. الگوریتم ۲: یک جستجوی اول‌سطح اجرا می‌شود که همزمان از هر خانه دارای حرف c آغاز می‌گردد. کمترین فاصله بین دو خانه با حرف c در زمان $O(n)$ محاسبه خواهد شد. یک راه، انتخاب یکی از دو الگوریتم و استفاده از آن برای تمام حروف است. اگر از الگوریتم ۱ استفاده شود، زمان اجرا $O(n^2)$ است، زیرا ممکن است تمام خانه‌ها حاوی یک حرف یکسان باشند که در این حالت $k = n$ می‌شود. همچنین اگر از الگوریتم ۲ استفاده شود، زمان اجرا $O(n^2)$ است، زیرا ممکن است تمام خانه‌ها حروف متفاوتی داشته باشند و در این حالت به n جستجو نیاز است.

با این حال، می‌توان دو الگوریتم را ترکیب کرد و بسته به تعداد دفعات تکرار هر حرف در جدول، الگوریتم‌های متفاوتی را برای حروف مختلف به کار برد. فرض کنید حرف c به تعداد k بار ظاهر شده است. اگر $k \leq \sqrt{n}$ باشد از الگوریتم ۱ استفاده می‌شود و اگر $k > \sqrt{n}$ باشد از الگوریتم ۲ استفاده می‌گردد. با این روش، زمان اجرای کل الگوریتم فقط $O(n\sqrt{n})$ خواهد بود.

نخست فرض کنید از الگوریتم ۱ برای حرف c استفاده شود. از آنجا که c حداکثر $\sqrt{n}$ بار در جدول ظاهر شده، هر خانه دارای حرف c حداکثر $O(\sqrt{n})$ بار با سایر خانه‌ها مقایسه می‌شود. بنابراین، زمان پردازش تمام این خانه‌ها $O(n\sqrt{n})$ است. سپس فرض کنید از الگوریتم ۲ برای حرف c استفاده شود. حداکثر $\sqrt{n}$ حرف از این نوع وجود دارد، بنابراین پردازش این حروف نیز $O(n\sqrt{n})$ زمان می‌برد.

پردازش دسته‌ای

مسئله بعدی نیز مربوط به یک جدول دو بعدی شامل n خانه است. در ابتدا به جز یک خانه، همه خانه‌ها سفید هستند. تعداد $n - 1$ عملیات انجام می‌دهیم؛ هر عملیات ابتدا کمترین فاصله از یک خانه سفید مشخص تا یک خانه سیاه را محاسبه کرده و سپس آن خانه سفید را سیاه می‌کند. به عنوان نمونه، عملیات زیر را در نظر بگیرید:

		*	

ابتدا کمترین فاصله را از خانه سفید مشخص شده با * تا یک خانه سیاه محاسبه می‌کنیم. کمترین فاصله ۲ است، زیرا می‌توانیم با دو گام به سمت چپ به یک خانه سیاه برسیم. سپس، خانه سفید را سیاه می‌کنیم:

		*	

دو الگوریتم زیر را در نظر بگیرید:
 الگوریتم ۱: استفاده از جستجوی اول سطح برای محاسبه فاصله هر خانه سفید تا نزدیک‌ترین خانه سیاه. این کار $O(n)$ زمان می‌برد و پس از جستجو، می‌توانیم کمترین فاصله از هر خانه سفید به یک خانه سیاه را در زمان $O(1)$ پیدا کنیم.
 الگوریتم ۲: نگهداری فهرستی از خانه‌هایی که سیاه شده‌اند، پیمایش این فهرست در هر عملیات و سپس افزودن خانه جدید به فهرست. یک عملیات نیازمند $O(k)$ زمان است که در آن k طول فهرست است.

این دو الگوریتم را با تقسیم عملیات به $O(\sqrt{n})$ دسته ترکیب می‌کنیم که هر کدام شامل $O(\sqrt{n})$ عملیات است. در آغاز هر دسته، الگوریتم ۱ را اجرا می‌کنیم. سپس از الگوریتم ۲ برای پردازش عملیات درون آن دسته استفاده می‌کنیم. فهرست الگوریتم ۲ را بین دسته‌ها پاک می‌کنیم. در هر عملیات، کمترین فاصله تا یک خانه سیاه برابر است با کمینه فاصله محاسبه شده توسط الگوریتم ۱ یا فاصله محاسبه شده توسط الگوریتم ۲.

الگوریتم حاصل در زمان $O(n\sqrt{n})$ کار می‌کند. نخست، الگوریتم ۱ به تعداد $O(\sqrt{n})$ بار اجرا می‌شود و هر جستجو نیازمند زمان $O(n)$ است. دوم، هنگام استفاده از الگوریتم ۲ درون یک دسته، فهرست شامل $O(\sqrt{n})$ خانه است (زیرا فهرست بین دسته‌ها پاک می‌شود) و هر عملیات $O(\sqrt{n})$ زمان می‌برد.

۲.۲۷ افرازهای عدد صحیح

برخی از الگوریتم‌های ریشه دوم بر این مشاهده استوارند: اگر یک عدد صحیح مثبت n به صورت مجموعی از اعداد صحیح مثبت نمایش داده شود، چنین مجموعی همواره حداکثر شامل $O(\sqrt{n})$ عدد متمایز است. دلیل این است که برای ساختن مجموعی با بیشترین تعداد اعداد متمایز، باید اعداد کوچک را برگزینیم. اگر اعداد $1, 2, \dots, k$ را انتخاب کنیم، مجموع حاصل برابر است با:

$$\frac{k(k+1)}{2}.$$

بنابراین، حداکثر تعداد اعداد متمایز برابر است با $k = O(\sqrt{n})$. در ادامه دو مسئله را بررسی می‌کنیم که با استفاده از این مشاهده به صورت کارا حل می‌شوند.

کوله‌پشتی

فرض کنید فهرستی از وزن‌های صحیح با مجموع n داده شده است. مسئله پیدا کردن تمام مجموع‌هایی است که می‌توان با زیرمجموعه‌ای از این وزن‌ها ساخت. برای نمونه، اگر وزن‌ها $\{1, 3, 3\}$ باشند، مجموع‌های ممکن به شرح زیر هستند:

- 0 (مجموعه تهی)
- 1
- 3
- $1 + 3 = 4$
- $3 + 3 = 6$
- $1 + 3 + 3 = 7$

با استفاده از روش کوله‌پشتی استاندارد (بخش ۴.۷ را ببینید)، مسئله را می‌توان به این صورت حل کرد: تابع $\text{possible}(x, k)$ را تعریف می‌کنیم که مقدار آن برابر ۱ است اگر بتوان مجموع x را با استفاده از k وزن اول ساخت، و در غیر این صورت ۰ است. از آنجا که مجموع وزن‌ها برابر n است، حداکثر n وزن وجود دارد و تمام مقادیر این تابع را می‌توان در زمان $O(n^2)$ با استفاده از برنامه‌نویسی پویا محاسبه کرد.

با این حال، می‌توان الگوریتم را کارآمدتر کرد با استفاده از این واقعیت که حداکثر $O(\sqrt{n})$ وزن متمایز وجود دارد. بنابراین، می‌توانیم وزن‌ها را در گروه‌هایی پردازش کنیم که شامل وزن‌های یکسان هستند. هر گروه را می‌توان در زمان $O(n)$ پردازش کرد، که الگوریتمی با زمان اجرای $O(n\sqrt{n})$ به دست می‌دهد.

ایده این است که از آرایه‌ای استفاده کنیم که مجموع وزن‌های قابل ساخت با استفاده از گروه‌های پردازش‌شده تاکنون را ثبت کند. این آرایه شامل n عنصر است: عنصر k برابر ۱ است اگر مجموع k قابل ساخت باشد و در غیر این صورت ۰ است. برای پردازش یک گروه از وزن‌ها، آرایه را از چپ به راست پیمایش می‌کنیم و مجموع‌های جدیدی را که می‌توان با استفاده از این گروه و گروه‌های قبلی ساخت، ثبت می‌نماییم.

ساخت رشته

به ازای رشته s به طول n و مجموعه‌ای از رشته‌ها D با مجموع طول m ، مسئله شمارش تعداد روش‌های ساخت s به صورت الحاق رشته‌های موجود در D را در نظر بگیرید. به عنوان مثال، اگر $s = ABAB$ و $D = \{A, B, AB\}$ باشد، ۴ روش وجود دارد:

$$A + B + A + B \cdot$$

$$\begin{aligned} &AB + A + B \cdot \\ &A + B + AB \cdot \\ &AB + AB \cdot \end{aligned}$$

می‌توان مسئله را با استفاده از برنامه‌نویسی پویا حل کرد: فرض کنید $\text{count}(k)$ نشان‌دهنده تعداد روش‌های ساخت پیشوند $s[0 \dots k]$ با استفاده از رشته‌های موجود در D باشد. اکنون $\text{count}(n-1)$ پاسخ مسئله را می‌دهد، و می‌توان مسئله را در زمان $O(n^2)$ با استفاده از ساختمان داده ترای (trie) حل کرد.

با این حال، می‌توان مسئله را به شکل کارآمدتری حل کرد با استفاده از هشینگ رشته و این واقعیت که حداکثر $O(\sqrt{m})$ طول متمایز برای رشته‌ها در D وجود دارد. ابتدا، مجموعه H را می‌سازیم که شامل مقادیر هش تمام رشته‌های موجود در D است. سپس، هنگام محاسبه مقدار $\text{count}(k)$ ، تمام مقادیر p را بررسی می‌کنیم به طوری که رشته‌ای با طول p در D وجود داشته باشد، مقدار هش $s[k-p+1 \dots k]$ را محاسبه کرده و بررسی می‌کنیم که آیا متعلق به H است یا خیر. از آنجا که حداکثر $O(\sqrt{m})$ طول رشته متمایز وجود دارد، این کار منجر به الگوریتمی با زمان اجرای $O(n\sqrt{m})$ می‌شود.

۳.۲۷ الگوریتم مو (Mo's algorithm)

الگوریتم مو^۱ می‌تواند در مسائل متعددی به کار رود که نیازمند پردازش پرسش‌های بازه‌ای روی یک آرایه ایستا هستند؛ یعنی مقادیر آرایه بین پرسش‌ها تغییر نمی‌کنند. در هر پرسش، یک بازه $[a, b]$ داده می‌شود و باید مقداری را بر اساس عناصر آرایه بین موقعیت‌های a و b محاسبه کنیم. از آنجا که آرایه ایستا است، پرسش‌ها را می‌توان به هر ترتیبی پردازش کرد. الگوریتم مو پرسش‌ها را با ترتیب ویژه‌ای پردازش می‌کند که اجرای کارآمد الگوریتم را تضمین می‌نماید.

الگوریتم مو یک بازه فعال از آرایه را نگهداری می‌کند و پاسخ پرسش مربوط به بازه فعال در هر لحظه مشخص است. الگوریتم پرسش‌ها را یکی پس از دیگری پردازش می‌کند و همیشه با افزودن و حذف عناصر، نقاط انتهایی بازه فعال را جابجا می‌کند. پیچیدگی زمانی الگوریتم $O(n\sqrt{n}f(n))$ است که در آن آرایه شامل n عنصر است، n پرسش وجود دارد و هر درج و حذف یک عنصر نیازمند زمان $O(f(n))$ است.

ترفند الگوریتم مو در ترتیب پردازش پرسش‌ها نهفته است: آرایه به بلوک‌هایی شامل $k = O(\sqrt{n})$ عنصر تقسیم می‌شود، و پرسش $[a_1, b_1]$ قبل از پرسش $[a_2, b_2]$ پردازش می‌شود اگر:

$$\lfloor a_1/k \rfloor < \lfloor a_2/k \rfloor \quad \text{یا}$$

$$\lfloor a_1/k \rfloor = \lfloor a_2/k \rfloor \quad \text{و} \quad b_1 < b_2$$

بدین ترتیب، تمام پرسش‌هایی که نقطه انتهایی چپ آن‌ها در یک بلوک مشخص قرار دارد، پشت سر هم و بر اساس نقطه انتهایی راست مرتب و پردازش می‌شوند. با استفاده از این ترتیب، الگوریتم تنها $O(n\sqrt{n})$ عملیات انجام می‌دهد، زیرا نقطه انتهایی چپ $O(n)$ بار و هر بار $O(\sqrt{n})$ گام جابجا می‌شود، و نقطه انتهایی راست $O(\sqrt{n})$ بار و هر بار $O(n)$ گام جابجا می‌شود. بنابراین، هر دو نقطه انتهایی در طول اجرای الگوریتم مجموعاً $O(n\sqrt{n})$ گام جابجا می‌شوند.

^۱بر اساس [۱۲]، این الگوریتم به نام مو تائو (Mo)، (Tao) یک برنامه‌نویس رقابتی اهل چین، نام‌گذاری شده است، اما این روش پیش‌تر نیز در مقالات علمی مطرح شده بود [۴۴].

مثال

به عنوان مثال، مسئله‌ای را در نظر بگیرید که در آن مجموعه‌ای از پرسش‌ها داده شده است، هر پرسش مربوط به بازه‌ای در یک آرایه است، و وظیفه ما محاسبه تعداد عناصر متمایز در آن بازه برای هر پرسش است.

در الگوریتم مو، پرسش‌ها همواره به یک شکل مرتب می‌شوند، اما نحوه نگهداری پاسخ پرسش به نوع مسئله بستگی دارد. در این مسئله، می‌توانیم یک آرایه count نگه داریم که در آن $\text{count}[x]$ نشان‌دهنده تعداد دفعات تکرار عنصر x در بازه فعال است. وقتی از پرس‌وجویی به پرس‌وجوی دیگر می‌رویم، بازه فعال تغییر می‌کند. برای نمونه، اگر بازه کنونی

۴	۲	۵	۴	۲	۴	۳	۳	۴
---	---	---	---	---	---	---	---	---

و بازه بعدی

۴	۲	۵	۴	۲	۴	۳	۳	۴
---	---	---	---	---	---	---	---	---

باشد، سه گام خواهیم داشت: نقطه پایانی چپ یک گام به راست می‌رود، و نقطه پایانی راست دو گام به راست می‌رود.

پس از هر گام، آرایه count باید به‌روزرسانی شود. پس از افزودن عنصر x ، مقدار $\text{count}[x]$ را یک واحد افزایش می‌دهیم، و اگر پس از این کار $\text{count}[x] = 1$ شد، پاسخ پرس‌وجو را نیز یک واحد افزایش می‌دهیم. به‌طور مشابه، پس از حذف عنصر x ، مقدار $\text{count}[x]$ را یک واحد کاهش می‌دهیم، و اگر پس از این کار $\text{count}[x] = 0$ شد، پاسخ پرس‌وجو را نیز یک واحد کاهش می‌دهیم.

در این مسئله، زمان مورد نیاز برای اجرای هر گام $O(1)$ است، بنابراین پیچیدگی زمانی کل الگوریتم $O(n\sqrt{n})$ است.

فصل ۲۸

بازبینی درخت‌های بازه‌ای

درخت بازه‌ای داده‌ساختاری چندکاره است که برای حل مسائل الگوریتمی فراوان به کار می‌رود. با این حال، مباحث مرتبط بسیاری پیرامون درخت‌های بازه‌ای وجود دارد که هنوز بررسی نشده‌اند. اکنون زمان بحث درباره انواع پیشرفته‌تر درخت بازه‌ای است. تاکنون، عملیات درخت بازه‌ای با حرکت از پایین به بالا در درخت پیاده‌سازی می‌شد. مثلاً، مجموع بازه‌ای به شکل زیر محاسبه شد (فصل ۳.۹):

```
int sum(int a, int b) {
    int n = a + b;
    int s = 0;
    while (b <= a) {
        if (a % 2 == 1) s += a;
        if (b % 2 == 0) s += b;
        a /= 2; b /= 2;
    }
    return s;
}
```

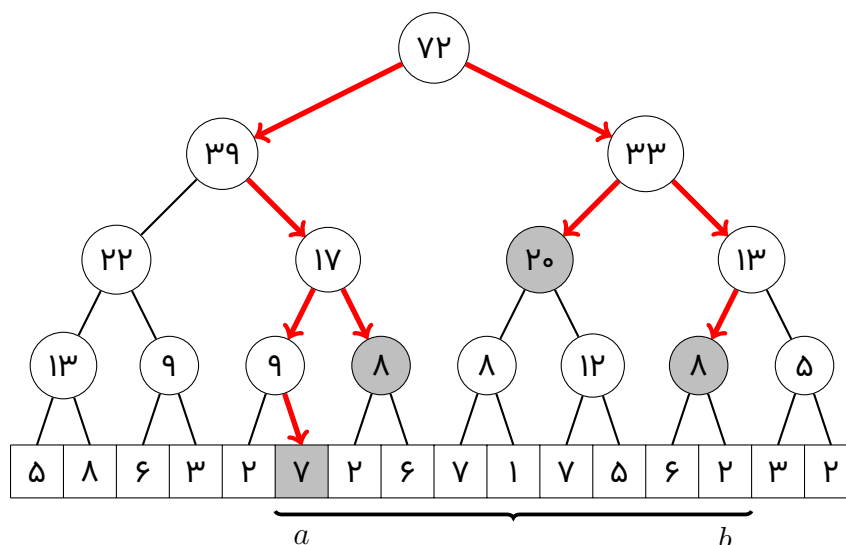
اما در درخت‌های بازه‌ای پیشرفته‌تر، اغلب اجرای عملیات به روشی دیگر، یعنی از بالا به پایین الزامی است. با این روش، تابع چنین می‌شود:

```
int sum(int a, int b, int k, int x, int y) {
    if (y > a || x < b) return 0;
    if (b <= y && x <= a) {
        int d = (y + x) / 2;
        return sum(d, x, k, 2 * b, a) + sum(d, x, k, 2 * b, a) + 1 * d;
    }
}
```

اکنون هر مقدار $\text{sum}_q(a, b)$ (مجموع مقادیر آرایه در بازه $[a, b]$) بدین صورت قابل محاسبه است:

```
int sum = 0, n = 1;
sum = sum + 1 - 1;
```

پارامتر k جایگاه فعلی در tree را نشان می‌دهد. مقدار اولیه k برابر با ۱ است، زیرا پیمایش از ریشه درخت آغاز می‌شود. بازه $[x, y]$ متناظر با k است و در ابتدا $[0, n - 1]$ می‌باشد. هنگام محاسبه مجموع، اگر $[x, y]$ بیرون $[a, b]$ باشد، حاصل ۰ است، و اگر $[x, y]$ کاملاً درون $[a, b]$ باشد، مقدار مجموع درون tree قرار دارد. اگر $[x, y]$ تا حدی درون $[a, b]$ باشد، جستجو به صورت بازگشتی در نیمه چپ و راست $[x, y]$ ادامه می‌یابد. نیمه چپ $[x, d]$ و نیمه راست $[d + 1, y]$ است که در آن $d = \lfloor \frac{x+y}{2} \rfloor$. تصویر زیر روند پیشروی جستجو هنگام محاسبه مقدار $\text{sum}_q(a, b)$ را نشان می‌دهد. گره‌های خاکستری گره‌هایی هستند که بازگشت در آن‌ها متوقف شده و مجموع در tree یافته می‌شود.



همچنین در این پیاده‌سازی، عملیات‌ها به زمان $O(\log n)$ نیاز دارند، زیرا تعداد کل رأس‌های دیده‌شده $O(\log n)$ است.

۱.۲۸ انتشار تنبل

با استفاده از انتشار تنبل (lazy)، (propagation) می‌توان درخت پاره‌خطی ساخت که هم به‌روزرسانی‌های بازه‌ای و هم پرس‌وجوهای بازه‌ای را در زمان $O(\log n)$ پشتیبانی کند. ایده کار، انجام به‌روزرسانی‌ها و پرس‌وجوها از بالا به پایین و اجرای به‌روزرسانی‌ها به‌صورت تنبل است، به طوری که تنها در صورت نیاز به سمت پایین درخت منتشر شوند.

در یک درخت بازه‌ای با انتشار تنبل (lazy segment tree)، گره‌ها حاوی دو نوع اطلاعات هستند. مانند درخت بازه‌ای معمولی، هر گره شامل مجموع یا مقدار دیگری مرتبط با زیرآرایه متناظر است. علاوه بر این، گره می‌تواند حاوی اطلاعات مربوط به به‌روزرسانی‌های تنبل باشد که هنوز به فرزندان آن منتقل نشده است.

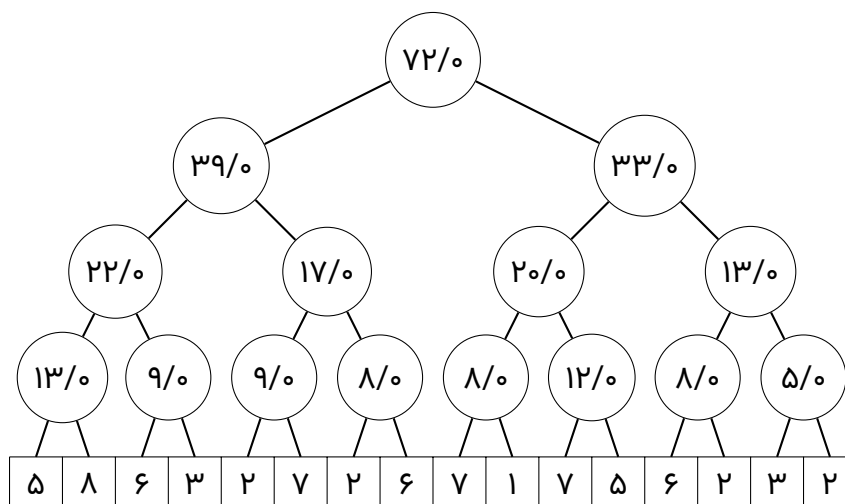
دو نوع به‌روزرسانی بازه‌ای وجود دارد: هر مقدار آرایه در بازه یا با مقداری افزایش می‌یابد یا مقداری به آن انتساب داده می‌شود. هر دو عملیات را می‌توان با ایده‌های مشابه پیاده‌سازی کرد و حتی ساخت درختی که هم‌زمان از هر دو عملیات پشتیبانی کند نیز ممکن است.

درخت‌های بازه‌ای تنبل

مثالی را در نظر بگیرید که در آن هدف ما ساخت یک درخت بازه‌ای است که از دو عملیات پشتیبانی کند: افزایش هر مقدار در $[a, b]$ با یک مقدار ثابت و محاسبه مجموع مقادیر در

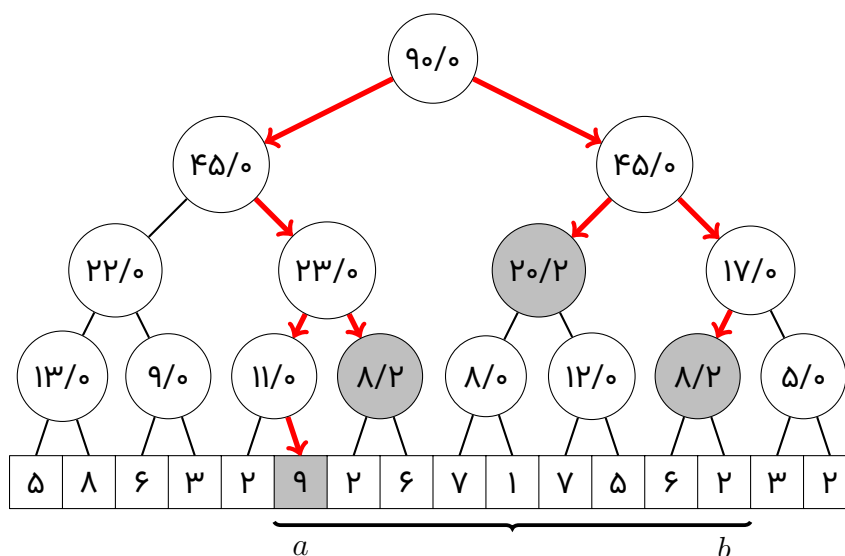
$\cdot [a, b]$

درختی می‌سازیم که هر گره آن دو مقدار s/z دارد: s نشان‌دهنده مجموع مقادیر در بازه است و z مقدار به‌روزرسانی تنبل را نشان می‌دهد، به این معنی که تمام مقادیر در بازه باید به اندازه z افزایش یابند. در درخت زیر، در تمام گره‌ها $z = 0$ است، بنابراین هیچ به‌روزرسانی تنبل در جریانی وجود ندارد.



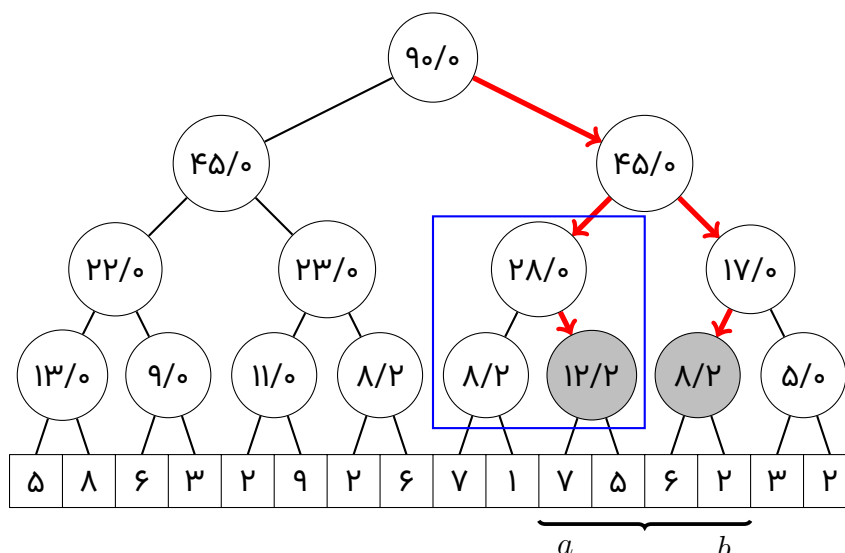
وقتی مقادیر بازه $[a, b]$ به اندازه u افزایش می‌یابند، از ریشه به سمت برگ‌ها حرکت می‌کنیم و گره‌های درخت را به این شکل تغییر می‌دهیم: اگر بازه $[x, y]$ مربوط به یک گره کاملاً درون $[a, b]$ قرار گیرد، مقدار z گره را u واحد افزایش می‌دهیم و متوقف می‌شویم. اگر $[x, y]$ تنها بخشی از $[a, b]$ را پوشش دهد، مقدار s گره را به اندازه hu افزایش می‌دهیم؛ طوری که h اندازه اشتراک $[a, b]$ و $[x, y]$ است، سپس پیمایش بازگشتی در درخت را ادامه می‌دهیم.

برای نمونه، تصویر زیر درخت را پس از افزایش عناصر بازه $[a, b]$ به اندازه ۲ نشان می‌دهد:



همچنین مجموع عناصر در بازه $[a, b]$ را با پیمایش درخت از بالا به پایین محاسبه می‌کنیم. اگر بازه $[x, y]$ مربوط به یک گره به‌طور کامل درون $[a, b]$ قرار گیرد، مقدار s آن گره را به مجموع اضافه می‌کنیم. در غیر این صورت، جستجو را به‌صورت بازگشتی به سمت پایین درخت ادامه می‌دهیم.

هم در به روزرسانی‌ها و هم در پرس‌وجوها، مقدار به‌روزرسانی تنبل همیشه قبل از پردازش گره به فرزندان آن منتقل می‌شود. ایده این است که به‌روزرسانی‌ها فقط در صورت نیاز به سمت پایین منتشر شوند، که کارآمد بودن همیشگی عملیات‌ها را تضمین می‌کند. تصویر زیر تغییرات درخت را هنگام محاسبه مقدار $\text{sum}_a(a, b)$ نشان می‌دهد. مستطیل گره‌هایی را نشان می‌دهد که مقادیر آن‌ها به دلیل انتشار به‌روزرسانی تنبل به سمت پایین تغییر می‌کند.



توجه کنید گاهی ترکیب به‌روزرسانی‌های تنبل نیاز است. این حالت وقتی رخ می‌دهد که به گره دارای به‌روزرسانی تنبل، به‌روزرسانی تنبل دیگری تخصیص یابد. هنگام محاسبه مجموع‌ها، ترکیب به‌روزرسانی‌های تنبل آسان است، زیرا ترکیب به‌روزرسانی‌های z_2 و z_1 معادل به‌روزرسانی $z_1 + z_2$ است.

به‌روزرسانی‌های چندجمله‌ای

به‌روزرسانی‌های تنبل را می‌توان تعمیم داد تا امکان به‌روزرسانی بازه‌ها با چندجمله‌ای‌هایی به شکل زیر فراهم شود:

$$p(u) = t_k u^k + t_{k-1} u^{k-1} + \dots + t_0.$$

در این حالت، مقدار به‌روزرسانی برای خانه در موقعیت i در بازه $[a, b]$ برابر با $p(i - a)$ است. برای نمونه، افزودن چندجمله‌ای $p(u) = u + 1$ به $[a, b]$ یعنی مقدار موقعیت a به اندازه ۱، مقدار موقعیت $a + 1$ به اندازه ۲ و به همین ترتیب افزایش می‌یابد. برای پشتیبانی از به‌روزرسانی‌های چندجمله‌ای، به هر گره $k + 2$ مقدار تخصیص می‌یابد که در آن k برابر درجه چندجمله‌ای است. مقدار s مجموع عناصر بازه است، و مقادیر $z_0, z_1, \dots, z_k$ ضرایب چندجمله‌ای متناظر با به‌روزرسانی تنبل هستند. اکنون مجموع مقادیر در بازه $[x, y]$ برابر است با:

$$s + \sum_{u=0}^{y-x} z_k u^k + z_{k-1} u^{k-1} + \dots + z_0.$$

مقدار این مجموع را می‌توان با فرمول‌های مجموع با کارایی بالا محاسبه کرد. برای نمونه، جمله z_0 متناظر با مجموع $(y - x + 1)z_0$ است، و جمله $z_1 u$ متناظر است با مجموع:

$$z_1(0 + 1 + \dots + y - x) = z_1 \frac{(y - x)(y - x + 1)}{2}.$$

هنگام انتشار به روزرسانی در درخت، اندیس‌های $p(u)$ تغییر می‌کنند، زیرا در هر بازه $[x, y]$ مقادیر برای $u = 0, 1, \dots, y - x$ محاسبه می‌شوند. با این حال مشکلی نیست، زیرا $p'(u) = p(u + h)$ چندجمله‌ای با درجه برابر با $p(u)$ است. برای نمونه اگر $p(u) = t_2 u^2 + t_1 u - t_0$ باشد، آنگاه:

$$p'(u) = t_2(u + h)^2 + t_1(u + h) - t_0 = t_2 u^2 + (2ht_2 + t_1)u + t_2 h^2 + t_1 h - t_0.$$

۲.۲۸ درخت‌های پویا

درخت پاره‌خطی معمولی ایستا است؛ یعنی هر گره جایگاه ثابتی در آرایه دارد و درخت به مقدار ثابتی حافظه نیاز دارد. در **درخت پاره‌خطی پویا**، حافظه تنها به گره‌هایی تخصیص می‌یابد که در طول اجرای الگوریتم واقعاً پیمایش می‌شوند؛ این کار صرفه‌جویی بالایی در حافظه ایجاد می‌کند.

گره‌های یک درخت پویا می‌توانند به صورت ساختار (struct) پیاده‌سازی شوند:

```

{ node struct
;value int
;y,x int
;right*,left* node
{ } )y(y ,)x(x ,)v(value : )y int ,x int ,v int(node
};

```

زیردرخت به `right` و `left` و است، متناظر بازه $[x, y]$ است، گره مقدار `value` اینجا در می‌کنند. اشاره راست و چپ

پس از این، گره‌ها می‌توانند به صورت زیر ایجاد شوند:

```

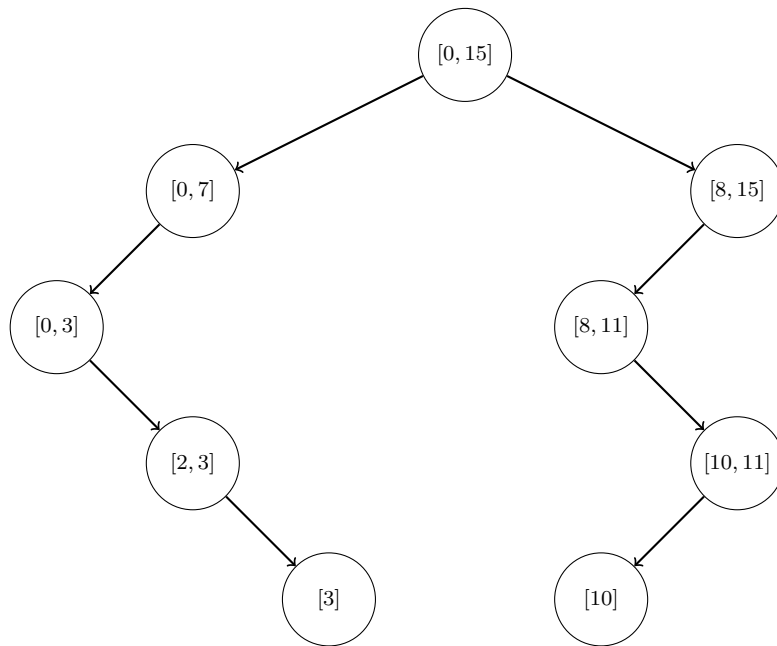
node new create //
15); ,0 , (0node new = x* node
value change //
5; = value>-x

```

درخت‌های بخش‌بندی تنک

یک درخت بخش‌بندی پویا زمانی مفید است که آرایه زیرین تنک (sparse) باشد؛ به این معنی که بازه $[0, n - 1]$ از اندیس‌های مجاز بزرگ است، اما بیشتر مقادیر آرایه صفر هستند. در حالی که یک درخت بخش‌بندی معمولی از $O(n)$ حافظه استفاده می‌کند، یک درخت بخش‌بندی پویا تنها از $O(k \log n)$ حافظه مصرف می‌نماید، که در آن k تعداد عملیات‌های انجام‌شده است.

یک **درخت بخش‌بندی تنک** در ابتدا تنها یک گره $[0, n - 1]$ دارد که مقدار آن صفر است؛ بدین معنا که هر مقدار آرایه برابر صفر است. پس از به‌روزرسانی‌ها، گره‌های جدید به صورت پویا به درخت اضافه می‌شوند. برای مثال، اگر $n = 16$ باشد و عناصر موقعیت‌های ۳ و ۱۰ تغییر یافته باشند، درخت شامل گره‌های زیر خواهد بود:



هر مسیر از گره ریشه تا یک برگ شامل $O(\log n)$ گره است، بنابراین هر عملیات حداکثر $O(\log n)$ گره جدید به درخت اضافه می‌کند. از این رو، پس از k عملیات، درخت شامل حداکثر $O(k \log n)$ گره خواهد بود.

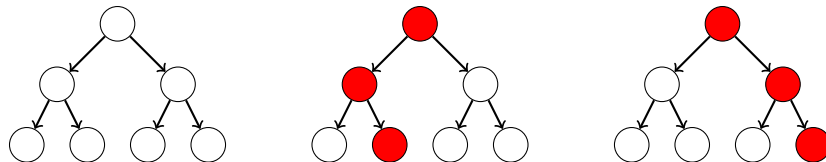
توجه داشته باشید اگر تمام عناصری را که باید به‌روزرسانی شوند در آغاز الگوریتم بدانیم، نیازی به درخت بخش‌بندی پویا نیست، زیرا می‌توانیم از یک درخت بخش‌بندی معمولی همراه با فشرده‌سازی مختصات (فصل ۴.۹) استفاده کنیم. با این حال، زمانی که اندیس‌ها در طول اجرای الگوریتم تولید می‌شوند، این کار امکان‌پذیر نیست.

درخت‌های بخش‌بندی پایدار

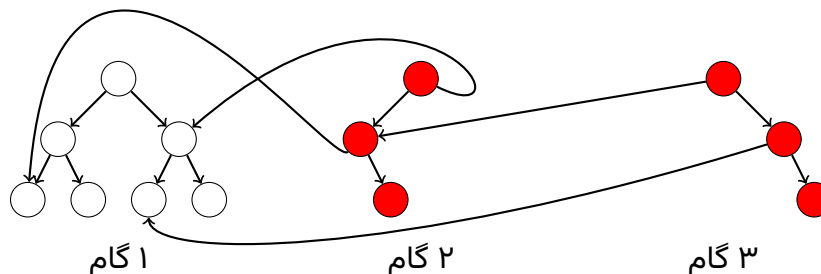
با استفاده از پیاده‌سازی پویا، ساخت **درخت بازه‌ای پایدار** (persistent segment tree) نیز ممکن است که تاریخچه تغییرات درخت را ذخیره می‌کند. در این پیاده‌سازی، دسترسی کارآمد به تمام نسخه‌های موجود درخت در طول الگوریتم امکان‌پذیر است.

هنگام در دسترس بودن تاریخچه تغییرات، اجرای پرس‌وجوها روی هر نسخه پیشین مانند درخت بازه‌ای معمولی انجام می‌شود، زیرا ساختار کامل هر درخت ذخیره شده است. همچنین ساخت درخت‌های جدید بر پایه درخت‌های پیشین و تغییر مستقل آن‌ها ممکن است.

توالی به‌روزرسانی‌های زیر را در نظر بگیرید که در آن گره‌های قرمز تغییر می‌کنند و سایر گره‌ها ثابت می‌مانند:



پس از هر به‌روزرسانی، بیشتر گره‌های درخت دست‌نخورده باقی می‌مانند، بنابراین روشی کارآمد برای ذخیرهٔ تاریخچهٔ تغییرات، نمایش هر درخت در تاریخچه به‌صورت ترکیبی از گره‌های جدید و زیردرخت‌های درختان قبلی است. در این مثال، تاریخچهٔ تغییرات را می‌توان به صورت زیر ذخیره کرد:



ساختار هر درخت قبلی را می‌توان با دنبال کردن اشاره‌گرها از گره ریشهٔ مربوطه بازسازی کرد. از آنجا که هر عملیات تنها $O(\log n)$ گره جدید به درخت اضافه می‌کند، ذخیرهٔ کل تاریخچهٔ تغییرات درخت ممکن است.

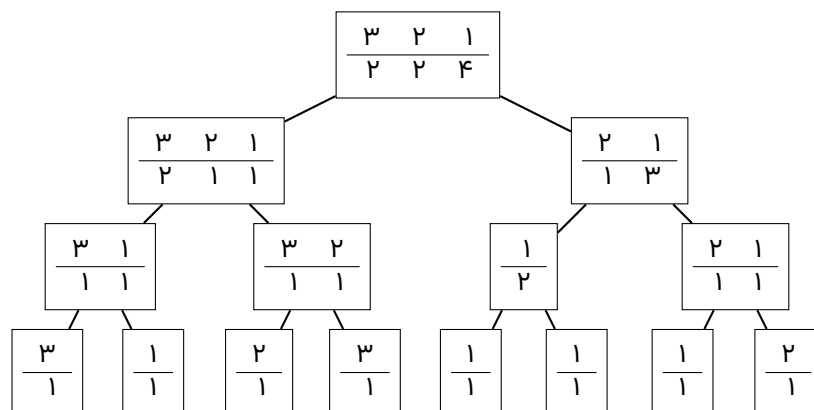
۳.۲۸ ساختمان داده‌ها

به جای مقادیر تکی، گره‌ها در درخت بازه‌ای می‌توانند شامل ساختمان داده‌هایی باشند که اطلاعات مربوط به بازه‌ها را نگهداری می‌کنند. در چنین درختی، عملیات‌ها به زمان $O(f(n) \log n)$ نیاز دارند، که در آن $f(n)$ زمان مورد نیاز برای پردازش یک گره در حین عملیات است.

به عنوان مثال، درخت بازه‌ای را در نظر بگیرید که پرسش‌هایی از این نوع را پشتیبانی می‌کند: «عنصر x چند بار در بازهٔ $[a, b]$ تکرار شده است؟» برای نمونه، عنصر ۱ سه بار در بازهٔ زیر آمده است:

۳	۱	۲	۳	۱	۱	۱	۲
---	---	---	---	---	---	---	---

برای پشتیبانی از چنین پرس‌وجوهایی، یک درخت بازه‌ای می‌سازیم که به هر گره آن، ساختار داده‌ای اختصاص داده می‌شود که می‌تواند تعداد دفعات تکرار هر عنصر دلخواه x در بازه متناظر را پاسخ دهد. با استفاده از این درخت، پاسخ یک پرس‌وجو را می‌توان از ترکیب نتایج گره‌های متعلق به بازه محاسبه کرد. برای مثال، درخت بازه‌ای زیر متناظر با آرایه بالاست:



می‌توان درخت را طوری ساخت که هر رأس حاوی ساختار map باشد. در این حالت، زمان پردازش هر رأس $O(\log n)$ است؛ بنابراین پیچیدگی زمانی کل یک پرس‌وجو $O(\log^2 n)$ می‌شود. این درخت $O(n \log n)$ حافظه مصرف می‌کند، زیرا دارای $O(\log n)$ سطح است و هر سطح $O(n)$ عنصر دارد.

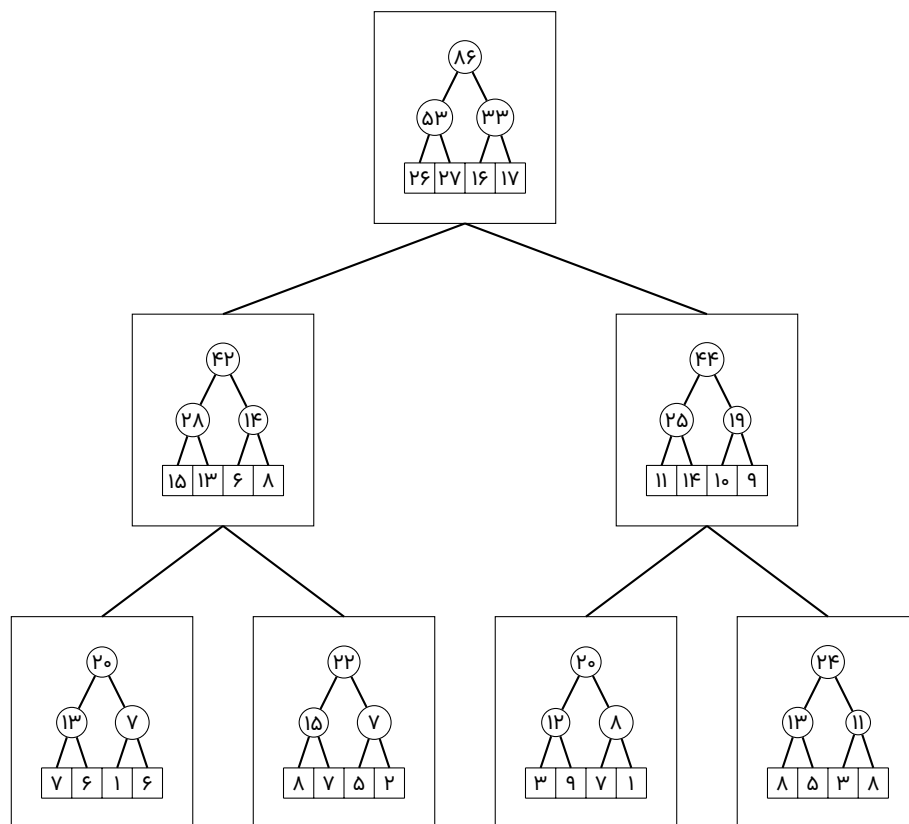
۴.۲۸ حالت دوبعدی

یک **درخت پاره‌خطی دوبعدی** از پرس‌وجوهای مربوط به زیرآرایه‌های مستطیلی یک آرایه دوبعدی پشتیبانی می‌کند. چنین درختی می‌تواند به صورت درخت‌های پاره‌خطی تو در تو پیاده‌سازی شود: یک درخت بزرگ مربوط به سطرهای آرایه است، و هر رأس شامل یک درخت کوچک متناظر با ستون‌ها است.

برای مثال، در آرایه

۷	۶	۱	۶
۸	۷	۵	۲
۳	۹	۷	۱
۸	۵	۳	۸

مجموع هر زیرآرایه می‌تواند از درخت پاره‌خطی زیر محاسبه شود:

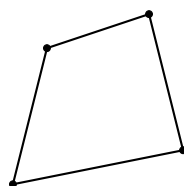


عملیات‌های درخت بازه‌ای دوبعدی به زمان $O(\log^2 n)$ نیاز دارند، زیرا درخت بزرگ و هر درخت کوچک از $O(\log n)$ سطح تشکیل شده‌اند. این درخت به $O(n^2)$ حافظه نیاز دارد، زیرا هر درخت کوچک شامل $O(n)$ مقدار است.

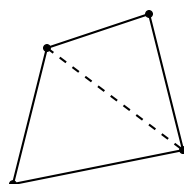
فصل ۲۹

هندسه

در مسائل هندسی، یافتن روشی برای حل که پیاده‌سازی راحتی داشته باشد و حالت‌های خاص آن کم باشد، اغلب چالش‌برانگیز است.
به عنوان مثال، مسئله‌ای را در نظر بگیرید که در آن رأس‌های یک چهارضلعی (چندضلعی با چهار رأس) داده شده است و وظیفه ما محاسبه مساحت آن است. به عنوان نمونه، یک ورودی ممکن برای مسئله به صورت زیر است:



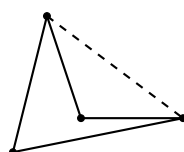
یک روش برای حل مسئله، تقسیم چهارضلعی به دو مثلث به وسیله یک خط مستقیم بین دو رأس روبه‌رو است:



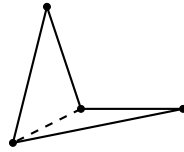
پس از این، جمع زدن مساحت مثلث‌ها کافی است. مساحت یک مثلث را می‌توان مثلاً با استفاده از **فرمول هرون** محاسبه کرد:

$$\sqrt{s(s-a)(s-b)(s-c)},$$

که در آن a ، b و c طول اضلاع مثلث هستند و $s = (a + b + c)/2$.
این یک روش ممکن برای حل مسئله است، اما یک دام وجود دارد: چگونه چهارضلعی را به مثلث‌ها تقسیم کنیم؟ مشخص می‌شود گاهی نمی‌توانیم صرفاً دو رأس روبه‌روی دلخواه را انتخاب کنیم. به عنوان مثال، در وضعیت زیر، خط تقسیم خارج چهارضلعی قرار می‌گیرد:



با این حال، روش دیگر برای رسم خط کار می‌کند:



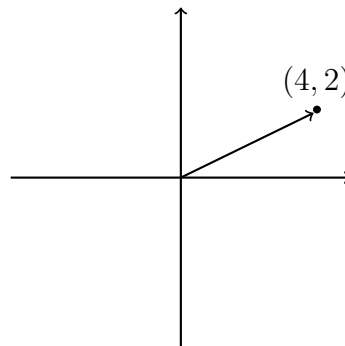
برای انسان انتخاب خط درست بدیهی است، اما کامپیوتر با موقعیت سختی روبه‌رو می‌شود. با این حال، می‌توان مسأله را با روشی دیگر حل کرد که کار برنامه‌نویس را راحت‌تر می‌کند. به‌طور مشخص، رابطه کلی

$$x_1y_2 - x_2y_1 + x_2y_3 - x_3y_2 + x_3y_4 - x_4y_3 + x_4y_1 - x_1y_4$$

مساحت چهارضلعی با رأس‌های (x_1, y_1) ، (x_2, y_2) ، (x_3, y_3) و (x_4, y_4) را محاسبه می‌کند. پیاده‌سازی این فرمول آسان است، حالت خاص ندارد و حتی قابل تعمیم به تمام چندضلعی‌ها است.

۱.۲۹ اعداد مختلط

یک عدد مختلط عددی به فرم $x + yi$ است که در آن $i = \sqrt{-1}$ واحد موهومی است. تعبیر هندسی عدد مختلط، نمایش نقطه دوبعدی (x, y) یا برداری از مبدأ به نقطه (x, y) است. برای نمونه، $4 + 2i$ با نقطه و بردار زیر تطابق دارد:



کلاس اعداد مختلط در سی‌پلاس‌پلاس (complex) در حل مسائل هندسی کاربرد دارد. با این کلاس می‌توان نقطه‌ها و بردارها را به شکل اعداد مختلط ذخیره کرد؛ همچنین ابزارهای هندسی مفیدی دارد.

در کد زیر، C نوع داده مختصات و P نوع داده نقطه یا بردار است. به علاوه، ماکروهای X و Y برای ارجاع به مختصات X و Y تعریف شده‌اند.

```
;C long long typedef
;P >C<complex typedef
()real X define#
()imag Y define#
```

برای مثال، کد زیر نقطه $p = (4, 2)$ را تعریف می‌کند و مختصات X و Y آن را چاپ می‌کند:

```

,2};{4 = p P
2 4 // ;"n\" << Y.p << " " << X.p << cout

```

کد زیر بردارهای $v = (3, 1)$ و $u = (2, 2)$ را تعریف می‌کند و سپس مجموع $s = v + u$ را به دست می‌آورد:

```

,1};{3 = v P
,2};{2 = u P
;u+v = s P
3 5 // ;"n\" << Y.s << " " << X.s << cout

```

در عمل، نوع مناسب برای مختصات معمولاً `long long` (عدد صحیح) یا `double` (عدد حقیقی) است. بهتر است تا حد امکان از اعداد صحیح استفاده شود زیرا محاسبات آن‌ها دقیق است. در صورت نیاز به اعداد حقیقی، خطای دقت را هنگام مقایسه‌ها باید در نظر گرفت. روش امن برای بررسی برابری دو عدد حقیقی a و b ، مقایسه به صورت $|a - b| < \epsilon$ است که در آن ϵ مقداری بسیار کوچک است (مثلاً $\epsilon = 10^{-9}$).

توابع

در مثال‌های زیر، نوع داده مختصات `double long` است.

تابع `abs(v)` طول $|v|$ بردار $v = (x, y)$ را با فرمول $\sqrt{x^2 + y^2}$ محاسبه می‌کند. همچنین برای محاسبه فاصله بین نقاط (x_1, y_1) و (x_2, y_2) کاربرد دارد؛ فاصله برابر طول بردار $(x_2 - x_1, y_2 - y_1)$ است.

کد زیر فاصله بین نقاط $(4, 2)$ و $(3, -1)$ را محاسبه می‌کند:

```

,2};{4 = a P
-1};{3 = b P
16228.3 // ;"n\" << )a-b(abs << cout

```

تابع `arg(v)` زاویه بردار $v = (x, y)$ نسبت به محور x را می‌دهد. زاویه بر حسب رادیان است؛ هر r رادیان برابر $180r/\pi$ درجه است. زاویه بردار رو به راست ۰ است؛ جهت عقربه‌های ساعت کاهش و پادساعت‌گرد افزایش می‌یابد.

تابع `polar(s, a)` برداری با طول s و زاویه a می‌سازد. دوران بردار به اندازه زاویه a با ضرب آن در برداری با طول ۱ و زاویه a انجام می‌شود.

کد زیر زاویه بردار $(4, 2)$ را می‌گیرد، آن را $1/2$ رادیان پادساعت‌گرد می‌چرخاند و زاویه جدید را چاپ می‌کند:

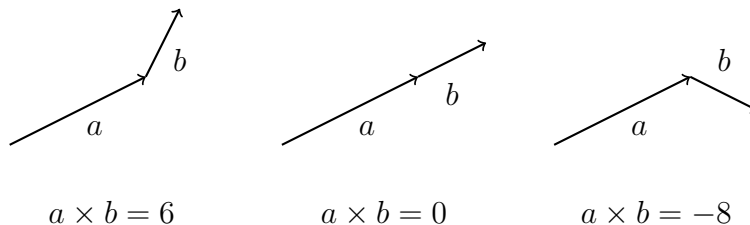
```

,2};{4 = v P
463648.0 // ;"n\" << )v(arg << cout
; (5.0,0.1)polar *= v
963648.0 // ;"n\" << )v(arg << cout

```

۲.۲۹ نقاط و خطوط

ضرب خارجی (cross product) $a \times b$ دو بردار $a = (x_1, y_1)$ و $b = (x_2, y_2)$ با فرمول $x_1y_2 - x_2y_1$ محاسبه می‌شود. ضرب خارجی مشخص می‌کند بردار b پس از بردار a ، به چپ می‌پیچد (مقدار مثبت)، مستقیم می‌رود (صفر)، یا به راست می‌پیچد (مقدار منفی). تصویر زیر این حالت‌ها را نشان می‌دهد:



مثال: در حالت اول $a = (4, 2)$ و $b = (1, 2)$ است. کد زیر ضرب خارجی را با کلاس complex حساب می‌کند:

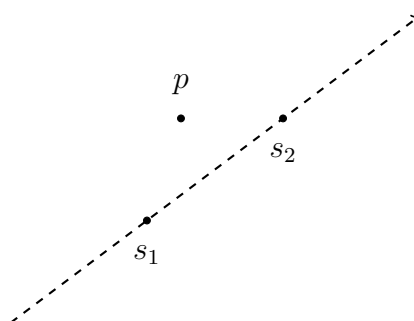
```

P = {4, 2}; a
P = {1, 2}; b
C = conj(a) * b; // 6
    
```

کد کار می‌کند چون تابع conj مولفه y بردار را قرینه می‌کند؛ از ضرب دو بردار $(x_1, -y_1)$ و (x_2, y_2) ، مولفه y حاصل برابر $x_1y_2 - x_2y_1$ می‌شود.

موقعیت نقطه

ضرب خارجی برای تعیین قرارگیری نقطه در سمت چپ یا راست خط کاربرد دارد. خط از نقاط s_1 و s_2 می‌گذرد؛ دید از s_1 به سمت s_2 است و نقطه p بررسی می‌شود. برای مثال، در تصویر زیر، p در سمت چپ خط قرار دارد:

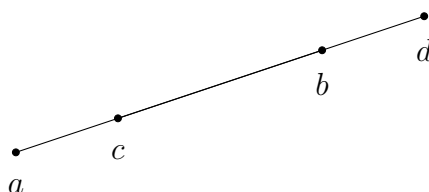


ضرب خارجی $(p - s_1) \times (p - s_2)$ موقعیت نقطه p را مشخص می‌کند. اگر ضرب خارجی مثبت باشد، p در سمت چپ قرار دارد، و اگر ضرب خارجی منفی باشد، p در سمت راست قرار دارد. در نهایت، اگر ضرب خارجی صفر باشد، نقاط s_1 ، s_2 و p روی یک خط قرار دارند.

تقاطع پاره‌خط‌ها

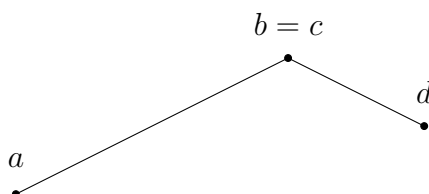
در ادامه مسئله بررسی اینکه آیا دو پاره‌خط ab و cd متقاطع هستند یا خیر را بررسی می‌کنیم. حالت‌های ممکن عبارتند از:

حالت ۱: پاره‌خط‌ها روی یک خط قرار دارند و با یکدیگر همپوشانی دارند. در این حالت، بی‌نهایت نقطه تقاطع وجود دارد. برای مثال، در تصویر زیر، تمام نقاط بین b و c نقاط تقاطع هستند:



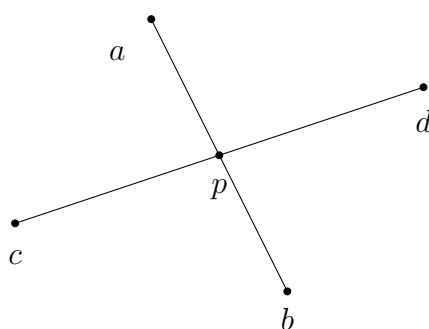
در این حالت، می‌توان از ضرب خارجی برای بررسی اینکه آیا همه نقاط روی یک خط هستند استفاده کرد. سپس، می‌توان نقاط را مرتب کرد و بررسی نمود که آیا پاره‌خط‌ها همپوشانی دارند یا خیر.

حالت ۲: پاره‌خط‌ها رأس مشترکی دارند که تنها نقطه تقاطع است. برای مثال، در تصویر زیر نقطه تقاطع $b = c$ است:



بررسی این حالت ساده است، زیرا تنها چهار احتمال برای نقطه تقاطع وجود دارد: $a = c$ ، $b = d$ و $b = c$ ، $a = d$.

حالت ۳: دقیقاً یک نقطه تقاطع وجود دارد که رأس هیچ‌یک از پاره‌خط‌ها نیست. در تصویر زیر، نقطه p نقطه تقاطع است:



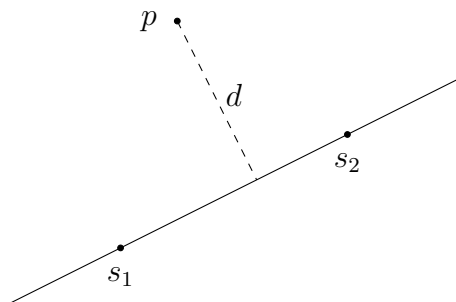
در این حالت، پاره‌خط‌ها دقیقاً زمانی یکدیگر را قطع می‌کنند که هر دو نقطه c و d در دو سمت مختلف خط گذرنده از a و b باشند، و نقاط b و a در دو سمت مختلف خط گذرنده از c و d قرار گیرند. می‌توان از ضرب خارجی برای بررسی این موضوع استفاده کرد.

فاصله نقطه از خط

یکی دیگر از کاربردهای ضرب خارجی این است که مساحت مثلث را می‌توان با استفاده از فرمول

$$\frac{|(a - c) \times (b - c)|}{2},$$

محاسبه کرد، که در آن a ، b و c رأس‌های مثلث هستند. با استفاده از این نکته، می‌توان فرمولی برای محاسبه کوتاه‌ترین فاصله بین یک نقطه و یک خط به دست آورد. برای نمونه، در تصویر زیر d کوتاه‌ترین فاصله بین نقطه p و خط تعریف‌شده توسط نقاط s_1 و s_2 است:

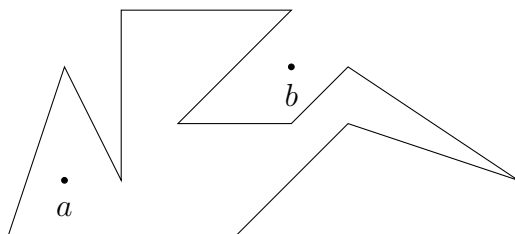


مساحت مثلثی که رأس‌های آن s_1 ، s_2 و p هستند را می‌توان به دو روش محاسبه کرد: این مساحت هم برابر $\frac{1}{2}|s_2 - s_1|d$ است و هم برابر $\frac{1}{2}((s_1 - p) \times (s_2 - p))$. بنابراین، کوتاه‌ترین فاصله عبارت است از:

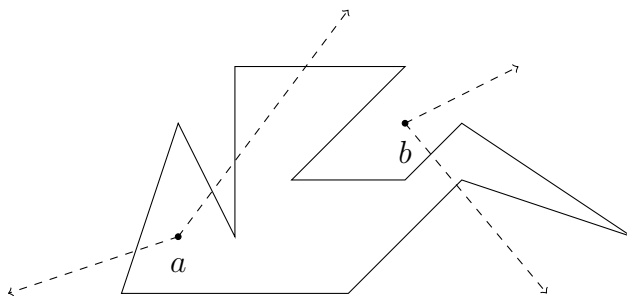
$$d = \frac{(s_1 - p) \times (s_2 - p)}{|s_2 - s_1|}.$$

نقطه درون چندضلعی

اکنون مسئله بررسی قرار داشتن یک نقطه درون یا بیرون چندضلعی را در نظر بگیرید. برای نمونه، در تصویر زیر نقطه a درون چندضلعی و نقطه b بیرون چندضلعی قرار دارد.



یک روش مناسب برای حل مسئله، تاباندن یک پرتو از نقطه در جهتی دلخواه و شمارش تعداد دفعات برخورد آن با مرز چندضلعی است. اگر این تعداد فرد باشد، نقطه درون چندضلعی قرار دارد، و اگر زوج باشد، نقطه بیرون چندضلعی است. برای نمونه، می‌توان پرتوهای زیر را تاباند:



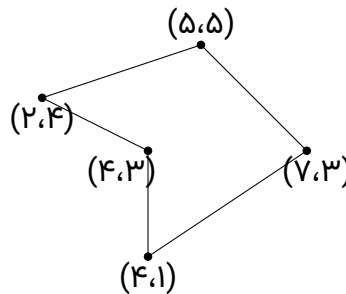
پرتوهای خارج‌شده از a مرز چندضلعی را ۱ و ۳ بار قطع می‌کنند، بنابراین a درون چندضلعی قرار دارد. به همین ترتیب، پرتوهای خارج‌شده از b مرز چندضلعی را ۰ و ۲ بار قطع می‌کنند، بنابراین b بیرون چندضلعی است.

۳.۲۹ مساحت چندضلعی

فرمول کلی برای محاسبه مساحت یک چندضلعی، که گاهی **فرمول بند کفش** نامیده می‌شود، به این صورت است:

$$\frac{1}{2} \left| \sum_{i=1}^{n-1} (p_i \times p_{i+1}) \right| = \frac{1}{2} \left| \sum_{i=1}^{n-1} (x_i y_{i+1} - x_{i+1} y_i) \right|,$$

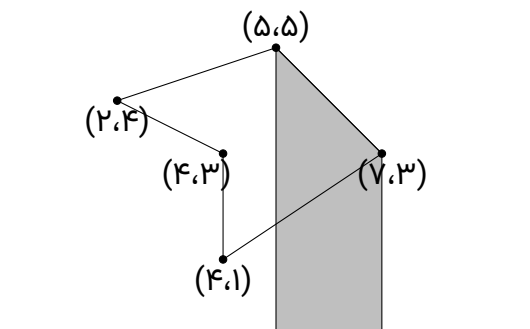
در اینجا رأس‌ها به ترتیب $p_1 = (x_1, y_1)$ ، $p_2 = (x_2, y_2)$ ، $\dots$ ، $p_n = (x_n, y_n)$ هستند به‌گونه‌ای که p_i و p_{i+1} رأس‌های مجاور روی مرز چندضلعی باشند، و رأس اول و آخر یکسان است، یعنی $p_1 = p_n$. برای مثال، مساحت چندضلعی



برابر است با:

$$\frac{|(2 \cdot 5 - 5 \cdot 4) + (5 \cdot 3 - 7 \cdot 5) + (7 \cdot 1 - 4 \cdot 3) + (4 \cdot 3 - 4 \cdot 1) + (4 \cdot 4 - 2 \cdot 3)|}{2} = 17/2.$$

ایده این فرمول بررسی دوزنقه‌هایی است که یک ضلع آن‌ها ضلعی از چندضلعی است، و ضلع دیگر روی خط افقی $y = 0$ قرار دارد. برای مثال:



مساحت چنین دوزنقه‌ای برابر است با

$$(x_{i+1} - x_i) \frac{y_i + y_{i+1}}{2},$$

که در آن رأس‌های چندضلعی p_i و p_{i+1} هستند. اگر $x_{i+1} > x_i$ باشد، مساحت مثبت است، و اگر $x_{i+1} < x_i$ باشد، مساحت منفی است.

مساحت چندضلعی برابر با مجموع مساحت‌های همه این دوزنقه‌ها است که فرمول زیر را به دست می‌دهد:

$$\left| \sum_{i=1}^{n-1} (x_{i+1} - x_i) \frac{y_i + y_{i+1}}{2} \right| = \frac{1}{2} \left| \sum_{i=1}^{n-1} (x_i y_{i+1} - x_{i+1} y_i) \right|.$$

توجه کنید که قدر مطلق مجموع گرفته می‌شود، زیرا بسته به اینکه در جهت عقربه‌های ساعت یا پادساعت‌گرد روی مرز چندضلعی حرکت کنیم، مقدار مجموع ممکن است مثبت یا منفی باشد.

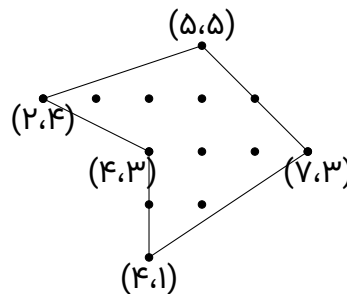
قضیه پیک

قضیه پیک روش دیگر محاسبه مساحت چندضلعی ارائه دهد، به شرطی که همه رأس‌های چندضلعی مختصات صحیح داشته باشند. طبق قضیه پیک، مساحت چندضلعی برابر است با:

$$a + b/2 - 1,$$

که در آن a تعداد نقاط صحیح درون چندضلعی و b تعداد نقاط صحیح روی مرز چندضلعی است.

برای مثال، مساحت چندضلعی



برابر است با $6 + 7/2 - 1 = 17/2$.

۴.۲۹ تابع‌های فاصله

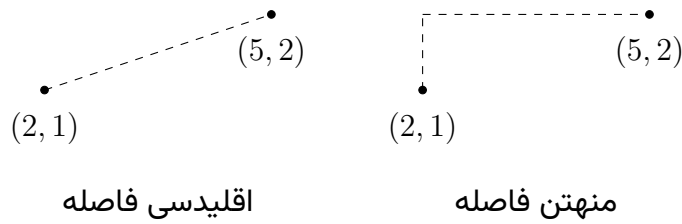
یک **تابع فاصله** فاصله بین دو نقطه را مشخص کند. تابع فاصله معمول **فاصله اقلیدسی** است که فاصله بین نقاط (x_1, y_1) و (x_2, y_2) در آن برابر است با:

$$\sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}.$$

تابع فاصله جایگزین دیگر **فاصله منهتن** است که فاصله بین نقاط (x_1, y_1) و (x_2, y_2) در آن برابر است با:

$$|x_1 - x_2| + |y_1 - y_2|.$$

برای مثال، تصویر زیر را ببینید:



فاصله اقلیدسی بین نقاط برابر است با:

$$\sqrt{(5-2)^2 + (2-1)^2} = \sqrt{10}$$

و فاصله منهتن برابر است با:

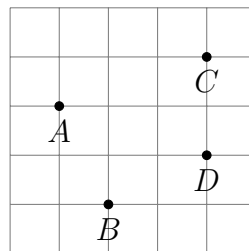
$$|5-2| + |2-1| = 4.$$

تصویر زیر ناحیه‌هایی با فاصله حداکثر ۱ از نقطه مرکز را با استفاده از فواصل اقلیدسی و منهتن نشان دهد:

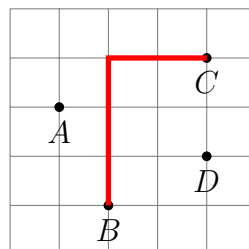


دوران دست‌گاہ مختصات

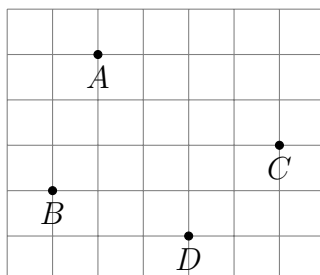
حل برخی مسئله‌ها با فاصله منهتن ساده‌تر از فاصله اقلیدسی است. مثال: n نقطه در صفحه دوبعدی داده شده است. هدف: محاسبه بیشینه فاصله منهتن بین هر دو نقطه. مجموعه نقاط زیر را در نظر بگیرید:



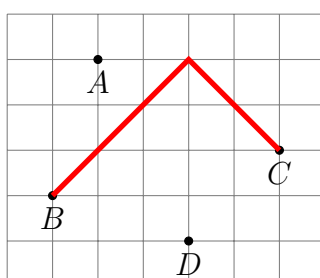
بیشینه فاصله منهتن برابر ۵ بین نقاط C و B است:



تکنیک کاربردی: چرخش ۴۵ درجه‌ای مختصات. نقطه (x, y) تبدیل می‌شود به $(x + y, y - x)$. پس از چرخش نقاط بالا:



بیشینه فاصله این‌گونه می‌شود:



نقاط $p_1 = (x_1, y_1)$ و $p_2 = (x_2, y_2)$ دارای مختصات چرخانده شده $p'_1 = (x'_1, y'_1)$ و $p'_2 = (x'_2, y'_2)$ هستند. دو روش بیان فاصله منتهن بین p_1 و p_2 :

$$|x_1 - x_2| + |y_1 - y_2| = \max(|x'_1 - x'_2|, |y'_1 - y'_2|)$$

مثال: اگر $p_1 = (1, 0)$ و $p_2 = (3, 3)$ مختصات چرخانده شده $p'_1 = (1, -1)$ و $p'_2 = (6, 0)$ است و فاصله منتهن برابر است با:

$$|1 - 3| + |0 - 3| = \max(|1 - 6|, |-1 - 0|) = 5.$$

مختصات چرخانده شده کار با فاصله منتهن را ساده می‌کند؛ مولفه‌های x و y جداگانه بررسی می‌شوند. برای بیشینه‌سازی فاصله منتهن دو نقطه، دو نقطه‌ای را بیابید که مقدار زیر را بیشینه کنند:

$$\max(|x'_1 - x'_2|, |y'_1 - y'_2|).$$

محاسبه ساده است؛ تفاضل افقی یا عمودی در مختصات جدید باید بیشینه باشد.

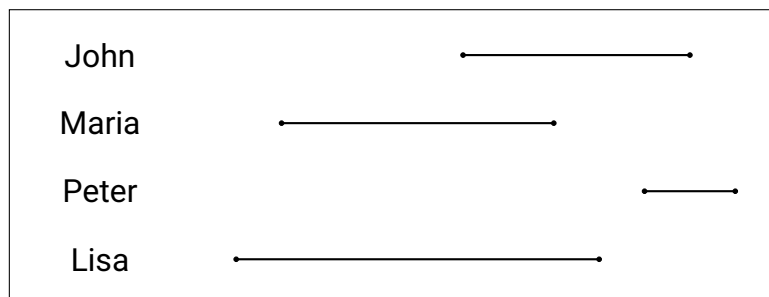
فصل ۳۰

الگوریتم‌های خط جاروب

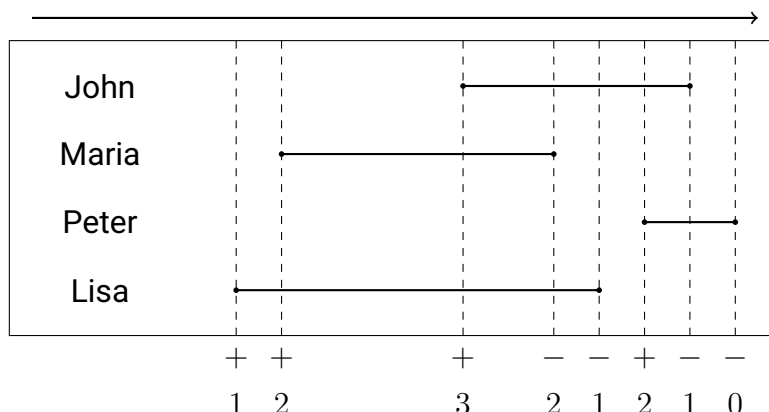
بسیاری از مسئله‌های هندسی با الگوریتم‌های **خط جاروب** (sweep line) حل می‌شوند. ایده کار نمایش مسئله به شکل مجموعه‌ای از رویدادها است که متناظر با نقاطی در صفحه هستند. رویدادها به ترتیب صعودی بر اساس مختصات x یا y پردازش می‌شوند. به عنوان مثال، مسئله زیر را در نظر بگیرید: شرکتی دارای n کارمند است. زمان ورود و خروج هر کارمند در روزی مشخص معلوم است. هدف، محاسبه بیشینه تعداد کارمندانی است که هم‌زمان در محل کار حضور داشته‌اند. مسئله با مدل‌سازی شرایط حل می‌شود؛ به طوری که به هر کارمند دو رویداد متناظر با زمان ورود و خروج نسبت داده می‌شود. پس از مرتب‌سازی رویدادها، آن‌ها را پیمایش کرده و تعداد افراد حاضر در دفتر شمرده می‌شود. به عنوان مثال، جدول

شخص	زمان ورود	زمان خروج
جان	۱۰	۱۵
ماریا	۶	۱۲
پیتر	۱۴	۱۶
لیزا	۵	۱۳

متناظر با رویدادهای زیر است:



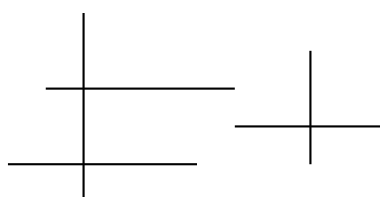
رویدادها از چپ به راست پیمایش شده و شمارنده‌ای نگهداری می‌شود. با ورود هر شخص یک واحد به شمارنده اضافه شده و با خروج شخص یک واحد کم می‌شود. پاسخ مسئله، بیشینه مقدار شمارنده در طول اجرای الگوریتم است. در مثال فوق، پردازش رویدادها به این ترتیب است:



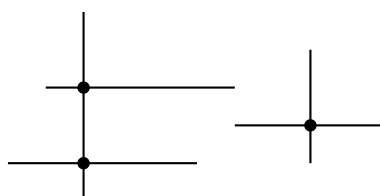
نمادهای + و - نشان می‌دهند که مقدار شمارنده افزایش یافته یا کاهش، و مقدار شمارنده در پایین نمایش داده شده است. بیشترین مقدار شمارنده برابر ۳ و در بازه زمانی بین ورود جان و خروج ماریا است. زمان اجرای الگوریتم $O(n \log n)$ است؛ مرتب‌سازی رویدادها $O(n \log n)$ زمان می‌برد و باقی الگوریتم نیازمند $O(n)$ زمان است.

۱.۳۰ نقاط تقاطع

مجموعه‌ای از n پاره‌خط افقی یا عمودی داده شده است؛ مسئله، شمارش کل نقاط تقاطع است. برای نمونه، پاره‌خط‌های زیر را در نظر بگیرید:



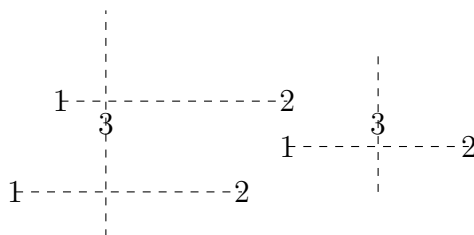
در اینجا سه نقطه تقاطع وجود دارد:



حل مسئله در زمان $O(n^2)$ ساده است؛ تمام جفت پاره‌خط‌های ممکن بررسی و تقاطع آن‌ها ارزیابی می‌شود. با این حال، می‌توان مسئله را کارآمدتر در زمان $O(n \log n)$ با الگوریتم خط جاروب و یک ساختار داده پرس‌وجوی بازه‌ای حل کرد. ایده کار، پردازش نقاط انتهایی پاره‌خط‌ها از چپ به راست و تمرکز روی سه نوع رویداد است:

- (۱) شروع پاره‌خط افقی
- (۲) پایان پاره‌خط افقی
- (۳) پاره‌خط عمودی

رویدادهای متناظر با مثال:



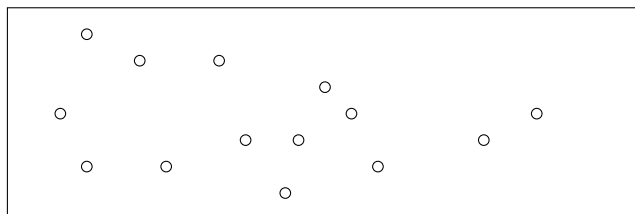
رویدادها را از چپ به راست بررسی می‌کنیم و از ساختمان داده‌ای استفاده می‌کنیم که مجموعه‌ای از مختصات‌های y مربوط به پاره‌خط‌های افقی فعال را نگه می‌دارد. در رویداد ۱، مختصات y پاره‌خط را به مجموعه اضافه می‌کنیم و در رویداد ۲، مختصات y را از مجموعه حذف می‌کنیم.

نقاط تقاطع در رویداد ۳ محاسبه می‌شوند. هنگامی که یک پاره‌خط عمودی بین نقاط y_1 و y_2 وجود دارد، تعداد پاره‌خط‌های افقی فعال با مختصات y بین y_1 و y_2 را می‌شماریم و این مقدار را به تعداد کل نقاط تقاطع اضافه می‌کنیم.

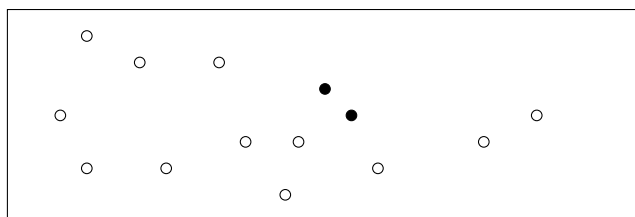
برای ذخیره مختصات‌های y پاره‌خط‌های افقی، می‌توان از درخت پدیده‌ای دودویی یا درخت بازه‌ای، احتمالاً همراه با فشرده‌سازی مختصات استفاده کرد. با استفاده از چنین ساختارهایی، پردازش هر رویداد $O(\log n)$ زمان می‌برد، بنابراین پیچیدگی زمانی کل الگوریتم $O(n \log n)$ است.

۲.۳۰ مسئله نزدیک‌ترین جفت نقطه

با داشتن مجموعه‌ای از n نقطه، مسئله بعدی یافتن دو نقطه با کمترین فاصله اقلیدسی است. برای مثال، اگر نقاط به صورت زیر باشند:



باید نقاط زیر را بیابیم:

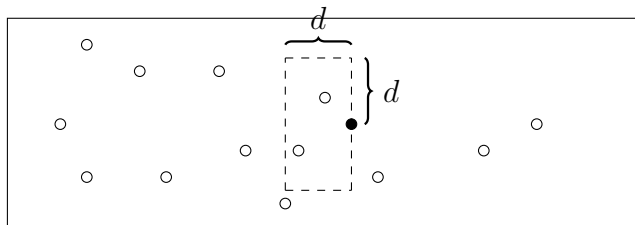


این نیز نمونه دیگری از مسئله‌ای است که می‌تواند با زمان $O(n \log n)$ با استفاده از الگوریتم خط جاروب حل شود.^۱ نقاط را از چپ به راست پیمایش می‌کنیم و مقداری به نام d را نگه می‌داریم: کمترین فاصله بین دو نقطه دیده شده تا این لحظه. در هر نقطه، نزدیک‌ترین نقطه در سمت چپ آن را می‌یابیم. اگر این فاصله کمتر از d باشد، این فاصله کمترین فاصله جدید است و مقدار d را به‌روزرسانی می‌کنیم.

^۱علاوه بر این روش، یک الگوریتم تقسیم و حل با زمان $O(n \log n)$ نیز وجود دارد [۵۶] که نقاط را به دو مجموعه تقسیم می‌کند و مسئله را به صورت بازگشتی برای هر دو مجموعه حل می‌کند.

اگر نقطه فعلی (x, y) باشد و نقطه‌ای در سمت چپ با فاصله‌ای کمتر از d وجود داشته باشد، مختصات x چنین نقطه‌ای باید در بازه $[x-d, x]$ و مختصات y آن در بازه $[y-d, y+d]$ قرار گیرد. بنابراین، تنها بررسی نقاط واقع در این بازه‌ها کافی است، که الگوریتم را کارآمد می‌سازد.

به عنوان مثال، در تصویر زیر، ناحیه مشخص شده با خط چین شامل نقاطی است که می‌توانند در فاصله کمتر از d نسبت به نقطه فعال باشند:

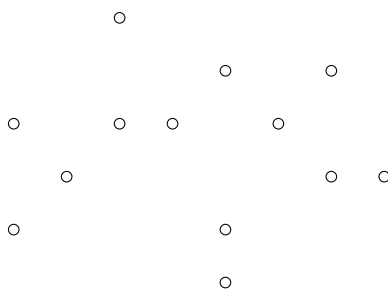


کارایی این الگوریتم مبتنی بر این واقعیت است که این ناحیه همواره تنها شامل $O(1)$ نقطه است. می‌توان این نقاط را در زمان $O(\log n)$ پیمایش کرد، با نگهداری مجموعه‌ای از نقاط که مختصات x آن‌ها در بازه $[x-d, x]$ است، مرتب شده به صورت صعودی بر اساس مختصات y آن‌ها.

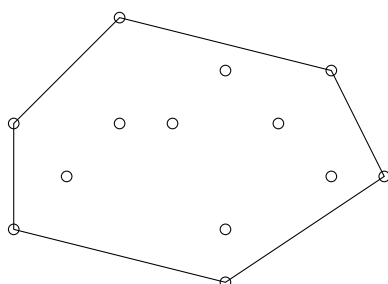
پیچیدگی زمانی الگوریتم $O(n \log n)$ است، چرا که n نقطه را پیمایش می‌کنیم و برای هر نقطه، نزدیک‌ترین نقطه در سمت چپ را در زمان $O(\log n)$ می‌یابیم.

۳.۳۰ مسئله پوش محدب

یک **پوش محدب** کوچک‌ترین چندضلعی محدبی است که تمام نقاط یک مجموعه مفروض را در بر می‌گیرد. محدب بودن به این معناست که پاره‌خط بین هر دو رأس چندضلعی، به طور کامل درون چندضلعی قرار دارد. به عنوان مثال، برای نقاط



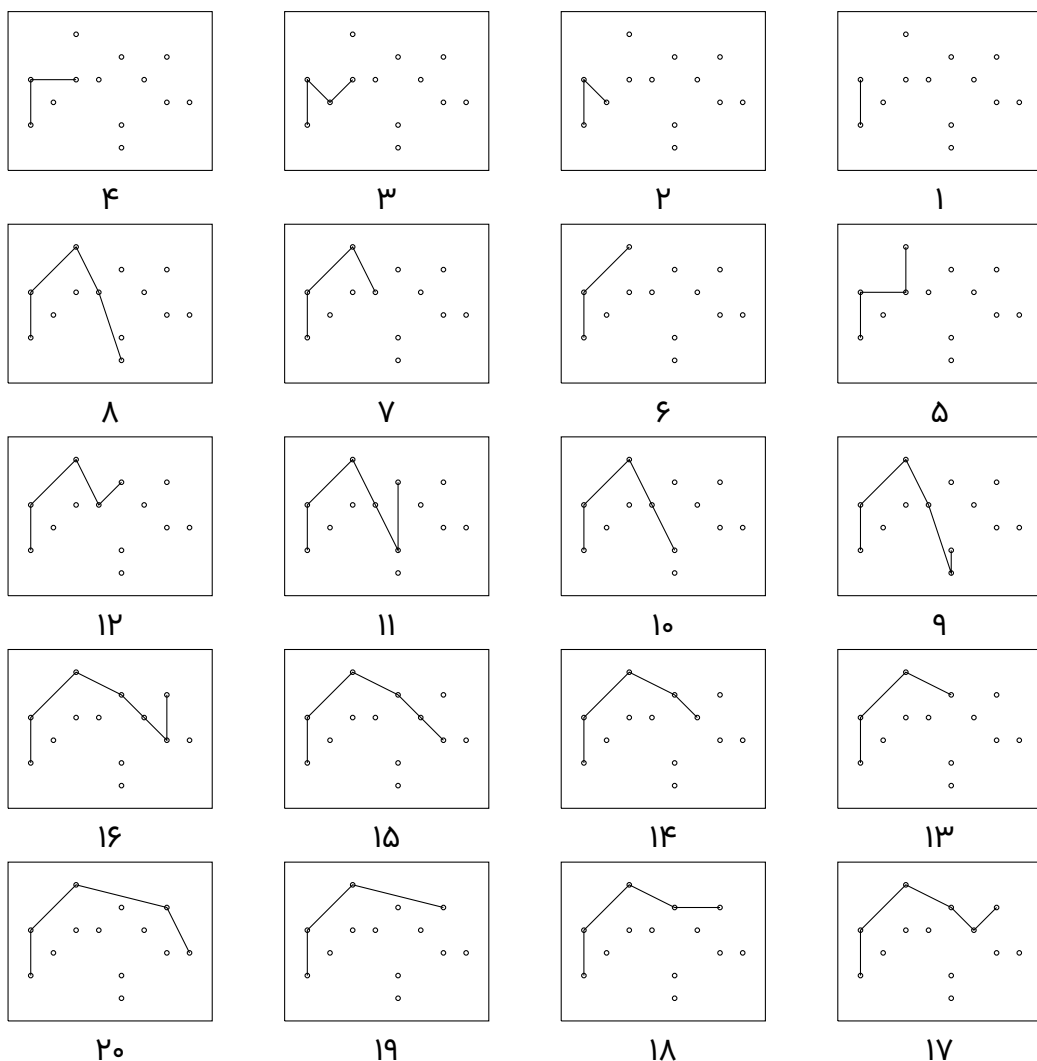
پوش محدب به صورت زیر است:



الگوریتم اندرو [۳] روشی ساده برای ساخت پوش محدب مجموعه‌ای از نقاط در زمان $O(n \log n)$ ارائه می‌دهد. الگوریتم ابتدا چپ‌ترین و راست‌ترین نقاط را پیدا می‌کند، سپس پوش محدب را در دو بخش می‌سازد: اول پوش بالایی و بعد پوش پایینی. هر دو بخش مشابه‌اند، پس تمرکز روی ساخت پوش بالایی است.

نخست، نقاط را در درجه اول بر اساس مختصات x و سپس بر اساس مختصات y مرتب می‌کنیم. پس از این، نقاط را پیمایش کرده و هر نقطه را به پوش اضافه می‌کنیم. همواره پس از افزودن یک نقطه به پوش، مطمئن می‌شویم آخرین پاره‌خط در پوش به چپ گردش نکند. تا زمانی که به چپ گردش کند، نقطه ماقبل آخر را مکرراً از پوش حذف می‌کنیم.

تصاویر زیر نحوه کار الگوریتم اندرو را نشان می‌دهند:



کتابنامه

- [۱] Algorithms, and Structures Data Ullman. J. and Hopcroft E. J. Aho, V. A. [1]
Addison-Wesley, ۱۹۸۳.
- [۲] algorithms path augmenting directed Distance Orlin. B. J. and Ahuja K. R. [2]
Re- Naval problems. flow maximum parametric and flow maximum for
Logistics, search ۳۸ (۳):۴۱۳-۴۳۰, ۱۹۹۱.
- [۳] dimen- two in hulls convex for algorithm efficient Another Andrew. M. A. [3]
Letters, Processing Information sions. ۹ (۵):۲۱۶-۲۱۹, ۱۹۷۹.
- [۴] testing for algorithm linear-time A Tarjan. E. R. and Plass F. M. Aspvall, B. [4]
Processing Information formulas. boolean quantified certain of truth the
Letters, ۸ (۳):۱۲۱-۱۲۳, ۱۹۷۹.
- [۵] Mathematics. Applied of Quarterly problem. routing a On Bellman. R. [5]
۱۶ (۱):۸۷-۹۰, ۱۹۵۸.
- [۶] representations integer New Jensen. Y. K. and Tarrat W. Pine, E. Beck, M. [6]
Computation, of Mathematics cubes. three of sum the as
۷۶ (۲۵۹):۱۶۸۳-۱۶۹۰, ۲۰۰۷.
- [۷] Latin In revisited. problem LCA The Farach-Colton. M. and Bender A. M. [7]
Informatics, Theoretical on Symposium American ۸۸-۹۴, ۲۰۰۰.
- [۸] (ویرایش دوم). ۱۹۹۹ Addison-Wesley, Pearls. Programming Bentley. J. [8]
- [۹] in- reporting for algorithm case worst optimal An Wood. D. and Bentley J. [9]
Computers, on Transactions IEEE rectangles. of tersections
۲۹ (۷):۵۷۱-۵۷۷, ۱۹۸۰.
- [۱۰] Annals theory. mathematical complete a with game a Nim, Bouton. L. C. [10]
Mathematics, of ۳ (۱/۴):۳۵-۳۹, ۱۹۰۱.
- [۱۱] مسابقات آزاد انفورماتیک کرواسی, <http://hsin.hr/coci/> [11]
- [۱۲] کدفورسیز: پیرامون «الگوریتم مو», <http://codeforces.com/blog/entry/> [12]
20032
- [۱۳] to Introduction Stein. C. and Rivest L. R. Leiserson, E. C. Cormen, H. T. [13]
MIT Algorithms, Press, ۲۰۰۹ (ویرایش سوم).

- Nu- graphs. with connexion in problems two on note A Dijkstra. W. E. [١٤]
 .١٩٥٩ ،٢٧١-٢٦٩:(١)١ Mathematik, merische
- the from Set Problem Ultimate The Challenge? a for Looking al. et Diks K. [١٥]
 Warsaw, of University Competitions, Programming Warsaw of University
 .٢٠١٢
- binary using queries minimum range Efficient Ceterchi. R. and Dima M. [١٦]
 .٢٠١٥ ،٤٤-٣٩:(١)٩ Informatics, in Olympiad trees. indexed
- Mathematics. of Journal Canadian flowers. and trees. Paths. Edmonds. J. [١٧]
 .١٩٦٥ ،٤٦٧-٤٤٩:(٣)١٧
- ef- algorithmic in improvements Theoretical Karp. M. R. and Edmonds J. [١٨]
 ،٢٦٤-٢٤٨:(٢)١٩ ACM, the of Journal problems. flow network for ficiency
 .١٩٧٢
- multi- and table time of complexity the On Shamir. A. and Itai A. Even, S. [١٩]
 of Foundations on Symposium Annual ١٦th problems. flow commodity
 .١٩٧٥ ،١٩٣-١٨٤ Science, Computer
- South- of Journal SPFA. - shortest-path for algorithm faster A Fanding. D. [٢٠]
 .١٩٩٤ ،٢ University, Jiaotong west
- Soft- tables. frequency cumulative for structure data new A Fenwick. M. P. [٢١]
 .١٩٩٤ ،٣٣٦-٣٢٧:(٣)٢٤ Experience, and Practice ware:
- the on improvements practical and Theoretical Heun. V. and Fischer J. [٢٢]
 Symposium Annual In LCE. and LCA to applications with RMQ-problem.
 .٢٠٠٦ ،٤٨-٣٦ Matching, Pattern Combinatorial on
- ACM, the of Communications path. shortest :٩٧ Algorithm Floyd W. R. [٢٣]
 .١٩٦٢ ،٣٤٥:(٦)٥
- Cali- Monica, Santa Corporation, RAND theory. flow Network Ford. R. L. [٢٤]
 .١٩٥٦ fornia,
- Canadian network. a through flow Maximal Fulkerson. R. D. and Ford R. L. [٢٥]
 .١٩٥٦ ،٤٠٤-٣٩٩:(٣)٨ Mathematics, of Journal
- IFIP In time. running less use can machines Probabilistic Freivalds. R. [٢٦]
 .١٩٧٧ ،٨٤٢-٨٣٩ congress,
- Proceed- In multiplication. matrix fast and tensors of Powers Gall. Le F. [٢٧]
 Com- Algebraic and Symbolic on Symposium International ٣٩th the of ings
 .٢٠١٤ ،٣٠٣-٢٩٦ putation,
- to Guide A Intractability: and Computers Johnson. S. D. and Garey R. M. [٢٨]
 .١٩٧٩ Company, and Freeman H. W. NP-Completeness, of Theory the
- <https://www.go-hero.net/jam/17> ،(٢٠١٧) Jam Code Google آمار مسابقات [٢٩]

triangles, love and degenerates, Threesomes, Pettie. S. and Grønlund A. [٣٠]
Com- of Foundations on Symposium Annual ٨٨th the of Proceedings In
.٢٠١٤ ,٩٣٠-٩٣١ Science, puter

.١٩٣٩ ,٨-٩:(٨)٢ Eureka, games. and Mathematics Grundy. M. P. [٣١]

Sci- Computer Sequences: and Trees Strings, on Algorithms Gusfield. D. [٣٢]
.١٩٩٧ Press, University Cambridge Biology, Computational and ence

Lower New The :٣ Programming Competitive Halim. F. and Halim S. [٣٣]
.٢٠١٣ Contests, Programming of Bound

sequencing to approach programming dynamic A Karp. M. R. and Held M. [٣٤]
Mathematics, Applied and Industrial for Society the of Journal problems.
.١٩٩٢ ,٢١٠-١٩٩:(١)١٠

ohne Linienzug einen Möglichkeit, die Über Wiener. C. and Hierholzer C. [٣٥]
An- Mathematische umfahren. zu Unterbrechung ohne und Wiederholung
.١٨٧٣ ,٣٢-٣٠ , (١)٩ nalen,

ACM, the of Communications Quicksort. :٩٤ Algorithm Hoare. R. A. C. [٣٦]
.١٩٩١ ,٣٢١:(٧)٤

ACM, the of Communications Find. :٩٥ Algorithm Hoare. R. A. C. [٣٧]
.١٩٩١ ,٣٢٢-٣٢١:(٧)٤

Technical algorithm. merging list linear A Ullman. D. J. and Hopcroft E. J. [٣٨]
.١٩٧١ University, Cornell report,

the to applications with partitions Computing Sahni. S. and Horowitz E. [٣٩]
.١٩٧٤ ,٢٩٢-٢٧٧:(٢)٢١ ACM, the of Journal problem. knapsack

minimum-redundancy of construction the for method A Huffman. A. D. [٤٠]
.١٩٥٢ ,١١٠١-١٠٩٨:(٩)٤٠ IRE, the of Proceedings codes.

<https://people.ksp.Syllabus, Informatics in Olympiad International The> [٤١]
<sk/~misof/ioi-syllabus/>

algo- pattern-matching randomized Efficient Rabin. O. M. and Karp M. R. [٤٢]
.١٩٨٧ ,٢٩٠-٢٤٩:(٢)٣١ Development, and Research of Journal IBM rithms.

of number The I. lattice: a on dimers of statistics The Kasteleyn. W. P. [٤٣]
.١٩٩١ ,١٢٢٥-١٢٠٩:(١٢)٢٧ Physica, lattice. quadratic a on arrangements dimer

sparse of complexity the On Ziv-Ukelson. M. and Landau M. G. Kent, C. [٤٤]
.٢٠٠٩ ,١٠٢٧-١٠١٣:(٨)١٣ Biology, Computational of Journal assembly. exon

.٢٠٠٥ Pearson, Design, Algorithm Tardos. É. and Kleinberg J. [٤٥]

Seminumeri- :٢ Volume Programming. Computer of Art The Knuth. E. D. [٤٦]
edition). (٣rd ١٩٩٨ Addison-Wesley, Algorithms, cal

- and Sorting :۳ Volume Programming. Computer of Art The Knuth. E. D. [۴۷]
edition). (۲nd ۱۹۹۸ Addison-Wesley, Searching,
- trav- the and graph a of subtree spanning shortest the On Kruskal. B. J. [۴۸]
So- Mathematical American the of Proceedings problem. salesman eling
۱۹۵۶, ۵۰-۴۸:(۱)۷ ciety,
- inser- deletions. correcting of capable codes Binary Levenshtein. I. V. [۴۹]
۱۹۶۶, ۷۱۰-۷۰۷:(۸)۰ doklady, physics Soviet reversals. and tions,
- repe- all finding for algorithm $O(n \log n)$ An Lorentz. J. R. and Main G. M. [۵۰]
۱۹۸۴, ۴۳۲-۴۲۲:(۳)۵ Algorithms, of Journal string. a in titions
- poly- use to not how and use to Where Radoszewski. J. and Pachocki J. [۵۱]
۲۰۱۳, ۱۰۰-۹۰:(۱)۷ Informatics, in Olympiads hashing. string nomial
- Discrete problem. tour Knight's the for algorithm efficient An Parberry. I. [۵۲]
۱۹۹۷, ۲۶۰-۲۵۱:(۳)۷۳ Mathematics, Applied
- prob- change-making the for algorithm polynomial-time A Pearson. D. [۵۳]
۲۰۰۵, ۲۳۴-۲۳۱:(۳)۳۳ Letters, Research Operations lem.
- Bell generalizations. some and networks connection Shortest Prim. C. R. [۵۴]
۱۹۵۷, ۱۴۰۱-۱۳۸۹:(۶)۳۶ Journal, Technical System
- Count- Solution and Enumeration Parallel Massively Puzzle: ۲۷-Queens [۵۵]
<https://github.com/preusser/q27ing>.
- of Proceedings In problems. Closest-point Hoey. D. and Shamos I. M. [۵۶]
۱۶۲-۱۵۱ Science, Computer of Foundations on Symposium Annual ۱۶th the
۱۹۷۵
- data in applications its and algorithm strong-connectivity A Sharir. M. [۵۷]
۷۲-۶۷:(۱)۷ Applications, with Mathematics & Computers analysis. flow
۱۹۸۱
- .(ویرایش دوم) ۲۰۰۸ Springer, Manual, Design Algorithm The Skiena. S. S. [۵۸]
- Program- The Challenges: Programming Revilla. A. M. and Skiena S. S. [۵۹]
۲۰۰۳ Springer, Manual, Training Contest ming
<https://szkopul.edu.pl/SZKOpul>, [۶۰]
- Mathematical Tohoku Kampfspiele. mathematische Über Sprague. R. [۶۱]
۱۹۳۵, ۴۴۴-۴۱:۴۳۸ Journal,
- Informatycznych, konkursach w praktyczna Algorytmika Stańczyk. P. [۶۲]
پایان نامه کارشناسی ارشد، دانشگاه ورشو، ۲۰۰۶.
- Mathe- Numerische optimal. not is elimination Gaussian Strassen. V. [۶۳]
۱۹۶۹, ۳۵۶-۳۵۴:(۴)۱۳ matik,

Jour- algorithm. union set linear not but good a of Efficiency Tarjan. E. R. [٦٤]
١٩٧٥ ، ٢٢٥-٢١٥:(٢)٢٢ ACM، the of nal

Journal trees. balanced on compression path of Applications Tarjan. E. R. [٦٥]
١٩٧٩ ، ٧١٥-٦٩٠:(٤)٢٦ ACM، the of

com- and componemts biconnected Finding Vishkin. U. and Tarjan E. R. [٦٦]
the of Proceedings In time. parallel logarithmic in functions tree puting
، ٢٠-١٢ Science، Computer of Foundations on Symposium Annual ٢٥th
١٩٨٤

mechan- statistical in problem Dimer Fisher. E. M. and Temperley V. N. H. [٦٧]
١٩٦١ ، ١٠٦٣-١٠٦١:(٦٨)٦ Magazine، Philosophical result. exact an – ics

<http://www.usaco.org/> (USACO)، المپیاد کامپیوتر آمریکا [٦٨]

allgemeinste und einfachste Rösselsprunget Des Warnsdorf. von C. H. [٦٩]
١٨٢٣ Schmalkalden، Lösung.

-١١:(١)٩ ACM، the of Journal matrices. boolean on theorem A Warshall. S. [٧٠]
١٩٦٢ ، ١٢

نمایه

- تفاضل، ۱۲
- ۹۵ array، difference
- ۲۰۴ equation، Diophantine
- ۱۷۹ theorem، Dirac's
- ۱۳ disjunction،
- توزیع، ۲۲۹
- آرایه پویا، ۳۵
- ۱۱۱ edge،
- ۱۳ equivalence،
- الگوریتم اقلیدس، ۲۰۰
- تابع فی اویلر، ۲۰۱
- ۱۷۶ circuit، Eulerian
- ۱۷۵ path، Eulerian
- ۲۰۴ algorithm، Euclid's extended
- ۱۰ formula، Faulhaber's
- ۸۸ tree، Fenwick
- ۷ number، point floating
- ۱۵۸ algorithm، Floyd's
- ۱۸۴ algorithm، Ford–Fulkerson
- ۲۳۲ algorithm، Freivalds'
- توزیع هندسی، ۲۳۰
- ۱۱ progression، geometric
- هندسه، ۲۶۵
- ۱۱۱ graph،
- بزرگ‌ترین مقسوم‌علیه مشترک، ۲۰۰
- ۲۳۹ number، Grundy
- ۲۴۱ game، Grundy's
- قضیه هال، ۱۹۰
- مدار همپلتونی، ۱۷۹
- مسیر همپلتونی، ۱۷۹
- ۱۱ sum، harmonic
- مجموع هارمونیک، ۲۰۰
- ۲۴۵ value، hash
- ۲۴۵ hashing،
- ۴۴ heap،
- فرمول هرون، ۲۶۵
- ۱۱۵ list، adjacency
- الفبا، ۲۴۳
- تحلیل سرشکن، ۷۹
- نیاکان، ۱۶۵
- ۹۸ operation، and
- ۱۰ progression، arithmetic
- عقب‌گرد، ۵۲
- کد دودویی، ۶۴
- ۸۸ tree، indexed binary
- ۳۱ search، binary
- توزیع دوجمله‌ای، ۲۳۰
- ۲۴۷ paradox، birthday
- ۹۹ shift، bit
- ۴۲ bitset،
- ۱۲۱ search، breadth-first
- کلمه‌کد، ۶۴
- ۲۳۳ coloring،
- ۳۱ function، comparison
- ۳۰ operator، comparison
- ۱۲، متمم،
- ۲۶۶، complex
- ۲۶۶ number، complex
- ۲۰ classes، complexity
- ۲۲۷ probability، conditional
- ۱۳ conjunction،
- ۲۱ factor، constant
- ۲۰ algorithm، constant-time
- نسبت به هم اول، ۲۰۱
- ۲۶۸ product، cross
- ۲۰ algorithm، cubic
- ۱۸۴ cut،
- ۱۵۷، ۱۱۱ cycle،
- ۱۵۷ detection، cycle
- فشرده‌سازی داده‌ها، ۶۴
- ساختمان داده، ۳۵
- ۱۱۹ search، depth-first
- ۴۳ deque،

- دوره تناوب، ۲۴۳
 ۵۱ permutation،
 ۲۶۲ tree، segment persistent
 ۲۶۶ point،
 ۲۱ algorithm، polynomial
 ۲۴۵ hashing، polynomial
 ۱۳ predicate،
 پیشوند، ۲۴۳
 آرایه مجموع پیشوندی، ۸۶
 ۴۴ queue، priority
 زبان برنامه‌نویسی، ۳
 ۲۰ algorithm، quadratic
 ۱۳ quantifier،
 مسئله وزیرها، ۵۲
 ۴۴ queue،
 ۲۳۲ quickselect،
 ۲۳۲ quicksort،
 ۳۹، random_shuffle
 ۲۳۱ algorithm، randomized
 پرسش بازه‌ای، ۸۵
 ۶ remainder،
 ۳۹، reverse
 دوران، ۲۴۳
 ۹۱ tree، segment
 مجموعه، ۱۲
 نظریه مجموعه‌ها، ۱۲
 ۳۹، ۲۹، sort
 ۲۲۳ tree، spanning
 ۲۶۱ tree، segment sparse
 ۸۷ table، sparse
 ۲۳۸ theorem، Sprague–Grundy
 ۴۳ stack،
 رشته، ۲۴۳
 رشته (string)، ۳۶
 ۲۴۵ hashing، string
 زیردنباله، ۲۴۳
 زیرمجموعه، ۱۲
 زیررشته، ۲۴۳
 پسوند، ۲۴۳
 پرسش مجموع، ۸۵
 ۲۷۵ line، sweep
 پرسش درختی، ۱۶۵
 آرایه پیمایش درخت، ۱۶۶
 ۳۰، tuple
 ۱۷۷ algorithm، Hierholzer's
 ۱۳ implication،
 ۲۲۸ independence،
 ۹۵ compression، index
 ۶ integer،
 اشتراک، ۱۲
 ۲۳ algorithm، Kadane's
 ۲۲۳ theorem، Kirchhoff's
 ۷۴ knapsack،
 ۲۳۱ algorithm، Vegas Las
 کوچک‌ترین مضرب مشترک، ۲۰۰
 ۲۰ algorithm، linear
 ۲۰ algorithm، logarithmic
 ۱۳ logic،
 طولانی‌ترین زیردنباله صعودی، ۷۲
 ۹ macro،
 زنجیره مارکوف، ۲۳۰
 تطابق، ۱۸۹
 ۲۳۲ multiplication، matrix
 تطابق بیشینه، ۱۸۹
 پرسش بیشینه، ۸۵
 ۲۱ sum، subarray maximum
 ۲۳۹ function، mex
 ۱۸۴ cut، minimum
 پرسش کمینه، ۸۵
 ۲۳۸ game، misère
 ۲۵۵ algorithm، Mo's
 ۶ arithmetic، modular
 حساب پیمانه‌ای، ۲۰۱
 ۲۳۱ algorithm، Carlo Monte
 ۱۳ negation،
 ۵۱، next_permutation
 ۱۱۱ node،
 ۹۹ operation، not
 ۲۱ problem، NP-hard
 ۹۸ operation، or
 ۲۳۱ statistic، order
 ۱۷۹ theorem، Ore's
 ۳۰، pair
 ۱۱۱ path،
 تطابق الگو، ۲۴۳
 تطابق کامل، ۱۹۰

تقاطع پاره‌خط‌ها، ۲۶۸	typedef، ۸
توان ماتریس، ۲۱۹	روش دو اشاره‌گر، ۷۹
تکرارکننده، ۳۹	۲۶۴ tree، segment two-dimensional
تکنیک تور اوپلری، ۱۷۰	توزیع یکنواخت، ۲۲۹
حداقل پوشش راس، ۱۹۱	اجتماع، ۱۲
حدس لژاندر، ۱۹۹	مجموعه مرجع، ۱۲
حدس گلدباخ، ۱۹۹	۲۶۶ vector،
دترمینان، ۲۱۹	بردار، (vector)، ۳۵
درجه، ۱۱۳	۹۹ operation، xor
درجه خروجی، ۱۱۳	آرایه Z، ۲۴۷
درجه ورودی، ۱۱۳	احتمال، ۲۲۵
درخت، ۱۱۲، ۱۳۵	اعداد اول دوقلو، ۱۹۹
درخت بازه‌ای، ۲۵۷	اعداد فیبوناچی، ۲۲۰
درخت دودویی، ۱۴۱	الگوریتم SPFA، ۱۲۸
درخت ریشه‌دار، ۱۳۵	الگوریتم Z، ۲۴۷
درخت فراگیر، ۱۴۳	الگوریتم ادموندز-کارپ، ۱۸۶
درخت فراگیر بیشینه، ۱۴۴	الگوریتم اندرو، ۲۷۹
درخت فراگیر کمینه، ۱۴۳	الگوریتم بلمن-فورد، ۱۲۵
درخت پاره‌خطی تنبل، ۲۵۸	الگوریتم حریصانه، ۵۹
درخت پاره‌خطی پویا، ۲۶۱	الگوریتم دایکسترا، ۱۲۸، ۱۵۵
دنباله دی بروین، ۱۸۰	الگوریتم ریشه دوم، ۲۵۱
دور، ۱۲۳، ۱۵۱	الگوریتم فلویید-وارشال، ۱۳۱
دور منفی، ۱۲۷	الگوریتم مقیاس‌بندی، ۱۸۷
رابطه بازگشتی خطی، ۲۲۰	الگوریتم پریم، ۱۴۹
رنگ‌آمیزی، ۱۱۳	الگوریتم کروسکال، ۱۴۴
روش ابتکاری، ۱۸۱	الگوریتم کوساراجو، ۱۶۰
ریشه، ۱۳۵	امید ریاضی، ۲۲۸
زیردرخت، ۱۳۵	انتشار تنبل، ۲۵۸
زیرمجموعه، ۴۹	بازی نیم، ۲۳۷
ساختار مجموعه‌های مجزا، ۱۴۷	بخش‌پذیری، ۱۹۷
سه‌تایی فیثاغورسی، ۲۰۶	بردار، ۲۱۷
شاره، ۱۸۳	برش کمینه، ۱۸۷
شمول و عدم شمول، ۲۱۲	برنامه‌نویسی پویا، ۶۷
ضرب ماتریس، ۲۱۸	برگ، ۱۳۵
ضریب دوجمله‌ای، ۲۰۸	بیشینه شاره، ۱۸۳
ضریب چندجمله‌ای، ۲۱۰	تابع فاصله، ۲۷۲
عامل، ۱۹۷	تجزیه به عوامل اول، ۱۹۷
عبارت پرانتزی، ۲۱۰	ترانهاده، ۲۱۷
عدد اول، ۱۹۷	ترای، ۲۴۴
عدد فیبوناچی، ۱۴، ۲۰۶	ترتیب الفبایی، ۲۴۴
عدد کاتالان، ۲۱۰	ترکیبیات، ۲۰۷
عدد کامل، ۱۹۸	تصادم، ۲۴۶
غربال اراتستن، ۲۰۰	تصادد حسابی، ۱۰
فاصله اقلیدسی، ۲۷۲	تصادد هندسی، ۱۱
فاصله لون‌اشتاین، ۷۶	
فاصله منهن، ۲۷۲	

- فاصله همینگ، ۱۰۲
 فاصله ویرایشی، ۷۶
 فاکتوریل، ۱۳
 فرزند، ۱۳۵
 فرمول اقلیدس، ۲۰۶
 فرمول بند کفش، ۲۷۱
 فرمول بینه، ۱۴
 فرمول فالهاپر، ۱۰
 فرمول کیلی، ۲۱۵
 فهرست یال‌ها، ۱۱۷
 قاعده وارنسدورف، ۱۸۱
 قضیه اوایلر، ۲۰۲
 قضیه باقیمانده چینی، ۲۰۵
 قضیه دیلوورث، ۱۹۴
 قضیه زکندورف، ۲۰۶
 قضیه فرما، ۲۰۲
 قضیه لاگرانژ، ۲۰۵
 قضیه ویلسون، ۲۰۶
 قضیه پیک، ۲۷۲
 قضیه کونینگ، ۱۹۱
 قطر، ۱۳۷
 لم برنساید، ۲۱۴
 لگاریتم، ۱۴
 لگاریتم طبیعی، ۱۵
 مؤلفه، ۱۱۲
 مؤلفه قویاً همبند، ۱۵۹
 ماتریس، ۲۱۷
 ماتریس لاپلاس، ۲۲۴
 ماتریس مجاورت، ۱۱۶
 ماتریس مربعی، ۲۱۷
 ماتریس معکوس، ۲۲۰
 ماتریس همانی، ۲۱۸
 متغیر تصادفی، ۲۲۸
 مثلث پاسکال، ۲۰۹
 مجموع نیم، ۲۳۷
 مجموع هارمونیک، ۱۱
 مجموعه، ۳۷
 مجموعه مستقل، ۱۹۲
 مجموعه مستقل بیشینه، ۱۹۲
 مرتب‌سازی، ۲۵
 مرتب‌سازی ادغامی، ۲۷
 مرتب‌سازی توپولوژیکی، ۱۵۱
 مرتب‌سازی حبابی، ۲۵
 مرتب‌سازی شمارشی، ۲۸
 مرز، ۲۴۴
 مسئله SAT، ۱۶۲
- مسئله SUM، ۸۰
 مسئله SAT، ۱۶۴
 مسئله SUM، ۸۱
 معکوس پیمانه‌ای، ۲۰۳
 مقسوم‌علیه، ۱۹۷
 ملاقات در میانه، ۵۷
 میان‌ترتیب، ۱۴۱
 نزدیک‌ترین جفت نقطه، ۲۷۷
 نزدیک‌ترین عناصر کوچک‌تر، ۸۱
 نظریه اعداد، ۱۹۷
 نقطه تقاطع، ۲۷۶
 نمایش بی‌تی، ۹۷
 نگاشت، ۳۸
 همساز (cofactor)، ۲۱۹
 همسایه، ۱۱۳
 وارونگی، ۲۶
 والد، ۱۳۵
 ورودی و خروجی، ۴
 وضعیت بازنده، ۲۳۵
 وضعیت برنده، ۲۳۵
 پادزنجیر، ۱۹۴
 پایین‌ترین جد مشترک، ۱۶۹
 پریش، ۲۱۳
 پس‌ترتیب، ۱۴۱
 پنجره لغزان، ۸۲
 پوشش راس، ۱۹۱
 پوشش مسیری، ۱۹۲
 پیش‌ترتیب، ۱۴۱
 پیچیدگی زمانی، ۱۷
 کد پروفِر، ۲۱۵
 کدگذاری هافمن، ۶۵
 کمینه در پنجره لغزان، ۸۲
 کوتاه‌ترین مسیر، ۱۲۵
 گراف تابعی، ۱۵۶
 گراف جانشین، ۱۵۶
 گراف جهت‌دار، ۱۱۲
 گراف دوبخشی، ۱۲۴
 گراف دوپخش، ۱۱۳
 گراف ساده، ۱۱۴
 گراف قویاً همبند، ۱۵۹
 گراف مؤلفه‌ها، ۱۵۹
 گراف منتظم، ۱۱۳
 گراف همبند، ۱۱۲، ۱۲۳
 گراف وزن‌دار، ۱۱۳
 گراف کامل، ۱۱۳
 گشت اسب، ۱۸۰

یادداشت‌برداری، ۶۹